LAPORAN TUGAS AKHIR

IMPLEMENTASI *COMPOSABLE ARCHITECTURE* BERBASIS *STRATEGY* DAN *DECORATOR PATTERNS* PADA ALGORITMA *FUZZY* C-MEANS UNTUK KLASTERISASI KINERJA KARYAWAN

SKRIPSI

Diajukan Sebagai Pedoman Pelaksanaan Penilaian Tugas Akhir pada Jurusan Informatika Fakultas Teknik Universitas Jenderal Soedirman

Oleh:

GHAZA INDRA PRATAMA

H1D022073

KEMENTERIAN PENDIDIKAN TINGGI, SAINS, DAN TEKNOLOGI

UNIVERSITAS JENDERAL SOEDIRMAN

FAKULTAS TEKNIK

JURUSAN INFORMATIKA

PURWOKERTO

2026

LEMBAR PENGESAHAN

Tugas Akhir dengan Judul:

IMPLEMENTASI *COMPOSABLE ARCHITECTURE* BERBASIS *STRATEGY* DAN *DECORATOR PATTERNS* PADA ALGORITMA *FUZZY* C-MEANS UNTUK KLASTERISASI KINERJA KARYAWAN

Oleh:

GHAZA INDRA PRATAMA

H1D022073

Diajukan untuk memenuhi salah satu persyaratan memperoleh gelar Sarjana Komputer pada Jurusan Informatika Fakultas Teknik Universitas Jenderal Soedirman

Diterima dan disetujui

Pada tanggal : .....

Dosen Pembimbing I	**Dosen Pembimbing II**
Drs. Ir. Eddy Maryanto, M.Cs	**Dwi Kurnia Wibowo, S.Kom.,M.Kom**
NIP. 196711101993031025	NIP.199607102024061002

Mengetahui,

Dekan Fakultas Teknik

Prof. Dr. Eng. Ir. Agus Martoyo, S.T., M.T., IPU., ASEAN Eng.

NIP. 197109202006041001

LEMBAR PERNYATAAN KEASLIAN SKRIPSI

Saya yang bertanda tangan di bawah ini:

Nama : Ghaza Indra Pratama

NIM : H1D022073

Program studi : Informatika

Dengan ini, saya menyatakan bahwa dalam skripsi ini tidak terdapat karya yang pernah diajukan untuk memperoleh gelar kesarjanaan di suatu perguruan tinggi dan sepanjang pengetahuan saya juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang lain, kecuali yang secara tertulis diacu dalam naskah ini dan disebutkan dalam daftar pustaka. Demikian pernyataan ini yang saya buat dengan sebenar-benarnya tanpa ada unsur paksaan dari pihak manapun.

Purwokerto, 26 Juni 2026

Ghaza Indra Pratama

HALAMAN PERSEMBAHAN

Lembar persembahan ini merupakan wujud ungkapan terima kasih yang paling dalam sekaligus dedikasi atas segala bentuk cinta, pengorbanan, dan ilmu yang telah mengiringi proses perjuangan saya. Karya ini saya persembahkan secara khusus kepada:

1. Tuhan Yang Maha Esa
2. Bapak Prof. Dr. Eng. Ir. Agus Maryoto, ST., M.T., IPU., selaku Dekan Fakultas Teknik, Universitas Jenderal Soedirman.
3. Bapak Dr. Ir. Lasmedi Afuan, S.T., M.Cs., IPM., selaku Ketua Jurusan Informatika, Fakultas Teknik, Universitas Jenderal Soedirman.
4. Bapak Drs. Ir. Eddy Maryanto, M.Cs., selaku Dosen Pembimbing I dan, Bapak Dwi Kurnia Wibowo, S.Kom., M.Kom, selaku Dosen Pembimbing II yang dengan penuh perhatian telah memberikan bimbingan dan arahan selama penyusunan proposal ini.
5. Orang tua dan keluarga tercinta atas doa, semangat, serta dukungan yang tiada harganya.
6. Rekan-rekan seperjuangan di Jurusan Informatika angkatan 2022, serta seluruh pihak yang telah memberikan dukungan dan masukan.
7. Sosok luar biasa yang selalu setia menjadi pendengar terbaik di masa-masa sulit, menemani setiap dinamika suka dan duka, serta tanpa lelah mengalirkan dukungan, doa, dan motivasi hingga skripsi ini dapat rampung dengan indah.

HALAMAN MOTO

"You have power over your mind - not outside events. Realize this, and you will find strength." — Marcus Aurelius, *Meditations*

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan tugas akhir penelitian dengan judul "Implementasi *composable architecture* berbasis *strategy* dan *decorator patterns* pada algoritma *fuzzy c-means* untuk klasterisasi kinerja karyawan." Laporan tugas akhir ini disusun sebagai salah satu syarat untuk menempuh Tugas Akhir pada Jurusan Informatika, Fakultas Teknik, Universitas Jenderal Soedirman.

Dalam penyusunan tugas akhir ini, penulis mendapatkan berbagai dukungan, arahan, serta masukan dari banyak pihak. Oleh karena itu, dengan penuh hormat dan rasa syukur, penulis menyampaikan terima kasih kepada:

1. Bapak Prof. Dr. Eng. Ir. Agus Maryoto, ST., M.T., IPU., selaku Dekan Fakultas Teknik, Universitas Jenderal Soedirman.
2. Bapak Dr. Ir. Lasmedi Afuan, S.T., M.Cs., IPM., selaku Ketua Jurusan Informatika, Fakultas Teknik, Universitas Jenderal Soedirman.
3. Bapak Drs. Ir. Eddy Maryanto, M.Cs., selaku Dosen Pembimbing I dan, Bapak Dwi Kurnia Wibowo, S.Kom., M.Kom, selaku Dosen Pembimbing II yang dengan penuh perhatian telah memberikan bimbingan dan arahan selama penyusunan proposal ini.
4. Orang tua dan keluarga tercinta atas doa, semangat, serta dukungan yang tiada harganya.
5. Rekan-rekan seperjuangan di Jurusan Informatika angkatan 2022, serta seluruh pihak yang telah memberikan dukungan dan masukan.

Penulis menyadari proposal ini memiliki keterbatasan, sehingga kritik dan saran sangat diharapkan untuk perbaikan ke depan. Semoga karya ini dapat berkontribusi pada pengembangan ilmu, khususnya dalam data mining berbasis website.

Purwokerto, 26 Juni 2026

Ghaza Indra Pratama

DAFTAR ISI

LEMBAR PENGESAHAN i
LEMBAR PERNYATAAN KEASLIAN SKRIPSI ii
HALAMAN PERSEMBAHAN iii
HALAMAN MOTO iv
KATA PENGANTAR v
DAFTAR ISI vi
DAFTAR GAMBAR x
DAFTAR TABEL xiii
ABSTRAK xiv
ABSTRACT xv
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 3
1.3 Batasan Masalah 3
1.4 Tujuan Penelitian 4
1.5 Manfaat Penelitian 4
1.6 Luaran Penelitian 5
BAB II TINJAUAN PUSTAKA 7
2.1 Manajemen Kinerja Sumber Daya Manusia 7
2.1.1 Konsep Penilaian Kinerja 7
2.1.2 *Key Performance Indicator* (KPI) 8
2.1.3 *Human Resource Information System* (HRIS) 8
2.2 Data Mining dan Klasterisasi 9
2.2.1 Data Mining 9

2.2.2 Clustering..... 9
2.3 Algoritma Fuzzy C-Means (FCM) 10
2.3.1 Konsep Dasar FCM 10
2.3.2 Tahapan Algoritma FCM 11
2.3.3 Validasi Cluster (*Silhouette Coefficient*)..... 12
2.4 Arsitektur Perangkat Lunak Modern 14
2.4.1 Composable Architecture..... 14
2.4.2 Modular Monolith..... 14
2.5 Pola Desain Perangkat Lunak (*Software Design Patterns*)..... 15
2.5.1 Strategy Pattern..... 15
2.5.2 Decorator Pattern 16
2.6 Teknologi Pengembangan..... 16
2.6.1 NestJS *Framework*..... 16
2.6.2 React JS Framework 17
2.6.3 PostgreSQL..... 18
2.6.4 Metode Waterfall 19
2.6.5 Unified Modeling Language (UML) 20
2.6.6 Entity Relationship Diagram (ERD)..... 22
2.6.7 Black-Box Testing 23
2.6.8 Integration Testing 23
2.7 Penelitian Sejenis 24
BAB III METODOLOGI PENELITIAN..... 28
3.1 Waktu dan Tempat Penelitian 28
3.2 Alat Penelitian 28
3.3 Data Penelitian 28
3.3.1 Studi Pustaka..... 29

3.3.2 Observasi 29
3.3.3 Wawancara 29
3.4 Tahapan Penelitian 29
3.4.1 Identifikasi Masalah 30
3.4.2 Pembangunan Sistem Metode Waterfall 31
3.4.3 Penyusunan Laporan Penelitian 34
3.5 Jadwal Penelitian 34
BAB IV HASIL DAN PEMBAHASAN 36
4.1 Analisis Kebutuhan 36
4.1.1 Analisis Kebutuhan Sistem 36
4.1.2 Analisis Data Karyawan 37
4.2 Perancangan Sistem 39
4.2.1 Perancangan *Composable Architecture* 40
4.2.2 Perancangan *Strategy Pattern* 42
4.2.3 Perancangan *Decorator Pattern* 44
4.2.4 Perancangan Algoritma FCM 47
4.2.5 Perancangan Antarmuka Pengguna (*Mockup*) 51
4.2.6 Use Case Diagram 76
4.2.7 Activity Diagram 78
4.2.8 Class Diagram 112
4.2.9 Entity Relationship Diagram 120
4.3 Implementasi 130
4.3.1 Implementasi *Strategy Pattern* 130
4.3.2 Implementasi *Decorator Pattern* 141
4.3.3 Pra-pemrosesan Data 145
4.3.4 Implementasi Algoritma FCM 153

4.3.5 Implementasi *Silhouette Coefficient* 168
4.3.6 Implementasi Antarmuka Pengguna 177
4.4 Pengujian 217
4.4.1 Pengujian Fungsional 217
4.4.2 Pengujian Integrasi Modul 227
4.4.3 Pengujian *Validitas Cluster* 230
BAB V PENUTUP 234
5.1 Kesimpulan 234
5.2 Saran 235
DAFTAR PUSTAKA 236

DAFTAR GAMBAR

Gambar 1. Metode Waterfall 19
Gambar 2. Tahapan Penelitian 30
Gambar 3. Pembangunan Sistem Metode Waterfall 31
Gambar 4. Struktur Jabatan PT Arete Logistik Indonesia 38
Gambar *5*. Arsitektur Modular Sistem 40
Gambar 6. Flowchart Pemilihan Strategy 43
Gambar 7. Diagram Alur Sistem FCM 48
Gambar 8. Diagram Alur Algoritma FCM 50
Gambar 9. Halaman *Login* 52
Gambar 10. Halaman KPI *Dashboard* 53
Gambar 11. Halaman Manajemen Karyawan 55
Gambar 12. Halaman Manajemen Departemen 56
Gambar 13. Halaman Manajemen Jabatan 57
Gambar 14. Halaman Jadwal Kerja 58
Gambar 15. Halaman *Monitor* Presensi 59
Gambar 16. Halaman *Master Approval Chain — Tab Template* 60
Gambar 17. Halaman *Master Approval Chain — Tab* Peserta Tambahan 60
Gambar 18. Halaman *Master Approval Chain — Tab Preview Chain* 61
Gambar 19. Halaman Master Variabel KPI 62
Gambar 20. Halaman *Assignment* Bobot KPI 63
Gambar 21. Halaman Periode Penilaian KPI 64
Gambar 22. *Modal* Konfigurasi FCM 65
Gambar 23. *Modal* FCM Selesai 66
Gambar 24. Halaman Hasil Klasterisasi KPI 67
Gambar 25. Halaman *Input* Penilaian KPI 68
Gambar 26. *Modal Input* Nilai KPI 69
Gambar 27. Halaman *Approval* Pengajuan 70
Gambar 28. Halaman Hasil Klasterisasi KPI Atasan 71
Gambar 29. *Modal* Detail Nilai KPI 72
Gambar 30. Halaman Pengajuan Izin 73

Gambar 31. Halaman Presensi Saya 74
Gambar 32. Halaman KPI Saya 75
Gambar 33. *Use Case Diagram* HRIS Modul Pendukung 76
Gambar 34. *Use Case Diagram* HRIS Modul KPI 77
Gambar 35. *Activity Diagram* Manajemen Karyawan 79
Gambar 36. *Activity Diagram* Manajemen Departemen 81
Gambar 37. *Activity Diagram* Manajemen Jabatan 83
Gambar 38. *Activity Diagram* Manajemen Jadwal Kerja 85
Gambar 39. *Activity Diagram Monitor* Presensi 87
Gambar 40. *Activity Diagram* Konfigurasi *Approval Chain* 89
Gambar 41. *Activity Diagram* Manajemen Variabel KPI 91
Gambar 42. *Activity Diagram* Manajemen *Assignment* Bobot 93
Gambar 43. *Activity Diagram* Manajemen Periode Penilaian 95
Gambar 44. *Activity Diagram* Konfigurasi Parameter FCM 97
Gambar 45. *Activity Diagram* Lihat Hasil Klasterisasi 98
Gambar 46. *Activity Diagram* Lihat Hasil Klasterisasi Bawahan 99
Gambar 47. *Activity Diagram* Lihat KPI Saya 100
Gambar 48. *Activity Diagram* Lihat Presensi Saya 101
Gambar 49. *Activity Diagram* Pengajuan Izin 102
Gambar 50. *Activity Diagram Approval* Izin 104
Gambar 51. *Activity Diagram Input* Nilai KPI Bawahan 106
Gambar 52. *Activity Diagram Input* Nilai KPI (sebagai pengganti) 108
Gambar 53. *Activity Diagram* Proses FCM 110
Gambar 54. *Class Diagram Strategy Pattern* 113
Gambar 55. *Class Diagram Decorator Pattern* 114
Gambar 56. *Class Diagram* FCM dan LCG 116
Gambar 57. *Class Diagram Service Architecture Overview* 118
Gambar 58. ERD Organisasi & *User Management* 120
Gambar 59. ERD KPI 122
Gambar 60. ERD *Auth* & Permission 124
Gambar 61. ERD Presensi & Jadwal Kerja 126
Gambar 62. ERD *Approval*, Izin & Notifikasi 128

Gambar 63. Halaman Daftar Master Variabel KPI 178
Gambar 64. *Modal* Tambah Variabel KPI 179
Gambar 65. Halaman Daftar *Assignment* Bobot 181
Gambar 66. *Modal* Detail Validasi Bobot Per Jabatan 182
Gambar 67. *Modal* Preview Decorator Pattern 182
Gambar 68. Halaman Daftar Periode Penilaian KPI 189
Gambar 69. *Modal* Buat Periode KPI Baru 189
Gambar 70. *Modal* Monitor Progress Input KPI 190
Gambar 71. *Modal* FCM 191
Gambar 72. Halaman *Input* Penilaian KPI 198
Gambar 73. *Modal* Detail *Progress* Input KPI 198
Gambar 74. *Modal Input* Nilai KPI untuk Karyawan dengan Jabatan *Driver* .. 199
Gambar 75. Halaman KPI Dashboard 205
Gambar 76. Halaman Hasil Klasterisasi KPI 209
Gambar 77. Halaman KPI Saya 212

DAFTAR TABEL

Tabel 1. Tabel Nilai Silhouette 13
Tabel 2. Komponen Use Case Diagram..... 21
Tabel 3. Komponen Activity Diagram..... 22
Tabel 4. Penelitian Sejenis..... 24
Tabel 5. Jadwal Penelitian 35
Tabel 6. Kebutuhan Fungsional Sistem 36
Tabel 8. Distribusi Karyawan Per Jabatan dan Departemen 38
Tabel 9. Variabel KPI Layer Global 45
Tabel 10. Variabel KPI Layer Departemen..... 45
Tabel 11. Variabel KPI Layer Jabatan..... 45
Tabel 12. Parameter FCM..... 47
Tabel 13. perbandingan kombinasi parameter..... 139
Tabel 14. Tabel Nilai *Silhouette* 176
Tabel 15. Black Box Testing — Aktor HR..... 217
Tabel 16. Black Box Testing — Aktor Atasan..... 223
Tabel 17. Black Box Testing — Aktor Karyawan 225
Tabel 18. Pengujian Integrasi Modul..... 228
Tabel 19. Hasil Pengujian Sensitivitas Parameter 231
Tabel 20. Hasil Pengujian Reproducibility..... 233

ABSTRAK

IMPLEMENTASI *COMPOSABLE ARCHITECTURE* BERBASIS *STRATEGY* DAN *DECORATOR PATTERNS* PADA ALGORITMA *FUZZY* C-MEANS UNTUK KLASTERISASI KINERJA KARYAWAN

GHAZA INDRA PRATAMA
H1D022073

Sistem penilaian kinerja karyawan yang umum diterapkan sering menghadapi keterbatasan dalam menangani variasi variabel KPI antar departemen dan jabatan, terutama ketika organisasi memiliki kategori karyawan dengan struktur kerja non-konvensional seperti karyawan mitra. Keterbatasan ini disebabkan oleh implementasi variabel penilaian secara *hardcode* dalam kode program. Penelitian ini mengembangkan sistem klasterisasi kinerja karyawan dengan arsitektur *composable* berbasis *Strategy Pattern* dan *Decorator Pattern* yang diintegrasikan dengan algoritma *Fuzzy C-Means*, dengan studi kasus pada PT Arete Logistik Indonesia. *Strategy Pattern* diimplementasikan dengan tiga *concrete strategy* untuk menangani karakteristik jabatan yang berbeda, sementara *Decorator Pattern* diimplementasikan dalam bentuk fungsional untuk komposabilitas resolusi variabel KPI. *Reproducibility* hasil klasterisasi dijamin melalui *Linear Congruential Generator* dengan *seed* deterministik dari identitas periode penilaian. Pengujian *Black Box* pada 29 kebutuhan pengguna dengan 60 skenario uji menghasilkan status valid pada seluruh skenario. Pengujian validitas cluster menghasilkan *Silhouette Coefficient* sebesar 0,6522 untuk konfigurasi tiga cluster yang berada pada kategori Struktur Sedang dan stabil terhadap variasi pangkat pembobot. Pengujian *reproducibility* pada lima eksekusi berulang menghasilkan nilai dan label cluster yang identik. Sistem ini memberikan fleksibilitas konfigurasi variabel KPI tanpa duplikasi kode sekaligus menjamin sifat *reproducibility* hasil klasterisasi.

Kata Kunci: *Strategy Pattern*, *Decorator Pattern*, *Fuzzy C-Means*, Klasterisasi Kinerja Karyawan, *Silhouette Coefficient*, *Linear Congruential Generator*

ABSTRACT

IMPLEMENTING COMPOSABLE ARCHITECTURE WITH STRATEGY AND DECORATOR PATTERNS FOR FUZZY C-MEANS-BASED EMPLOYEE PERFORMANCE CLUSTERING

GHAZA INDRA PRATAMA
H1D022073

Employee performance evaluation systems commonly applied today often face limitations in handling KPI variable variations across departments and job positions, especially when organizations have employee categories with non-conventional work structures such as partner employees. These limitations are caused by hardcoded implementation of evaluation variables in program code. This research developed an employee performance clustering system with composable architecture based on Strategy Pattern and Decorator Pattern integrated with the Fuzzy C-Means algorithm, with a case study at PT Arete Logistik Indonesia. The Strategy Pattern is implemented with three concrete strategies to handle different job characteristics, while the Decorator Pattern is implemented in functional form for the composability of KPI variable resolution. The reproducibility of clustering results is guaranteed through a Linear Congruential Generator with a deterministic seed derived from the assessment period identity. Black box testing on 29 user requirements with 60 test scenarios produced valid status on all scenarios. Cluster validity testing produced a Silhouette Coefficient of 0.6522 for the three-cluster configuration which falls into the Medium Structure category and remains stable against fuzziness parameter variations. Reproducibility testing on five repeated executions produced identical values and cluster labels. This system provides flexibility in KPI variable configuration without code duplication while ensuring the reproducibility of clustering results.

Keywords: *Strategy Pattern, Decorator Pattern, Fuzzy C-Means, Employee Performance Clustering, Silhouette Coefficient, Linear Congruential Generator*

BAB I
PENDAHULUAN

1.1 Latar Belakang

Sumber daya manusia merupakan salah satu faktor penting dalam suatu organisasi atau perusahaan. Agar kegiatan manajemen dapat berjalan dengan baik maka perusahaan harus memiliki karyawan yang berpengetahuan dan berketerampilan tinggi serta berusaha untuk mengelola perusahaan seoptimal mungkin agar kinerja karyawan semakin meningkat (Teknologi, 2021). Dalam lingkungan bisnis yang kompetitif, pengelolaan kinerja melalui sistem Human Resource Information System (HRIS) merupakan kebutuhan mutlak untuk menjamin produktivitas dan pengembangan karyawan (Suhada & Ali, t.t.). HRIS merupakan sistem yang dikembangkan menggunakan teknologi yang terintegrasi untuk mengumpulkan, menyimpan, dan menganalisis informasi yang bersangkutan dengan karyawan di dalam sebuah perusahaan. Analisis performa kinerja karyawan dapat dimanfaatkan untuk memberikan insentif, promosi, demosi, dan atau pelatihan yang dibutuhkan, maka dari itu penilaain harus bersifat objektif (Fatinah, 2022).

Pada PT Arete Logistik Indonesia, terdapat tantangan khusus dalam pengelolaan SDM, yaitu adanya struktur jabatan karyawan yang heterogen. Karyawan dibagi menjadi tenaga manajemen dan mitra pengemudi (driver). Kedua kelompok ini memiliki parameter keberhasilan yang sangat berbeda; Personel manajemen dievaluasi berdasarkan pencapaian KPI (Key Performance Indicators) dan kompetensi manajerial, sedangkan mitra pengemudi dievaluasi berdasarkan aspek teknis seperti keselamatan berkendara, efisiensi bahan bakar dan jumlah perjalanan. Ketidakseragaman variabel penilaian ini menyebabkan metode evaluasi standar sering kali tidak relevan dan menimbulkan bias subjektivitas (Rizal dkk., 2022).

Untuk mengatasi bias tersebut, diperlukan metode komputasi cerdas seperti clustering. Clustering merupakan metode analisis data dengan mengelompokkan data berdasarkan kecocokan karakteristiknya. Algoritma clustering yang biasa digunakan adalah K-Means, K-Medoids, dan Fuzzy C-

Means (Syukron dkk., 2022). Algoritma Fuzzy C-Means (FCM) dinilai tepat karena memiliki kemampuan mengelompokkan data dengan derajat keanggotaan, sehingga mampu merepresentasikan kinerja karyawan yang seringkali berada pada area samar (tidak mutlak baik atau buruk) (Zahro dkk., 2024). Namun, implementasi algoritma FCM pada struktur data yang dinamis dan berbeda-beda antar jabatan karyawan menimbulkan tantangan teknis tersendiri dalam pengembangan perangkat lunak.

Pengembangan sistem HRIS konvensional seringkali menggunakan logika pemrograman prosedural yang kaku. Ketika dihadapkan pada variasi kriteria penilaian yang heterogen, pengembang cenderung menggunakan percabangan logika yang berlebihan. Hal ini mengakibatkan kode program menjadi rumit, sulit dipelihara, dan tidak fleksibel terhadap perubahan struktur organisasi (Mohlala dkk., 2024). Oleh karena itu, diperlukan pendekatan arsitektur perangkat lunak yang adaptif. Sebagaimana dijelaskan dalam literatur modern, arsitektur ini memungkinkan sistem untuk beralih dari struktur monolitik yang kaku menjadi kapabilitas bisnis yang modular, sehingga sistem dapat disesuaikan secara presisi terhadap kebutuhan spesifik tanpa perombakan total (Kumar & Padur, 2025). Dalam penelitian ini, penggunaan Strategy Pattern memungkinkan sistem memilih algoritma penilaian yang sesuai dengan peran karyawan secara otomatis, sementara Decorator Pattern memungkinkan penyusunan variabel input untuk algoritma FCM secara dinamis dan bertingkat (Kevin & Kurniawan, 2023). Integrasi kedua pola tersebut diharapkan dapat mengoptimalkan kinerja algoritma FCM, sehingga mampu menghasilkan klasterisasi kinerja karyawan yang akurat dan relevan meskipun terdapat variasi kriteria penilaian antar jabatan.

Berdasarkan uraian permasalahan tersebut, penelitian ini bertujuan untuk mengimplementasikan Composable Architecture berbasis Strategy dan Decorator Pattern pada algoritma Fuzzy C-Means guna mengakomodasi kompleksitas evaluasi kinerja karyawan dengan karakteristik jabatan karyawan yang heterogen. Melalui implementasi arsitektur ini, diharapkan terbentuk mekanisme klasterisasi yang tidak hanya adaptif terhadap variasi kriteria penilaian, tetapi juga akurat secara analitik, sehingga pada akhirnya dapat

memberikan dasar pengambilan keputusan manajemen SDM yang lebih objektif dan terstandarisasi.

1.2 Rumusan Masalah

Berdasarkan latar belakang masalah yang telah diuraikan, maka rumusan masalah dalam penelitian ini adalah:

a. Bagaimana merancang dan mengimplementasikan Composable Architecture berbasis Strategy dan Decorator Pattern untuk menangani kompleksitas logika penilaian kinerja yang heterogen antara tenaga manajemen dan mitra pengemudi?
b. Bagaimana menerapkan algoritma Fuzzy C-Means (FCM) pada struktur data dinamis yang dihasilkan oleh arsitektur tersebut agar dapat menghasilkan klasterisasi kinerja karyawan yang objektif dan akurat?

1.3 Batasan Masalah

Agar penelitian ini dapat dilakukan secara lebih terarah, maksimal, dan spesifik, maka ditetapkan beberapa batasan masalah sebagai berikut:

a. Penelitian ini secara khusus memfokuskan pada evaluasi kinerja karyawan pada PT Arete Logistik Indonesia, dengan karakteristik variabel penilaian yang berbeda untuk masing-masing kategori.
b. Implementasi sistem menerapkan Composable Architecture dengan pola desain Strategy Pattern untuk menangani pemilihan logika penilaian berdasarkan jabatan, dan Decorator Pattern untuk penyusunan variabel penilaian dinamis. Algoritma inti yang digunakan untuk pengelompokan kinerja adalah Fuzzy C-Means (FCM).
c. Meskipun sistem dirancang sebagai HRIS terintegrasi, fokus mendalam penelitian ini (terutama pada penerapan algoritma) dibatasi pada modul-modul yang relevan dengan penilaian kinerja, yaitu modul KPI/Appraisal. Modul lain hanya dibahas sebagai data pendukung atau output sederhana.
d. Sistem dikembangkan berbasis website (web-based application) menggunakan framework NestJS untuk sisi backend guna mendukung implementasi arsitektur modular, dan tidak mencakup pengembangan aplikasi mobile (Android/iOS).

e. Sistem ini dirancang untuk digunakan oleh tiga jenis pengguna utama, yaitu Administrator (pengelola konfigurasi sistem), HR/Manajemen (penilai/evaluator), dan Karyawan (sebagai objek yang dinilai dan pengguna fitur mandiri).
f. Hasil akhir dari sistem berupa klasterisasi atau pengelompokan status kinerja karyawan (misalnya: High Performer, Standard, Low Performer) yang berfungsi sebagai sistem pendukung keputusan (decision support) untuk rekomendasi insentif atau evaluasi karir, dan tidak menggantikan keputusan final manajemen perusahaan.

1.4 Tujuan Penelitian

Adapun tujuan dari penelitian ini adalah sebagai berikut:

a. Membangun sistem evaluasi kinerja dengan arsitektur perangkat lunak yang adaptif dan modular menggunakan pola desain Strategy dan Decorator untuk mengakomodasi variasi kriteria penilaian.
b. Mengimplementasikan algoritma Fuzzy C-Means untuk melakukan pengelompokan (clustering) kinerja karyawan secara otomatis berdasarkan parameter yang telah disesuaikan dengan karakteristik jabatan.

1.5 Manfaat Penelitian

Melalui penelitian ini, diharapkan dapat diperoleh berbagai manfaat yang dirinci sebagai berikut:

a. Bagi Perusahaan (PT Arete Logistik Indonesia):

   Menyediakan sistem evaluasi kinerja yang objektif dan terstandarisasi yang mampu menangani penilaian karyawan dengan karakteristik jabatan berbeda (manajemen dan mitra pengemudi) secara otomatis. Dengan adanya sistem ini, bias subjektivitas dalam penilaian dapat diminimalkan, serta proses pengambilan keputusan terkait insentif, promosi, atau pelatihan dapat dilakukan dengan lebih akurat dan efisien dibandingkan proses manual.
b. Bagi Peneliti:

   Menerapkan konsep rekayasa perangkat lunak tingkat lanjut melalui implementasi Composable Architecture menggunakan pola desain Strategy dan Decorator ke dalam kasus nyata, sekaligus menguji kemampuan

algoritma Fuzzy C-Means dalam klasterisasi data kinerja. Penelitian ini juga menjadi sarana untuk mengasah keterampilan teknis dalam pengembangan backend modern menggunakan framework NestJS, serta sebagai syarat pemenuhan kelulusan di Jurusan Informatika.

c. Bagi Karyawan dan Manajemen HR:

   Memberikan transparansi dan keadilan dalam proses penilaian kinerja, di mana setiap karyawan dinilai berdasarkan parameter yang relevan dengan tugas dan fungsinya masing-masing. Bagi tim HR, sistem ini menyajikan visualisasi pengelompokan kinerja yang mudah dipahami, sehingga memudahkan dalam memonitor produktivitas SDM secara real-time.

d. Bagi Pembaca:

   Menjadi sumber referensi teknis mengenai penerapan Composable Architecture dan pola desain (Design Patterns) dalam menangani kompleksitas logika bisnis pada sistem informasi SDM. Selain itu, penelitian ini diharapkan dapat memberikan wawasan baru terkait integrasi arsitektur perangkat lunak dengan algoritma data mining (Fuzzy C-Means), serta menjadi acuan bagi peneliti lain yang tertarik mengembangkan sistem adaptif di masa mendatang.

1.6 Luaran Penelitian

Penelitian ini diharapkan menghasilkan beberapa luaran sebagai berikut:

a. Perangkat Lunak Sistem Evaluasi Kinerja Berbasis Web Sebuah aplikasi HRIS (Human Resource Information System) yang berfungsi sebagai alat bantu klasterisasi kinerja karyawan secara objektif. Sistem ini dirancang dengan mengimplementasikan Composable Architecture menggunakan pola desain Strategy dan Decorator untuk menangani struktur penilaian yang dinamis antara karyawan manajemen dan mitra pengemudi. Selain itu, sistem ini terintegrasi dengan algoritma Fuzzy C-Means (FCM) yang berfungsi melakukan proses kalkulasi dan pengelompokan data kinerja secara otomatis, sehingga menghasilkan keluaran berupa status kinerja yang akurat dan terstandarisasi.

b. Dokumentasi Ilmiah dalam Bentuk Laporan Tugas Akhir Laporan ini akan menguraikan secara komprehensif dan terstruktur seluruh proses penelitian, mulai dari analisis kebutuhan karyawan yang heterogen, perancangan arsitektur perangkat lunak yang modular, penerapan pola desain dalam kode program, hingga analisis hasil klasterisasi algoritma. Laporan ini juga menyajikan hasil pengujian validitas sistem, sehingga diharapkan dapat menjadi bahan studi dan referensi teknis bagi pengembangan sistem informasi SDM yang adaptif di masa depan.

BAB II
TINJAUAN PUSTAKA

2.1 Manajemen Kinerja Sumber Daya Manusia

2.1.1 Konsep Penilaian Kinerja

Penilaian kinerja karyawan adalah bagian penting dari proses pengelolaan sumber daya manusia di perusahaan. Proses evaluasi ini bertujuan untuk menilai prestasi dan kontribusi karyawan dalam memenuhi standar kinerja yang telah ditetapkan (Rustiawan dkk., 2023). Secara umum, penilaian kinerja didefinisikan sebagai sistem formal untuk meninjau dan mengevaluasi kinerja individu atau tim. Kegiatan ini tidak hanya sekadar memberikan nilai, tetapi juga berfungsi sebagai media komunikasi antara manajemen dan karyawan untuk menyelaraskan tujuan individu dengan tujuan perusahaan (Fitriana, 2022).

Oleh karena itu, sistem pendukung keputusan dalam penilaian kinerja menjadi faktor krusial yang mempengaruhi validitas dan akurasi hasil penilaian. Keakuratan data penilaian menjadi syarat mutlak, karena hasil penilaian kinerja ini berfungsi sebagai acuan bagi perusahaan dalam evaluasi karyawan, mulai dari pemberian insentif, kenaikan gaji, promosi bahkan hingga demosi. Keputusan yang diambil berdasarkan data yang tidak valid dapat menyebabkan ketidakpuasan karyawan, penurunan motivasi, dan tingginya tingkat perputaran karyawan (turnover) (Anindita & Rahayu, 2021).

Namun, tantangan utama dalam penilaian kinerja adalah menjaga objektivitas ketika dihadapkan pada karakteristik jabatan yang beragam. Setiap posisi dalam perusahaan memiliki indikator keberhasilan yang berbeda; misalnya, staf administrasi dinilai dari ketelitian dokumen, sementara staf lapangan atau pengemudi dinilai dari aspek keselamatan dan efisiensi operasional. Perbedaan mendasar inilah yang menuntut adanya metode penilaian yang adaptif dan terstruktur agar evaluasi dapat berjalan secara adil dan relevan sesuai dengan tanggung jawab masing-masing karyawan.

2.1.2 *Key Performance Indicator* (KPI)

Key Performance Indicator (KPI) merupakan ukuran berskala dan kuantitatif yang digunakan untuk mengevaluasi kinerja karyawan dalam mencapai target perusahaan. KPI juga digunakan untuk menentukan tujuan yang terukur, melihat tren, serta mendukung pengambilan keputusan (Nurwahidah dkk., 2021). Indikator ini harus memenuhi kriteria SMART (Specific, Measurable, Achievable, Relevant, Time-bound) agar dapat berfungsi efektif sebagai alat ukur keberhasilan operasional maupun strategis perusahaan.

Namun, penerapan KPI dalam perusahaan yang memiliki struktur organisasi kompleks seringkali menghadapi tantangan variabilitas. Setiap divisi atau jenjang jabatan memerlukan indikator yang spesifik dan berbeda. Heterogenitas indikator inilah yang menuntut sistem penilaian kinerja untuk bersifat fleksibel, mampu mengakomodasi berbagai jenis variabel input tanpa harus merombak struktur dasar penilaian secara keseluruhan.

2.1.3 *Human Resource Information System* (HRIS)

Human Resource Information System (HRIS) didefinisikan sebagai sistem terintegrasi yang dirancang untuk mengumpulkan, menyimpan, dan menganalisis informasi terkait sumber daya manusia dalam suatu organisasi. Sistem ini menggabungkan manajemen sumber daya manusia (MSDM) sebagai disiplin ilmu dengan teknologi informasi dasar (Hossain dkk., 2025). Fungsi utama HRIS adalah membantu pengambilan keputusan yang efektif dengan menyediakan data yang akurat, tepat waktu, dan relevan bagi manajemen maupun karyawan.

Dalam perkembangannya, HRIS telah bertransformasi dari sekadar alat administrasi pencatatan data menjadi sistem yang mencakup modul penilaian kinerja. Penerapan HRIS berbasis website menjadi standar modern karena menawarkan aksesibilitas tinggi dan sentralisasi data (Ali Quaosar & Rahman, 2021). Namun, tantangan terbesar dalam pengembangan modul penilaian kinerja pada HRIS adalah bagaimana menerjemahkan kebijakan penilaian yang dinamis ke dalam logika pemrograman yang statis. Oleh karena itu, diperlukan perancangan

arsitektur perangkat lunak yang matang agar sistem HRIS tidak hanya berfungsi sebagai tempat penyimpanan data, tetapi juga mampu melakukan pemrosesan analitik yang kompleks secara otomatis.

2.2 Data Mining dan Klasterisasi

2.2.1 Data Mining

Data mining didefinisikan sebagai proses ekstraksi informasi dan pengetahuan implisit yang berpotensi berguna dari sekumpulan data besar yang bersifat kompleks, tidak lengkap, maupun acak. Berbeda dengan metode penelitian tradisional, teknologi data mining dapat langsung mengolah data tanpa memerlukan asumsi atau hipotesis awal yang kaku. Tujuannya adalah untuk menemukan pola pengetahuan yang valid, bersifat praktis untuk diterapkan, serta menawarkan kebaruan atas pola-pola data yang sebelumnya tidak diketahui (Wu dkk., 2021).

Secara fungsional, data mining dapat dikategorikan menjadi dua, yaitu prediktif dan deskriptif. prediktif bertujuan untuk memprediksi nilai masa depan berdasarkan pola data historis, sedangkan deskriptif bertujuan untuk menemukan pola atau struktur yang dapat dimengerti manusia dari data yang ada (Hapsari dkk., 2025). Dalam konteks evaluasi kinerja, penelitian ini memanfaatkan fungsi deskriptif data mining melalui teknik pengelompokan atau klasterisasi. Teknik ini digunakan untuk mengidentifikasi kategori kinerja karyawan secara alami berdasarkan kemiripan atribut data, tanpa adanya label kelas yang ditentukan sebelumnya.

2.2.2 Clustering

Clustering atau pengklasteran merupakan metode analisis data yang bertujuan untuk mengelompokkan objek-objek ke dalam beberapa kelompok atau klaster berdasarkan kemiripan karakteristik yang dimilikinya. Berbeda dengan klasifikasi yang bersifat supervised learning, clustering termasuk dalam kategori unsupervised learning karena tidak melibatkan variabel target atau label kelas dalam proses pembelajarannya. Artinya, algoritma ini tidak digunakan untuk memprediksi nilai tertentu, melainkan untuk menemukan struktur yang tersembunyi di dalam data

dengan memaksimalkan kesamaan data dalam satu klaster dan meminimalkan kesamaan antar klaster (Nugraha dkk., 2022).

Secara umum, pendekatan clustering dapat dibedakan menjadi dua jenis, yaitu Hard Clustering dan Soft Clustering. Pada Hard Clustering, setiap data harus masuk secara tegas ke dalam satu klaster saja. Sebaliknya, pada Soft Clustering, sebuah data dapat menjadi anggota dari beberapa klaster sekaligus dengan derajat keanggotaan yang bervariasi antara 0 hingga 1. Konsep Soft Clustering ini seringkali lebih relevan untuk diterapkan pada cluster yang memiliki banyak variable, seperti penilaian kinerja manusia, di mana batas antara kriteria "Baik" dan "Cukup" seringkali bersifat samar (Ruan & Joe-Wong, 2022).

2.3 Algoritma Fuzzy C-Means (FCM)

2.3.1 Konsep Dasar FCM

Fuzzy C-Means (FCM) adalah algoritma pengklasteran data yang diperkenalkan oleh Jim Bezdek pada tahun 1981. Algoritma ini merupakan pengembangan dari metode K-Means, di mana keberadaan data dalam suatu klaster ditentukan oleh nilai derajat keanggotaan (*membership degree*). Jika pada K-Means sebuah data hanya bisa menjadi anggota satu klaster secara mutlak (0 atau 1), maka pada FCM setiap titik data memiliki derajat keanggotaan antara 0 hingga 1 untuk setiap klaster yang ada. Hal ini memungkinkan sebuah data menjadi anggota dari beberapa klaster sekaligus dengan bobot yang berbeda, yang sangat berguna untuk menangani data dengan batasan yang tidak tegas (*fuzzy*) (Wulandari & Yogantara, 2022).

2.3.2 Tahapan Algoritma FCM

Mengacu pada logika perhitungan algoritma *Fuzzy C-Means*, berikut adalah langkah-langkah detailnya (Nasrullah dkk., 2024):

1. Inisialisasi Parameter

   Menentukan parameter awal sebelum proses iterasi dimulai:

   - Jumlah klaster (c): Jumlah kelompok yang akan dibentuk.
   - Pangkat pembobot (w): Eksponen fuzzy ($w > 1$) yang mengatur tingkat kekaburan.
   - Error terkecil (ξ): Batas toleransi konvergensi.
   - Fungsi objektif awal: $P_0 = 0$.
   - Iterasi awal: t = 1.

2. Pembangkitan Partisi Awal (μ_{ik})

   Membangkitkan matriks partisi awal μ_{ik} secara acak untuk $i = 1,2,\dots,n$ dan $k = 1,2,\dots,$k. Nilai μ_{ik} merepresentasikan derajat keanggotaan data ke-i pada klaster ke-k.

3. Perhitungan Pusat Klaster (V_{kj})

   Menghitung titik pusat klaster ke-k (V_{kj}) untuk setiap atribut, menggunakan persamaan berikut:

$$V_{kj} = \frac{\sum_{i=1}^{n}(\mu_{ik})^{w} x_{ij}}{\sum_{i=1}^{n}(\mu_{ik})^{w}}$$

   Keterangan:

   - x_{ij} : Data ke-i pada atribut ke-j.
   - μ_{ik} : Derajat keanggotaan data ke-i pada klaster ke-k.
   - w : Pangkat pembobot.

4. Perhitungan Fungsi Objektif (P_t)

   Menghitung fungsi objektif pada iterasi ke-t (P_t) untuk mengukur total varian dalam klaster menggunakan rumus :

$$p_t = \sum_{i=1}^{n}\sum_{k=1}^{c}\sum_{j=1}^{m}(\mu_{ik})^{w}(x_{ij} - v_{kj})^2$$

5. Pembaruan Matriks Partisi (μ_{ik})

   Memperbarui nilai derajat keanggotaan (μ_{ik}) untuk iterasi selanjutnya.

$$\mu_{ik} = \frac{(\sum_{j=1}^{m}(x_{ij} - v_{kj})^2)^{\frac{-1}{w-1}}}{\sum_{k=1}^{c}(\sum_{j=1}^{m}(x_{ij} - v_{kj})^2)^{\frac{-1}{w-1}}}$$

6. Pengecekan Konvergensi

   Memeriksa selisih fungsi objektif antara iterasi saat ini (p_t) dan iterasi sebelumnya (p_{t-1}). Jika:

$$|p_t - p_{t-1}| < \xi$$

   Maka proses iterasi berhenti (konvergen). Jika tidak, iterasi dilanjutkan (t = t + 1) dan kembali ke langkah 3.

2.3.3 Validasi Cluster (*Silhouette Coefficient*)

Validasi cluster merupakan tahapan krusial untuk mengevaluasi kualitas pengelompokan data yang dihasilkan oleh algoritma. Mengingat clustering bersifat unsupervised (tidak memiliki label target), validasi internal diperlukan untuk mengukur seberapa baik struktur objek dalam setiap kelompok. Penelitian ini menggunakan metode Silhouette Coefficient yang mengukur kualitas cluster berdasarkan dua aspek, yaitu cohesion (kepadatan dalam satu cluster) dan separation (pemisahan antar cluster) (Rochman dkk., 2022).

proses perhitungan nilai *Silhouette* (s) untuk setiap objek data ke-*i* dilakukan melalui 4 tahapan sistematis berikut :

1. Menghitung Rata-rata Jarak Intra-Cluster (a(i))

   Langkah pertama adalah menghitung rata-rata jarak antara objek *i* dengan seluruh objek lain yang berada dalam satu cluster yang sama (misal *cluster* A). Nilai ini merepresentasikan tingkat kedekatan (cohesion) objek terhadap kelompoknya sendiri.

$$a(i) = \frac{1}{|A| - 1} \sum_{j \in A, j \neq i} d(i, j)$$

2. Menghitung Rata-rata Jarak ke Cluster Lain (*d(i,C)*)

   Selanjutnya, menghitung rata-rata jarak antara objek i dengan seluruh objek yang berada di cluster lain (misal cluster C).

$$d(i,C) = \frac{1}{|C|}\sum_{j \in C} d(i,j)$$

3. Menentukan Jarak Tetangga Terdekat (*b(i)*)

   Dari hasil perhitungan langkah kedua, dipilih nilai rata-rata jarak yang paling minimum dari semua kemungkinan cluster lain ($C \neq A$). Nilai minimum ini disebut sebagai *b(i)* yang merepresentasikan jarak pemisahan (separation) terdekat.

$$b(i) = \min_{C \neq A} d(i,C)$$

4. Menghitung Silhouette Coefficient (*s(i)*)

   Nilai akhir silhouette untuk objek *i* dihitung dengan membandingkan nilai separasi (*b(i)*) dan kohesi (*a(i)*) menggunakan rumus:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

Nilai *Silhouette Coefficient* berkisar antara -1 hingga 1. Semakin nilai mendekati 1, maka kualitas pengelompokan dinilai semakin baik. kualitas struktur *cluster* dikategorikan sebagaimana disajikan pada Tabel 2.1 (Rochman dkk., 2022).

Tabel 1. Tabel Nilai *Silhouette*

Nilai Silhouette (SC)	Kriteria Struktur (Structure Quality)
$0{,}7 < SC \leq 1$	Close Structure (Struktur Kuat)
$0{,}5 < SC \leq 0{,}7$	Medium Structure (Struktur Sedang)
$0{,}25 < SC \leq 0{,}5$	Tensile Structure (Struktur Lemah)
$SC \leq 0{,}25$	Unstructured (Tidak Terstruktur)

2.4 Arsitektur Perangkat Lunak Modern

2.4.1 Composable Architecture

Composable Enterprise Architecture (CEA) merupakan paradigma desain perangkat lunak modern yang memandang sistem bukan sebagai struktur monolitik yang kaku, melainkan sebagai kumpulan komponen fungsional independen yang dapat disusun, diganti, atau dikombinasikan ulang. Pendekatan ini muncul sebagai respon terhadap keterbatasan arsitektur tradisional yang memiliki ketergantungan komponen yang ketat (*tightly coupled*), sehingga sulit untuk dimodifikasi atau dikembangkan secara cepat. Arsitektur ini berfokus pada pembagian kapabilitas bisnis menjadi komponen yang dapat di-*deploy* secara mandiri untuk meningkatkan fleksibilitas (*agility*). Inti dari arsitektur ini adalah penggunaan Packaged Business Capabilities (PBCs) yang mana membungkus logika bisnis, data, dan proses yang spesifik—seperti manajemen inventaris atau penagihan—ke dalam unit otonom . Interaksi antar komponen ini difasilitasi melalui pengembangan berbasis API-first, di mana setiap modul berkomunikasi menggunakan antarmuka standar yang telah disepakati (API contracts)(Rusum & Anasuri, 2023).

Dalam konteks pengembangan sistem informasi SDM yang kompleks, Composable Architecture menawarkan solusi teknis untuk menangani heterogenitas kebutuhan. Dengan memisahkan logika bisnis yang berbeda ke dalam modul-modul terisolasi, risiko kegagalan sistem secara menyeluruh dapat diminimalisir. Selain itu, pendekatan ini memungkinkan setiap modul untuk diperbarui atau diganti secara selektif tanpa mengganggu operasional sistem lainnya, yang secara signifikan meningkatkan efisiensi pemeliharaan dan kecepatan inovasi.

2.4.2 Modular Monolith

Modular Monolith adalah pola arsitektur perangkat lunak yang secara strategis menggabungkan kesederhanaan struktur monolitik dengan keuntungan modularitas dari microservices. Arsitektur ini mengorganisir sistem ke dalam modul-modul yang terhubung secara longgar, di mana

setiap modul memiliki batas yang tegas dan dependensi yang eksplisit, namun tetap digunakan sebagai satu unit kesatuan (Su & Li, 2024).

Pendekatan ini berbeda dengan arsitektur monolitik tradisional yang seringkali memiliki ketergantungan kode yang rumit. Dalam Modular Monolith, sistem mempertahankan skema basis data yang terpadu dan berjalan dalam satu proses aplikasi , sehingga menghilangkan kompleksitas infrastruktur terdistribusi yang biasanya ditemukan pada microservices. Hal ini menawarkan keunggulan berupa skalabilitas dan pemeliharaan yang lebih baik dibandingkan monolith tradisional, sekaligus menjadi alternatif yang lebih efisien bagi tim yang ingin menghindari overhead operasional dari microservices (Su & Li, 2024).

Penerapan Modular Monolith ini diimplementasikan menggunakan framework NestJS. NestJS mendukung penuh karakteristik Modular Monolith melalui fitur @Module yang memungkinkan pengkapsulan logika bisnis ke dalam konteks yang terisolasi. Dengan demikian, meskipun seluruh kode berjalan pada satu server, logika penilaian kinerja dapat dipisahkan secara tegas sesuai dengan domain bisnisnya, selaras dengan prinsip modularitas yang menekankan pada kohesi internal yang tinggi dan ketergantungan antar modul yang minimal.

2.5 Pola Desain Perangkat Lunak (*Software Design Patterns*)

2.5.1 Strategy Pattern

Strategy Pattern merupakan pola desain perilaku (*behavioral design pattern*) yang digunakan untuk mendefinisikan sekumpulan algoritma yang memiliki tujuan serupa, membungkusnya ke dalam kelas-kelas terpisah, dan membuatnya dapat saling dipertukarkan (*interchangeable*). Penerapan pola ini bertujuan untuk memecah kelas yang menanggung terlalu banyak tanggung jawab (bloated class) dengan menempatkan logika spesifik ke dalam objek tersendiri, sehingga struktur kode menjadi lebih representatif dan terprediksi (Kevin & Kurniawan, 2023).

Pola ini sangat efektif untuk menghindari penggunaan logika kondisional yang kompleks yang seringkali muncul ketika sistem harus

menangani variasi proses bisnis yang dinamis. Dalam struktur Strategy Pattern, terdapat tiga komponen utama:

1. Strategy Interface

   Antarmuka umum yang mendefinisikan kontrak metode bagi semua algoritma konkret.

2. Concrete Strategy

   Kelas implementasi yang memuat logika algoritma spesifik.

3. Context

   Kelas pengguna yang berinteraksi dengan strategi melalui antarmuka tanpa perlu mengetahui detail implementasinya.

2.5.2 Decorator Pattern

Decorator Pattern merupakan pola desain struktural (*structural design pattern*) yang memungkinkan penambahan perilaku atau fungsionalitas baru pada suatu objek secara dinamis tanpa harus mengubah struktur dari kelas aslinya. Pola ini bekerja dengan cara "melampirkan" fungsionalitas tambahan ke objek saat runtime, menjadikannya alternatif yang lebih fleksibel dibandingkan teknik pewarisan (inheritance) tradisional yang bersifat statis(Kurmangali dkk., 2022).

Kelebihan utama pola ini adalah kemampuannya mengatasi masalah "ledakan kelas" (*class explosion*). Dalam pendekatan pewarisan biasa, penambahan variasi fitur seringkali memaksa pengembang membuat banyak sub-kelas baru untuk setiap kombinasi kemungkinan. Sebaliknya, Decorator Pattern memungkinkan fitur-fitur tersebut dibungkus (*wrapped*) secara independen, sehingga sistem menjadi lebih mudah dipelihara (*maintainable*) dan terbuka untuk perluasan fitur di masa depan tanpa memodifikasi kode yang sudah ada(Kurmangali dkk., 2022).

2.6 Teknologi Pengembangan

2.6.1 NestJS *Framework*

NestJS adalah kerangka kerja (*framework*) sisi server (*server-side*) berbasis Node.js yang dirancang untuk membangun aplikasi yang efisien dan berskala besar. NestJS efektif digunakan dalam perancangan sistem informasi manajemen modern karena kemampuannya menangani

pencatatan dan monitoring tugas secara sistematis (Muhamad Saepuloh & Seliwati, 2022). Keunggulan utama NestJS terletak pada arsitektur modular dan performanya. NestJS memiliki performa yang sangat baik dalam menangani permintaan API. Penelitian tersebut menunjukkan bahwa NestJS lebih efisien dalam penggunaan sumber daya CPU dan memori dibandingkan micro-framework sejenis seperti Lumen, dengan selisih efisiensi memori mencapai 1% dan CPU sebesar 6% pada beban pengujian tertentu. Efisiensi ini menjadikan NestJS pilihan yang tepat untuk aplikasi yang membutuhkan resource optimal (Trias Hanggara dkk., 2022).

Secara teknis, NestJS mengorganisir kode ke dalam modul-modul (Modules) dan memanfaatkan sistem Dependency Injection (DI) yang kuat. Fitur modularitas ini selaras dengan konsep Composable Architecture, di mana logika bisnis dapat dipisahkan secara tegas. Dukungan penuh terhadap TypeScript dan arsitektur yang terinspirasi dari Angular juga memastikan kode yang dihasilkan lebih terstruktur, mudah dipelihara, dan minim kesalahan (bug) dibandingkan pengembangan menggunakan JavaScript murni.

2.6.2 React JS Framework

React JS adalah pustaka (*library*) JavaScript bersifat *open-source* yang dikembangkan oleh Meta (Facebook) untuk membangun antarmuka pengguna (*User Interface*) yang interaktif dan dinamis, khususnya untuk aplikasi satu halaman (*Single Page Application*). React JS merupakan teknologi fundamental dalam arsitektur web modern yang memungkinkan pengembang untuk membangun aplikasi yang cepat dan berskala besar melalui pendekatan deklaratif (Fariz dkk., 2022).

Karakteristik utama yang menjadikan React JS relevan dalam penelitian ini adalah:

1. Component-Based Architecture

   React membangun antarmuka pengguna dengan memecahnya menjadi bagian-bagian kecil yang disebut Components. Setiap komponen bersifat independen, dapat digunakan kembali (reusable), dan mengelola logikanya sendiri (state). Pendekatan ini sangat selaras

dengan konsep Composable Architecture yang diterapkan pada sistem ini, di mana antarmuka antar modul dapat disusun dari komponen-komponen UI yang modular tanpa duplikasi kode yang berlebihan.

2. Virtual DOM (Document Object Model)

   Untuk meningkatkan performa, React menggunakan mekanisme Virtual DOM, yaitu representasi memori ringan dari DOM asli peramban. Saat terjadi perubahan data, React tidak langsung merender ulang seluruh halaman. Sebaliknya, algoritma diffing React akan membandingkan versi Virtual DOM baru dengan versi sebelumnya, dan hanya memperbarui elemen DOM asli yang benar-benar berubah. Efisiensi ini sangat krusial untuk memastikan sistem HRIS tetap responsif saat memproses data penilaian karyawan dalam jumlah besar.

3. Unidirectional Data Flow

   React menerapkan aliran data satu arah (one-way data binding), di mana data mengalir dari komponen induk (parent) ke komponen anak (child) melalui properti (props). Struktur ini membuat logika aplikasi lebih terprediksi dan mempermudah proses debugging ketika terjadi kesalahan kalkulasi pada tampilan sistem.

2.6.3 PostgreSQL

PostgreSQL adalah sistem manajemen basis data relasional (*Relational Database Management System* / RDBMS) bersifat *open-source* yang dikenal karena stabilitas, skalabilitas, dan kepatuhan terhadap standar SQL. Stonebraker et al. (1986), sebagai inisiator awal pengembangan teknologi ini, merancang PostgreSQL untuk menangani beban kerja kompleks yang tidak dapat diakomodasi oleh sistem basis data tradisional(Makris dkk., 2021).

Dalam konteks pengembangan sistem HRIS dengan arsitektur Modular Monolith, pemilihan PostgreSQL didasarkan pada dua faktor utama:

1. ACID Compliance

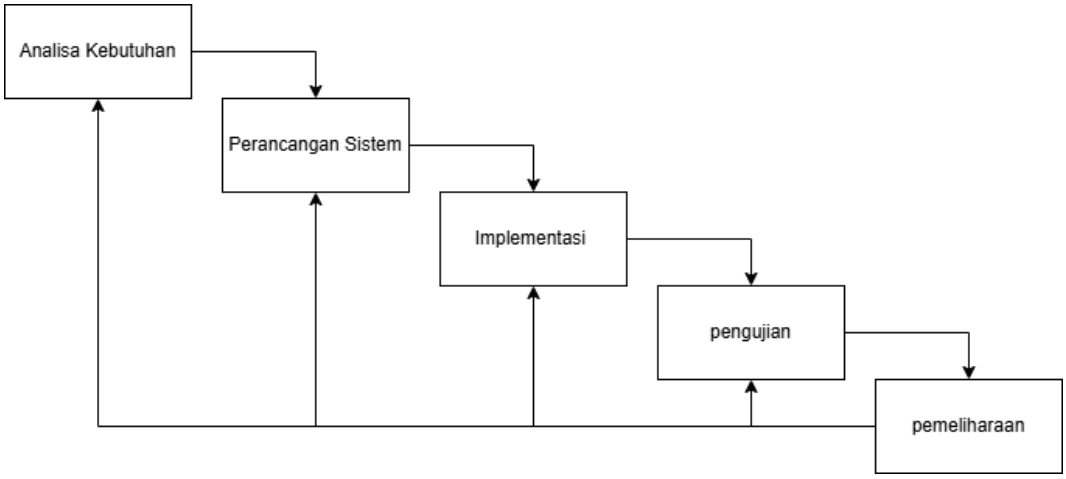
   Menjamin integritas data transaksi tetap akurat dan konsisten meskipun terjadi kegagalan sistem.

2. Dukungan JSONB

   PostgreSQL memiliki kemampuan hybrid untuk menyimpan data semi-terstruktur (JSON) di dalam tabel relasional. Fitur ini sangat relevan untuk menyimpan konfigurasi parameter penilaian Fuzzy C-Means yang dinamis atau struktur log aktivitas pengguna tanpa harus mengubah skema tabel secara kaku.

2.6.4 Metode Waterfall

Metode waterfall merupakan model pengembangan perangkat lunak yang bersifat sistematis dan berurutan, dimulai dari analisis kebutuhan pengguna, kemudian dilanjutkan dengan tahap perencanaan, permodelan, pembangunan, hingga penyerahan sistem kepada pengguna (Nur Adiya et al., 2024). Proses ini diakhiri dengan dukungan atau pemeliharaan terhadap perangkat lunak yang telah dihasilkan. Kelebihan dari metode waterfall terletak pada pembagian tahapan yang jelas dan terstruktur, sehingga memudahkan pengelolaan serta pengendalian dalam proses pengembangan. Tahapan dalam metode waterfall dapat dilihat pada Gambar 1.



Gambar 1. Metode Waterfall

Metode *waterfall* menurut Sommerville memiliki lima tahapan yang terdiri dari (Adi Kurniyanti & Murdiani, 2022):

1. Analisa Kebutuhan

   Pada tahap ini dilakukan proses komunikasi untuk membantu pengembang memahami perangkat lunak yang diinginkan pengguna sekaligus menetapkan batasan sistem. Pemahaman tersebut umumnya

diperoleh melalui wawancara, diskusi, maupun survei langsung. Data yang terkumpul kemudian dianalisis guna memperoleh informasi yang diperlukan.

2. Perancangan Sistem

   Setelah tahap spesifikasi kebutuhan selesai, langkah berikutnya adalah menyiapkan desain sistem. Tahap ini berfungsi untuk merancang dan mendefinisikan arsitektur sistem secara menyeluruh sehingga dapat menjadi acuan dalam proses pengembangan selanjutnya.

3. Implementasi

   Pada tahap ini, sistem dibangun dalam bentuk program-program kecil yang disebut unit atau modul. Setiap unit dikembangkan secara terpisah, kemudian digabungkan dan diintegrasikan pada tahap berikutnya hingga membentuk sistem yang utuh.

4. Pengujian

   Unit yang telah dikembangkan pada tahap sebelumnya digabungkan ke dalam sistem setelah melalui proses pengujian masing-masing unit. Setelah integrasi selesai, dilakukan pengujian menyeluruh terhadap sistem untuk memastikan tidak terdapat kesalahan atau kegagalan.

5. Pemeliharaan

   Pada tahap terakhir, perangkat lunak yang telah selesai dikembangkan dijalankan dan dipelihara. Pemeliharaan mencakup perbaikan terhadap kesalahan yang tidak terdeteksi pada tahap sebelumnya serta pengembangan atau peningkatan sistem sesuai kebutuhan baru yang muncul.

2.6.5 Unified Modeling Language (UML)

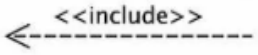
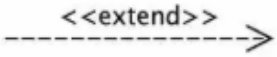
Pemodelan sistem merupakan tahap penting dalam analisis dan perancangan perangkat lunak karena memberikan gambaran yang jelas mengenai alur kerja, interaksi, serta struktur sistem yang akan dibangun. Dengan pemodelan, pengembang dapat memahami kebutuhan sistem secara lebih terstruktur dan memvisualisasikan hubungan antar komponen sebelum diimplementasikan. Dalam penelitian ini, pemodelan sistem dilakukan menggunakan pendekatan berorientasi objek dengan Unified Modeling

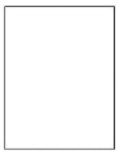
Language (UML). UML adalah sebuah standar pemodelan visual yang digunakan untuk mendeskripsikan, merancang, dan mendokumentasikan sistem perangkat lunak sehingga aspek fungsional maupun strukturalnya dapat direpresentasikan secara komprehensif (Hidayati et al., 2023). Diagram yang digunakan dalam penelitian ini meliputi use case diagram, sequence diagram, dan class diagram untuk menggambarkan kebutuhan fungsional, alur proses, serta struktur kelas dalam sistem.

1. *Use Case Diagram* (Diagram Use Case)

   Use case diagram digunakan untuk menggambarkan hubungan antara aktor dengan fungsionalitas sistem (Ramdany, 2024). Diagram ini menampilkan apa saja layanan yang disediakan sistem serta bagaimana interaksi antara pengguna dengan sistem. Komponen dalam *use case diagram* dapat dilihat pada Tabel 2 dibawah.

Tabel 2. Komponen *Use Case Diagram*

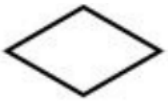
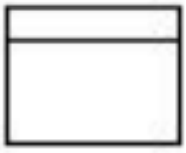
Gambar	**Nama**	**Keterangan**
	*Actor*	Menspesifikasikan himpunan peran yang pengguna mainkan ketika berinteraksi dengan *use case.*
	*Include*	Menunjukkan bahwa suatu *use case* seluruhnya merupakan fungsionalitas dari *use case* lainnya.
	*Extend*	Menunjukkan bahwa suatu *use case* merupakan tambahan fungsional dari *use case* lainnya jika suatu kondisi terpenuhi.
	Association	Apa yang menghubungkan antara objek satu dengan objek lainnya.
	*Use Case*	Deskripsi dari urutan aksi-aksi yang ditampilkan sistem yang menghasilkan suatu hasil yang terukur bagi suatu aktor.

	*System*	Menspesifikasikan paket yang menampilkan sistem secara terbatas.

2. *Activity Diagram* (Diagram Aktivitas)

Activity diagram menggambarkan alur aktivitas atau proses dalam sistem dari awal hingga akhir. Diagram ini menunjukkan urutan aktivitas, keputusan, aliran kontrol yang menggambarkan bagaimana suatu proses berjalan (Wayahdi & Ruziq, 2023). Komponen activity diagram dapat dilihat pada Tabel 3 dibawah.

Tabel 3. Komponen *Activity Diagram*

Gambar	Nama	Keterangan
	Status Awal	Sebuah diagram aktivitas memiliki sebuah status awal.
	Aktivitas	Aktivitas yang dilakukan sistem, aktivitas biasanya diawali dengan kata kerja.
	Percabangan/ *Decision*	Percabangan dimana ada pilihan aktivitas yang lebih dari satu.
	Penggabungan/ *Join*	Penggabungan dimana lebih dari satu aktivitas digabungkan menjadi satu.
	Status Akhir	Status akhir yang dilakukan sistem, sebuah diagram aktivitas memiliki sebuah status akhir.
	*Swimlane*	*Swimlane* memisahkan organisasi bisnis yang bertanggung jawab terhadap aktivitas yang terjadi.

2.6.6 Entity Relationship Diagram (ERD)

Entity Relationship Diagram (ERD) adalah model konseptual yang digunakan untuk mendeskripsikan struktur basis data secara logis. Chen (1976) memperkenalkan ERD sebagai metode untuk memetakan dunia nyata ke dalam skema basis data melalui entitas (entities) dan hubungan (relationships) antar entitas tersebut (Togatorop dkk., 2021).

Dalam penelitian ini, ERD digunakan untuk merancang struktur penyimpanan data yang ternormalisasi guna menghindari redundansi. Diagram ini akan memvisualisasikan relasi antara entitas utama. Mengingat sistem ini menerapkan pola Strategy Pattern, ERD juga berperan penting untuk memetakan bagaimana hasil penilaian dari algoritma yang berbeda yang disimpan dalam struktur tabel yang terpadu namun tetap dapat dibedakan relasinya.

2.6.7 Black-Box Testing

Black-box testing merupakan metode pengujian perangkat lunak yang berfokus pada fungsi sistem tanpa memperhatikan struktur internal maupun kode program. Pengujian dilakukan dengan memberikan berbagai input dan mengamati output yang dihasilkan untuk memastikan kesesuaiannya dengan spesifikasi yang telah ditentukan. Tujuan dari metode ini adalah memverifikasi bahwa perangkat lunak berjalan sesuai kebutuhan, di mana skenario uji disusun berdasarkan persyaratan sistem dan hasilnya dibandingkan dengan output yang diharapkan (P. P. Adi et al., 2020).

2.6.8 Integration Testing

Integration Testing adalah teknik pengujian perangkat lunak yang berfokus pada verifikasi interaksi dan pertukaran data antara berbagai komponen atau modul yang berbeda dalam sebuah aplikasi. Tujuan utama dari pengujian ini adalah untuk mengidentifikasi masalah atau bug yang timbul ketika komponen-komponen yang berbeda digabungkan dan berinteraksi satu sama lain (Xiong dkk., 2022).

Secara spesifik, metode ini menguji modul perangkat lunak untuk memastikan kebenaran komunikasi di antaranya. Integration testing umumnya dilaksanakan setelah seluruh modul menyelesaikan tahap unit testing. Proses ini dapat dilakukan secara bertahap, modul demi modul, dengan mengikuti urutan logika sistem yang tepat. Fokus utama dari pengujian ini adalah untuk menemukan cacat yang terjadi pada saat interaksi antar modul yang terintegrasi.

2.7 Penelitian Sejenis

Penelitian ini mengacu pada studi-studi terdahulu yang relevan sebagai rujukan, pertimbangan, sekaligus dasar perbandingan dalam pengembangannya. Kajian penelitian serupa disajikan pada Tabel 5.

Tabel 4. Penelitian Sejenis

No	Peneliti (Tahun)	Judul	Hasil	Perbedaan
1	Lapendy, Resky, & Surianto (2025)	Analisis Metode Fuzzy C-Means (FCM) dalam Menentukan Performansi Kinerja Karyawan	Menghasilkan 3 klaster kinerja dengan nilai validasi *Silhouette Coefficient* tertinggi sebesar 0,5653 menggunakan 5 variabel penilaian utama.	Fokus pada analisis pola data dan penentuan kriteria dominan menggunakan alat bantu simulasi (Matlab) pada dataset statis. Penulis membangun sistem informasi berbasis web (real application) dengan integrasi pola desain.
2	Fitriyani, Riana, & Primasari (2024)	Penerapan Metode Fuzzy C-Means dan TOPSIS Dalam Rekomendasi Perpanjangan Kontrak Karyawan	Mengembangkan sistem pendukung keputusan untuk rekomendasi perpanjangan kontrak dengan membagi data karyawan menjadi beberapa klaster menggunakan FCM, lalu merankingnya menggunakan TOPSIS.	Fokus penelitian adalah sistem pendukung keputusan (SPK) untuk kasus spesifik perpanjangan kontrak. Penulis membangun sistem HRIS menyeluruh dengan arsitektur Modular Monolith untuk menangani penilaian kinerja harian yang kompleks.
3	Yusuf, Hayati, & Sugiarti (2023)	Sistem Pendukung Keputusan Penilaian Kinerja Pegawai dengan Algoritma Fuzzy C-Means dan Hungarian	Menerapkan FCM untuk mengelompokkan pegawai ke dalam 3 klaster (Kurang, Baik, Sangat Baik) dengan nilai validasi Silhouette Coefficient	Fokus penelitian menggabungkan FCM dengan algoritma Hungarian untuk optimasi penugasan kerja. Penulis berfokus pada fleksibilitas arsitektur (Modular Monolith) untuk

			0,6035, serta menggunakan algoritma Hungarian untuk optimasi pembagian tugas.	menangani struktur penilaian yang dinamis dan heterogen.
4	Yudhistira, Aldino, & Darwis (2022)	Analisis Klasterisasi Penilaian Kinerja Pegawai Menggunakan Metode Fuzzy C-Means	Menghasilkan 5 klaster kinerja dan menemukan pola bahwa variabel komitmen dan kedisiplinan adalah faktor dominan dalam penentuan reward.	Menggunakan simulasi Matlab untuk menganalisis data perilaku pegawai pada instansi pemerintahan. Penulis menerapkan FCM dalam arsitektur Modular Monolith yang mampu beradaptasi terhadap perubahan variabel tanpa mengubah kode inti.
5	Rakhmawaty, Nasution, & Amijaya (2022)	Perbandingan Metode K-Means Dan Metode Fuzzy C-Means (FCM) Pada Analisis Kinerja Pegawai PT. Cemara Khatulistiwa Persada Bontang	Membuktikan FCM lebih unggul dibanding K-Means dengan nilai Silhouette Index 0,59604 (vs K-Means 0,4907) dan Davies Bouldin Index 0,58985. Menghasilkan 3 klaster kinerja: Tinggi (3 orang), Sedang (25 orang), dan Rendah (22 orang).	Fokus pada komparasi performa algoritma (K-Means vs FCM) menggunakan data statis perusahaan. Penulis berfokus pada implementasi FCM ke dalam sistem HRIS nyata dengan arsitektur Modular Monolith yang menangani penilaian dinamis.
6	Sari, Iskandar, & Prasetya (2022)	Analisa Metode Fuzzy C Means Untuk Klasterisasi Kinerja Teknisi CCAN Telkom Kediri	Menerapkan FCM untuk mengelompokkan kinerja 37 teknisi guna mengatasi masalah penilaian manual yang lambat. Penelitian ini bertujuan membantu manajer dalam	Fokus pada implementasi FCM untuk Teknisi Lapangan pada satu divisi spesifik. Penulis membangun sistem HRIS skala perusahaan yang menangani penilaian multi-divisi (Manajemen &

			pengambilan keputusan promosi atau evaluasi.	Operasional) secara terintegrasi.
7	Pramitasari & Nataliani (2021)	Perbandingan Clustering Karyawan Berdasarkan Nilai Kinerja dengan Algoritma K-Means dan Fuzzy C-Means	Algoritma FCM terbukti memiliki akurasi lebih tinggi (76%) dibandingkan K-Means (44%) dalam mengelompokkan data kinerja yang bersifat samar.	Penelitian ini menggunakan data homogen (satu jenis kriteria). Penulis menangani data heterogen (Supir vs Manajemen) menggunakan arsitektur dinamis.
8	Rahmadhani, Djunaidy, & Mukhlason (2021)	Evaluasi Kinerja Pemasok Menggunakan Fuzzy C-Means Clustering dan AHP di CV Delta Raya	Menghasilkan 3 klaster pemasok berdasarkan model RFM (Recency, Frequency, Monetary). Klaster K2 diidentifikasi sebagai yang terbaik dengan skor rata-rata tertinggi (0,6635), dan analisis lanjutan menggunakan DEA menemukan 12 pemasok paling potensial .	Fokus penelitian adalah evaluasi kinerja pemasok (Supplier) menggunakan kombinasi FCM dan AHP/DEA. Penulis menerapkan FCM untuk penilaian kinerja karyawan (SDM) yang diintegrasikan ke dalam sistem informasi berbasis web dengan arsitektur Modular Monolith.
9	Himah & Sulaiman (2021)	Implementasi Metode Fuzzy C-Means dan TOPSIS dalam Evaluasi Kinerja Keuangan Perusahaan Perbankan di Indonesia Berdasarkan Rasio Keuangan	Mengimplementasikan metode FCM untuk mengelompokkan kinerja keuangan perbankan dan TOPSIS untuk perankingan berdasarkan rasio keuangan (ROA, ROE, dll)	Fokus pada evaluasi kinerja keuangan (perbankan) menggunakan kombinasi FCM dan TOPSIS. Penulis berfokus pada penilaian kinerja karyawan dalam sistem HRIS berarsitektur Modular Monolith.
10	Sanjaya & Nataliani (2021)	Perbandingan Pembobotan Kriteria dan Seleksi	Membandingkan akurasi FCM dengan pembobotan fitur	Fokus pada teknik optimasi input (seleksi fitur) untuk meningkatkan akurasi

		Kriteria pada Pengelompokan Kinerja Karyawan dengan Fuzzy C-Means	(akurasi 0,8462) vs seleksi fitur (akurasi 1). Menemukan bahwa "Kedisiplinan" dan "Kejujuran" adalah fitur paling kritis.	algoritma pada satu jenis dataset. Penulis berfokus pada arsitektur sistem yang mampu beradaptasi dengan berbagai jenis dataset (heterogen) secara dinamis.
No	**Peneliti (Tahun)**	**Judul**	**Hasil**	**Perbedaan**
4	Ghaza Indra Pratama (2025)	Implementasi composable architecture berbasis strategy dan decorator patterns pada algoritma fuzzy c-means untuk klasterisasi kinerja karyawan	Terwujudnya sistem HRIS yang mampu menangani penilaian kinerja multi-jabatan (heterogen) secara objektif dalam satu platform terpadu.	Mengintegrasikan algoritma FCM ke dalam arsitektur Modular Monolith dengan Strategy dan Decorator Pattern untuk fleksibilitas logika penilaian.

BAB III
METODOLOGI PENELITIAN

3.1 Waktu dan Tempat Penelitian

Waktu penelitian berlangsung selama empat bulan, terhitung dari bulan Januari hingga April 2026. Penelitian ini dilaksanakan di PT. Arete Logistik Indonesia.

3.2 Alat Penelitian

Penelitian ini memanfaatkan berbagai alat, baik perangkat keras maupun perangkat lunak, untuk mendukung proses perancangan, pembangunan, dan pengujian sistem. Adapun alat penelitian yang digunakan adalah sebagai berikut:

1. Perangkat Keras

   ASUS TUF Gaming Laptop F-15 dengan spesifikasi:

   - Prosesor : Intel(R) i7-11600H
   - RAM : 16,0 GB
   - Penyimpanan : SSD 512 GB
   - GPU : NVIDIA RTX 3050

2. Perangkat Lunak
   - Sistem Operasi Windows 11 Home
   - Visual Studio Code
   - Draw.io
   - Microsoft Word

3.3 Data Penelitian

Metode pengumpulan data digunakan untuk mendapatkan informasi dan data yang akurat dalam pembangunan sistem penilaian kinerja karyawan yang heterogen pada PT Arete Logistik Indonesia, penulis menggunakan metode sebagai berikut (Ardiansyah dkk., 2023):

3.3.1 Studi Pustaka

Penulis mengumpulkan dan mempelajari referensi teori dari buku, jurnal ilmiah, dan artikel terpercaya yang berkaitan dengan:

- Konsep Composable Architecture dan penerapannya pada pengembangan backend.
- Pola desain perangkat lunak (Software Design Patterns), khususnya Strategy Pattern dan Decorator Pattern.
- Logika matematis algoritma Fuzzy C-Means (FCM) untuk klasterisasi data.
- Dokumentasi teknis framework NestJS.

3.3.2 Observasi

Penulis melakukan pengamatan langsung terhadap proses penilaian kinerja yang saat ini berjalan di perusahaan. Fokus observasi dilakukan pada dokumen atau formulir penilaian (KPI) yang digunakan untuk dua kategori karyawan berbeda, yaitu staf manajemen dan mitra pengemudi, guna memahami struktur data dan variabel penilaian yang digunakan.

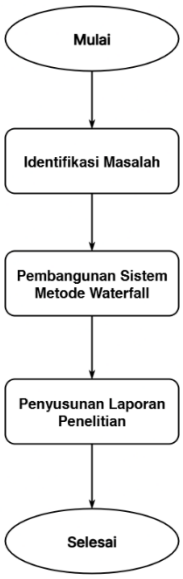
3.3.3 Wawancara

Penulis melakukan tanya jawab dengan pihak HRD dan Manajer Operasional untuk mengetahui kriteria spesifik yang menjadi bobot penilaian. Hal ini penting untuk menentukan variabel apa saja yang akan dimasukkan ke dalam Decorator serta batasan nilai (threshold) yang dianggap baik atau buruk.

3.4 Tahapan Penelitian

Penelitian ini menerapkan metode pengembangan perangkat lunak *Waterfall*. Pemilihan metode ini didasarkan pada sifatnya yang sistematis dan berurutan, sehingga setiap tahap dapat dilaksanakan secara terstruktur mulai dari analisis kebutuhan hingga pemeliharaan sistem. Pendekatan ini dianggap sesuai karena pengembangan HRIS dengan arsitektur *Modular Monolith* menuntut kejelasan spesifikasi sejak awal, baik terkait pemetaan kriteria penilaian yang heterogen (antara manajemen dan mitra pengemudi), penentuan parameter algoritma *Fuzzy C-Means*, maupun desain pola *Strategy* dan

Decorator, guna memastikan sistem yang dihasilkan mampu menangani dinamika penilaian kinerja secara akurat sesuai rancangan. Tahapan penelitian secara keseluruhan digambarkan pada Gambar 2 dan dijelaskan lebih lanjut pada uraian berikut:



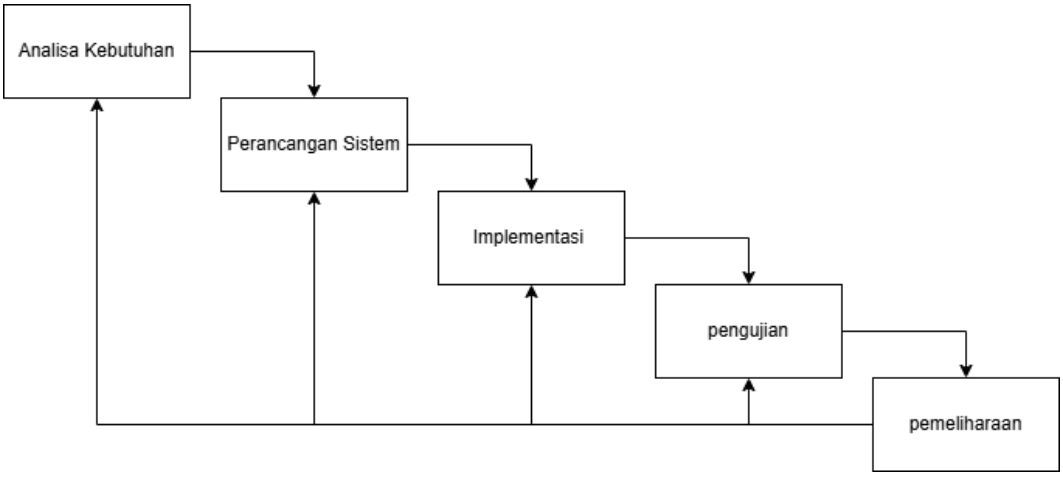
Gambar 2. Tahapan Penelitian

3.4.1 Identifikasi Masalah

Tahap identifikasi masalah dilakukan untuk menentukan fokus penelitian berdasarkan permasalahan nyata yang terjadi dalam proses evaluasi kinerja karyawan di PT Arete Logistik Indonesia. Masalah yang diidentifikasi antara lain kompleksitas pengelolaan penilaian yang heterogen antara staf manajemen (berbasis target finansial) dan mitra pengemudi (berbasis ritase dan keselamatan), serta belum adanya sistem yang memiliki fleksibilitas arsitektur untuk menangani penambahan variabel penilaian dinamis (seperti bonus atau sanksi) tanpa merombak ulang kode program. Dari identifikasi masalah ini kemudian ditetapkan rumusan masalah, tujuan penelitian, dan batasan sistem yang akan dikembangkan.

3.4.2 Pembangunan Sistem Metode Waterfall

Penelitian berjudul "Implementasi composable architecture berbasis strategy dan decorator patterns pada algoritma fuzzy c-means untuk klasterisasi kinerja karyawan" ini menerapkan metode pengembangan perangkat lunak waterfall. Pemilihan metode ini didasarkan pada sifatnya yang sistematis dan berurutan, sehingga setiap tahap dapat dilaksanakan secara jelas mulai dari analisis kebutuhan hingga pemeliharaan system. Tahapan penelitian secara keseluruhan digambarkan pada gambar dan dijelaskan lebih lanjut pada uraian berikut:



Gambar 3. Pembangunan Sistem Metode Waterfall

1. Analisa Kebutuhan

   Tahap ini bertujuan untuk mengidentifikasi dan mendefinisikan kebutuhan fungsional serta non-fungsional sistem secara mendalam. Proses analisis dimulai dengan memetakan atribut data karyawan yang heterogen guna memastikan setiap variabel penilaian yang berbeda antara staf manajemen dan mitra pengemudi teridentifikasi dengan jelas. Bersamaan dengan itu, dilakukan analisis kebutuhan arsitektur untuk menentukan spesifikasi modularitas yang diperlukan, sehingga logika penilaian yang spesifik untuk setiap peran dapat dipisahkan secara tegas dan tidak menimbulkan konflik logika dalam sistem yang terintegrasi.

2. Perancangan Sistem

   Setelah kebutuhan terdefinisi, tahap selanjutnya adalah merancang cetak biru (blueprint) sistem sebagai panduan teknis sebelum proses

pengkodean. Perancangan ini difokuskan pada desain Composable Architecture menggunakan framework NestJS untuk memastikan fleksibilitas sistem di masa depan. Penulis menyusun pemodelan visual menggunakan Unified Modeling Language (UML), dengan fokus utama pada penggunaan Class Diagram. Diagram ini digunakan secara komprehensif untuk memetakan hubungan logika bisnis (antara Interface, Strategy, dan Decorator) sekaligus memodelkan struktur entitas basis data (database entities) secara berbasis objek. Pendekatan ini dipilih agar struktur penyimpanan data selaras dengan logika aplikasi, sehingga mampu mengakomodasi variasi nilai penilaian antar jabatan tanpa merusak integritas sistem. Selain itu, disusun pula Sequence Diagram untuk menggambarkan alur interaksi data dari input pengguna hingga menghasilkan output klaster.

3. Implementasi

Tahap implementasi merupakan proses penerjemahan desain sistem menjadi baris kode program yang dapat dieksekusi. Penulis menggunakan bahasa pemrograman TypeScript dengan basis framework NestJS untuk membangun sisi backend sistem. Fokus utama pada tahap ini meliputi penulisan kode untuk Strategy Pattern pada Service Layer guna menangani pemilihan logika bisnis berdasarkan jabatan secara otomatis, serta penerapan Decorator Pattern untuk menyusun vektor input penilaian secara dinamis. Selain aspek arsitektur, fungsi matematis dari algoritma Fuzzy C-Means (FCM) juga dikonversi ke dalam algoritma pemrograman pada tahap ini agar sistem mampu melakukan kalkulasi klasterisasi data kinerja secara akurat.

Karena penelitian ini melibatkan algoritma Data Mining, penulis menambahkan tahapan spesifik untuk pengolahan data:

a. Pra-pemrosesan Data

Data mentah kinerja karyawan seringkali memiliki skala yang berbeda. Penulis melakukan proses Normalisasi agar setiap variabel memiliki rentang nilai yang setara (0 hingga 1) sebelum diproses oleh algoritma FCM.

b. Proses Klasterisasi

Data yang telah dinormalisasi akan diproses menggunakan algoritma FCM dengan tahapan:

- Penentuan jumlah cluster (c).
- Penentuan matriks partisi awal secara acak.
- Perhitungan pusat cluster (centroid).
- Pembaruan derajat keanggotaan.
- Iterasi hingga mencapai konvergensi (nilai error minimum).

4. Pengujian

Pengujian Untuk memastikan sistem berjalan sesuai tujuan, pengujian dilakukan dalam dua aspek:

a. Pengujian Fungsional

Tahap pengujian akan dilakukan untuk memastikan perangkat lunak berjalan sesuai dengan kebutuhan dan spesifikasi yang telah ditentukan. Metode yang akan digunakan adalah black-box testing, di mana pengujian difokuskan pada fungsi input dan output sistem tanpa melihat struktur internal kode. Pengujian ini akan mencakup validasi hasil diagnosis dan fungsionalitas antarmuka pengguna untuk memastikan sistem dapat berjalan dengan baik.

b. Pengujian Integrasi Modul

Pendekatan ini dipilih untuk memverifikasi validitas interaksi dan pertukaran data antar komponen yang telah disusun dalam arsitektur Modular Monolith. Mengingat sistem ini menerapkan pola desain yang kompleks seperti Strategy dan Decorator Pattern di lingkungan framework NestJS, pengujian integrasi menjadi langkah krusial untuk memastikan bahwa mekanisme Dependency Injection antar modul berjalan tanpa hambatan. Fokus utama pada tahap ini adalah mendeteksi potensi kesalahan (bug) atau kegagalan komunikasi pada antarmodul sebelum sistem digunakan secara menyeluruh. Pengujian ini menjamin bahwa aliran data—mulai dari input pengguna pada frontend ReactJS, pemrosesan logika algoritma

Fuzzy C-Means di backend, hingga penyimpanan ke basis data—terintegrasi dengan benar dan bebas dari anomali fungsional.

c. Pengujian Validitas Cluster

Mengukur kualitas hasil pengelompokan yang dilakukan oleh algoritma FCM. Penulis menggunakan metode Silhouette Coefficient untuk mengetahui seberapa baik objek ditempatkan dalam cluster-nya (seberapa dekat dengan cluster sendiri dan seberapa jauh dari cluster lain).

5. Pemeliharaan

Pada tahap akhir, perangkat lunak yang telah selesai dikembangkan dijalankan dan memasuki fase pemeliharaan. Pemeliharaan yang akan dilakukan meliputi perbaikan terhadap kesalahan yang belum terdeteksi pada tahap sebelumnya serta pengembangan sistem lebih lanjut sesuai kebutuhan baru yang muncul.

3.4.3 Penyusunan Laporan Penelitian

Tahap penyusunan laporan penelitian dilakukan secara paralel dengan proses pembangunan sistem. Pada tahap ini, setiap aktivitas penelitian mulai dari analisis kebutuhan, perancangan, implementasi, hingga pengujian didokumentasikan secara sistematis. Laporan memuat analisis dan pembahasan hasil yang diperoleh, evaluasi kinerja sistem, serta kesimpulan yang menjawab rumusan masalah dan tujuan penelitian. Selain itu, disertakan pula saran untuk pengembangan lebih lanjut agar sistem dapat terus disempurnakan.

3.5 Jadwal Penelitian

Penelitian direncanakan berlangsung selama empat bulan dengan distribusi waktu yang dapat dilihat pada Tabel dibawah.

Tabel 5. Jadwal Penelitian

Tahap Penelitian	Januari 2026	Februari 2026	Maret 2026	April 2026
Identifikasi Masalah				
Pembangunan Sistem Metode *Waterfall*				
Analisis Kebutuhan				
Perancangan Sistem				
Implementasi				
Pengujian				
Pemeliharaan				
Penyusunan Laporan Penelitian				

BAB IV HASIL DAN PEMBAHASAN

4.1 Analisis Kebutuhan

Tahap analisis kebutuhan yang dilakukan melalui studi pustaka, observasi, dan wawancara di PT Arete Logistik Indonesia mengungkap bahwa perusahaan belum memiliki sistem penilaian kinerja yang baku, sehingga evaluasi selama ini bersifat informal, subjektif, dan tidak terdokumentasi dengan baik. Masalah ini diperparah oleh heterogenitas jabatan antara tenaga manajemen dan mitra pengemudi (*driver*) yang memiliki karakteristik pekerjaan sangat berbeda. Akibatnya, penilaian tidak dapat menggunakan variabel yang seragam, yang kemudian menjadi tantangan utama dalam merancang sistem evaluasi kinerja yang adil, objektif, dan relevan bagi seluruh karyawan.

4.1.1 Analisis Kebutuhan Sistem

Berdasarkan hasil analisis kebutuhan yang telah dilakukan, kebutuhan sistem diidentifikasi dan dikelompokkan menjadi kebutuhan fungsional. Kebutuhan fungsional mendeskripsikan fungsi-fungsi yang harus dapat dilakukan oleh sistem berdasarkan peran pengguna. Kebutuhan fungsional sistem disajikan pada Tabel 6 berikut

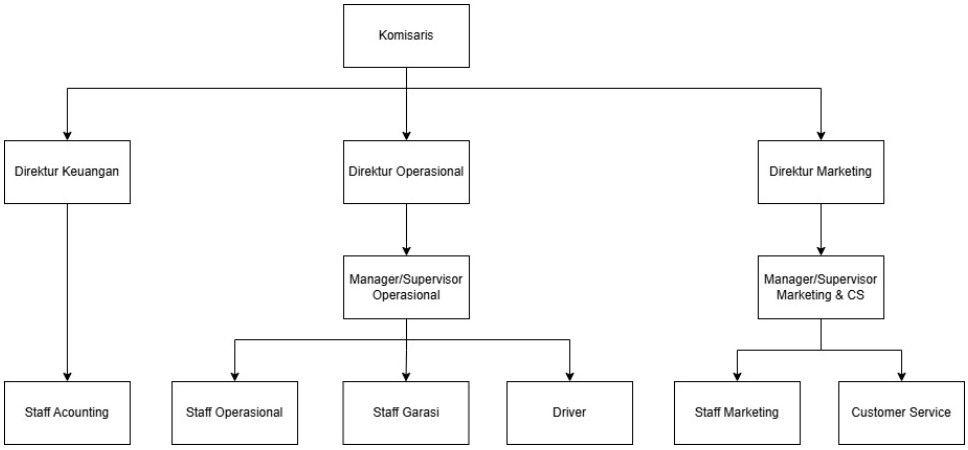
Tabel 6. Kebutuhan Fungsional Sistem

Pengguna	Kode	Deskripsi
HR	[URS-HR-01]	HR dapat mengakses sistem dengan melakukan login menggunakan kredensial yang valid
	[URS-HR-02]	HR dapat keluar dari sistem dengan melakukan logout untuk mengakhiri sesi
	[URS-HR-03]	HR dapat mengelola data pengguna sistem meliputi penambahan, pengubahan, dan penonaktifan akun pengguna
	[URS-HR-04]	HR dapat mengelola data departemen yang menjadi dasar assignment variabel KPI pada cakupan departemen dalam sistem
	[URS-HR-05]	HR dapat mengelola data jabatan yang menjadi dasar pemilihan logika penilaian secara dinamis dalam sistem
	[URS-HR-06]	HR dapat mengelola jadwal kerja karyawan sebagai dasar perhitungan presensi
	[URS-HR-07]	HR dapat mengelola data presensi karyawan
	[URS-HR-08]	HR dapat mengkonfigurasi chain approval untuk pengajuan izin karyawan
	[URS-HR-09]	HR dapat mengelola variabel KPI meliputi penambahan, pengubahan, dan penonaktifan variabel secara dinamis tanpa mengubah kode program

	[URS-HR-10]	HR dapat mengelola assignment variabel KPI ke tiga cakupan yaitu global, departemen, dan jabatan
	[URS-HR-11]	HR dapat membuka dan menutup periode penilaian secara terkontrol
	[URS-HR-12]	HR dapat mengkonfigurasi parameter FCM meliputi jumlah cluster, pangkat pembobot, error minimum, dan maksimum iterasi
	[URS-HR-13]	HR dapat menjalankan proses klasterisasi FCM setelah periode ditutup
	[URS-HR-14]	HR dapat melihat hasil klasterisasi kinerja seluruh karyawan beserta nilai Silhouette Coefficient
	[URS-HR-15]	HR dapat mengisi nilai KPI karyawan sebagai pengganti atasan jika diperlukan
Atasan	[URS-A-01]	Atasan dapat mengakses sistem dengan melakukan login menggunakan kredensial yang valid
	[URS-A-02]	Atasan dapat keluar dari sistem dengan melakukan logout untuk mengakhiri sesi
	[URS-A-03]	Atasan dapat memproses pengajuan izin bawahan berdasarkan chain approval yang telah dikonfigurasi
	[URS-A-04]	Atasan dapat melihat daftar bawahan beserta status penilaian pada periode yang sedang aktif
	[URS-A-05]	Atasan dapat melihat variabel KPI yang berlaku untuk setiap bawahan secara otomatis berdasarkan jabatan dan departemennya
	[URS-A-06]	Atasan dapat mengisi dan menyimpan draft nilai KPI per variabel untuk setiap bawahannya
	[URS-A-07]	Atasan dapat mengsubmit penilaian untuk mengunci data nilai KPI bawahan
	[URS-A-08]	Atasan dapat melihat hasil klasterisasi kinerja bawahannya berdasarkan periode yang dipilih
Karyawan	[URS-K-01]	Karyawan dapat mengakses sistem dengan melakukan login menggunakan kredensial yang valid
	[URS-K-02]	Karyawan dapat keluar dari sistem dengan melakukan logout untuk mengakhiri sesi
	[URS-K-03]	Karyawan dapat mengajukan izin melalui sistem
	[URS-K-04]	Karyawan dapat melihat status pengajuan izin dan data presensi dirinya sendiri
	[URS-K-05]	Karyawan dapat melihat variabel KPI yang berlaku untuk jabatannya pada periode yang sedang aktif
	[URS-K-06]	Karyawan dapat melihat hasil klasterisasi kinerja dirinya sendiri beserta derajat keanggotaan terhadap masing-masing cluster

4.1.2 Analisis Data Karyawan

Data karyawan yang digunakan dalam penelitian ini bersumber dari PT Arete Logistik Indonesia. Berdasarkan hasil observasi, perusahaan memiliki struktur organisasi yang terdiri dari beberapa tingkatan jabatan sebagaimana digambarkan pada Gambar 4 berikut.



Gambar 4. Struktur Jabatan PT Arete Logistik Indonesia

Berdasarkan struktur organisasi tersebut, jabatan di perusahaan terbagi menjadi dua kelompok dalam konteks penelitian ini. Kelompok pertama adalah jabatan yang berperan sebagai penilai, yaitu Komisaris, Direktur Utama, Direktur Keuangan, Direktur Operasional, dan Direktur Marketing. Kelompok kedua adalah jabatan yang menjadi objek penilaian kinerja, dengan total 45 karyawan aktif yang terbagi ke dalam tiga departemen dan 8 jabatan. Distribusi karyawan yang menjadi objek penilaian kinerja disajikan pada Tabel 8 berikut.

Tabel 7. Distribusi Karyawan Per Jabatan dan Departemen

No	Departemen	Jabatan	Jumlah
1	Operasional	Manager/Supervisor Operasional	1
2	Operasional	Staff Operasional	11
3	Operasional	Staff Garasi	5
4	Operasional	Driver	23
5	Marketing	Manager/Supervisor Marketing & CS	1
6	Marketing	Staff Marketing	1
7	Marketing	Customer Service	1
8	Finance	Staff Accounting	2

Berdasarkan tabel di atas, karyawan didominasi oleh jabatan Driver dengan 23 orang atau sekitar 50% dari total karyawan, diikuti Staff Operasional dengan 11 orang. Kondisi ini mencerminkan karakteristik PT Arete Logistik Indonesia sebagai perusahaan logistik yang operasionalnya bertumpu pada armada pengiriman. Heterogenitas data karyawan tidak hanya terletak pada jumlah, tetapi juga pada variabel KPI yang digunakan

untuk menilai kinerja masing-masing jabatan, di mana Driver dinilai berdasarkan aspek teknis operasional seperti jumlah ritase, efisiensi bahan bakar, dan keselamatan berkendara, sementara Staff Accounting dinilai berdasarkan aspek keuangan seperti akurasi data dan kepatuhan pajak. Perbedaan variabel penilaian antar jabatan inilah yang menjadi dasar penerapan *Strategy Pattern* dan *Decorator Pattern* dalam sistem, di mana variabel KPI yang berlaku untuk setiap karyawan ditentukan secara dinamis berdasarkan jabatan dan departemennya masing-masing.

Untuk mengakomodasi heterogenitas tersebut, variabel KPI dalam sistem diorganisasikan ke dalam tiga cakupan *assignment*, yaitu cakupan global yang berlaku untuk seluruh karyawan tanpa terkecuali, cakupan departemen yang berlaku untuk seluruh karyawan dalam satu departemen tertentu, dan cakupan jabatan yang berlaku eksklusif untuk satu jabatan tertentu. Kombinasi ketiga cakupan ini memungkinkan sistem menangani variasi variabel penilaian antar jabatan secara fleksibel tanpa perlu merombak struktur kode program ketika terjadi perubahan konfigurasi penilaian.

4.2 Perancangan Sistem

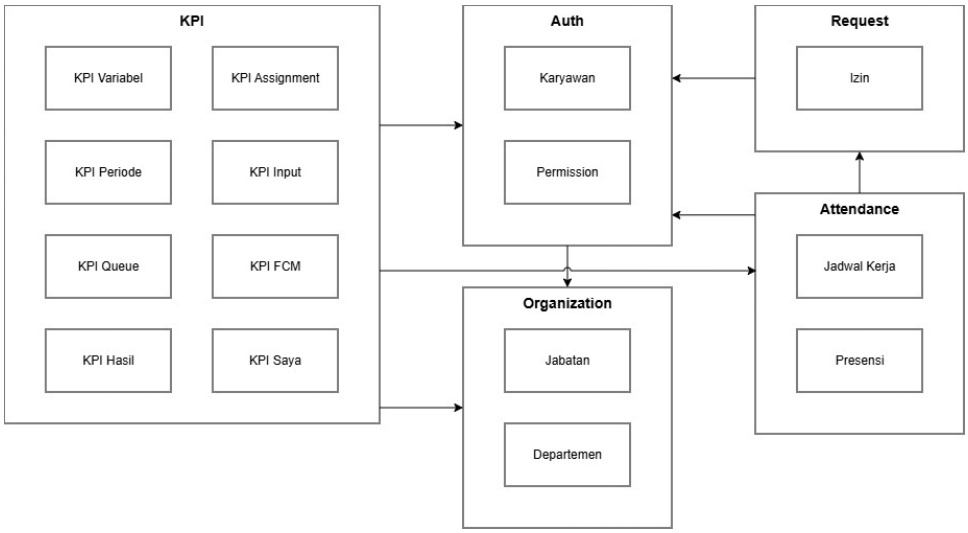
Tahap perancangan sistem dilakukan berdasarkan hasil analisis kebutuhan untuk menyusun cetak biru sistem sebagai panduan teknis sebelum implementasi, dengan fokus pada desain *Composable Architecture* menggunakan *framework* NestJS dengan pendekatan *Modular Monolith*, di mana setiap domain bisnis dipisahkan ke dalam modul-modul yang terisolasi namun tetap berjalan dalam satu kesatuan aplikasi. Pemodelan sistem dilakukan menggunakan *Unified Modeling Language* (UML) yang meliputi *Use Case Diagram*, *Activity Diagram*, *Sequence Diagram*, dan *Class Diagram* untuk menggambarkan kebutuhan fungsional, alur proses, dan struktur kelas, sementara struktur penyimpanan data dirancang menggunakan *Entity Relationship Diagram* (ERD) untuk memastikan integritas dan konsistensi data antar modul. Perancangan juga mencakup penyusunan *mockup* antarmuka pengguna serta desain arsitektur inti sistem yaitu *Composable Architecture*,

Strategy Pattern, *Decorator Pattern*, dan algoritma *Fuzzy C-Means* yang menjadi fokus utama penelitian ini.

4.2.1 Perancangan *Composable Architecture*

Sistem evaluasi kinerja karyawan ini dirancang menggunakan pendekatan *Composable Architecture* dengan pola *Modular Monolith*. Pendekatan ini dipilih karena mampu menggabungkan kesederhanaan arsitektur monolitik dengan keunggulan modularitas, di mana setiap domain bisnis dipisahkan ke dalam modul-modul yang terisolasi namun tetap berjalan dalam satu kesatuan aplikasi. Berbeda dengan *Microservices* yang memisahkan modul ke dalam proses dan infrastruktur yang berbeda, *Modular Monolith* memisahkan tanggung jawab di level kode melalui mekanisme *Dependency Injection* NestJS tanpa *overhead* komunikasi jaringan antar modul.

Sistem ini terdiri dari lima kelompok modul sebagaimana digambarkan pada arsitektur modular sistem pada Gambar 5 berikut.



Gambar 5. Arsitektur Modular Sistem

Sistem ini terdiri dari beberapa kelompok modul pendukung. Modul *auth* bertanggung jawab atas autentikasi, manajemen data karyawan, dan sistem *permission*, di mana proses *login* dan *logout* menghasilkan *JWT token* yang digunakan sebagai *guard* oleh seluruh modul, sementara sistem *permission*-nya menentukan hak akses setiap pengguna berdasarkan jabatan sehingga HR, Atasan, dan Karyawan hanya dapat mengakses fitur sesuai

perannya. Modul *organization* mengelola struktur organisasi perusahaan melalui sub-modul jabatan dan departemen, yang menjadi fondasi mekanisme *Strategy Pattern* dan *Decorator Pattern* pada modul KPI. Modul *attendance* mengelola jadwal kerja dan presensi karyawan, di mana data presensi menjadi sumber data otomatis untuk variabel KPI global yang diambil oleh modul *kpi-datasource* saat periode penilaian dibuka. Modul *request* mengelola pengajuan izin karyawan yang berpengaruh langsung terhadap data presensi, yang selanjutnya mempengaruhi nilai variabel presensi dalam perhitungan skor akhir FCM.

Modul *kpi* merupakan inti dari penelitian ini dan terdiri dari sembilan sub-modul yang mengikuti alur *lifecycle* penilaian kinerja. Modul *kpi-variabel* menyimpan definisi variabel KPI yang dapat dikonfigurasi secara dinamis, sedangkan modul *kpi-assignment* menghubungkan variabel ke tiga cakupan (global, departemen, dan jabatan) sekaligus memvalidasi total bobot setiap jabatan sebelum periode dapat dibuka. Modul *kpi-datasource* menyediakan *registry* sumber data otomatis, modul *kpi-periode* mengelola *lifecycle* periode penilaian mulai dari pembuatan hingga penutupan yang memicu proses klasterisasi, dan modul *kpi-input* mengimplementasikan *Strategy Pattern* serta *Decorator Pattern* untuk menentukan variabel KPI yang berlaku bagi setiap karyawan secara dinamis. Modul *kpi-queue* menerima *trigger* dari *kpi-periode* saat periode ditutup dan menjalankan proses klasterisasi secara *asynchronous* melalui *Bull Queue*, modul *kpi-fcm* menjalankan algoritma *Fuzzy C-Means* dan menghitung *Silhouette Coefficient*, modul *kpi-hasil* menyimpan dan menampilkan hasil klasterisasi kepada HR dan Atasan, sementara modul *kpi-saya* memungkinkan karyawan melihat hasil klasterisasi kinerjanya sendiri berdasarkan identitas yang sedang *login*.

Penerapan *Composable Architecture* dalam sistem ini tercermin dari dua aspek utama, yaitu variabel KPI, bobot, dan konfigurasi parameter FCM dapat diubah melalui konfigurasi *database* tanpa perlu mengubah kode program, serta logika pemilihan variabel penilaian dirancang menggunakan *Strategy Pattern* sehingga penambahan logika baru untuk jabatan baru

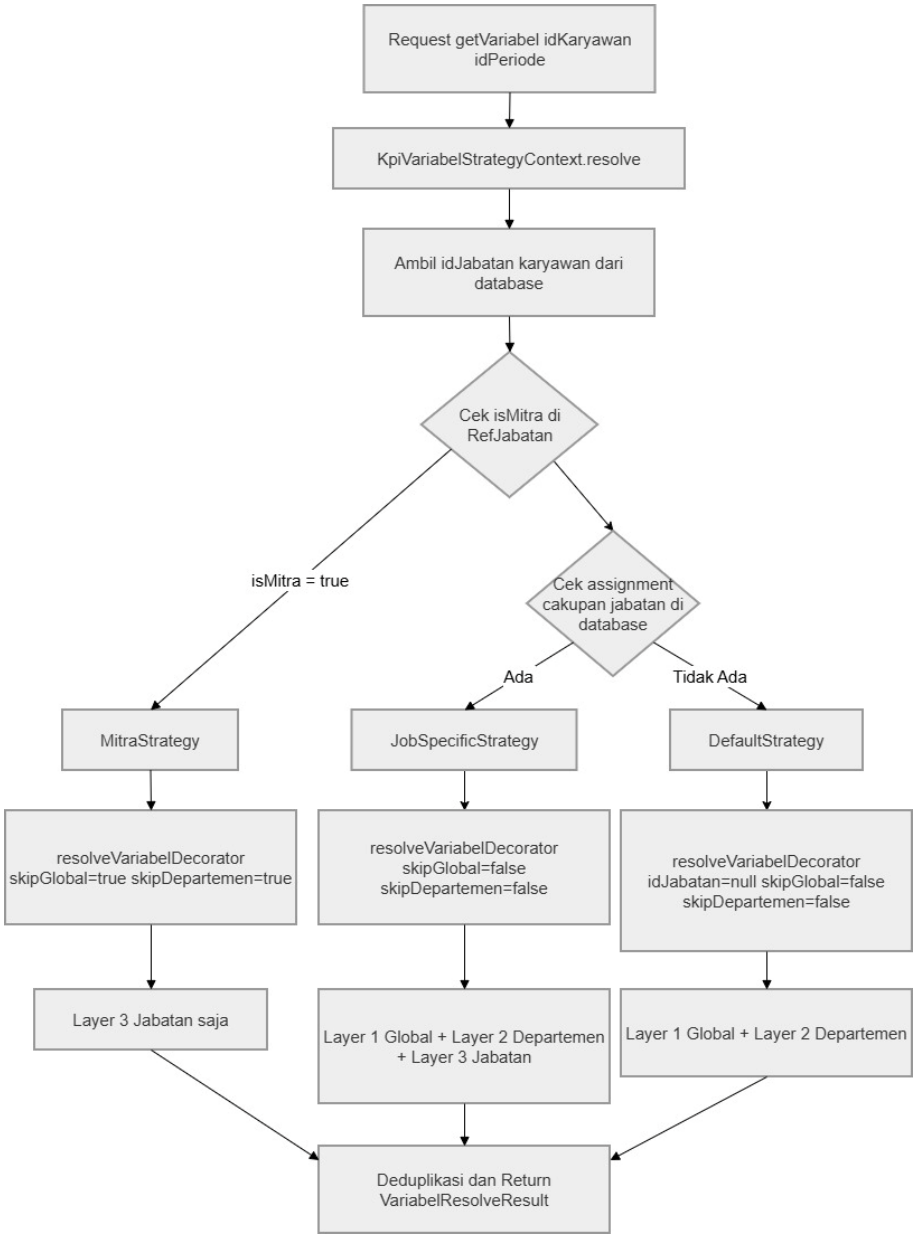
cukup dilakukan dengan membuat *concrete strategy* baru tanpa menyentuh kode yang sudah ada. Kedua aspek ini menjadikan sistem adaptif terhadap perubahan struktur organisasi dan kebutuhan bisnis tanpa memerlukan perombakan arsitektur secara keseluruhan.

4.2.2 Perancangan *Strategy Pattern*

Strategy Pattern diterapkan dalam sistem ini untuk menangani heterogenitas logika pemilihan variabel KPI antar jabatan secara dinamis. Tanpa *Strategy Pattern*, sistem harus menggunakan percabangan kondisional yang kaku untuk menentukan variabel yang berlaku bagi setiap jabatan sehingga penambahan jabatan baru akan selalu membutuhkan perubahan kode program, sedangkan dengan *Strategy Pattern* logika pemilihan variabel dienkapsulasi ke dalam kelas-kelas *strategy* yang dapat dipertukarkan sehingga sistem dapat beradaptasi terhadap perubahan konfigurasi jabatan tanpa menyentuh kode inti.

Sistem ini mengimplementasikan tiga *strategy* yang masing-masing mengimplementasikan *interface IKpiVariabelStrategy* dengan kontrak *method resolve()*. *DefaultStrategy* digunakan untuk jabatan yang tidak memiliki *assignment* cakupan jabatan di *database* dan menjalankan *resolve* variabel dengan dua *layer* yaitu global dan departemen, *JobSpecificStrategy* digunakan untuk jabatan karyawan tetap yang memiliki *assignment* cakupan jabatan di *database* dan menjalankan *resolve* variabel dengan tiga *layer* yaitu global, departemen, dan jabatan, sedangkan *MitraStrategy* digunakan khusus untuk mitra pengemudi yang memiliki *flag isMitra = true* pada data jabatan di *database* dan hanya menggunakan *layer* jabatan tanpa mengikutsertakan *layer* global maupun departemen karena mitra pengemudi dievaluasi berdasarkan parameter operasional yang sepenuhnya berbeda dari karyawan tetap. Pemilihan *strategy* dilakukan secara dinamis oleh *KpiVariabelStrategyContext* berdasarkan data jabatan karyawan di *database*.

Alur pemilihan *strategy* digambarkan pada Gambar 6 berikut.



Gambar 6. *Flowchart* Pemilihan *Strategy*

Ketika atasan membuka halaman penilaian untuk seorang karyawan, sistem memanggil *KpiVariabelStrategyContext.resolve()* dengan parameter *idKaryawan* dan *idPeriode*, kemudian *Context* mengambil data jabatan karyawan dari *database* dan melakukan pemeriksaan secara bertahap. Pemeriksaan pertama mengecek apakah jabatan karyawan memiliki *flag isMitra = true*; jika ya, *Context* memilih *MitraStrategy* yang memanggil *resolveVariabelDecorator* dengan parameter *skipGlobal=true* dan

skipDepartemen=true sehingga hanya *layer* jabatan yang diproses. Jika *isMitra = false*, *Context* melanjutkan pemeriksaan kedua yaitu mengecek keberadaan *assignment* cakupan jabatan di *database*, di mana jika ditemukan akan memilih *JobSpecificStrategy* yang memanggil *resolveVariabelDecorator* dengan ketiga *layer* aktif, dan jika tidak ditemukan akan memilih *DefaultStrategy* yang memanggil *resolveVariabelDecorator* hanya dengan *layer* global dan departemen.

Sebagai contoh, karyawan dengan jabatan *Driver* memiliki *flag isMitra = true* sehingga sistem memilih *MitraStrategy* dan hanya memproses variabel eksklusif jabatan *Driver* seperti *On-Time Delivery*, Total Ritase, Efisiensi BBM, *POD Return Speed*, Kepuasan Pelanggan, *Safety Score*, dan Perawatan Truk dengan total bobot 100%. Sebaliknya, karyawan dengan jabatan *Staff Accounting* yang memiliki *assignment* cakupan jabatan akan menggunakan *JobSpecificStrategy* dengan tiga *layer* aktif, sedangkan jabatan yang hanya memiliki variabel global dan departemen akan menggunakan *DefaultStrategy*.

4.2.3 Perancangan *Decorator Pattern*

Decorator Pattern diterapkan dalam sistem ini untuk menyusun variabel KPI yang berlaku bagi setiap karyawan secara dinamis dan bertingkat. Pendekatan ini memungkinkan sistem menggabungkan variabel dari beberapa *layer assignment* secara komposabel, di mana setiap *layer* dapat menambahkan atau menggantikan variabel dari *layer* sebelumnya tanpa mengubah struktur kode program. Implementasi *Decorator Pattern* dalam sistem ini diwujudkan melalui fungsi *resolveVariabelDecorator* yang dipanggil oleh setiap *strategy* dengan parameter yang berbeda sesuai dengan jenis *strategy*-nya.

Sistem ini mengorganisasikan variabel KPI ke dalam tiga cakupan *assignment* yang membentuk hierarki *layer* sebagai berikut:

Layer 1 Global berlaku untuk seluruh karyawan tetap tanpa terkecuali. Variabel pada *layer* ini merupakan indikator dasar yang relevan bagi semua jabatan. Variabel KPI pada *layer global* disajikan pada Tabel 9 berikut.

Tabel 8. Variabel KPI *Layer Global*

Variabel	Tipe	Arah	Bobot
Presensi	%	↑	10%

***Layer 2* Departemen** berlaku untuk seluruh karyawan dalam satu departemen tertentu dan dapat menggantikan variabel *global* jika terdapat variabel yang sama. Variabel KPI pada *layer* departemen disajikan pada Tabel 10 berikut.

Tabel 9. Variabel KPI *Layer* Departemen

Departemen	Variabel	Tipe	Arah	Bobot
Operasional	*Incident Response*	Jam	↓	10%
Marketing	*Complaint Rate*	Jumlah	↓	15%
Marketing	*SLA Compliance*	%	↑	15%
Finance	-	-	-	-

***Layer 3* Jabatan** berlaku eksklusif untuk satu jabatan tertentu dan memiliki prioritas tertinggi dapat menggantikan variabel dari *layer* departemen maupun *global* jika terdapat variabel yang sama. Variabel KPI pada *layer* jabatan disajikan pada Tabel 11 berikut.

Tabel 10. Variabel KPI *Layer* Jabatan

Jabatan	Variabel	Tipe	Arah	Bobot
Manager/Supervisor Operasional	*Fleet Utility*	%	↑	25%
	Order Fulfillment	%	↑	25%
	SLA Compliance	%	↑	15%
	Complaint Rate	Jumlah	↓	10%
	Budget Absorption	%	↑	5%
Manager/Supervisor Marketing & CS	*Repeat Order Rate*	%	↑	20%
	Order Fulfillment	%	↑	15%
	Response Time	Jam	↓	10%
	Budget Absorption	%	↑	10%
	New Volume Order	Jumlah	↑	5%
Staff Accounting	*Data Accuracy*	%	↑	25%
	AR Collection	Hari	↓	20%
	Invoice Processed	Dokumen	↑	20%
	Tax Compliance	%	↑	15%
	Days to Close	Hari	↓	5%
	Payroll Accuracy	%	↑	5%
Staff Operasional	*Order Fulfillment*	%	↑	25%
	Fleet Utility	%	↑	25%
	Document Accuracy	%	↑	15%
	Complaint Rate	Jumlah	↓	10%
	On-Time Dispatch	%	↑	5%

Staff Garasi	*Fleet Downtime Rate*	%	↓	20%
	Schedule Compliance	%	↑	20%
	Servis Turnaround Time	Hari	↓	15%
	Inventory Accuracy	%	↑	15%
	Vendor Management	Skala 1-5	↑	5%
	Budget Absorption	%	↑	5%
Customer Service	*Response Time*	Jam	↓	20%
	Order Fulfillment	%	↑	15%
	Repeat Order Rate	%	↑	10%
	Issue Resolution Rate	%	↑	10%
	First Contact Resolution	%	↑	5%
Staff Marketing	*Campaign Execution Rate*	%	↑	20%
	Lead Generation	Jumlah	↑	15%
	Content Output	Jumlah	↑	15%
	Market Research Report	Jumlah	↑	10%
Driver	*On-Time Delivery*	%	↑	25%
	Total Ritase	Trip	↑	20%
	Efisiensi BBM	Km/Liter	↑	15%
	POD Return Speed	Hari	↓	10%
	Kepuasan Pelanggan	Skala 1-5	↑	20%
	Safety Score	Poin 0-100	↑	5%
	Perawatan Truk	Skala 1-5	↑	5%

Mekanisme penggabungan ketiga *layer* dilakukan melalui struktur *variabelMap* yang menggunakan *idVariabel* sebagai kunci. *Layer* global dimasukkan terlebih dahulu sebagai prioritas terendah, kemudian *layer* departemen menggantikan variabel global jika *idVariabel* sama, dan terakhir *layer* jabatan menggantikan variabel departemen maupun global jika *idVariabel* sama. Hasil akhirnya adalah daftar variabel unik tanpa duplikasi yang mencerminkan konfigurasi penilaian yang tepat untuk setiap karyawan.

4.2.4 Perancangan Algoritma FCM

Algoritma *Fuzzy C-Means* (FCM) diterapkan dalam sistem ini untuk melakukan klasterisasi kinerja karyawan berdasarkan skor akhir yang dihasilkan dari proses penilaian KPI. FCM dipilih karena kemampuannya dalam melakukan *soft clustering*, di mana setiap karyawan memiliki derajat keanggotaan terhadap setiap *cluster* antara 0 hingga 1, sehingga mampu merepresentasikan kinerja karyawan yang seringkali berada pada area samar dan tidak dapat dikategorikan secara mutlak. Berbeda dengan *K-Means* yang bersifat *hard clustering*, FCM menghasilkan informasi yang lebih kaya karena karyawan dapat memiliki keanggotaan parsial di lebih dari satu *cluster* sekaligus.

Perancangan algoritma FCM dalam sistem ini mencakup dua aspek utama yaitu alur integrasi FCM ke dalam sistem dan alur algoritma FCM itu sendiri. Kedua aspek ini dirancang sebelum tahap implementasi untuk memastikan algoritma dapat berjalan dengan benar dan terintegrasi dengan baik ke dalam arsitektur *Modular Monolith* yang telah dirancang.

1. Parameter FCM

Sistem ini menggunakan parameter FCM yang dapat dikonfigurasi oleh HR melalui modul kpi-periode sebelum periode dibuka. Parameter yang digunakan disajikan pada Tabel 12 berikut.

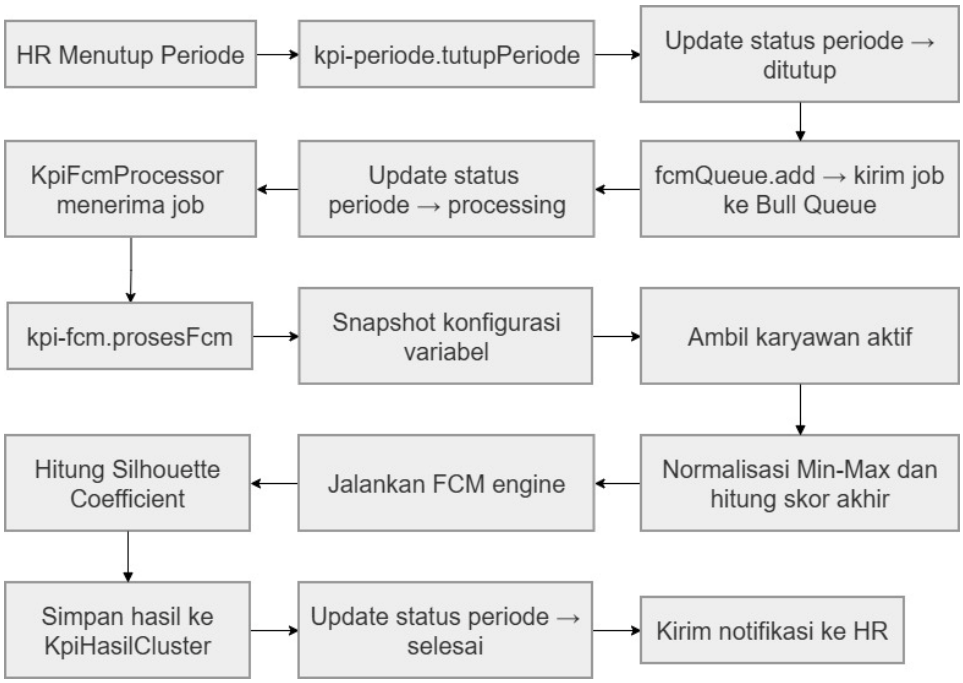
Tabel 11. Parameter FCM

Parameter	Nilai Default	Keterangan
Jumlah *cluster* (c)	3	*Fixed: High Performer, Standard, Low Performer*
Pangkat pembobot (m)	2.0	Eksponen fuzzy yang mengatur tingkat kekaburan
Maksimum iterasi	100	Batas maksimum iterasi jika belum konvergen
Error minimum (ε)	0.000001	Batas toleransi konvergensi
Seed	idPeriode	Seed deterministik via Linear Congruential Generator

Penggunaan *seed* deterministik berbasis idPeriode memastikan hasil klasterisasi bersifat *reproducible* untuk data dan periode yang sama, sistem selalu menghasilkan output yang identik meskipun dijalankan berulang kali.

2. Alur Integrasi FCM dalam Sistem

Proses FCM dijalankan secara otomatis setelah HR menutup periode penilaian. Alur integrasi FCM ke dalam sistem digambarkan pada Gambar 7 berikut.



Gambar 7. Diagram Alur Sistem FCM

Mengacu pada Gambar 7, proses dimulai ketika HR menutup periode melalui modul *kpi-periode*. Sistem kemudian mengirimkan *job* ke *Bull Queue* secara *asynchronous* sehingga proses klasterisasi berjalan di *background* tanpa mengganggu operasional sistem. *KpiFcmProcessor* menerima *job* dan mengorkestrasi enam langkah proses FCM. Langkah pertama adalah pembuatan *snapshot* seluruh konfigurasi *assignment* variabel yang aktif saat periode ditutup disimpan ke tabel *KpiVariabelSnapshot* untuk memastikan konsistensi hasil meskipun konfigurasi *assignment* diubah di kemudian hari. Langkah kedua adalah pengambilan seluruh karyawan aktif yang tidak masuk daftar *skip*. Langkah ketiga adalah normalisasi nilai dan perhitungan skor akhir per karyawan. Langkah keempat adalah menjalankan FCM *engine*. Langkah kelima adalah menghitung *Silhouette Coefficient* untuk mengukur kualitas hasil klasterisasi. Langkah keenam adalah menyimpan hasil ke tabel

KpiHasilCluster, memperbarui status periode menjadi selesai, dan mengirimkan notifikasi ke seluruh HR.

3. Normalisasi Min-Max

Sebelum data diproses oleh FCM *engine*, nilai KPI setiap karyawan dinormalisasi menggunakan metode *Min-Max* agar seluruh variabel berada dalam rentang 0 hingga 1. Normalisasi dilakukan dengan mempertimbangkan arah nilai variabel sebagai berikut:

a. Untuk variabel dengan arah lebih tinggi lebih baik:

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Keterangan :

- x_{norm} = nilai variabel setelah dinormalisasi
- x = nilai aktual variabel karyawan
- x_{min} = nilai minimum variabel yang dikonfigurasi
- x_{max} = nilai maksimum variabel yang dikonfigurasi

b. Untuk variabel dengan arah lebih rendah lebih baik:

$$x_{norm} = \frac{x_{max} - x}{x_{max} - x_{min}}$$

Keterangan :

- x_{norm} = nilai variabel setelah dinormalisasi
- x = nilai aktual variabel karyawan
- x_{min} = nilai minimum variabel yang dikonfigurasi
- x_{max} = nilai maksimum variabel yang dikonfigurasi

Hasil normalisasi kemudian dikalikan dengan bobot masing-masing variabel dan diakumulasikan menjadi satu skor akhir per karyawan dalam rentang 0 hingga 1 dengan rumus:

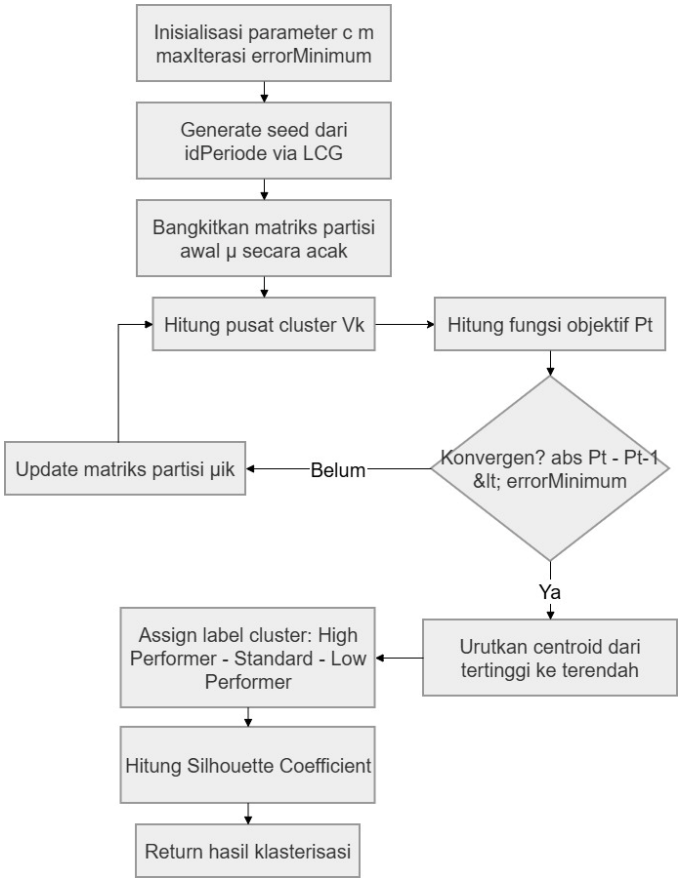
$$skor_{akhir} = \sum_{i=1}^{n} \frac{x_{norm_i} \times bobot_i}{100}$$

Keterangan :

- $skor_{akhir}$ = skor akhir karyawan dalam rentang 0 hingga 1
- x_{norm_i} = nilai normalisasi variabel ke-i
- $bobot_i$ = bobot variabel ke-i dalam persentase
- n = jumlah variabel KPI yang berlaku untuk karyawan

4. Alur Algoritma FCM

Alur algoritma FCM yang dijalankan oleh *fcm.engine.ts* digambarkan pada Gambar 8 berikut.



Gambar 8. Diagram Alur Algoritma FCM

Mengacu pada Gambar 8, algoritma FCM dimulai dengan inisialisasi parameter dan pembangkitan *seed* deterministik dari *idPeriode* menggunakan *Linear Congruential Generator* (LCG). Matriks partisi awal μ dibangkitkan secara acak menggunakan *seed* tersebut, kemudian dinormalisasi per karyawan sehingga jumlah derajat keanggotaan setiap karyawan terhadap seluruh *cluster* sama dengan 1. Selanjutnya algoritma

masuk ke tahap iterasi yang mengacu pada tahapan FCM sebagaimana telah dijelaskan pada 2.3.2, meliputi perhitungan pusat *cluster*, perhitungan fungsi objektif, pembaruan matriks partisi, dan pengecekan konvergensi. Iterasi berhenti ketika selisih fungsi objektif antar iterasi lebih kecil dari nilai *error* minimum atau maksimum iterasi telah tercapai.

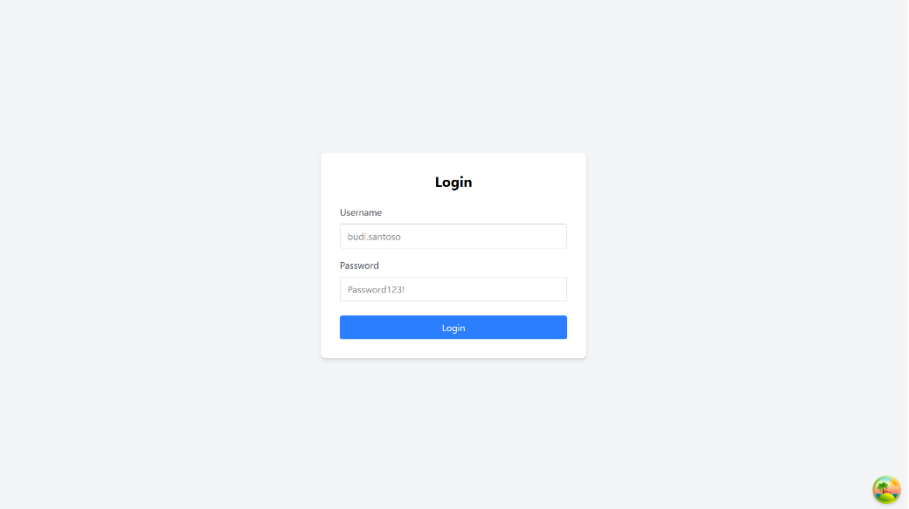
Setelah konvergen, sistem menentukan label *cluster* secara otomatis berdasarkan nilai *centroid cluster* dengan *centroid* tertinggi mendapat label *High Performer*, *cluster* tengah mendapat label *Standard*, dan *cluster* dengan *centroid* terendah mendapat label *Low Performer*. Kualitas hasil klasterisasi kemudian diukur menggunakan *Silhouette Coefficient* yang mengacu pada rumus sebagaimana telah dijelaskan pada 2.3.3.

4.2.5 Perancangan Antarmuka Pengguna (*Mockup*)

Perancangan antarmuka pengguna dilakukan untuk menggambarkan tampilan visual sistem yang akan dibangun sebelum tahap implementasi. Antarmuka dirancang berdasarkan kebutuhan fungsional yang telah diidentifikasi pada sub-bab 4.1.1, di mana *system* memiliki tiga peran pengguna yaitu HR, Atasan, dan Karyawan dengan hak akses yang berbeda. Setiap pengguna hanya dapat mengakses halaman yang sesuai dengan *permission* jabatannya, sebagaimana telah dirancang dalam modul *auth* pada sub-bab 4.2.1.

Perancangan antarmuka dikelompokkan berdasarkan peran pengguna untuk memudahkan pemahaman alur navigasi dan fungsionalitas yang tersedia bagi masing-masing pengguna. Bagian pertama menampilkan halaman *login* yang dapat diakses oleh seluruh peran pengguna, dilanjutkan dengan halaman-halaman yang dapat diakses oleh HR, Atasan, dan Karyawan secara berurutan.

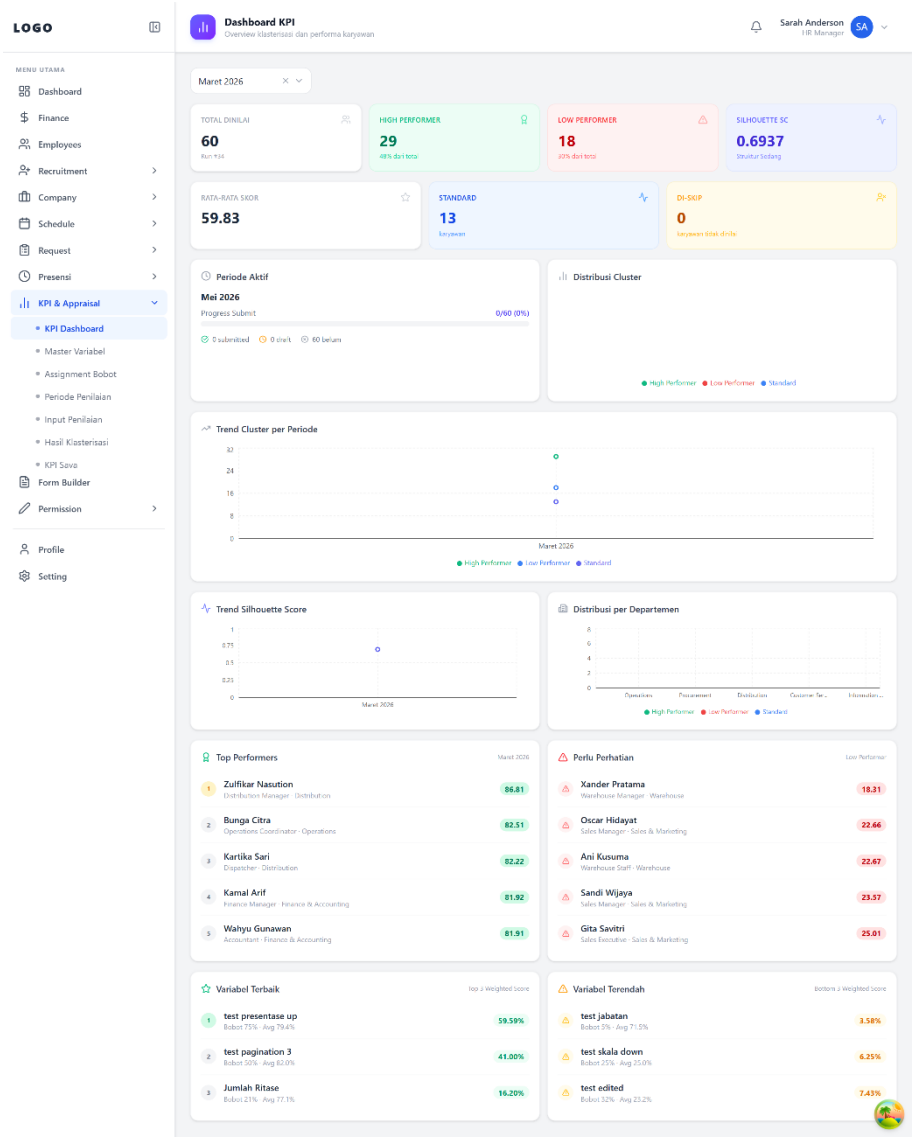
1. Halaman *Login* (URS-HR-01, URS-A-01, URS-K-01)



Gambar 9. Halaman *Login*

Halaman *login* merupakan halaman pertama yang ditampilkan kepada seluruh pengguna sistem sebelum dapat mengakses fitur-fitur yang tersedia. Halaman ini menampilkan *form login* yang terdiri dari dua *field input* yaitu *field Username* untuk memasukkan nama pengguna dan *field Password* untuk memasukkan kata sandi, serta tombol *Login* berwarna biru untuk memproses autentikasi. Tampilan halaman dirancang minimalis dengan *card form* yang terpusat di tengah layar pada latar belakang abu-abu terang, sehingga pengguna dapat langsung fokus pada proses *login* tanpa distraksi visual. Setelah pengguna berhasil melakukan autentikasi, sistem akan mengarahkan pengguna ke halaman *dashboard* sesuai dengan *permission* jabatan yang dimiliki.

2. Halaman KPI *Dashboard* (URS-HR-14)



Gambar 10. Halaman KPI *Dashboard*

Halaman *KPI Dashboard* merupakan halaman utama modul *KPI & Appraisal* yang dapat diakses oleh HR, di mana halaman ini menyajikan ringkasan komprehensif hasil klasterisasi kinerja karyawan secara visual. Pada bagian atas halaman terdapat *selector* periode yang memungkinkan HR memilih periode selesai yang ingin ditampilkan datanya.

Bagian atas konten menampilkan dua baris *summary card* yang merangkum indikator kunci hasil klasterisasi. Baris pertama menampilkan Total Dinilai, *High Performer*, *Low Performer*, dan

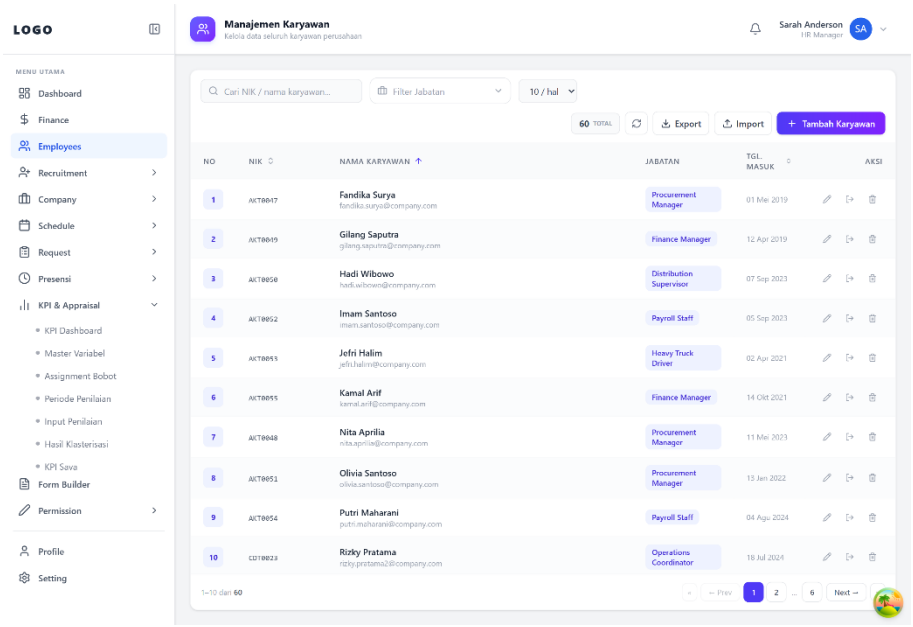
Silhouette SC beserta keterangan kategori kualitas klasterisasi. Baris kedua menampilkan Rata-rata Skor beserta tren perbandingan periode sebelumnya, jumlah karyawan *Standard*, dan jumlah karyawan yang Di-*Skip*.

Di bawah *summary card* terdapat dua panel berdampingan yaitu Periode Aktif dan Distribusi *Cluster*. Panel Periode Aktif menampilkan informasi periode penilaian yang sedang berjalan beserta *progress submit* karyawan dalam bentuk *progress bar* yang menunjukkan jumlah *submitted*, *draft*, dan belum diisi. Panel Distribusi *Cluster* menampilkan proporsi setiap *cluster* dalam bentuk *donut chart* dengan keterangan warna per label *cluster*.

Bagian tengah halaman menampilkan visualisasi tren dalam dua panel. Panel *Trend Cluster* per Periode menampilkan tren jumlah karyawan per *cluster* antar periode dalam bentuk *line chart*. Panel *Trend Silhouette Score* menampilkan tren kualitas klasterisasi antar periode, sedangkan panel Distribusi per Departemen menampilkan sebaran *cluster* per departemen dalam bentuk *stacked bar chart*.

Bagian bawah halaman menampilkan empat panel analisis. Panel *Top Performers* menampilkan lima karyawan dengan skor tertinggi beserta jabatan, departemen, dan skornya. Panel Perlu Perhatian menampilkan karyawan dengan label *Low Performer* beserta skornya sebagai bahan evaluasi HR. Panel Variabel Terbaik dan Variabel Terendah menampilkan analisis *weighted score* per variabel KPI untuk mengidentifikasi variabel dengan kontribusi tertinggi dan terendah terhadap skor akhir karyawan.

3. Halaman Manajemen Karyawan (URS-HR-03)

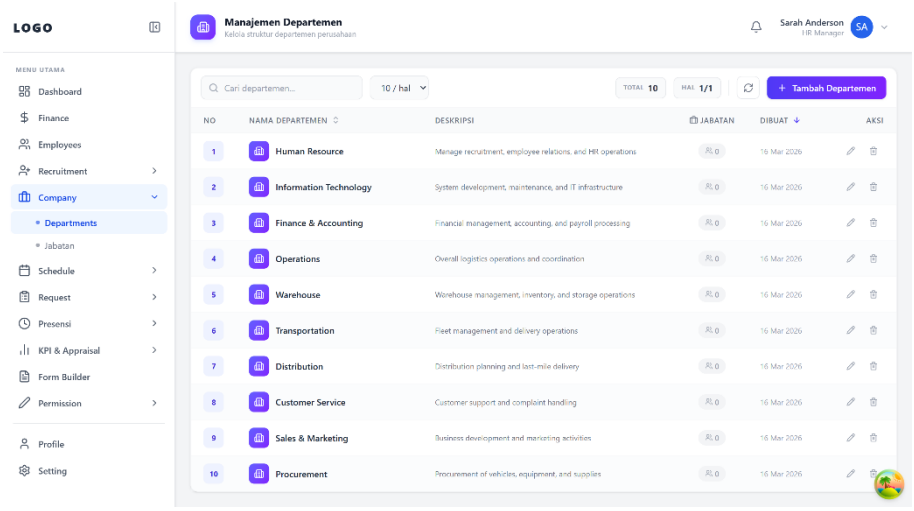


Gambar 11. Halaman Manajemen Karyawan

Halaman Manajemen Karyawan dapat diakses oleh HR untuk mengelola data seluruh karyawan perusahaan. Halaman ini menampilkan daftar karyawan dalam bentuk tabel yang dilengkapi dengan fitur pencarian berdasarkan NIK atau nama karyawan, *filter* berdasarkan jabatan, dan pengaturan jumlah data per halaman.

Tabel karyawan menampilkan kolom NIK, nama karyawan beserta *email*, jabatan yang ditampilkan dalam bentuk *badge* berwarna, tanggal masuk, dan kolom aksi yang berisi tombol *edit*, *reset password*, dan hapus. Bagian atas tabel menampilkan total keseluruhan karyawan beserta tombol *Export* untuk mengunduh data dan tombol *Import* untuk mengunggah data karyawan secara massal, serta tombol Tambah Karyawan untuk menambahkan karyawan baru. Navigasi halaman tersedia di bagian bawah tabel dalam bentuk *pagination*.

4. Halaman Manajemen Departemen (URS-HR-04)

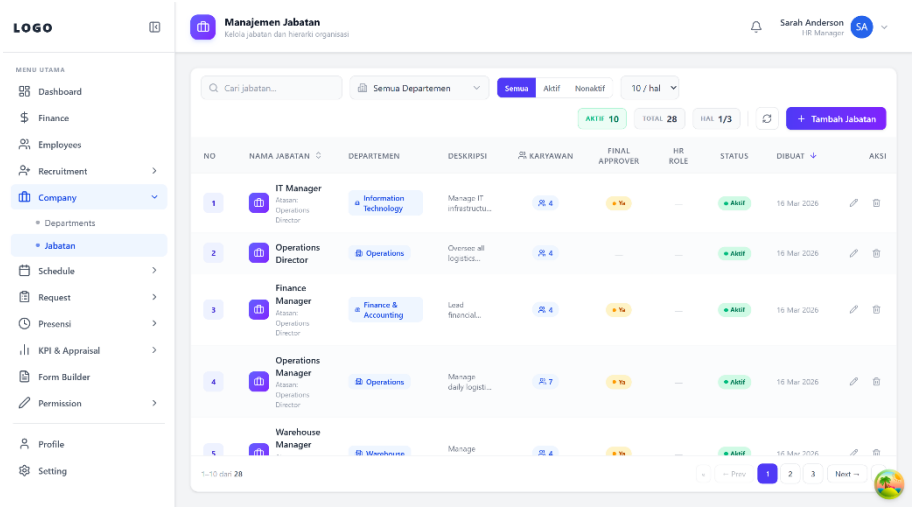


Gambar 12. Halaman Manajemen Departemen

Halaman Manajemen Departemen dapat diakses oleh HR untuk mengelola struktur departemen perusahaan. Halaman ini menampilkan daftar departemen dalam bentuk tabel yang dilengkapi dengan fitur pencarian berdasarkan nama departemen dan pengaturan jumlah data per halaman.

Tabel departemen menampilkan kolom nama departemen beserta ikon identitas, deskripsi singkat fungsi departemen, jumlah jabatan yang terdaftar dalam departemen tersebut, tanggal dibuat, dan kolom aksi yang berisi tombol *edit* dan hapus. Bagian atas tabel menampilkan total keseluruhan departemen beserta informasi halaman saat ini dan tombol Tambah Departemen untuk menambahkan departemen baru.

5. Halaman Manajemen Jabatan (URS-HR-05)

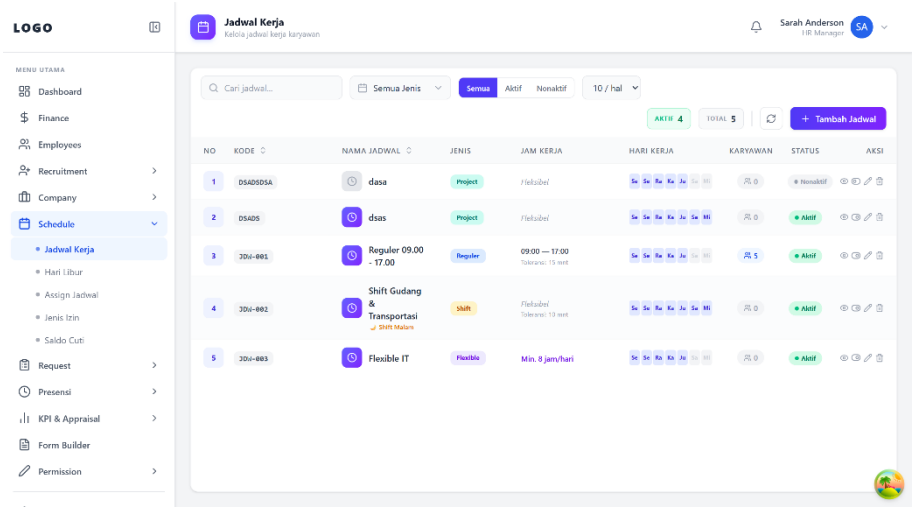


Gambar 13. Halaman Manajemen Jabatan

Halaman Manajemen Jabatan dapat diakses oleh HR untuk mengelola jabatan dan hierarki organisasi perusahaan. Halaman ini menampilkan daftar jabatan dalam bentuk tabel yang dilengkapi dengan fitur pencarian berdasarkan nama jabatan, *filter* berdasarkan departemen, *filter* status aktif atau nonaktif, dan pengaturan jumlah data per halaman.

Tabel jabatan menampilkan kolom nama jabatan beserta informasi atasan langsung, departemen yang ditampilkan dalam bentuk *badge* berwarna, deskripsi singkat, jumlah karyawan yang menduduki jabatan tersebut, status *Final Approver*, status *HR Role*, status aktif atau nonaktif, tanggal dibuat, dan kolom aksi yang berisi tombol *edit* dan hapus. Bagian atas tabel menampilkan jumlah jabatan aktif, total keseluruhan jabatan, informasi halaman saat ini, dan tombol Tambah Jabatan untuk menambahkan jabatan baru.

6. Halaman Jadwal Kerja (URS-HR-06)

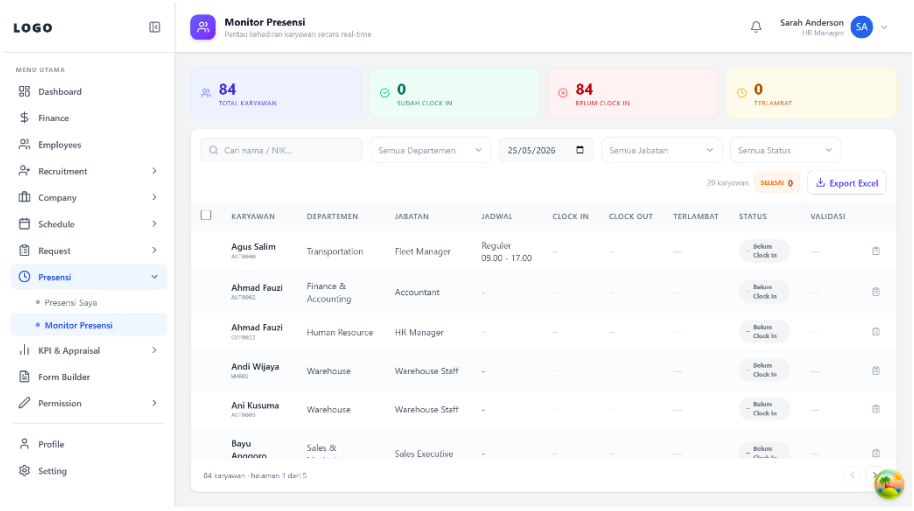


Gambar 14. Halaman Jadwal Kerja

Halaman Jadwal Kerja dapat diakses oleh HR untuk mengelola jadwal kerja karyawan yang menjadi dasar perhitungan presensi. Halaman ini menampilkan daftar jadwal kerja dalam bentuk tabel yang dilengkapi dengan fitur pencarian berdasarkan nama jadwal, *filter* berdasarkan jenis jadwal, *filter* status aktif atau nonaktif, dan pengaturan jumlah data per halaman.

Tabel jadwal kerja menampilkan kolom kode jadwal, nama jadwal beserta ikon jenis, jenis jadwal yang ditampilkan dalam bentuk *badge* berwarna seperti *Reguler*, *Shift*, *Project*, dan *Flexible*, jam kerja beserta toleransi keterlambatan, hari kerja yang ditampilkan dalam bentuk indikator hari aktif dan nonaktif, jumlah karyawan yang menggunakan jadwal tersebut, status aktif atau nonaktif, dan kolom aksi yang berisi tombol lihat detail, *edit*, dan hapus. Bagian atas tabel menampilkan jumlah jadwal aktif, total keseluruhan jadwal, dan tombol Tambah Jadwal untuk menambahkan jadwal kerja baru.

7. Halaman *Monitor* Presensi (URS-HR-07)

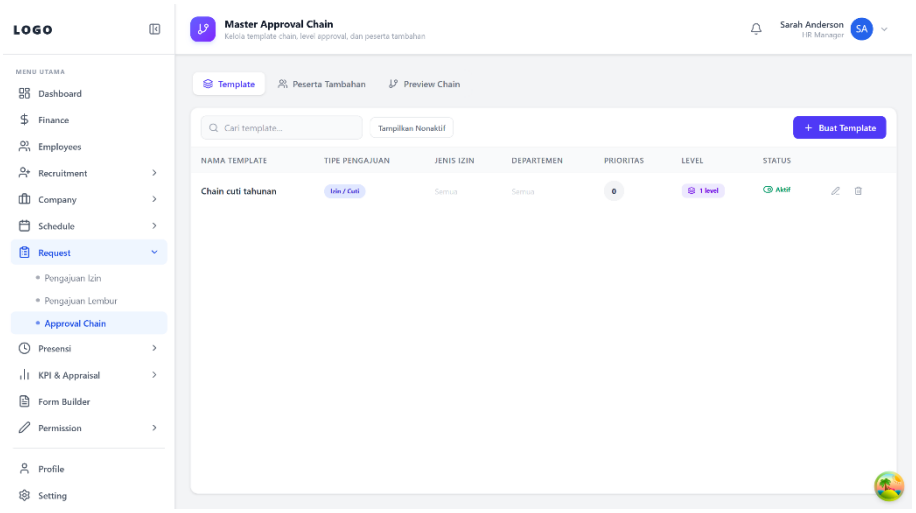


Gambar 15. Halaman *Monitor* Presensi

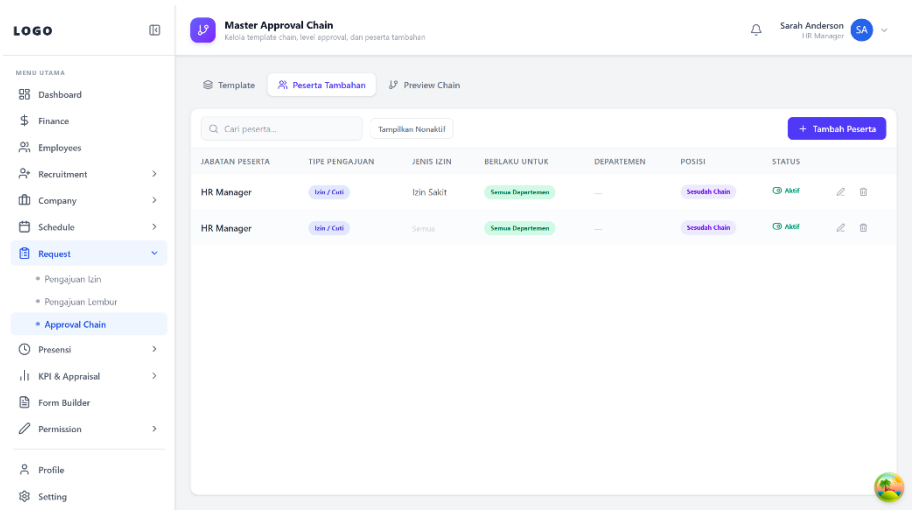
Halaman *Monitor* Presensi dapat diakses oleh HR untuk memantau kehadiran seluruh karyawan secara *real-time*. Bagian atas halaman menampilkan empat *summary card* yang merangkum kondisi kehadiran pada hari yang dipilih yaitu Total Karyawan, Sudah *Clock In*, Belum *Clock In*, dan Terlambat.

Halaman ini dilengkapi dengan fitur pencarian berdasarkan nama atau NIK karyawan, *filter* berdasarkan departemen, tanggal, jabatan, dan status kehadiran. Tabel presensi menampilkan kolom karyawan beserta NIK, departemen, jabatan, jadwal kerja, waktu *clock in*, waktu *clock out*, durasi keterlambatan, status kehadiran, dan kolom validasi. Bagian atas tabel menampilkan jumlah karyawan yang ditampilkan, indikator jumlah data yang selesai divalidasi, dan tombol *Export Excel* untuk mengunduh data presensi.

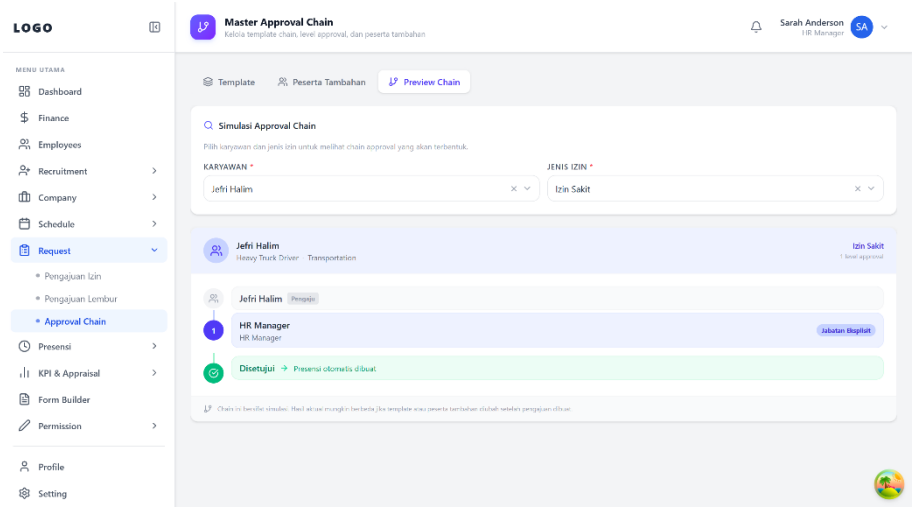
8. Halaman *Master Approval Chain* (URS-HR-08)



Gambar 16. Halaman *Master Approval Chain — Tab Template*



Gambar 17. Halaman *Master Approval Chain — Tab* Peserta Tambahan



Gambar 18. Halaman *Master Approval Chain — Tab Preview Chain*

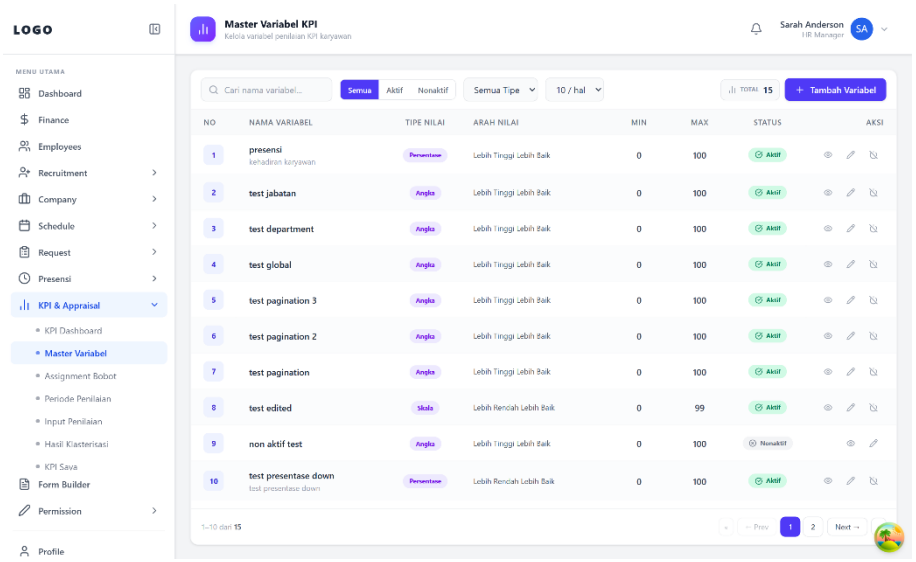
Halaman *Master Approval Chain* dapat diakses oleh HR untuk mengkonfigurasi alur persetujuan pengajuan izin karyawan. Halaman ini terdiri dari tiga *tab* yaitu *Template*, Peserta Tambahan, dan *Preview Chain*.

Tab Template menampilkan daftar *template chain approval* yang telah dikonfigurasi, di mana setiap *template* memiliki informasi nama *template*, tipe pengajuan, jenis izin, departemen yang berlaku, prioritas, jumlah *level approval*, dan status aktif atau nonaktif. HR dapat membuat *template* baru melalui tombol Buat *Template*.

Tab Peserta Tambahan menampilkan daftar jabatan yang ditambahkan sebagai peserta di luar *chain* utama, di mana setiap peserta memiliki informasi jabatan peserta, tipe pengajuan, jenis izin, cakupan departemen yang berlaku, posisi dalam *chain*, dan status aktif atau nonaktif. HR dapat menambahkan peserta baru melalui tombol Tambah Peserta.

Tab Preview Chain menyediakan fitur simulasi *chain approval*, di mana HR dapat memilih karyawan dan jenis izin untuk melihat susunan *chain approval* yang akan terbentuk secara dinamis beserta urutan *level approver* dan hasil akhir setelah disetujui.

9. Halaman Master Variabel KPI (URS-HR-09)

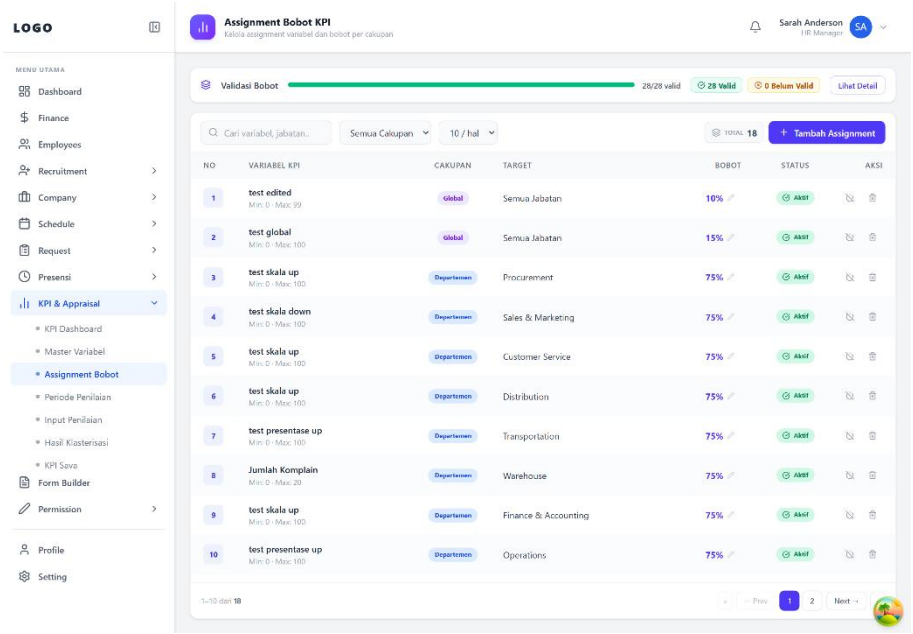


Gambar 19. Halaman Master Variabel KPI

Halaman Master Variabel KPI dapat diakses oleh HR untuk mengelola definisi variabel KPI yang digunakan dalam penilaian kinerja karyawan. Halaman ini menampilkan daftar variabel KPI dalam bentuk tabel yang dilengkapi dengan fitur pencarian berdasarkan nama variabel, *filter* status aktif atau nonaktif, *filter* berdasarkan tipe nilai, dan pengaturan jumlah data per halaman.

Tabel variabel KPI menampilkan kolom nama variabel beserta deskripsi singkat, tipe nilai yang ditampilkan dalam bentuk *badge* berwarna seperti Persentase, Angka, dan Skala, arah nilai yang menunjukkan apakah variabel bersifat lebih tinggi lebih baik atau lebih rendah lebih baik, nilai minimum, nilai maksimum, status aktif atau nonaktif, dan kolom aksi yang berisi tombol lihat detail, *edit*, dan nonaktifkan. Bagian atas tabel menampilkan total keseluruhan variabel dan tombol Tambah Variabel untuk menambahkan variabel KPI baru.

10. Halaman *Assignment* Bobot KPI (URS-HR-10)

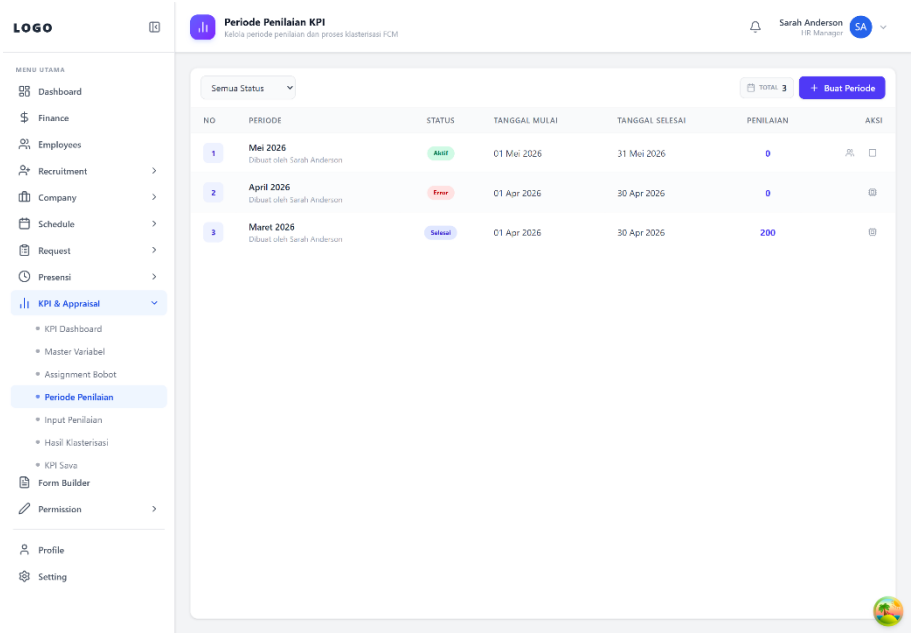


Gambar 20. Halaman *Assignment* Bobot KPI

Halaman *Assignment* Bobot KPI dapat diakses oleh HR untuk mengelola *assignment* variabel KPI ke tiga cakupan yaitu global, departemen, dan jabatan beserta bobotnya masing-masing. Bagian atas halaman menampilkan indikator Validasi Bobot dalam bentuk *progress bar* hijau yang menunjukkan jumlah jabatan yang telah memiliki total bobot valid sebesar 100%, di mana pada contoh ini seluruh 28 jabatan telah valid dan 0 jabatan belum valid. HR dapat melihat detail validasi per jabatan melalui tombol Lihat Detail.

Tabel *assignment* menampilkan daftar *assignment* variabel KPI yang dilengkapi dengan fitur pencarian berdasarkan nama variabel atau jabatan dan *filter* berdasarkan cakupan. Kolom yang ditampilkan meliputi nama variabel KPI beserta rentang nilai minimum dan maksimum, cakupan *assignment* yang ditampilkan dalam bentuk *badge* berwarna seperti *Global* dan Departemen, target cakupan yang menunjukkan jabatan atau departemen yang berlaku, bobot dalam persentase yang dapat di-*edit* langsung, status aktif atau nonaktif, dan kolom aksi yang berisi tombol nonaktifkan dan hapus. Bagian atas tabel menampilkan total keseluruhan *assignment* dan tombol Tambah *Assignment* untuk menambahkan *assignment* baru.

11. Halaman Periode Penilaian KPI (URS-HR-11)

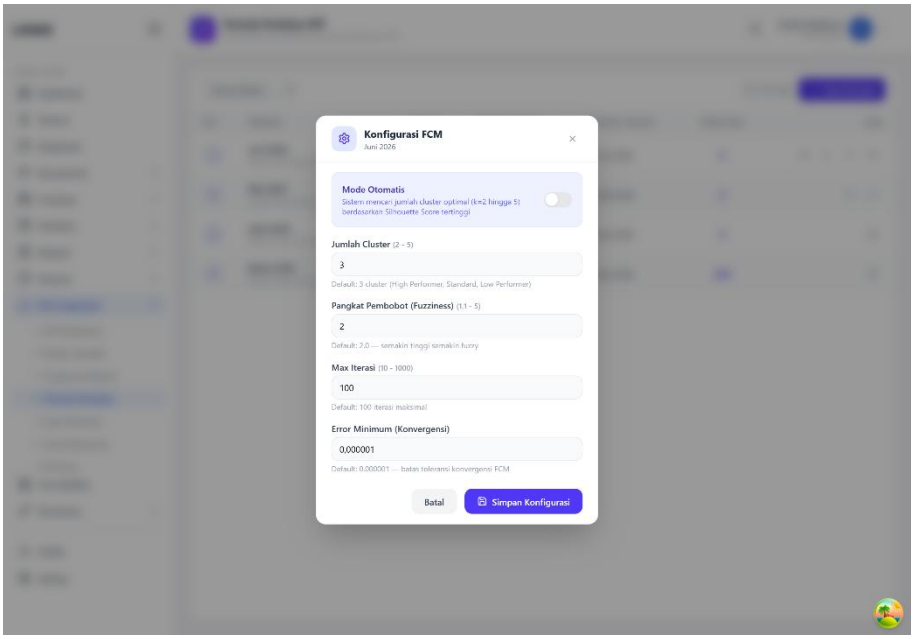


Gambar 21. Halaman Periode Penilaian KPI

Halaman Periode Penilaian KPI dapat diakses oleh HR untuk mengelola periode penilaian dan proses klasterisasi FCM. Halaman ini menampilkan daftar periode penilaian dalam bentuk tabel yang dilengkapi dengan *filter* berdasarkan status periode.

Tabel periode menampilkan kolom nama periode beserta informasi pembuat, status periode yang ditampilkan dalam bentuk *badge* berwarna di mana status Aktif berwarna hijau, *Error* berwarna merah, dan Selesai berwarna ungu, tanggal mulai, tanggal selesai, jumlah penilaian yang telah diinput, dan kolom aksi. Kolom aksi disesuaikan dengan status periode, yaitu periode berstatus Aktif menampilkan tombol *monitor progress input* dan tutup periode, sedangkan periode berstatus Selesai atau *Error* menampilkan tombol konfigurasi FCM untuk menjalankan ulang proses klasterisasi. Bagian atas tabel menampilkan total keseluruhan periode dan tombol Buat Periode untuk membuat periode penilaian baru.

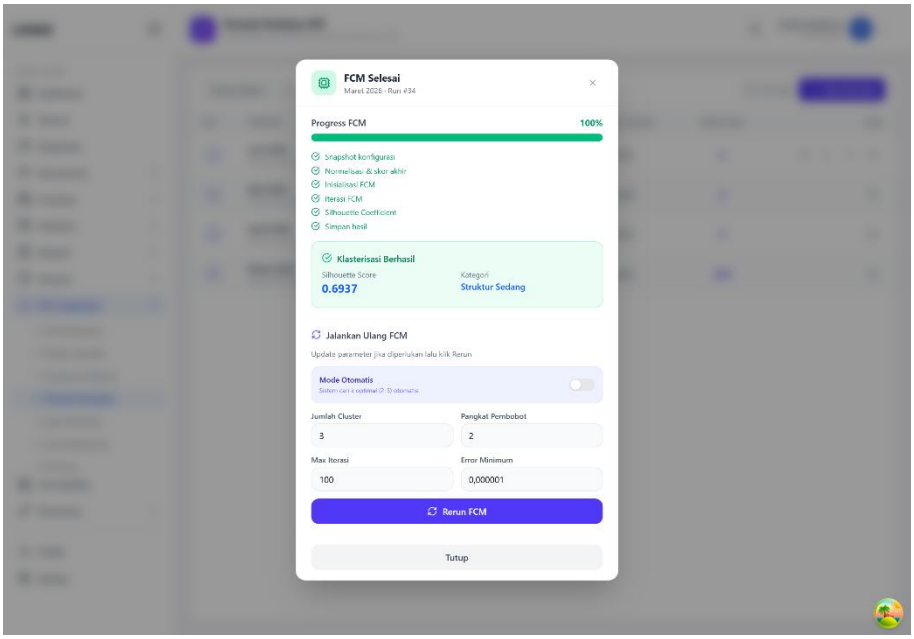
12. Modal Konfigurasi FCM (URS-HR-12)



Gambar 22. *Modal* Konfigurasi FCM

Modal Konfigurasi FCM muncul ketika HR mengklik tombol konfigurasi pada periode yang berstatus *draft*. *Modal* ini menampilkan empat parameter FCM yang dapat dikonfigurasi yaitu Jumlah *Cluster* dengan rentang 2 hingga 5 dan nilai *default* 3 *cluster* yang menghasilkan label *High Performer*, *Standard*, dan *Low Performer*, Pangkat Pembobot (*Fuzziness*) dengan rentang 1.1 hingga 5 dan nilai *default* 2.0, *Max* Iterasi dengan rentang 10 hingga 1000 dan nilai *default* 100 iterasi, serta *Error* Minimum (Konvergensi) dengan nilai *default* 0.000001 sebagai batas toleransi konvergensi FCM. Setiap parameter dilengkapi dengan keterangan nilai *default* untuk memandu HR dalam mengkonfigurasi. HR dapat menyimpan konfigurasi melalui tombol Simpan Konfigurasi atau membatalkan melalui tombol Batal.

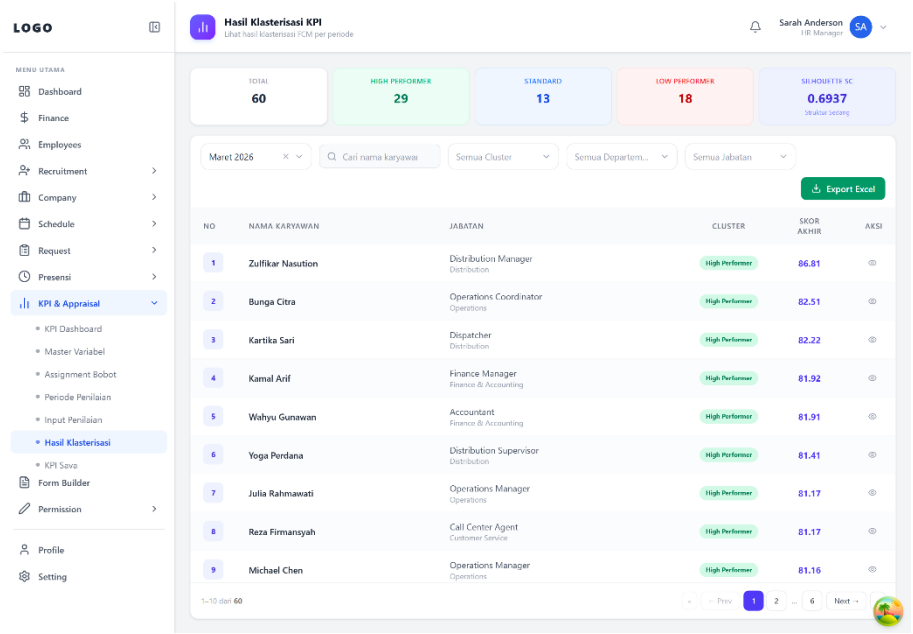
13. Modal FCM Selesai (URS-HR-13)



Gambar 23. *Modal* FCM Selesai

Modal FCM Selesai muncul setelah HR menutup periode dan proses klasterisasi FCM berhasil dijalankan. *Modal* ini menampilkan *progress bar* FCM yang telah mencapai 100% beserta daftar tahapan yang telah selesai dieksekusi yaitu *Snapshot* konfigurasi, Normalisasi dan skor akhir, Inisialisasi FCM, Iterasi FCM, *Silhouette Coefficient*, dan Simpan hasil. Bagian bawah *progress* menampilkan ringkasan hasil klasterisasi berupa nilai *Silhouette Score* dan kategori kualitas klasterisasi. Selain itu *modal* ini juga menyediakan fitur Jalankan Ulang FCM yang memungkinkan HR memperbarui parameter dan menjalankan ulang proses klasterisasi melalui tombol *Rerun* FCM.

14. Halaman Hasil Klasterisasi KPI (URS-HR-14, URS-HR-15)

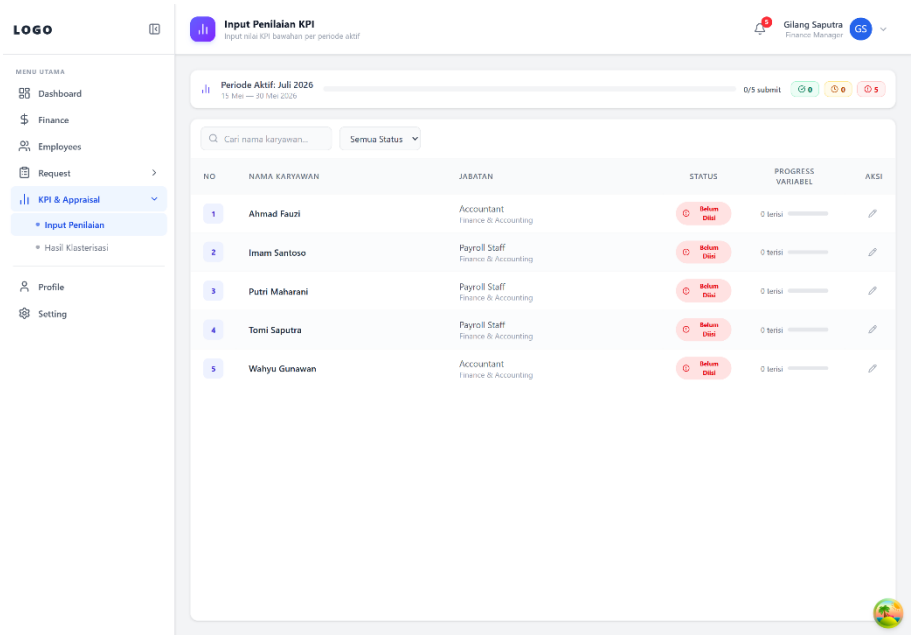


Gambar 24. Halaman Hasil Klasterisasi KPI

Halaman Hasil Klasterisasi KPI dapat diakses oleh HR untuk melihat hasil klasterisasi FCM per periode. Bagian atas halaman menampilkan lima *summary card* yang merangkum hasil klasterisasi yaitu Total karyawan yang dinilai, jumlah *High Performer*, jumlah *Standard*, jumlah *Low Performer*, dan nilai *Silhouette SC* beserta kategori kualitas klasterisasi.

Halaman ini dilengkapi dengan fitur *selector* periode, pencarian berdasarkan nama karyawan, *filter* berdasarkan *cluster*, departemen, dan jabatan, serta tombol *Export Excel* untuk mengunduh data hasil klasterisasi secara keseluruhan. Tabel hasil klasterisasi menampilkan kolom nama karyawan, jabatan beserta departemen, label *cluster* yang ditampilkan dalam bentuk *badge* berwarna, skor akhir dalam skala 0 hingga 100, dan kolom aksi berupa tombol lihat detail untuk melihat rincian nilai per variabel karyawan. Data ditampilkan diurutkan berdasarkan skor akhir tertinggi ke terendah.

15. Halaman *Input* Penilaian KPI (URS-A-04, URS-A-05)

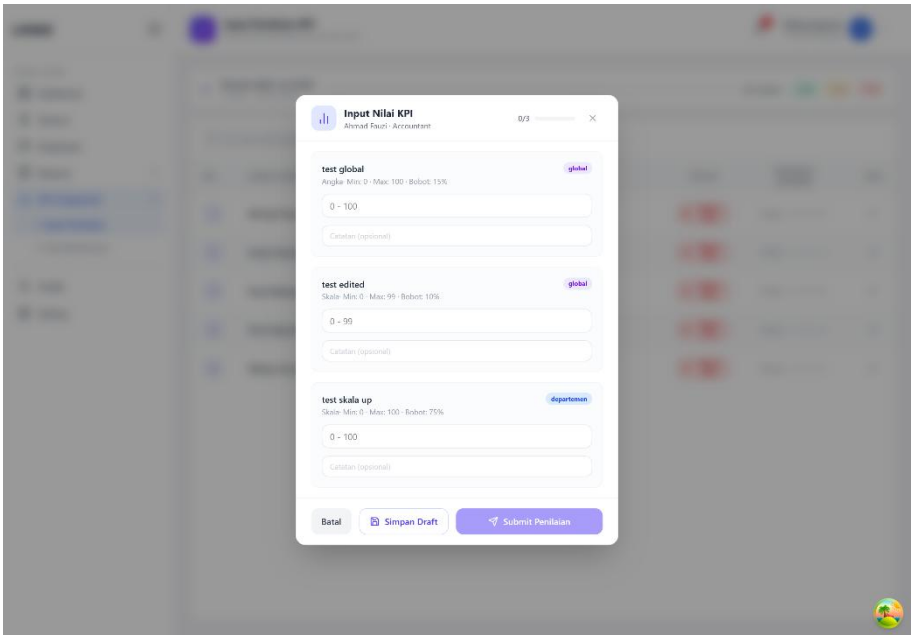


Gambar 25. Halaman *Input* Penilaian KPI

Halaman *Input* Penilaian KPI dapat diakses oleh Atasan untuk menginput nilai KPI bawahan pada periode yang sedang aktif. Bagian atas halaman menampilkan informasi periode aktif beserta rentang tanggal penilaian dan ringkasan *progress submit* yang menunjukkan jumlah bawahan yang sudah *submit*, sedang dalam status *draft*, dan belum diisi.

Tabel bawahan menampilkan daftar karyawan yang menjadi tanggung jawab atasan yang sedang *login*, di mana setiap baris menampilkan nama karyawan, jabatan beserta departemen, status penilaian yang ditampilkan dalam bentuk *badge* berwarna seperti Belum Diisi, *Draft*, dan *Submitted*, *progress* variabel yang menunjukkan jumlah variabel yang telah terisi beserta *progress bar*, dan tombol *edit* untuk membuka *form input* nilai. Halaman ini dilengkapi dengan fitur pencarian berdasarkan nama karyawan dan *filter* berdasarkan status penilaian.

16. *Modal Input* Nilai KPI (URS-A-06, URS-A-07)



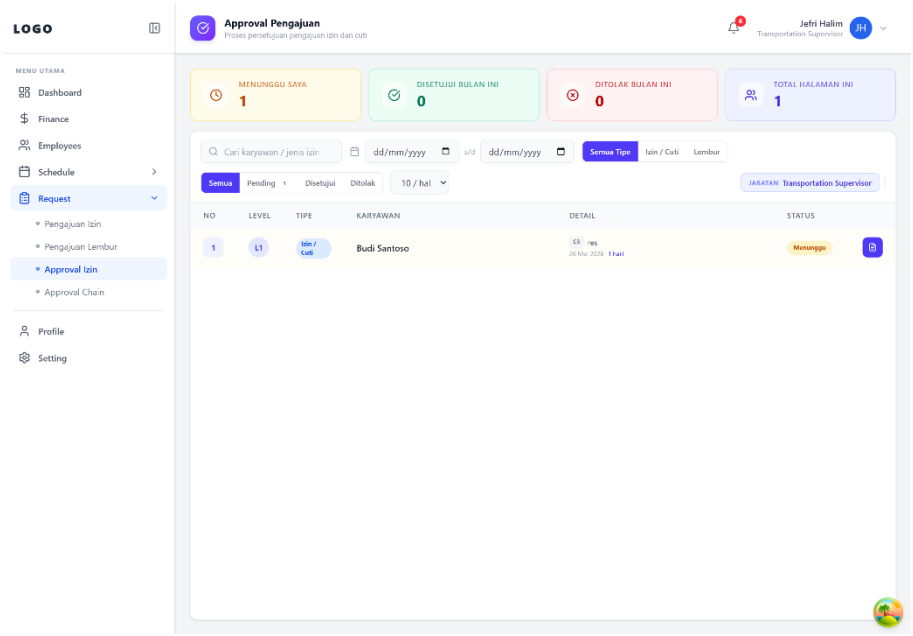
Gambar 26. *Modal Input* Nilai KPI

Modal Input Nilai KPI muncul ketika Atasan mengklik tombol *edit* pada salah satu bawahan di halaman *Input* Penilaian. *Modal* ini menampilkan nama karyawan beserta jabatan yang sedang dinilai dan indikator *progress* pengisian variabel di bagian atas.

Modal ini menampilkan seluruh variabel KPI yang berlaku untuk karyawan tersebut secara dinamis berdasarkan hasil *resolve Strategy Pattern* dan *Decorator Pattern*. Setiap variabel ditampilkan beserta informasi cakupan *assignment* dalam bentuk *badge* berwarna, yaitu *badge* ungu untuk variabel bercakupan global dan *badge* biru muda untuk variabel bercakupan departemen, tipe nilai, rentang nilai minimum hingga maksimum, dan bobot variabel. Atasan dapat mengisi nilai aktual pada *field input* yang telah dibatasi sesuai rentang minimum dan maksimum variabel, serta menambahkan catatan opsional untuk setiap variabel.

Bagian bawah *modal* menyediakan tiga tombol aksi yaitu tombol Batal untuk menutup *modal* tanpa menyimpan, tombol Simpan *Draft* untuk menyimpan nilai sementara tanpa mengunci data sesuai URS-A-06, dan tombol *Submit* Penilaian untuk mengunci seluruh nilai yang telah diisi sesuai URS-A-07.

17. Halaman *Approval* Pengajuan (URS-A-03)

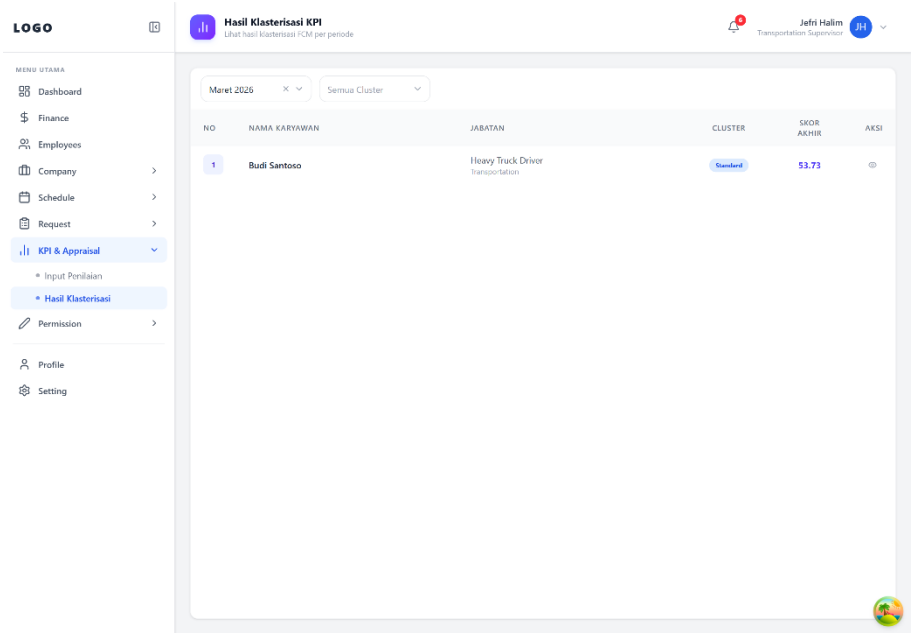


Gambar 27. Halaman *Approval* Pengajuan

Halaman *Approval* Pengajuan dapat diakses oleh Atasan untuk memproses pengajuan izin dan cuti bawahan berdasarkan *chain approval* yang telah dikonfigurasi. Bagian atas halaman menampilkan empat *summary card* yaitu Menunggu Saya yang menunjukkan jumlah pengajuan yang menunggu keputusan atasan yang sedang *login*, Disetujui Bulan Ini, Ditolak Bulan Ini, dan Total Halaman Ini.

Halaman ini dilengkapi dengan fitur pencarian berdasarkan nama karyawan atau jenis izin, *filter* berdasarkan rentang tanggal, *filter* tipe pengajuan yaitu Semua Tipe, Izin/Cuti, dan Lembur, serta *filter* status yaitu Semua, *Pending*, Disetujui, dan Ditolak. Bagian kanan atas tabel menampilkan informasi jabatan Atasan yang sedang *login* sebagai konteks *level approval*. Tabel pengajuan menampilkan kolom *level approval*, tipe pengajuan dalam bentuk *badge* berwarna, nama karyawan pengaju, detail pengajuan beserta tanggal dan durasi, dan status pengajuan. Atasan dapat memproses setiap pengajuan melalui tombol aksi pada kolom kanan tabel.

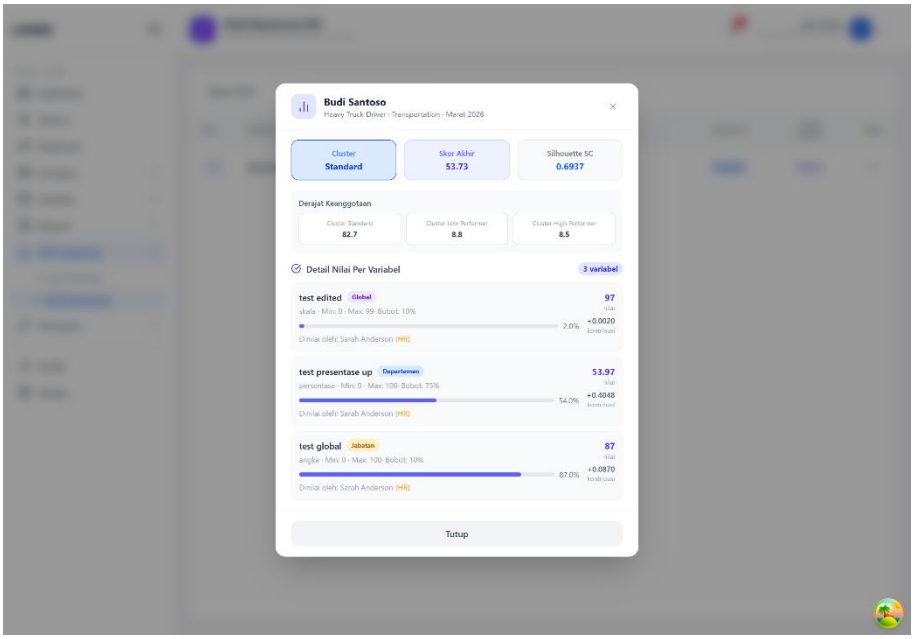
18. Halaman Hasil Klasterisasi KPI — Atasan (URS-A-08)



Gambar 28. Halaman Hasil Klasterisasi KPI Atasan

Halaman Hasil Klasterisasi KPI untuk peran Atasan menampilkan hasil klasterisasi FCM khusus bawahan langsung dari Atasan yang sedang *login*. Berbeda dengan tampilan HR yang menampilkan seluruh karyawan, halaman ini secara otomatis mem-*filter* data berdasarkan relasi atasan-bawahan sehingga Atasan hanya dapat melihat hasil klasterisasi kinerja bawahannya saja. Halaman ini dilengkapi dengan *selector* periode dan *filter* berdasarkan *cluster*. Tabel menampilkan kolom nama karyawan, jabatan beserta departemen, label *cluster*, skor akhir, dan tombol lihat detail.

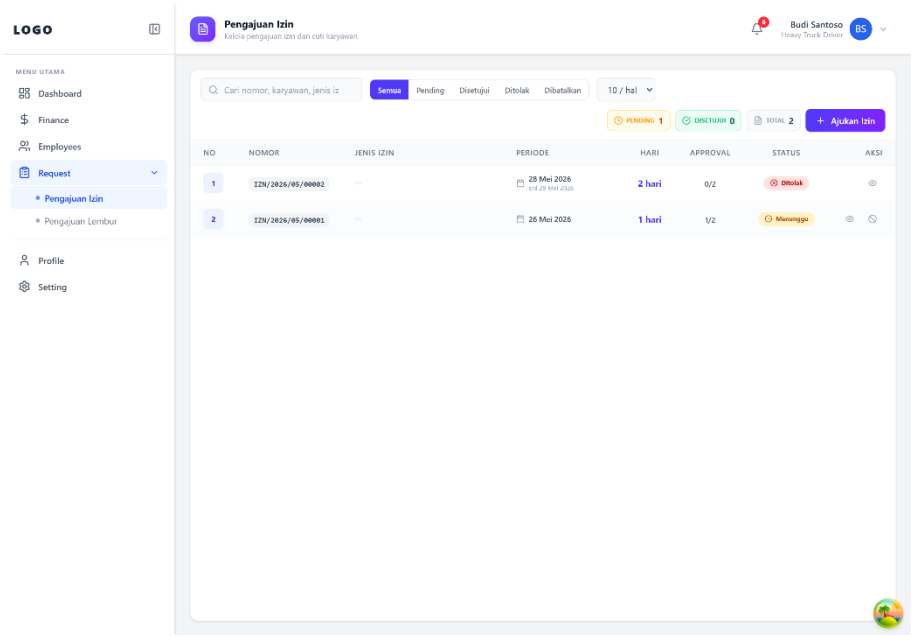
19. *Modal* Detail Nilai KPI (URS-A-08)



Gambar 29. *Modal* Detail Nilai KPI

Modal Detail Nilai KPI dapat diakses oleh Atasan dengan mengklik tombol lihat detail pada salah satu bawahan. *Modal* ini menampilkan nama karyawan, jabatan, departemen, dan periode penilaian di bagian atas. Bagian tengah menampilkan tiga indikator utama yaitu label *cluster*, skor akhir, dan nilai *Silhouette SC*. Di bawahnya terdapat bagian Derajat Keanggotaan yang menampilkan nilai keanggotaan karyawan terhadap setiap *cluster* dalam persentase, di mana hal ini merupakan keunggulan FCM sebagai *soft clustering*. Bagian bawah menampilkan Detail Nilai Per Variabel yang merinci setiap variabel KPI beserta cakupan *assignment* dalam bentuk *badge* berwarna, nilai aktual, persentase normalisasi, dan kontribusi skor beserta nama penilai.

20. Halaman Pengajuan Izin (URS-K-03)

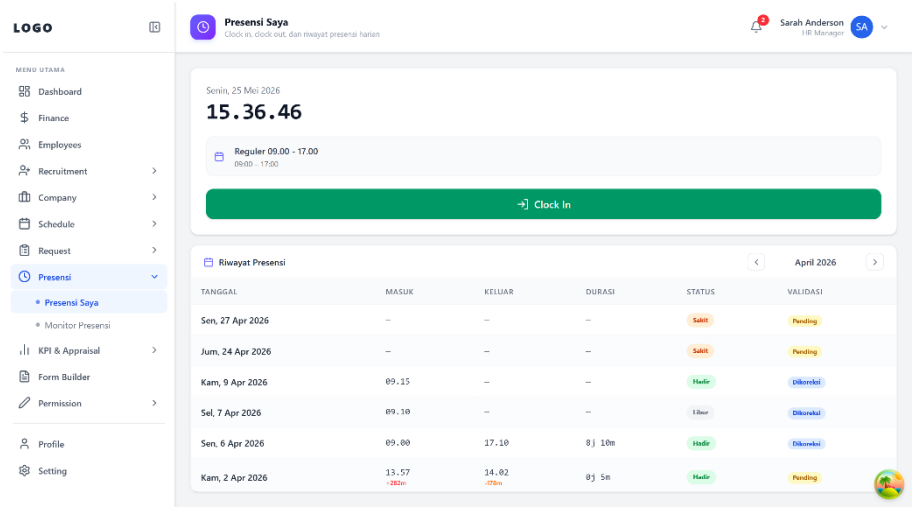


Gambar 30. Halaman Pengajuan Izin

Halaman Pengajuan Izin dapat diakses oleh Karyawan untuk mengajukan dan memantau status pengajuan izin. Halaman ini menampilkan riwayat pengajuan izin karyawan yang sedang *login* dalam bentuk tabel yang dilengkapi dengan fitur pencarian berdasarkan nomor, karyawan, atau jenis izin, serta *filter* status yaitu Semua, *Pending*, Disetujui, Ditolak, dan Dibatalkan.

Bagian atas tabel menampilkan ringkasan jumlah pengajuan berstatus *Pending* dan Disetujui beserta total keseluruhan pengajuan, serta tombol Ajukan Izin untuk membuat pengajuan baru. Tabel pengajuan menampilkan kolom nomor pengajuan, jenis izin, periode izin beserta tanggal mulai dan selesai, jumlah hari, *progress approval* yang menunjukkan *level* yang sudah menyetujui dari total *level* yang diperlukan, status pengajuan yang ditampilkan dalam bentuk *badge* berwarna seperti Ditolak berwarna merah dan Menunggu berwarna kuning, serta kolom aksi berupa tombol lihat detail dan batalkan pengajuan.

21. Halaman Presensi Saya (URS-K-04)

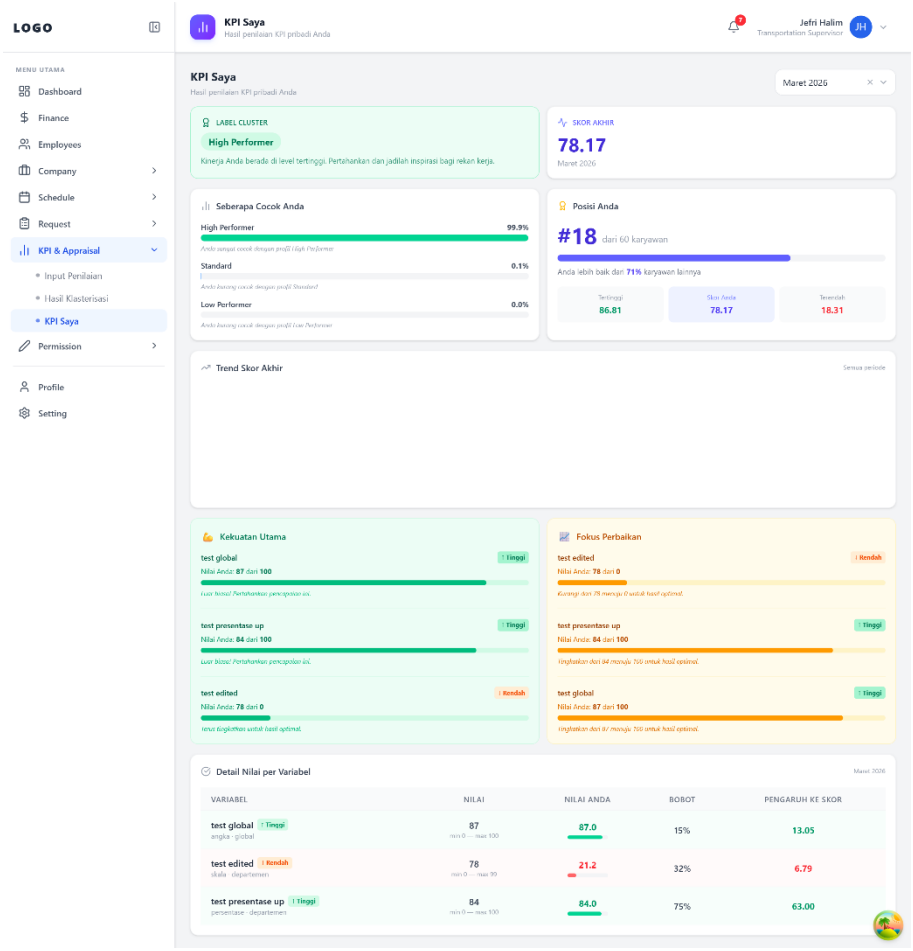


Gambar 31. Halaman Presensi Saya

Halaman Presensi Saya dapat diakses oleh Karyawan untuk melakukan *clock in*, *clock out*, dan memantau riwayat presensi harian. Bagian atas halaman menampilkan tanggal dan jam saat ini secara *real-time* beserta informasi jadwal kerja yang berlaku pada hari tersebut. Karyawan dapat melakukan *clock in* melalui tombol *Clock In* berwarna hijau yang mencolok, di mana tombol ini akan berubah menjadi tombol *Clock Out* setelah karyawan berhasil melakukan *clock in*.

Bagian bawah halaman menampilkan riwayat presensi karyawan yang dikelompokkan per bulan dengan navigasi panah untuk berpindah antar bulan. Tabel riwayat presensi menampilkan kolom tanggal, waktu masuk, waktu keluar, durasi kerja, status kehadiran yang ditampilkan dalam bentuk *badge* berwarna seperti Hadir, Sakit, dan Libur, serta status validasi yang menunjukkan apakah data presensi sudah divalidasi, sedang *pending*, atau telah dikoreksi oleh HR.

22. Halaman KPI Saya (URS-K-05, URS-K-06)



Gambar 32. Halaman KPI Saya

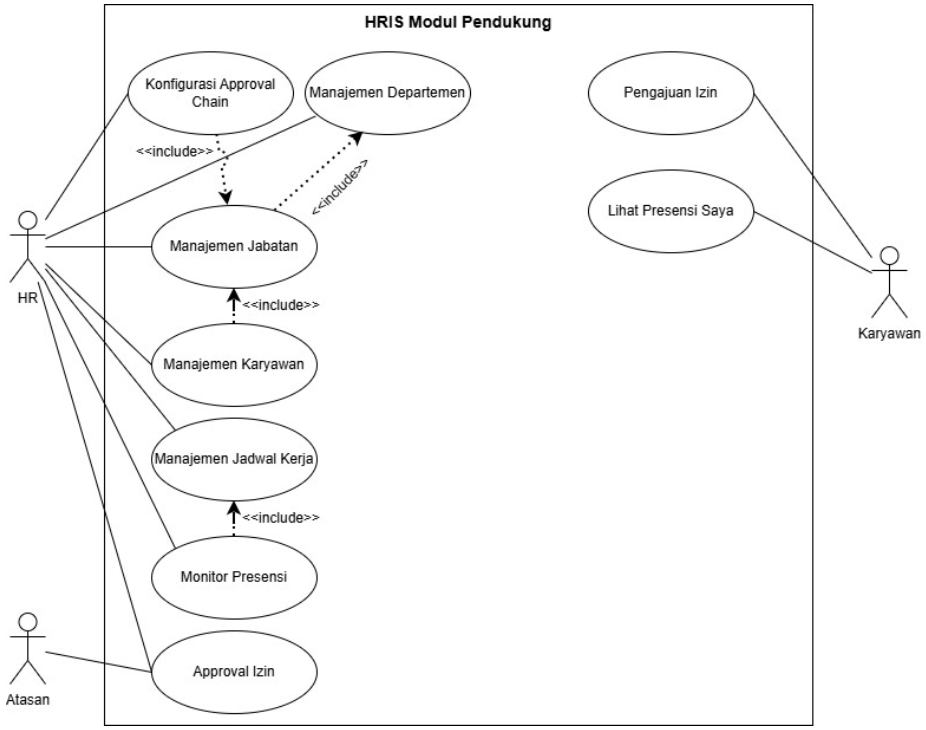
Halaman KPI Saya dapat diakses oleh Karyawan untuk melihat hasil penilaian KPI pribadi per periode, di mana bagian atas halaman terdapat *selector* periode yang memungkinkan karyawan memilih periode yang ingin ditampilkan. Halaman ini menampilkan Label *Cluster* yang menunjukkan hasil klasterisasi karyawan beserta pesan motivasi dan Skor Akhir yang menampilkan nilai skor karyawan pada periode tersebut. Panel Seberapa Cocok Anda menampilkan derajat keanggotaan karyawan terhadap setiap *cluster* dalam bentuk *progress bar* yang merupakan visualisasi konsep *soft clustering* FCM di mana setiap karyawan memiliki derajat keanggotaan terhadap semua *cluster* sekaligus, sedangkan Panel Posisi Anda menampilkan peringkat karyawan di antara seluruh karyawan beserta persentil dan perbandingan skor tertinggi, skor karyawan, dan skor terendah.

Bagian bawah halaman menampilkan *Trend* Skor Akhir yang menggambarkan perkembangan skor karyawan antar periode, panel Kekuatan Utama yang menunjukkan variabel KPI dengan performa terbaik beserta saran mempertahankan pencapaian, dan panel Fokus Perbaikan yang menunjukkan variabel yang perlu ditingkatkan beserta saran perbaikan. Bagian paling bawah menampilkan Detail Nilai per Variabel yang merinci setiap variabel KPI beserta nilai aktual, nilai karyawan yang dinormalisasi, bobot, dan pengaruh ke skor akhir.

4.2.6 Use Case Diagram

Use case diagram digunakan untuk menggambarkan hubungan antara aktor dengan fungsionalitas sistem berdasarkan kebutuhan fungsional yang telah diidentifikasi pada sub-bab 4.1.1. Sistem ini melibatkan tiga aktor yaitu HR, Atasan, dan Karyawan. Perancangan *use case diagram* dibagi menjadi dua diagram berdasarkan kelompok fungsionalitas sistem.

Diagram pertama menggambarkan *use case modul* pendukung sistem HRIS sebagaimana ditampilkan pada Gambar 33 berikut.

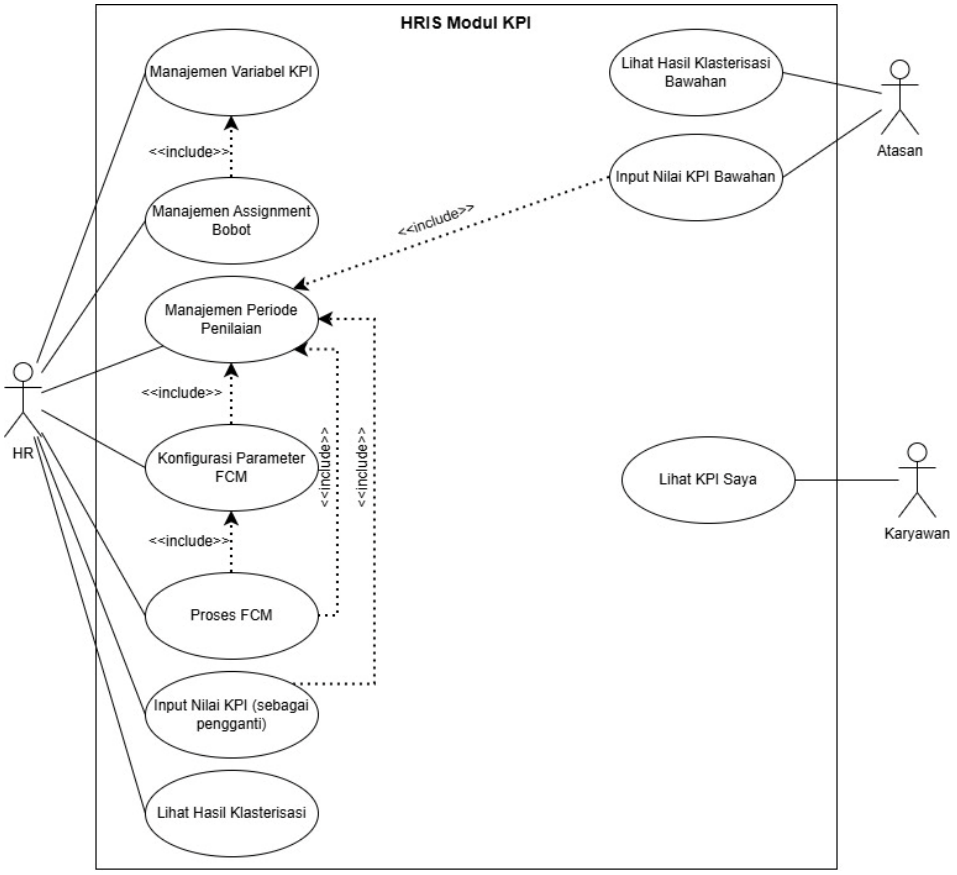


Gambar 33. *Use Case Diagram* HRIS Modul Pendukung

Diagram pertama mencakup fungsionalitas pengelolaan data master dan operasional harian sistem. HR dapat mengelola data karyawan, departemen,

jabatan, jadwal kerja, memantau presensi, mengkonfigurasi *approval chain*, dan memproses *approval* izin. Atasan dapat memproses *approval* izin bawahan. Karyawan dapat mengajukan izin dan memantau presensi pribadinya. Beberapa *use case* memiliki ketergantungan satu sama lain, di mana Manajemen Karyawan membutuhkan Manajemen Jabatan, Manajemen Jabatan membutuhkan Manajemen Departemen, *Monitor* Presensi membutuhkan Manajemen Jadwal Kerja, dan Konfigurasi *Approval Chain* membutuhkan Manajemen Jabatan.

Diagram kedua menggambarkan *use case* modul KPI yang menjadi fokus utama penelitian ini sebagaimana ditampilkan pada Gambar 34 berikut.



Gambar 34. *Use Case Diagram* HRIS Modul KPI

Diagram kedua mencakup fungsionalitas penilaian kinerja dan klasterisasi FCM. HR dapat mengelola variabel KPI, *assignment* bobot, periode penilaian, mengkonfigurasi parameter FCM, menjalankan proses FCM, melihat hasil klasterisasi, dan menginput nilai KPI sebagai pengganti

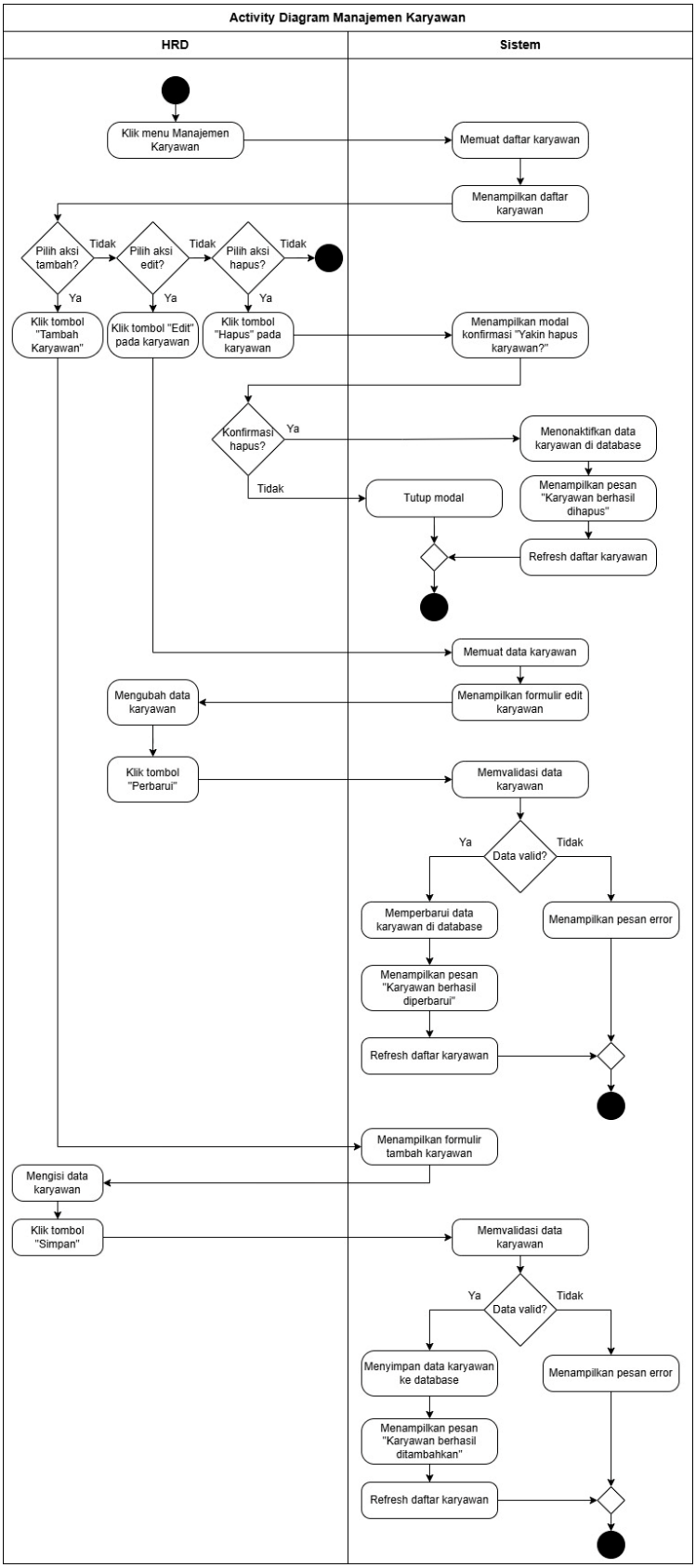
atasan jika diperlukan. Atasan dapat menginput nilai KPI bawahan dan melihat hasil klasterisasi bawahan. Karyawan dapat melihat hasil klasterisasi kinerja dirinya sendiri melalui halaman KPI Saya. Beberapa *use case* memiliki ketergantungan satu sama lain, di mana Manajemen *Assignment* Bobot membutuhkan Manajemen Variabel KPI, Konfigurasi Parameter FCM dan Proses FCM membutuhkan Manajemen Periode Penilaian, serta Proses FCM juga membutuhkan Konfigurasi Parameter FCM. *Input* Nilai KPI sebagai pengganti merupakan perluasan opsional dari *Input* Nilai KPI Bawahan yang hanya dilakukan HR dalam kondisi tertentu.

Di luar ketiga aktor yang tergambar pada use case diagram, terdapat pihak Staff IT yang bertindak sebagai pekerja lepas dengan tanggung jawab penuh atas keseluruhan aplikasi HRIS. Staff IT bertanggung jawab pada aspek teknis sistem, meliputi deployment, konfigurasi server, pemeliharaan aplikasi, penanganan permasalahan teknis, serta pengelolaan akses awal pengguna ke sistem. Peran ini tidak digambarkan dalam use case diagram karena aktivitasnya berada di luar interaksi fungsional bisnis, melainkan pada tingkat operasional teknis yang menopang berjalannya sistem secara keseluruhan.

4.2.7 Activity Diagram

Activity diagram digunakan untuk menggambarkan alur aktivitas dari setiap fungsionalitas sistem yang telah diidentifikasi pada *use case diagram* di sub-bab 4.2.6. Diagram ini memperlihatkan urutan aktivitas yang dilakukan oleh aktor dan respons yang diberikan oleh sistem, sehingga interaksi antara pengguna dan sistem dapat dipahami secara terperinci. Setiap *activity diagram* disajikan dalam bentuk dua kolom yang memisahkan tanggung jawab antara aktor dan sistem, di mana aktor merepresentasikan pengguna yang melakukan aksi dan sistem merepresentasikan respons yang diberikan terhadap aksi tersebut. Perancangan *activity diagram* dikelompokkan berdasarkan pembagian modul pada *use case diagram*, yaitu *activity diagram* untuk modul pendukung sistem HRIS dan *activity diagram* untuk modul KPI.

1. *Activity Diagram* Manajemen Karyawan



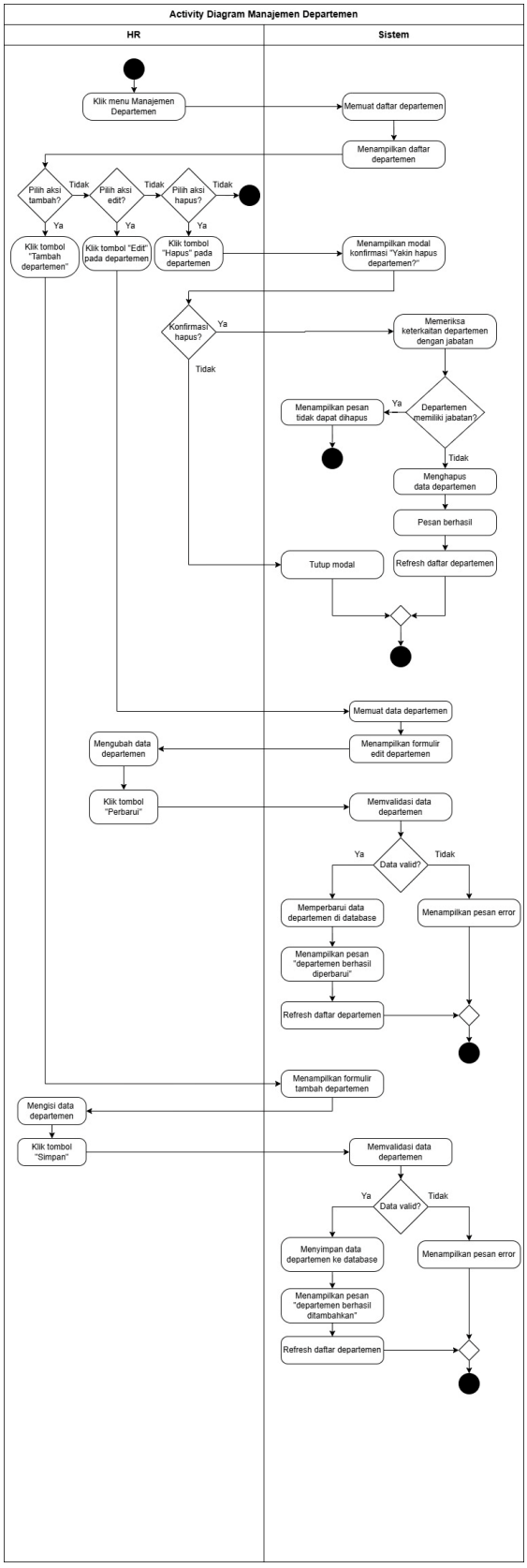
Gambar 35. *Activity Diagram* Manajemen Karyawan

Activity Diagram Manajemen Karyawan menggambarkan alur aktivitas pengelolaan data karyawan oleh aktor HR yang mencakup tiga operasi utama, yaitu penambahan, pembaruan, dan penghapusan data karyawan. Alur dimulai ketika HR mengakses menu Manajemen Karyawan dan sistem menampilkan daftar karyawan, kemudian HR dapat memilih salah satu dari tiga aksi pengelolaan, di mana apabila tidak ada aksi yang dipilih, alur berakhir tanpa operasi apa pun.

Pada aksi penambahan, HR menekan tombol Tambah Karyawan sehingga sistem menampilkan formulir tambah karyawan, kemudian HR mengisi data dan menekan tombol Simpan. Pada aksi pembaruan, HR menekan tombol *Edit* pada baris karyawan yang ingin diubah sehingga sistem memuat dan menampilkan formulir *edit* yang telah terisi, kemudian HR mengubah data dan menekan tombol Perbarui. Pada kedua aksi tersebut, sistem akan memvalidasi data yang dikirimkan, di mana jika valid sistem akan menyimpan atau memperbarui data ke basis data, menampilkan pesan keberhasilan, dan memperbarui daftar karyawan, sedangkan apabila tidak valid sistem menampilkan pesan kesalahan dan alur berakhir.

Pada aksi penghapusan, HR menekan tombol Hapus pada baris karyawan yang ingin dihapus sehingga sistem menampilkan *modal* konfirmasi. Apabila HR membatalkan penghapusan, sistem menutup *modal* tanpa perubahan pada data, sedangkan apabila HR mengonfirmasi penghapusan, sistem menonaktifkan data karyawan di basis data, menampilkan pesan keberhasilan, dan memperbarui daftar karyawan.

2. *Activity Diagram* Manajemen Departemen



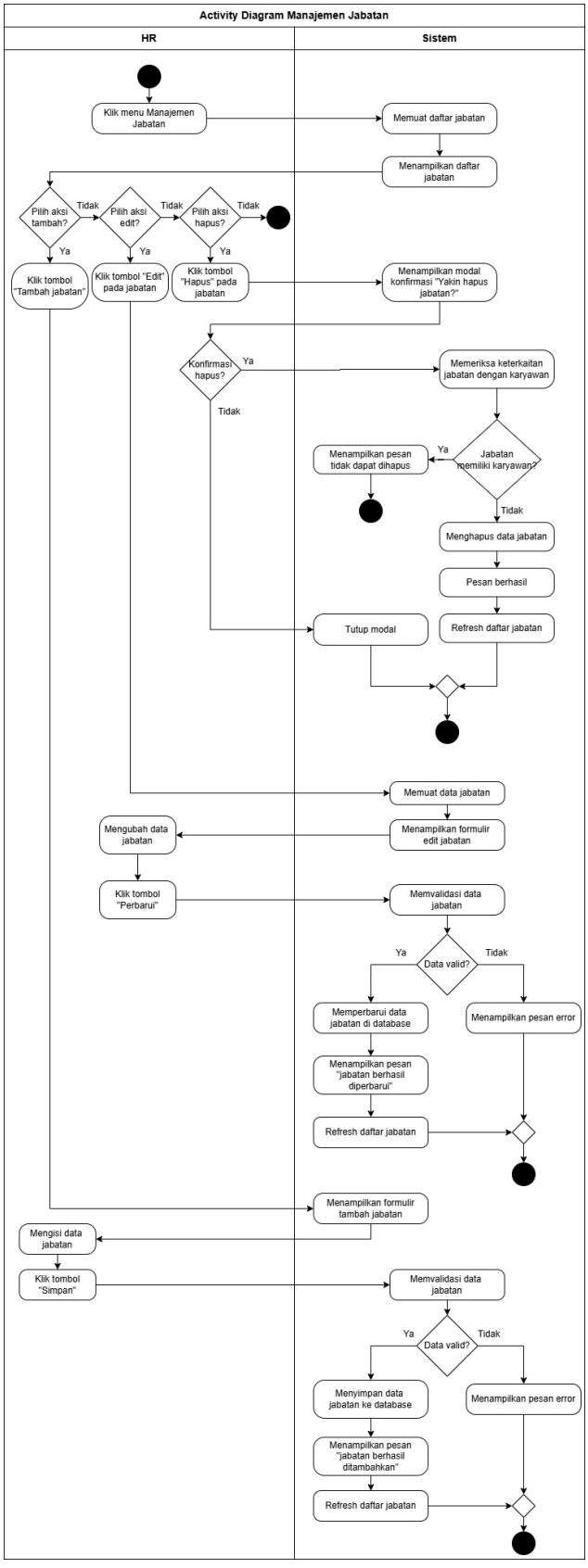
Gambar 36. *Activity Diagram* Manajemen Departemen

Activity Diagram Manajemen Departemen menggambarkan alur aktivitas pengelolaan data departemen oleh aktor HR. Alur dimulai ketika HR mengakses menu Manajemen Departemen dan sistem menampilkan daftar departemen yang tersedia, kemudian HR dapat memilih salah satu dari tiga aksi pengelolaan.

Pada aksi penambahan, HR menekan tombol Tambah Departemen sehingga sistem menampilkan formulir tambah departemen, kemudian HR mengisi data dan menekan tombol Simpan. Pada aksi pembaruan, HR menekan tombol *Edit* pada baris departemen yang ingin diubah sehingga sistem memuat dan menampilkan formulir *edit* yang telah terisi, kemudian HR mengubah data dan menekan tombol Perbarui. Pada kedua aksi tersebut, sistem akan memvalidasi data yang dikirimkan, di mana jika valid sistem akan menyimpan atau memperbarui data, menampilkan pesan keberhasilan, dan memperbarui daftar departemen, sedangkan apabila tidak valid sistem menampilkan pesan kesalahan dan alur berakhir.

Pada aksi penghapusan, HR menekan tombol Hapus pada baris departemen yang ingin dihapus sehingga sistem menampilkan *modal* konfirmasi. Apabila HR membatalkan penghapusan, sistem menutup *modal* tanpa perubahan pada data, sedangkan apabila HR mengonfirmasi penghapusan, sistem memeriksa keterkaitan departemen dengan jabatan untuk menjaga integritas data agar tidak terjadi referensi yang menggantung. Jika departemen masih memiliki jabatan terkait, sistem menampilkan pesan bahwa departemen tidak dapat dihapus dan alur berakhir, sedangkan apabila tidak memiliki jabatan terkait, sistem menghapus data, menampilkan pesan keberhasilan, dan memperbarui daftar departemen.

3. *Activity Diagram* Manajemen Jabatan



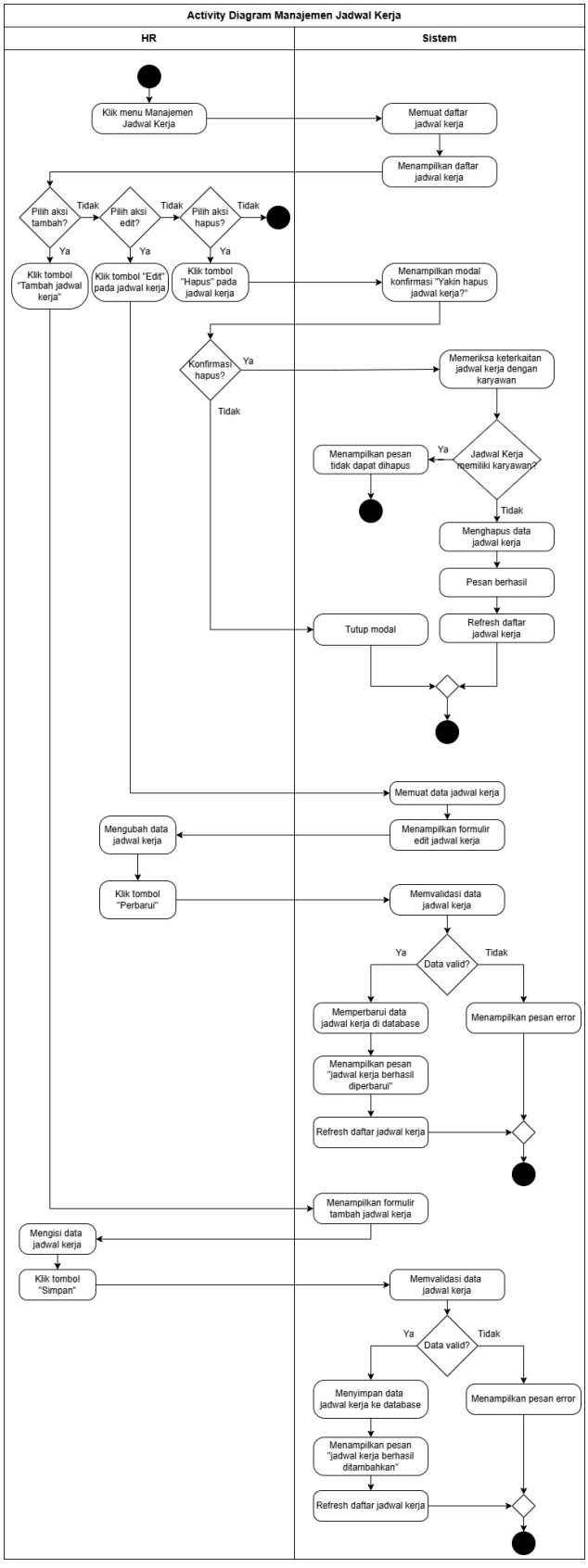
Gambar 37. *Activity Diagram* Manajemen Jabatan

Activity Diagram Manajemen Jabatan menggambarkan alur aktivitas pengelolaan data jabatan oleh aktor HR. Alur dimulai ketika HR mengakses menu Manajemen Jabatan dan sistem menampilkan daftar jabatan yang tersedia, kemudian HR dapat memilih salah satu dari tiga aksi pengelolaan.

Pada aksi penambahan, HR menekan tombol Tambah Jabatan sehingga sistem menampilkan formulir tambah jabatan, kemudian HR mengisi data dan menekan tombol Simpan. Pada aksi pembaruan, HR menekan tombol *Edit* pada baris jabatan yang ingin diubah sehingga sistem memuat dan menampilkan formulir *edit* yang telah terisi, kemudian HR mengubah data dan menekan tombol Perbarui. Pada kedua aksi tersebut, sistem akan memvalidasi data yang dikirimkan, di mana jika valid sistem akan menyimpan atau memperbarui data ke basis data, menampilkan pesan keberhasilan, dan memperbarui daftar jabatan, sedangkan apabila tidak valid sistem menampilkan pesan kesalahan dan alur berakhir.

Pada aksi penghapusan, HR menekan tombol Hapus pada baris jabatan yang ingin dihapus sehingga sistem menampilkan *modal* konfirmasi. Apabila HR membatalkan penghapusan, sistem menutup *modal* tanpa perubahan pada data, sedangkan apabila HR mengonfirmasi penghapusan, sistem memeriksa keterkaitan jabatan dengan karyawan untuk menjaga integritas data agar tidak terjadi referensi yang menggantung. Jika jabatan masih memiliki karyawan terkait, sistem menampilkan pesan bahwa jabatan tidak dapat dihapus dan alur berakhir, sedangkan apabila tidak memiliki karyawan terkait, sistem menghapus data, menampilkan pesan keberhasilan, dan memperbarui daftar jabatan.

4. *Activity Diagram* Manajemen Jadwal Kerja



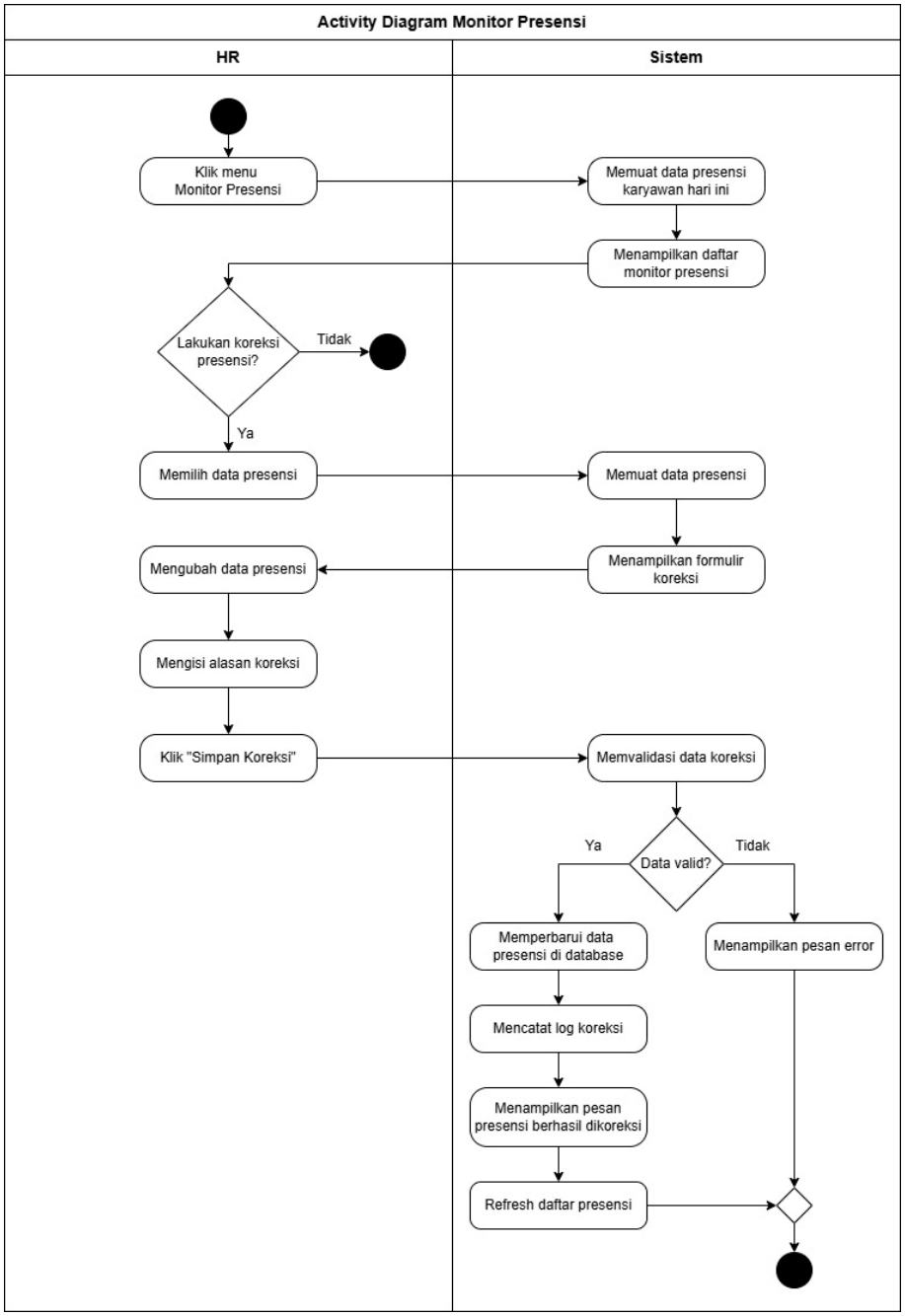
Gambar 38. *Activity Diagram* Manajemen Jadwal Kerja

Activity Diagram Manajemen Jadwal Kerja menggambarkan alur aktivitas pengelolaan data jadwal kerja oleh aktor HR. Alur dimulai ketika HR mengakses menu Manajemen Jadwal Kerja dan sistem menampilkan daftar jadwal kerja yang tersedia, kemudian HR dapat memilih salah satu dari tiga aksi pengelolaan.

Pada aksi penambahan, HR menekan tombol Tambah Jadwal Kerja sehingga sistem menampilkan formulir tambah jadwal kerja, kemudian HR mengisi data dan menekan tombol Simpan. Pada aksi pembaruan, HR menekan tombol *Edit* pada baris jadwal kerja yang ingin diubah sehingga sistem memuat dan menampilkan formulir *edit* yang telah terisi, kemudian HR mengubah data dan menekan tombol Perbarui. Pada kedua aksi tersebut, sistem akan memvalidasi data yang dikirimkan, di mana jika valid sistem akan menyimpan atau memperbarui data ke basis data, menampilkan pesan keberhasilan, dan memperbarui daftar jadwal kerja, sedangkan apabila tidak valid sistem menampilkan pesan kesalahan dan alur berakhir.

Pada aksi penghapusan, HR menekan tombol Hapus pada baris jadwal kerja yang ingin dihapus sehingga sistem menampilkan *modal* konfirmasi. Apabila HR membatalkan penghapusan, sistem menutup *modal* tanpa perubahan pada data, sedangkan apabila HR mengonfirmasi penghapusan, sistem memeriksa keterkaitan jadwal kerja dengan karyawan untuk menjaga integritas data agar tidak terjadi referensi yang menggantung. Jika jadwal kerja masih digunakan karyawan, sistem menampilkan pesan bahwa jadwal kerja tidak dapat dihapus dan alur berakhir, sedangkan apabila tidak digunakan karyawan, sistem menghapus data, menampilkan pesan keberhasilan, dan memperbarui daftar jadwal kerja.

5. *Activity Diagram Monitor* Presensi



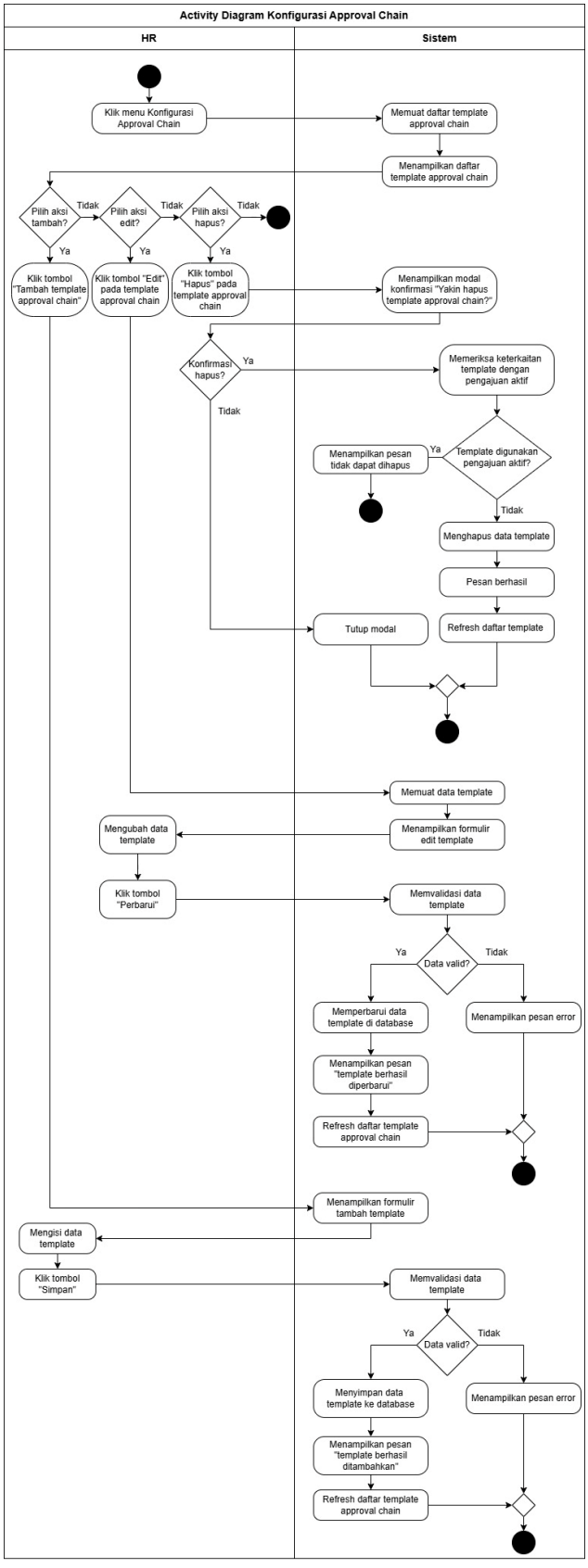
Gambar 39. *Activity Diagram Monitor* Presensi

Activity Diagram Monitor Presensi menggambarkan alur aktivitas pemantauan dan koreksi data presensi karyawan yang dilakukan oleh aktor HR. Alur aktivitas dimulai ketika HR mengakses menu *Monitor* Presensi, kemudian sistem memuat dan menampilkan daftar *monitor* presensi karyawan hari ini secara *real-time* yang mencakup status kehadiran, waktu *clock in*, waktu *clock out*, dan keterlambatan. HR

selanjutnya dapat memilih untuk melakukan koreksi data presensi atau keluar dari halaman tanpa melakukan aksi apa pun.

Pada aksi koreksi presensi, HR memilih data presensi karyawan yang ingin dikoreksi, kemudian sistem memuat data presensi dan menampilkan formulir koreksi. HR mengubah data presensi sesuai kebutuhan dan mengisi alasan koreksi sebagai dasar perubahan, lalu menekan tombol Simpan Koreksi. Sistem memvalidasi data koreksi yang dikirimkan, di mana proses pembaruan hanya dilakukan apabila data dinyatakan valid. Jika data valid, sistem memperbarui data presensi di basis data, mencatat *log* koreksi sebagai jejak audit untuk menjaga akuntabilitas perubahan data, menampilkan pesan keberhasilan, dan memperbarui daftar *monitor* presensi. Sebaliknya, apabila data tidak valid, sistem menampilkan pesan kesalahan dan alur aktivitas berakhir.

6. *Activity Diagram* Konfigurasi *Approval Chain*



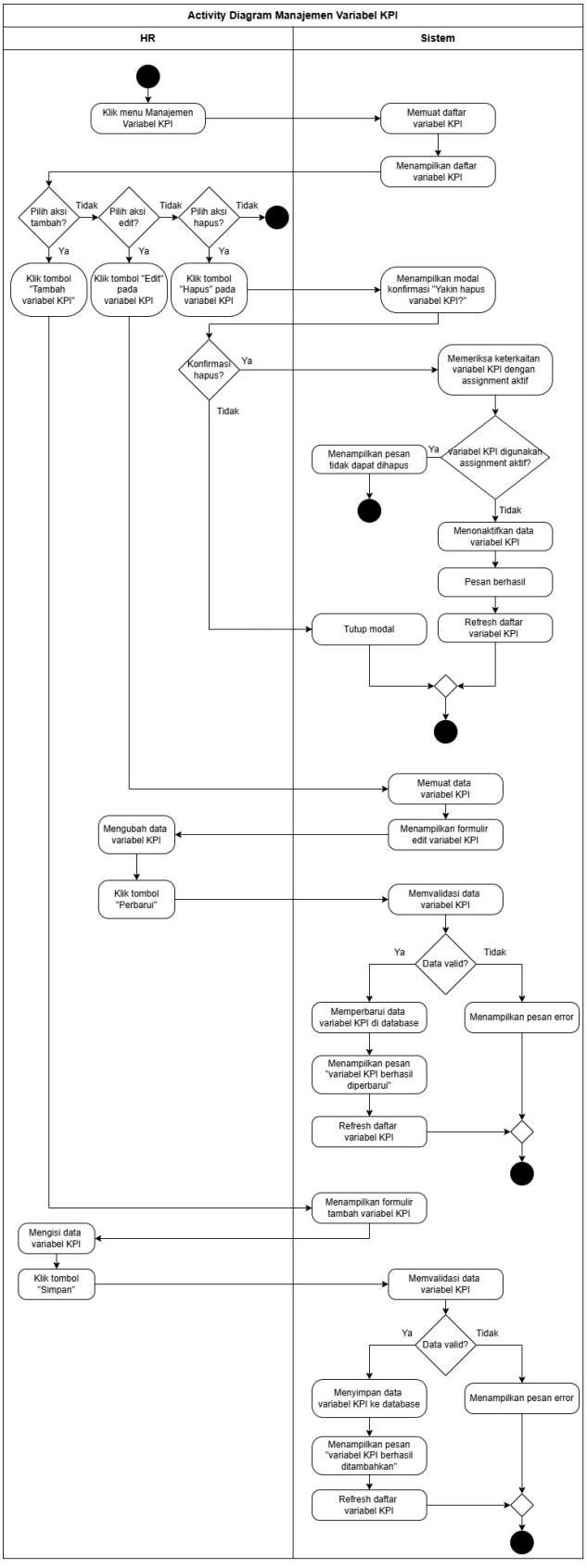
Gambar 40. *Activity Diagram* Konfigurasi *Approval Chain*

Activity Diagram Konfigurasi *Approval Chain* menggambarkan alur aktivitas pengelolaan *template approval chain* oleh aktor HR yang mencakup tiga operasi utama, yaitu penambahan, pembaruan, dan penghapusan data *template*. *Template approval chain* merupakan aturan yang menentukan alur persetujuan pengajuan izin atau lembur berdasarkan jenis pengajuan dan departemen tertentu. Alur dimulai ketika HR mengakses menu Konfigurasi *Approval Chain* dan sistem menampilkan daftar *template* yang tersedia, kemudian HR dapat memilih salah satu dari tiga aksi pengelolaan.

Pada aksi penambahan, HR menekan tombol Tambah *Template* sehingga sistem menampilkan formulir tambah *template*, kemudian HR mengisi data dan menekan tombol Simpan. Pada aksi pembaruan, HR menekan tombol *Edit* pada baris *template* yang ingin diubah sehingga sistem menampilkan formulir *edit* yang telah terisi, kemudian HR mengubah data dan menekan tombol Perbarui. Pada kedua aksi tersebut, sistem akan memvalidasi data yang dikirimkan, di mana jika valid sistem akan menyimpan atau memperbarui data ke basis data, menampilkan pesan keberhasilan, dan memperbarui daftar *template*, sedangkan apabila tidak valid sistem menampilkan pesan kesalahan.

Pada aksi penghapusan, HR menekan tombol Hapus pada baris *template* yang ingin dihapus sehingga sistem menampilkan *modal* konfirmasi. Apabila HR membatalkan, sistem menutup *modal* tanpa perubahan, sedangkan apabila HR mengonfirmasi, sistem memeriksa keterkaitan *template* dengan pengajuan aktif untuk menjaga integritas proses persetujuan yang sedang berjalan. Jika *template* masih digunakan pengajuan aktif, sistem menampilkan pesan tidak dapat dihapus, sedangkan apabila tidak digunakan, sistem menghapus data, menampilkan pesan keberhasilan, dan memperbarui daftar *template*.

7. *Activity Diagram* Manajemen Variabel KPI



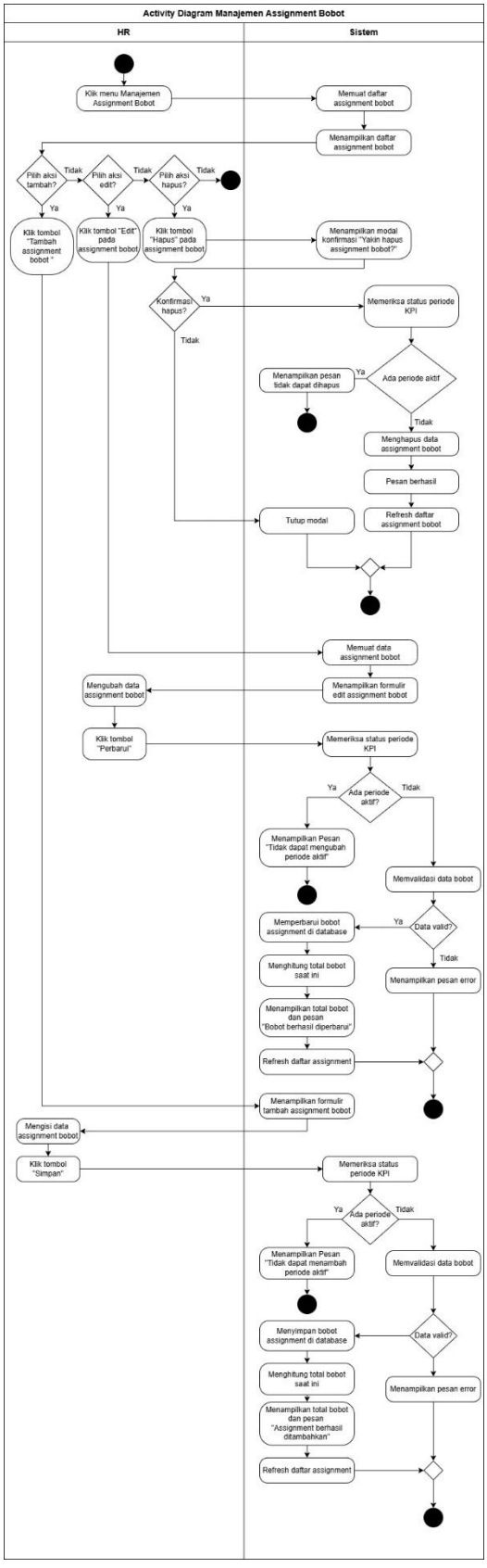
Gambar 41. *Activity Diagram* Manajemen Variabel KPI

Activity Diagram Manajemen Variabel KPI menggambarkan alur aktivitas pengelolaan data variabel KPI oleh aktor HR yang mencakup tiga operasi utama, yaitu penambahan, pembaruan, dan penghapusan data variabel KPI. Alur dimulai ketika HR mengakses menu Manajemen Variabel KPI dan sistem menampilkan daftar variabel KPI yang tersedia, kemudian HR dapat memilih salah satu dari tiga aksi pengelolaan, di mana apabila tidak ada aksi yang dipilih, alur berakhir tanpa operasi apa pun.

Pada aksi penambahan, HR menekan tombol Tambah Variabel KPI sehingga sistem menampilkan formulir tambah variabel KPI, kemudian HR mengisi data dan menekan tombol Simpan. Pada aksi pembaruan, HR menekan tombol *Edit* pada baris variabel KPI yang ingin diubah sehingga sistem menampilkan formulir *edit* yang telah terisi, kemudian HR mengubah data dan menekan tombol Perbarui. Pada kedua aksi tersebut, sistem akan memvalidasi data yang dikirimkan, di mana jika valid sistem akan menyimpan atau memperbarui data ke basis data, menampilkan pesan keberhasilan, dan memperbarui daftar variabel KPI, sedangkan apabila tidak valid sistem menampilkan pesan kesalahan.

Pada aksi penghapusan, HR menekan tombol Hapus pada baris variabel KPI yang ingin dihapus sehingga sistem menampilkan *modal* konfirmasi. Apabila HR membatalkan, sistem menutup *modal* tanpa perubahan, sedangkan apabila HR mengonfirmasi, sistem memeriksa keterkaitan variabel KPI dengan *assignment* aktif. Jika variabel KPI masih digunakan *assignment* aktif, sistem menampilkan pesan tidak dapat dihapus, sedangkan apabila tidak digunakan, sistem menonaktifkan data variabel KPI dengan mekanisme *soft delete*, menampilkan pesan keberhasilan, dan memperbarui daftar variabel KPI.

8. *Activity Diagram* Manajemen *Assignment* Bobot



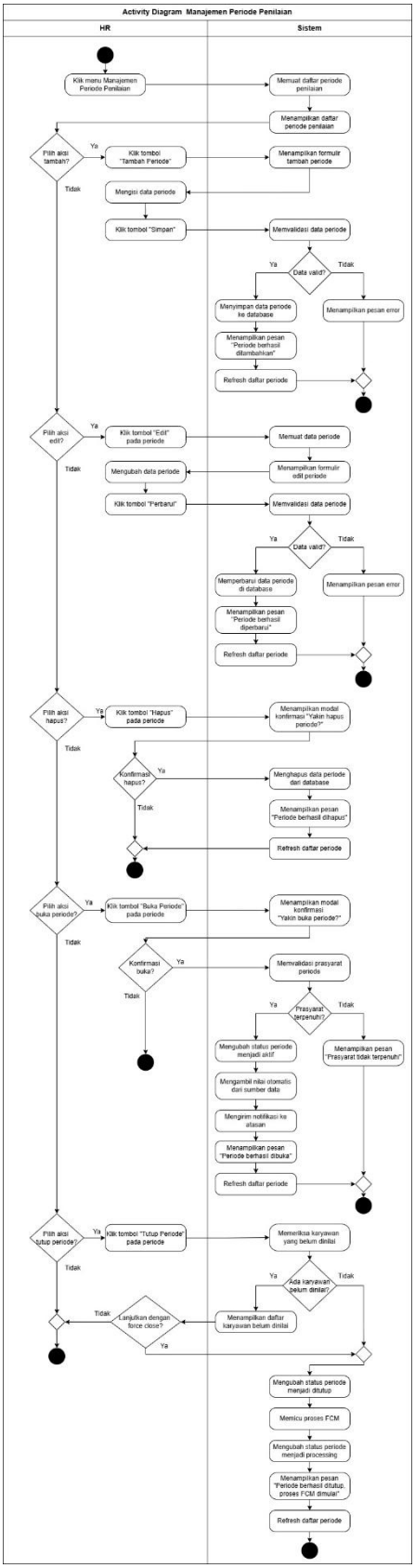
Gambar 42. *Activity Diagram* Manajemen *Assignment* Bobot

Activity Diagram Manajemen *Assignment* Bobot menggambarkan alur aktivitas pengelolaan *assignment* bobot variabel KPI oleh aktor HR yang mencakup tiga operasi utama, yaitu penambahan, pembaruan, dan penghapusan data *assignment*. Alur dimulai ketika HR mengakses menu Manajemen *Assignment* Bobot dan sistem menampilkan daftar *assignment* yang tersedia, kemudian HR dapat memilih salah satu dari tiga aksi pengelolaan atau keluar dari halaman tanpa operasi.

Pada aksi penambahan, HR menekan tombol Tambah *Assignment* Bobot lalu mengisi data pada formulir dan menekan tombol Simpan. Pada aksi pembaruan, HR menekan tombol *Edit* pada baris *assignment* lalu mengubah data dan menekan tombol Perbarui. Pada kedua aksi tersebut, sistem terlebih dahulu memeriksa status periode KPI sebagai mekanisme penjagaan agar bobot tidak dapat diubah saat periode berjalan, di mana apabila terdapat periode aktif sistem menampilkan pesan tidak dapat ditambahkan atau diubah, sedangkan apabila tidak ada sistem memvalidasi data. Jika data valid, sistem menyimpan atau memperbarui *assignment*, menghitung total bobot saat ini, menampilkan pesan keberhasilan beserta total bobot, dan memperbarui daftar *assignment*.

Pada aksi penghapusan, HR menekan tombol Hapus sehingga sistem menampilkan *modal* konfirmasi. Apabila HR membatalkan, sistem menutup *modal* tanpa perubahan, sedangkan apabila HR mengonfirmasi, sistem memeriksa status periode KPI, di mana jika terdapat periode aktif sistem menampilkan pesan tidak dapat dihapus, sedangkan apabila tidak ada, sistem menghapus data *assignment*, menampilkan pesan keberhasilan, dan memperbarui daftar *assignment*.

9. *Activity Diagram* Manajemen Periode Penilaian



Gambar 43. *Activity Diagram* Manajemen Periode Penilaian

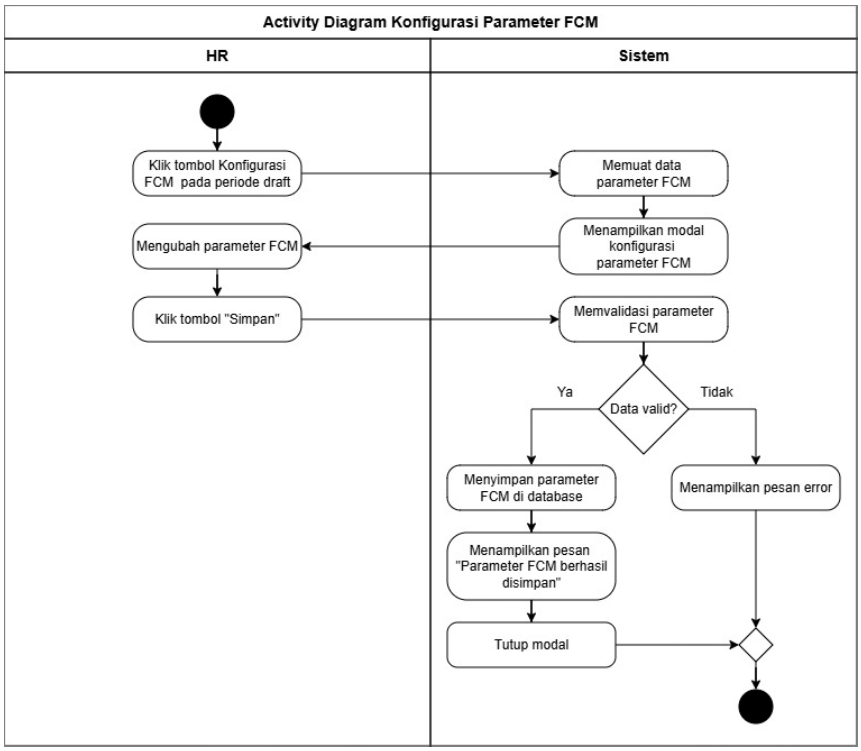
Activity Diagram Manajemen Periode Penilaian menggambarkan alur aktivitas pengelolaan periode penilaian KPI oleh aktor HR yang mencakup lima operasi utama, yaitu penambahan, pembaruan, penghapusan, pembukaan, dan penutupan periode. Periode penilaian merupakan entitas inti yang menjadi kerangka waktu pelaksanaan penilaian kinerja karyawan, di mana setiap periode memiliki rentang tanggal mulai dan tanggal selesai serta status yang berubah secara berurutan dari *draft* menjadi aktif, kemudian ditutup, *processing*, dan akhirnya selesai. Alur dimulai ketika HR mengakses menu Manajemen Periode Penilaian dan sistem menampilkan daftar periode beserta statusnya, kemudian HR dapat memilih salah satu dari lima aksi pengelolaan.

Pada aksi penambahan, HR menekan tombol Tambah Periode lalu mengisi data periode yang mencakup bulan, tahun, tanggal mulai, dan tanggal selesai, kemudian menekan tombol Simpan. Pada aksi pembaruan, HR menekan tombol *Edit* pada baris periode yang ingin diubah sehingga sistem menampilkan formulir *edit* yang telah terisi, kemudian HR mengubah data dan menekan tombol Perbarui. Pembaruan dan penghapusan hanya dapat dilakukan pada periode yang masih berstatus *draft*. Pada kedua aksi tersebut, sistem akan memvalidasi data yang dikirimkan dengan memeriksa kelengkapan data, kesesuaian rentang tanggal, dan keunikan kombinasi bulan dan tahun, di mana jika valid sistem akan menyimpan atau memperbarui data, menampilkan pesan keberhasilan, dan memperbarui daftar periode, sedangkan apabila tidak valid sistem menampilkan pesan kesalahan. Pada aksi penghapusan, HR menekan tombol Hapus sehingga sistem menampilkan *modal* konfirmasi, di mana apabila HR mengonfirmasi sistem akan menghapus data periode dari basis data.

Pada aksi pembukaan periode, sistem memvalidasi prasyarat yang mencakup tidak adanya periode lain yang sedang aktif dan total bobot variabel KPI untuk seluruh jabatan aktif harus sebesar 100% setelah resolusi tiga *layer* melalui mekanisme *Decorator Pattern*, di mana jika

terpenuhi sistem akan mengubah status periode menjadi aktif, mengambil nilai otomatis dari sumber data sistem, dan mengirim notifikasi kepada seluruh atasan aktif. Pada aksi penutupan periode, sistem memeriksa karyawan yang belum dinilai dan menyediakan opsi *force close* untuk menutup periode meskipun terdapat karyawan yang belum dinilai (karyawan tersebut akan dilewati dari proses klasterisasi). Apabila penutupan dilanjutkan, sistem mengubah status periode menjadi ditutup, memicu proses FCM secara *asynchronous* melalui antrian pekerjaan, dan mengubah status periode menjadi *processing*.

10. *Activity Diagram* Konfigurasi Parameter FCM

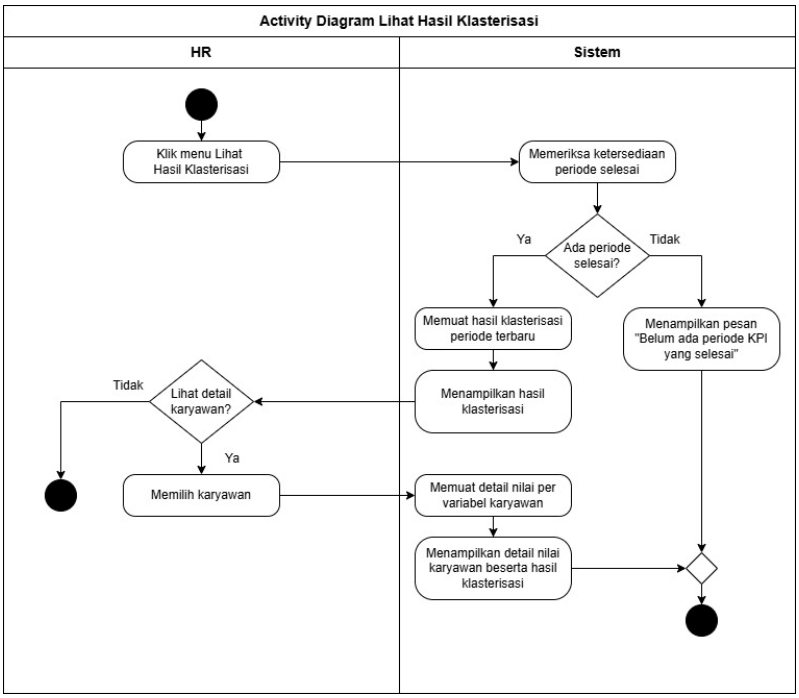


Gambar 44. *Activity Diagram* Konfigurasi Parameter FCM

Activity Diagram Konfigurasi Parameter FCM menggambarkan alur aktivitas pengaturan parameter algoritma *Fuzzy C-Means* oleh aktor HR pada periode penilaian yang berstatus *draft*. Parameter FCM yang dapat diatur mencakup jumlah *cluster*, pangkat pembobot, jumlah iterasi maksimum, dan ambang batas *error* minimum, di mana pengaturan ini hanya dapat dilakukan sebelum periode dibuka karena nilai parameter akan menjadi acuan proses klasterisasi kinerja karyawan ketika periode ditutup.

Alur dimulai ketika HR menekan tombol Konfigurasi FCM pada salah satu periode berstatus *draft* di halaman Manajemen Periode Penilaian, sehingga sistem memuat data parameter FCM yang saat ini berlaku dan menampilkan *modal* konfigurasi. HR mengubah parameter sesuai kebutuhan lalu menekan tombol Simpan, kemudian sistem memvalidasi parameter yang dikirimkan, di mana apabila valid sistem menyimpan parameter FCM ke basis data, menampilkan pesan keberhasilan, dan menutup *modal*, sedangkan apabila tidak valid sistem menampilkan pesan kesalahan dan alur berakhir.

11. *Activity Diagram* Lihat Hasil Klasterisasi



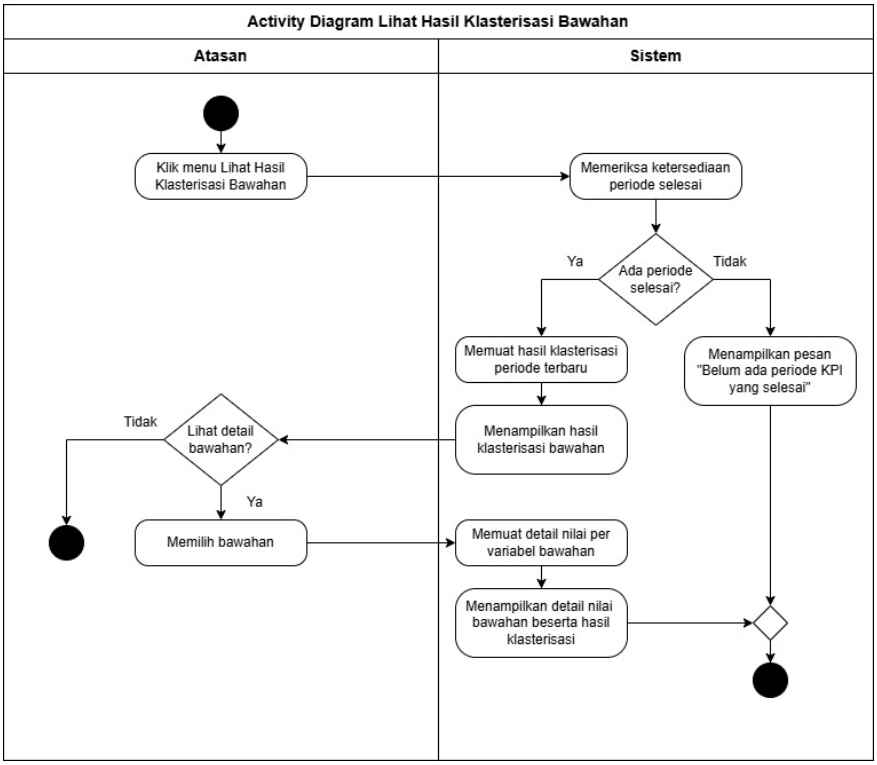
Gambar 45. *Activity Diagram* Lihat Hasil Klasterisasi

Activity Diagram Lihat Hasil Klasterisasi menggambarkan alur aktivitas pemantauan hasil klasterisasi kinerja karyawan oleh aktor HR setelah proses *Fuzzy C-Means* selesai. Alur dimulai ketika HR mengakses menu Lihat Hasil Klasterisasi, kemudian sistem memeriksa ketersediaan periode KPI yang sudah selesai, di mana apabila tidak terdapat periode yang sudah selesai sistem menampilkan pesan bahwa belum ada periode KPI yang selesai dan alur berakhir, sedangkan apabila terdapat periode selesai sistem memuat hasil klasterisasi dari periode terbaru secara otomatis yang mencakup daftar karyawan

beserta *cluster* yang ditempati, skor akhir, dan ringkasan jumlah karyawan per *cluster*.

HR selanjutnya dapat memilih untuk melihat detail nilai karyawan tertentu atau keluar dari halaman tanpa aksi. Pada aksi lihat detail karyawan, HR memilih karyawan yang ingin dilihat detailnya, kemudian sistem memuat dan menampilkan detail nilai per variabel karyawan tersebut beserta hasil klasterisasinya yang mencakup nilai aktual, nilai ternormalisasi, bobot, kontribusi terhadap skor akhir, dan derajat keanggotaan terhadap setiap *cluster* sebagai representasi konsep *soft clustering* pada *Fuzzy C-Means*.

12. *Activity Diagram* Lihat Hasil Klasterisasi Bawahan



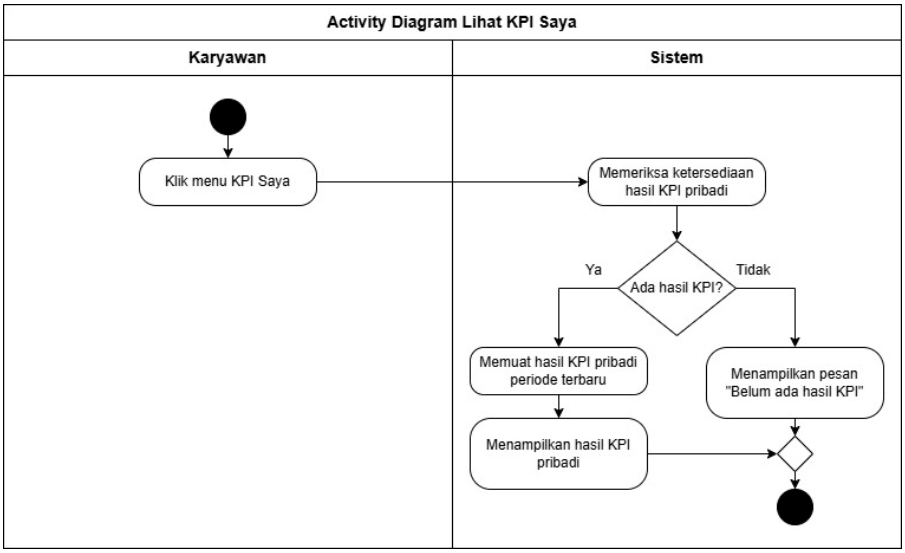
Gambar 46. *Activity Diagram* Lihat Hasil Klasterisasi Bawahan

Activity Diagram Lihat Hasil Klasterisasi Bawahan menggambarkan alur aktivitas pemantauan hasil klasterisasi kinerja karyawan oleh aktor Atasan setelah proses *Fuzzy C-Means* selesai. Atasan hanya dapat melihat hasil klasterisasi karyawan yang berada langsung di bawah jabatannya sebagai bawahan, di mana akses ini dibatasi oleh relasi atasan-bawahan pada struktur jabatan. Alur dimulai ketika Atasan

mengakses menu Lihat Hasil Klasterisasi Bawahan, kemudian sistem memeriksa ketersediaan periode KPI yang sudah selesai, di mana apabila tidak terdapat periode yang sudah selesai sistem menampilkan pesan bahwa belum ada periode KPI yang selesai dan alur berakhir, sedangkan apabila terdapat periode selesai sistem memuat hasil klasterisasi bawahan dari periode terbaru secara otomatis yang mencakup daftar bawahan beserta *cluster* yang ditempati, skor akhir, dan ringkasan jumlah bawahan per *cluster*.

Atasan selanjutnya dapat memilih untuk melihat detail nilai bawahan tertentu atau keluar dari halaman tanpa aksi. Pada aksi lihat detail bawahan, Atasan memilih bawahan yang ingin dilihat detailnya, kemudian sistem memuat dan menampilkan detail nilai per variabel bawahan tersebut beserta hasil klasterisasinya yang mencakup nilai aktual, nilai ternormalisasi, bobot, kontribusi terhadap skor akhir, dan derajat keanggotaan terhadap setiap *cluster* sebagai representasi konsep *soft clustering* pada *Fuzzy C-Means*.

13. *Activity Diagram* Lihat KPI Saya



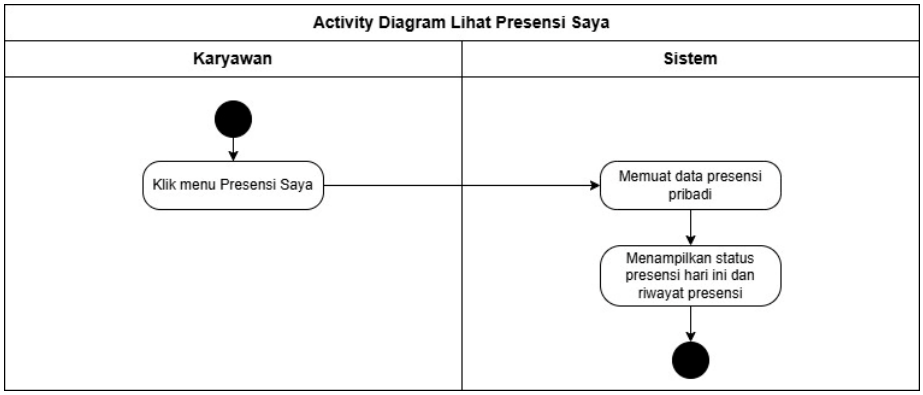
Gambar 47. *Activity Diagram* Lihat KPI Saya

Activity Diagram Lihat KPI Saya menggambarkan alur aktivitas pemantauan hasil penilaian KPI pribadi yang dilakukan oleh aktor Karyawan. Alur dimulai ketika Karyawan mengakses menu KPI Saya, kemudian sistem memeriksa ketersediaan hasil KPI pribadi karyawan

tersebut, di mana apabila Karyawan belum memiliki hasil KPI sistem menampilkan pesan bahwa belum ada hasil KPI dan alur berakhir.

Apabila Karyawan sudah memiliki hasil KPI, sistem memuat hasil KPI pribadi dari periode terbaru secara otomatis dan menampilkannya kepada Karyawan. Hasil KPI yang ditampilkan mencakup label *cluster* beserta deskripsi pencapaian kinerja, skor akhir, derajat keanggotaan terhadap setiap *cluster* sebagai representasi konsep *soft clustering* pada *Fuzzy C-Means*, posisi peringkat di antara seluruh karyawan beserta persentil, *trend* skor akhir antar periode, variabel KPI yang menjadi kekuatan utama dan fokus perbaikan, serta detail nilai per variabel KPI lengkap dengan nilai aktual, nilai ternormalisasi, bobot, dan kontribusi terhadap skor akhir.

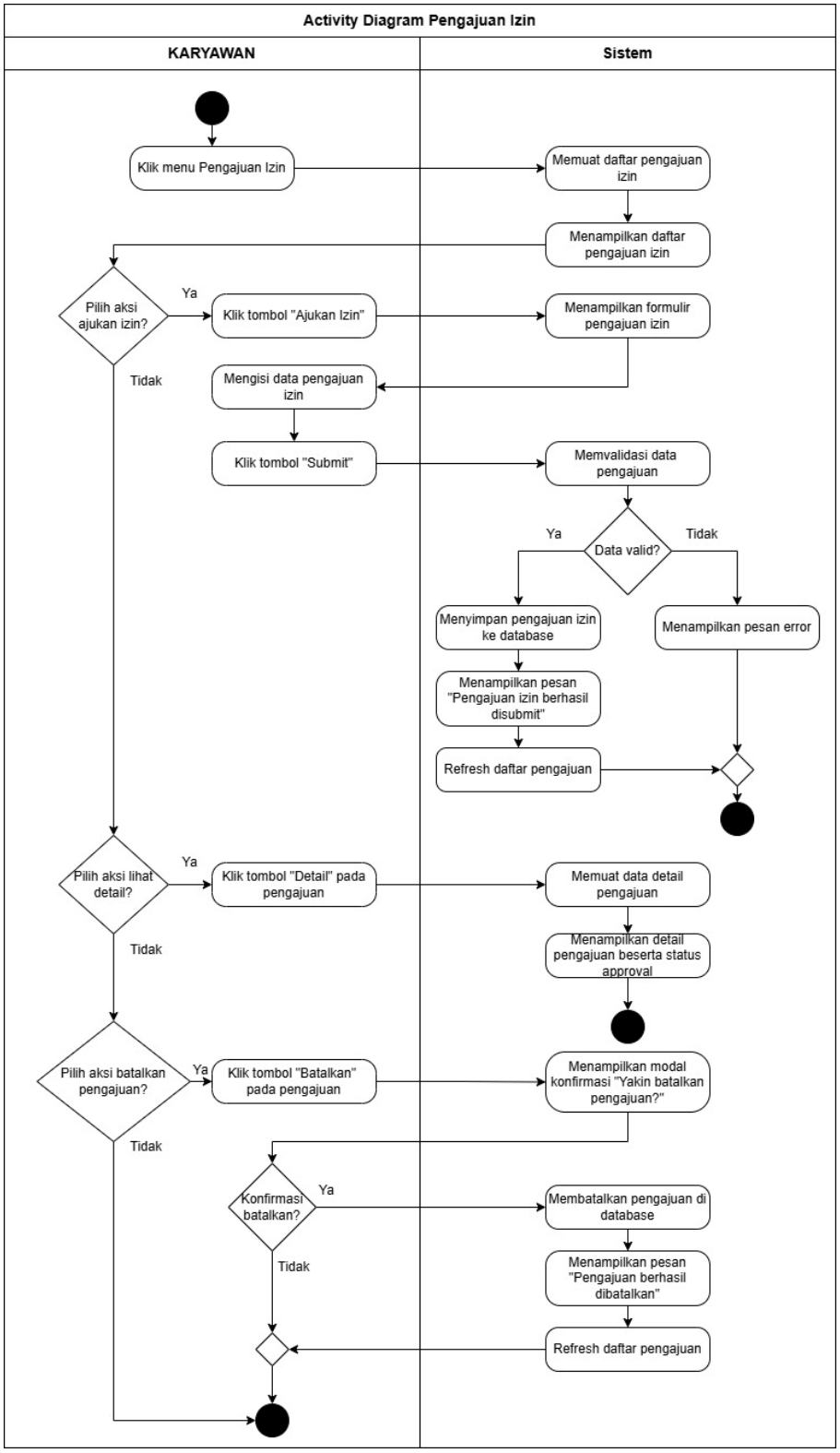
14. *Activity Diagram* Lihat Presensi Saya



Gambar 48. *Activity Diagram* Lihat Presensi Saya

Activity Diagram Lihat Presensi Saya menggambarkan alur aktivitas pemantauan data presensi pribadi yang dilakukan oleh aktor Karyawan. Alur aktivitas dimulai ketika Karyawan mengakses menu Presensi Saya, kemudian sistem memuat data presensi pribadi karyawan tersebut dan menampilkannya kepada Karyawan. Data presensi yang ditampilkan mencakup status presensi hari ini yang terdiri atas jadwal kerja, waktu *clock in*, waktu *clock out*, total jam kerja, status kehadiran, dan status validasi, serta riwayat presensi bulanan yang menampilkan rekapitulasi presensi pada bulan berjalan.

15. *Activity Diagram* Pengajuan Izin



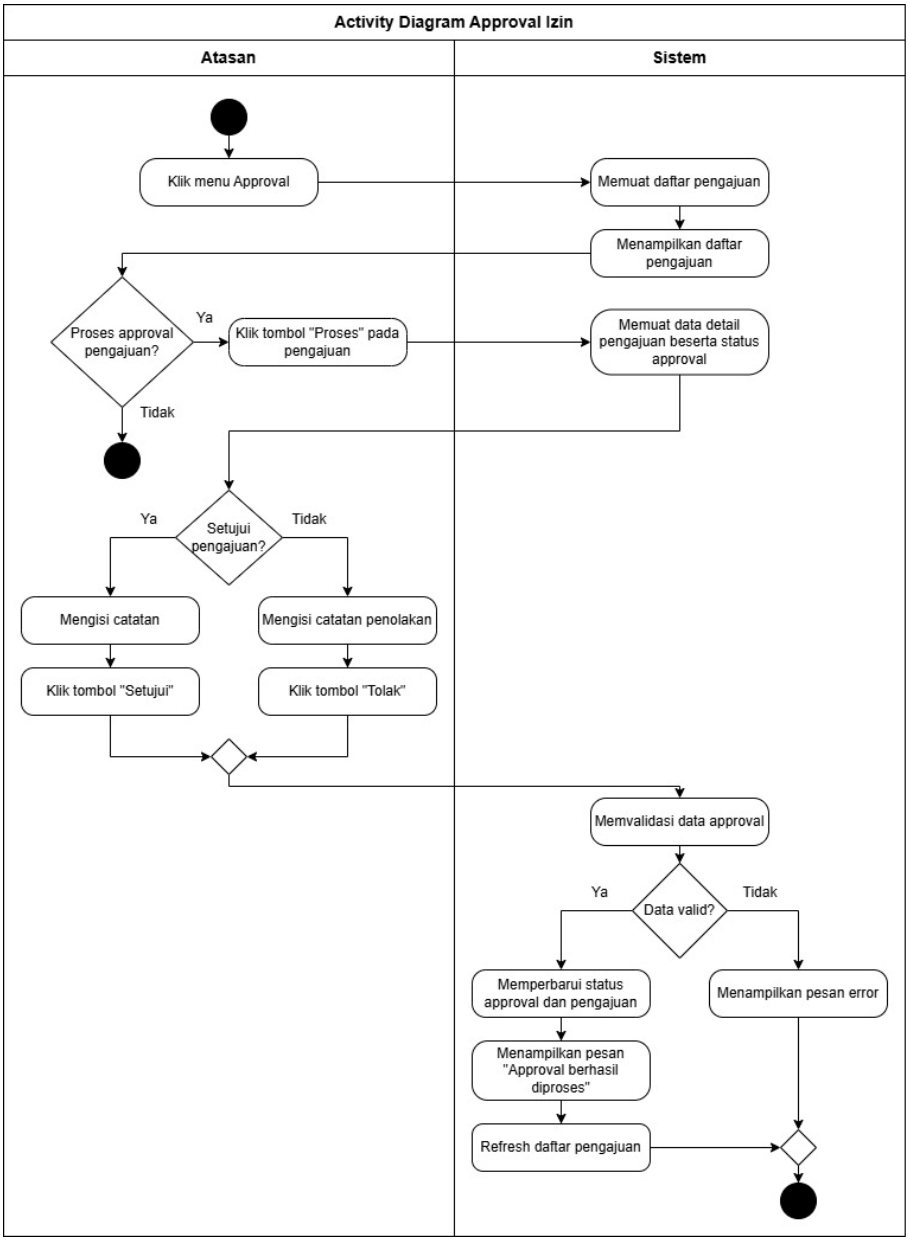
Gambar 49. *Activity Diagram* Pengajuan Izin

Activity Diagram Pengajuan Izin menggambarkan alur aktivitas pengelolaan pengajuan izin oleh aktor Karyawan yang mencakup tiga

operasi utama, yaitu pengajuan izin baru, pelihatan detail pengajuan, dan pembatalan pengajuan. Alur dimulai ketika Karyawan mengakses menu Pengajuan Izin dan sistem memuat serta menampilkan daftar pengajuan izin miliknya, kemudian Karyawan dapat memilih salah satu dari tiga aksi pengelolaan atau keluar dari halaman tanpa operasi. Pada aksi pengajuan izin, Karyawan menekan tombol Ajukan Izin sehingga sistem menampilkan formulir pengajuan izin, kemudian Karyawan mengisi data pengajuan yang mencakup jenis izin, tanggal mulai, tanggal selesai, keterangan, dan bukti pendukung apabila jenis izin yang dipilih mewajibkan bukti, lalu menekan tombol *Submit*. Sistem memvalidasi data dengan memeriksa kelengkapan data, kesesuaian rentang tanggal, dan kesesuaian jumlah hari dengan ketentuan jenis izin, di mana jika valid sistem akan menyimpan pengajuan ke basis data, menampilkan pesan keberhasilan, dan memperbarui daftar pengajuan, sedangkan apabila tidak valid sistem menampilkan pesan kesalahan dan alur berakhir.

Pada aksi pelihatan detail, Karyawan menekan tombol Detail pada baris pengajuan yang ingin dilihat sehingga sistem memuat dan menampilkan data detail pengajuan beserta status persetujuan dari masing-masing *approver* dalam *approval chain*, sehingga Karyawan dapat memantau perkembangan pengajuannya. Pada aksi pembatalan pengajuan, Karyawan menekan tombol Batalkan pada baris pengajuan yang ingin dibatalkan sehingga sistem menampilkan *modal* konfirmasi, di mana pembatalan hanya dapat dilakukan apabila pengajuan masih berstatus *pending* dan tanggal mulai pengajuan belum tiba. Apabila Karyawan mengonfirmasi pembatalan, sistem mengubah status pengajuan menjadi dibatalkan di basis data, menampilkan pesan keberhasilan, dan memperbarui daftar pengajuan, sedangkan apabila Karyawan membatalkan tindakan, alur berakhir.

16. *Activity Diagram Approval* Izin



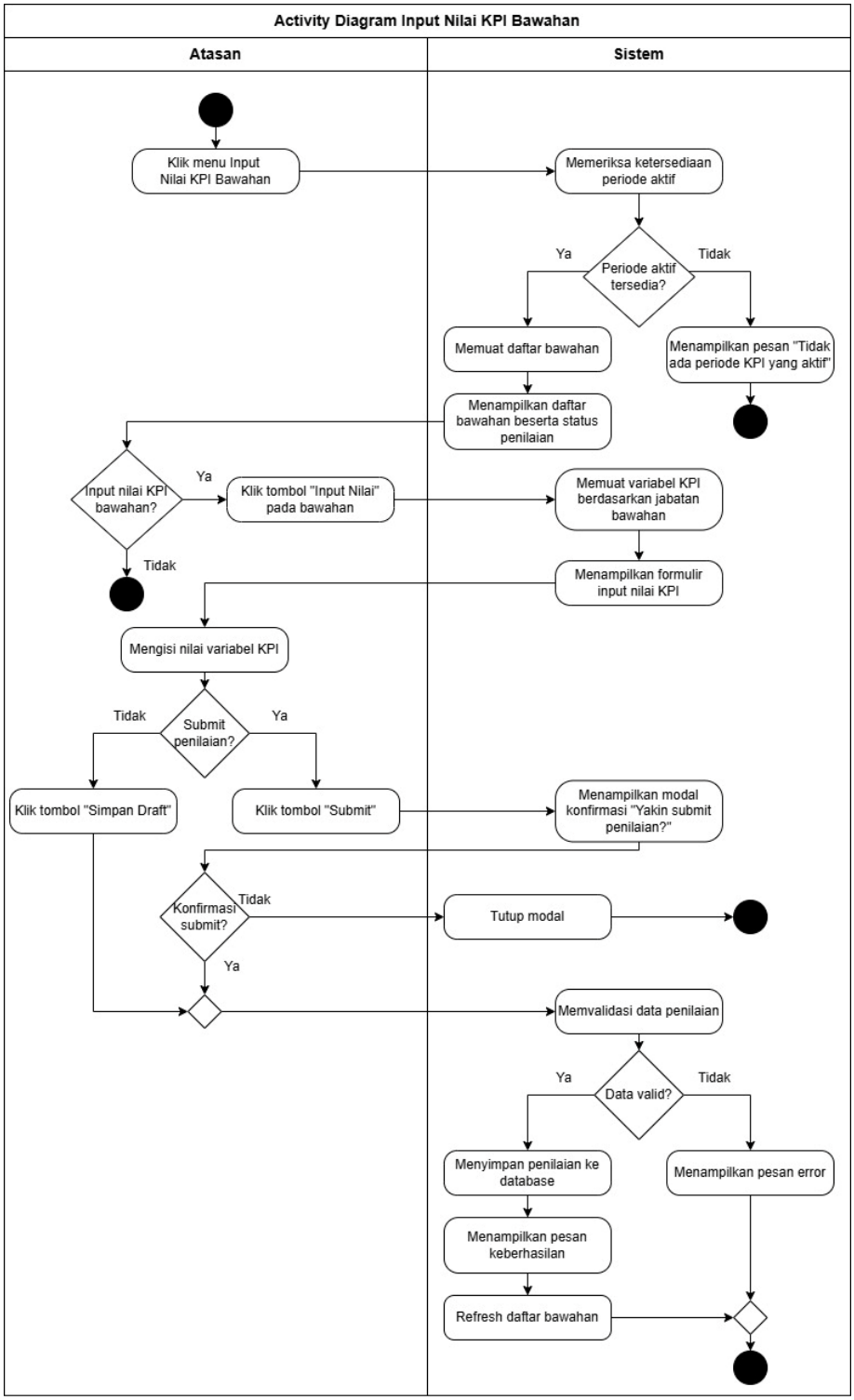
Gambar 50. *Activity Diagram Approval* Izin

Activity Diagram Approval Izin menggambarkan alur aktivitas pemrosesan persetujuan pengajuan izin yang dilakukan oleh aktor Atasan sebagai *approver* dalam *approval chain*. Alur aktivitas dimulai ketika Atasan mengakses menu *Approval*, kemudian sistem memuat dan menampilkan daftar pengajuan yang ditujukan kepadanya untuk diproses. Apabila Atasan tidak melakukan proses *approval*, alur aktivitas berakhir tanpa perubahan.

Apabila Atasan memilih untuk memproses *approval*, Atasan menekan tombol Proses pada baris pengajuan yang ingin diproses sehingga sistem memuat data detail pengajuan beserta status *approval* dari masing-masing *level* dalam *approval chain*. Atasan selanjutnya memutuskan untuk menyetujui atau menolak pengajuan tersebut berdasarkan informasi yang ditampilkan.

Apabila Atasan menyetujui pengajuan, Atasan mengisi catatan persetujuan kemudian menekan tombol Setujui. Sebaliknya, apabila Atasan menolak pengajuan, Atasan mengisi catatan penolakan sebagai alasan penolakan kemudian menekan tombol Tolak. Sistem selanjutnya memvalidasi data *approval* yang dikirimkan, di mana apabila data tidak valid, sistem menampilkan pesan kesalahan dan alur aktivitas berakhir. Apabila data valid, sistem memperbarui status *approval* pada *level* yang bersangkutan dan memperbarui status pengajuan secara keseluruhan, menampilkan pesan keberhasilan, dan memperbarui daftar pengajuan.

17. *Activity Diagram Input* Nilai KPI Bawahan



Gambar 51. *Activity Diagram Input* Nilai KPI Bawahan

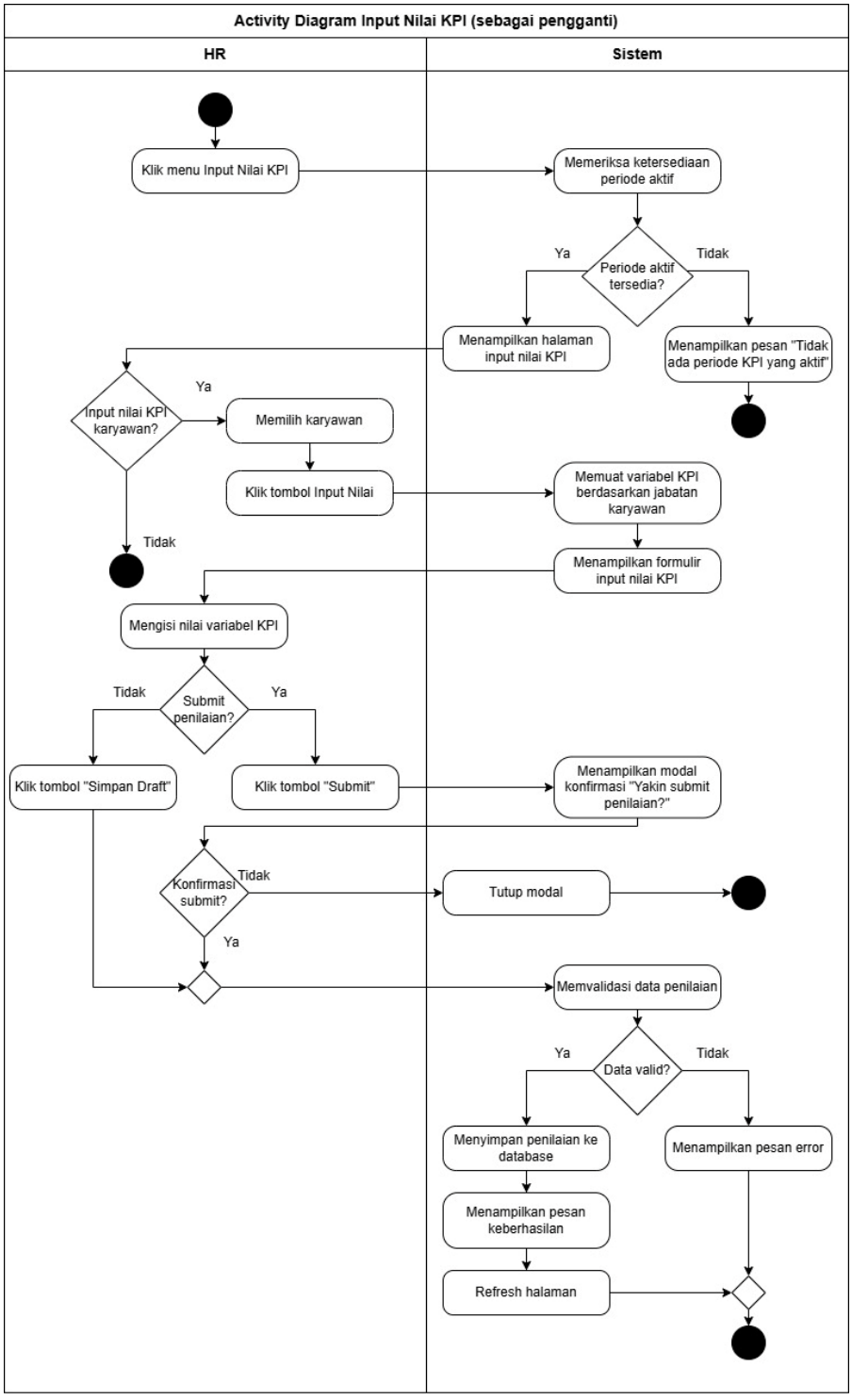
Activity Diagram Input Nilai KPI Bawahan menggambarkan alur aktivitas pemberian nilai KPI yang dilakukan oleh aktor Atasan untuk karyawan yang berada langsung di bawah jabatannya sebagai bawahan.

Alur dimulai ketika Atasan mengakses menu *Input* Nilai KPI Bawahan, kemudian sistem memeriksa ketersediaan periode KPI yang sedang aktif, di mana apabila tidak terdapat periode aktif atau hari ini berada di luar rentang tanggal periode aktif sistem menampilkan pesan bahwa tidak ada periode KPI yang aktif dan alur berakhir. Apabila terdapat periode aktif, sistem memuat daftar bawahan yang harus dinilai beserta status penilaiannya yang mencakup tiga status yaitu belum diisi, *draft*, dan *submitted*.

Pada aksi *input* nilai, Atasan menekan tombol *Input* Nilai pada baris bawahan yang ingin dinilai, kemudian sistem memuat variabel KPI berdasarkan jabatan bawahan tersebut dengan menerapkan mekanisme *Decorator Pattern* yang menggabungkan tiga *layer* variabel yaitu *layer* global, *layer* departemen, dan *layer* jabatan, di mana variabel yang sama pada *layer* yang lebih spesifik akan menggantikan variabel pada *layer* yang lebih umum. Sistem kemudian menampilkan formulir *input* nilai KPI yang berisi daftar variabel yang harus diisi beserta informasi pendukung seperti tipe nilai, rentang nilai, dan bobot variabel.

Atasan mengisi nilai untuk setiap variabel KPI, kemudian memilih untuk menyimpan sebagai *draft* melalui tombol Simpan *Draft*, atau melakukan *submit* penilaian melalui tombol *Submit* yang akan menampilkan *modal* konfirmasi karena penilaian yang sudah di-*submit* akan terkunci dan tidak dapat diubah lagi tanpa *unlock* dari HR. Sistem kemudian memvalidasi data penilaian dengan memeriksa kesesuaian nilai terhadap rentang yang ditentukan untuk setiap variabel, di mana apabila tidak valid sistem menampilkan pesan kesalahan dan alur berakhir, sedangkan apabila valid sistem menyimpan penilaian ke basis data dengan status sesuai pilihan, menampilkan pesan keberhasilan, dan memperbarui daftar bawahan.

18. *Activity Diagram Input* Nilai KPI (sebagai pengganti)



Gambar 52. *Activity Diagram Input* Nilai KPI (sebagai pengganti)

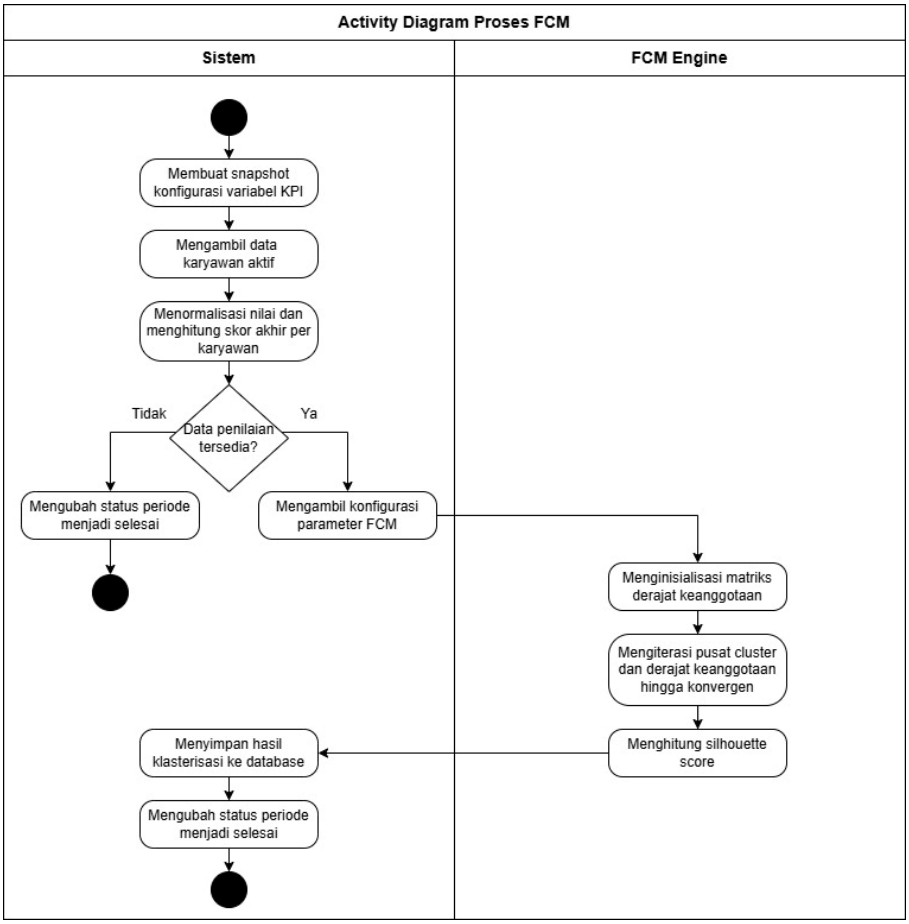
Activity Diagram Input Nilai KPI (sebagai pengganti) menggambarkan alur aktivitas pemberian nilai KPI yang dilakukan oleh aktor HR sebagai pengganti dalam kondisi tertentu, di mana HR dapat

memberikan nilai KPI untuk karyawan apabila atasan yang seharusnya menilai tidak dapat melakukan penilaian, seperti karena cuti panjang, *resign*, atau tidak menyelesaikan penilaian sampai batas waktu yang ditentukan. Alur dimulai ketika HR mengakses menu *Input* Nilai KPI, kemudian sistem memeriksa ketersediaan periode KPI yang sedang aktif, di mana apabila tidak terdapat periode aktif atau hari ini berada di luar rentang tanggal periode aktif sistem menampilkan pesan bahwa tidak ada periode KPI yang aktif dan alur berakhir. Apabila terdapat periode aktif, sistem menampilkan halaman *input* nilai KPI yang berisi panel pemilihan karyawan, kemudian HR dapat memilih untuk melakukan *input* nilai KPI atau keluar dari halaman tanpa aksi.

Pada aksi *input* nilai, HR memilih karyawan yang ingin dinilai melalui kolom pencarian karyawan kemudian menekan tombol *Input* Nilai, sehingga sistem memuat variabel KPI berdasarkan jabatan karyawan tersebut dengan menerapkan mekanisme *Decorator Pattern* yang menggabungkan tiga *layer* variabel yaitu *layer* global, *layer* departemen, dan *layer* jabatan, di mana variabel yang sama pada *layer* yang lebih spesifik akan menggantikan variabel pada *layer* yang lebih umum. Sistem kemudian menampilkan formulir *input* nilai KPI yang berisi daftar variabel yang harus diisi beserta informasi pendukung seperti tipe nilai, rentang nilai, dan bobot variabel.

HR mengisi nilai untuk setiap variabel KPI, kemudian memilih untuk menyimpan sebagai *draft* melalui tombol Simpan *Draft*, atau melakukan *submit* penilaian melalui tombol *Submit* yang akan menampilkan *modal* konfirmasi karena penilaian yang sudah di-*submit* akan terkunci dan tidak dapat diubah lagi. Sistem kemudian memvalidasi data penilaian dengan memeriksa kesesuaian nilai terhadap rentang yang ditentukan untuk setiap variabel, di mana apabila tidak valid sistem menampilkan pesan kesalahan dan alur berakhir, sedangkan apabila valid sistem menyimpan penilaian ke basis data dengan status sesuai pilihan, menampilkan pesan keberhasilan, dan memperbarui halaman.

19. *Activity Diagram* Proses FCM



Gambar 53. *Activity Diagram* Proses FCM

Activity Diagram Proses FCM menggambarkan alur aktivitas pelaksanaan algoritma *Fuzzy C-Means* untuk klasterisasi kinerja karyawan yang berjalan secara otomatis oleh sistem setelah HR menutup periode penilaian KPI. Proses ini berjalan secara *asynchronous* sehingga tidak melibatkan aktor manusia secara langsung, melainkan dikoordinasikan oleh dua komponen utama yaitu Sistem yang menangani orkestrasi data dan *FCM Engine* yang menangani algoritma klasterisasi inti.

Alur dimulai ketika sistem membuat *snapshot* konfigurasi variabel KPI yang berlaku pada saat itu untuk membekukan konfigurasi variabel sehingga proses klasterisasi tetap konsisten meskipun konfigurasi mengalami perubahan di masa depan. Sistem kemudian mengambil data karyawan aktif, melakukan normalisasi nilai dengan metode *min-*

max berdasarkan arah nilai variabel, dan menghitung skor akhir per karyawan sebagai akumulasi dari hasil perkalian nilai ternormalisasi dengan bobot masing-masing variabel KPI yang berlaku setelah resolusi tiga *layer* melalui mekanisme *Decorator Pattern*. Sistem kemudian memeriksa ketersediaan data penilaian, di mana apabila tidak terdapat data penilaian sistem mengubah status periode menjadi selesai dan alur berakhir tanpa hasil klasterisasi, sedangkan apabila terdapat data penilaian sistem mengambil konfigurasi parameter FCM yang mencakup jumlah *cluster*, pangkat pembobot, jumlah iterasi maksimum, dan ambang batas *error* minimum, lalu mengirimkan data skor akhir karyawan beserta konfigurasi parameter tersebut kepada *FCM Engine*.

FCM Engine memulai algoritma klasterisasi dengan menginisialisasi matriks derajat keanggotaan secara acak menggunakan generator bilangan acak deterministik, kemudian mengiterasi pusat *cluster* dan derajat keanggotaan hingga konvergen, di mana iterasi berhenti ketika perubahan derajat keanggotaan antar iterasi telah berada di bawah ambang batas *error* minimum atau ketika jumlah iterasi telah mencapai batas maksimum. *FCM Engine* kemudian menghitung *silhouette score* untuk mengevaluasi kualitas pengelompokan yang dihasilkan, di mana nilai *silhouette score* yang lebih tinggi mengindikasikan struktur klasterisasi yang lebih baik. Setelah *FCM Engine* menyelesaikan algoritma, sistem menerima hasil klasterisasi yang mencakup *cluster* karyawan, derajat keanggotaan setiap karyawan terhadap setiap *cluster*, dan *silhouette score*, kemudian menyimpan hasil ke basis data dan mengubah status periode menjadi selesai sebagai indikasi bahwa proses klasterisasi telah selesai dan hasil siap diakses oleh HR melalui menu Lihat Hasil Klasterisasi.

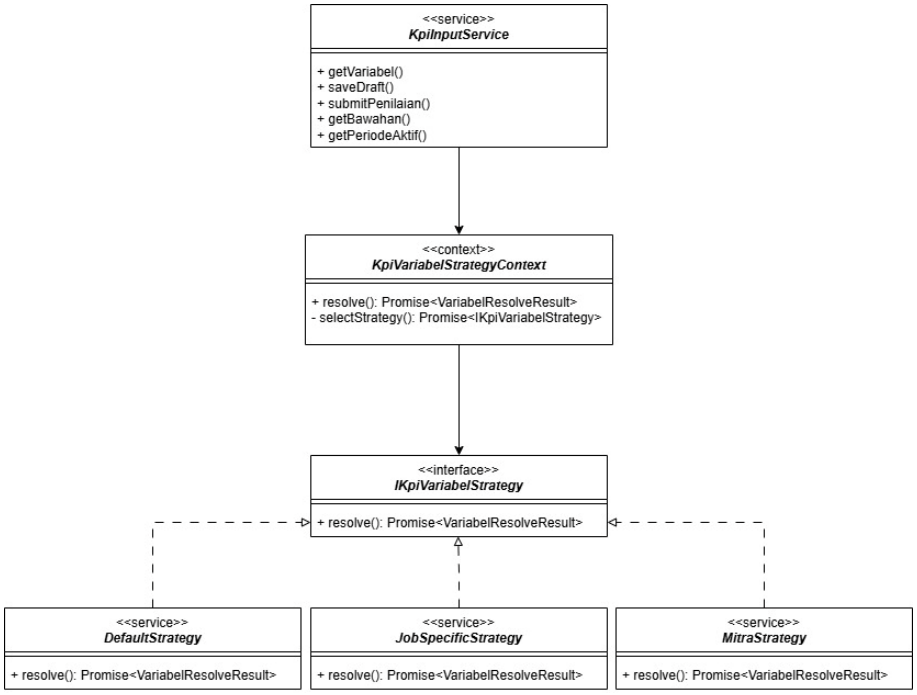
4.2.8 Class Diagram

Sub-bab ini menyajikan rancangan *class diagram* sistem yang dikembangkan, yang digunakan untuk menggambarkan struktur statis sistem meliputi *class-class* yang menyusun arsitektur, atribut dan *method* yang dimiliki, serta relasi antar *class*. Mengingat sistem yang dibangun mengimplementasikan dua pola desain utama yaitu *Strategy Pattern* dan *Decorator Pattern*, serta sebuah algoritma klasterisasi yang mengintegrasikan komponen *reproducibility*, maka *class diagram* pada penelitian ini disajikan dalam empat diagram terpisah agar setiap aspek arsitektur dapat dipahami secara fokus.

Pembagian keempat diagram dilakukan berdasarkan area tanggung jawab yang berbeda, yaitu diagram pertama berfokus pada implementasi *Strategy Pattern* untuk pemilihan strategi resolusi variabel KPI, diagram kedua menampilkan implementasi *Decorator Pattern* yang menyediakan resolusi variabel secara berlapis dan dapat dikomposisi, diagram ketiga menggambarkan arsitektur algoritma *Fuzzy C-Means* beserta komponen *Linear Congruential Generator* yang menjamin sifat deterministik hasil klasterisasi, dan diagram keempat menyajikan gambaran arsitektur *service layer* secara keseluruhan sebagai konteks integrasi antar komponen.

Beberapa keputusan desain perlu disampaikan agar pembacaan diagram menjadi lebih akurat. Pertama, beberapa komponen seperti *KpiVariabelDecorator*, *FcmEngine*, dan *LcgRandomGenerator* direalisasikan dalam bentuk fungsi modular pada implementasi nyata, namun dimodelkan sebagai *class* konseptual pada *class diagram* untuk menampilkan tanggung jawab arsitekturalnya secara visual. Kedua, atribut hasil *dependency injection* dari *framework* NestJS seperti *PrismaService* tidak ditampilkan pada kompartemen atribut karena merupakan komponen infrastruktur yang dikelola oleh *framework*. Ketiga, struktur *entity* dan relasi data yang lebih detail disajikan secara khusus pada sub-bab 4.2.9 (*Entity Relationship Diagram*).

1. *Class Diagram Strategy Pattern*



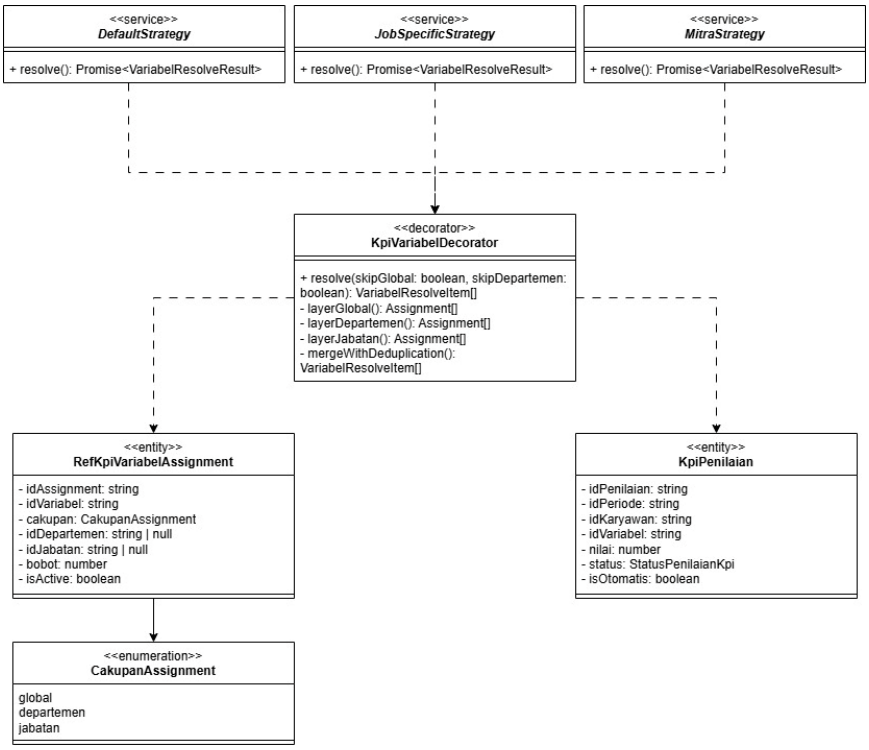
Gambar 54. *Class Diagram Strategy Pattern*

Class diagram pertama ini menggambarkan implementasi *Strategy Pattern* yang digunakan untuk memilih strategi resolusi variabel KPI per karyawan secara dinamis. Pendekatan ini diterapkan karena sistem perlu mengakomodasi tiga jenis karyawan yang memiliki kebutuhan resolusi variabel berbeda, yaitu karyawan dengan jabatan reguler, karyawan tanpa departemen, dan karyawan mitra, sehingga sistem dapat memilih algoritma resolusi yang sesuai pada saat *runtime* tanpa mengubah struktur *class* yang memanggilnya. Diagram ini terdiri dari enam *class*, di mana *IKpiVariabelStrategy* adalah *interface* yang mendefinisikan kontrak resolusi variabel dengan *method* utama *resolve()*, dan tiga *class concrete strategy* mengimplementasikan *interface* tersebut dengan algoritma resolusi yang berbeda, yaitu *DefaultStrategy* untuk karyawan tanpa departemen dengan resolusi dua *layer* (global dan jabatan), *JobSpecificStrategy* untuk karyawan reguler dengan resolusi tiga *layer* penuh (global, departemen, dan jabatan), serta *MitraStrategy* untuk karyawan mitra yang hanya melakukan resolusi pada *layer* jabatan. Selanjutnya, *KpiVariabelStrategyContext* berperan sebagai *context* yang memilih *strategy* yang tepat berdasarkan

karakteristik karyawan kemudian mendelegasikan eksekusi ke *strategy* terpilih, sedangkan *KpiInputService* bertindak sebagai pemanggil *context* tersebut yang merepresentasikan *layer service* yang menerima permintaan dari *controller*.

Relasi antar *class* pada diagram ini menggunakan dua jenis notasi, yaitu tiga *concrete strategy* terhubung ke *interface* melalui relasi *realization* dengan notasi *tree style* yang menunjukkan bahwa ketiganya mengimplementasikan kontrak yang sama, sedangkan hubungan antara *KpiInputService* dengan *KpiVariabelStrategyContext* dan antara *context* dengan *interface* menggunakan notasi *association* yang menggambarkan bahwa *context* menggunakan *strategy* melalui *interface* tanpa terikat pada implementasi tertentu. Desain ini memberikan tiga manfaat utama, yaitu kemudahan untuk menambah *strategy* baru tanpa mengubah *class* yang sudah ada, fleksibilitas dalam memilih algoritma resolusi pada saat *runtime*, dan kejelasan tanggung jawab masing-masing *class* yang sesuai dengan prinsip *Single Responsibility*.

2. *Class Diagram Decorator Pattern*



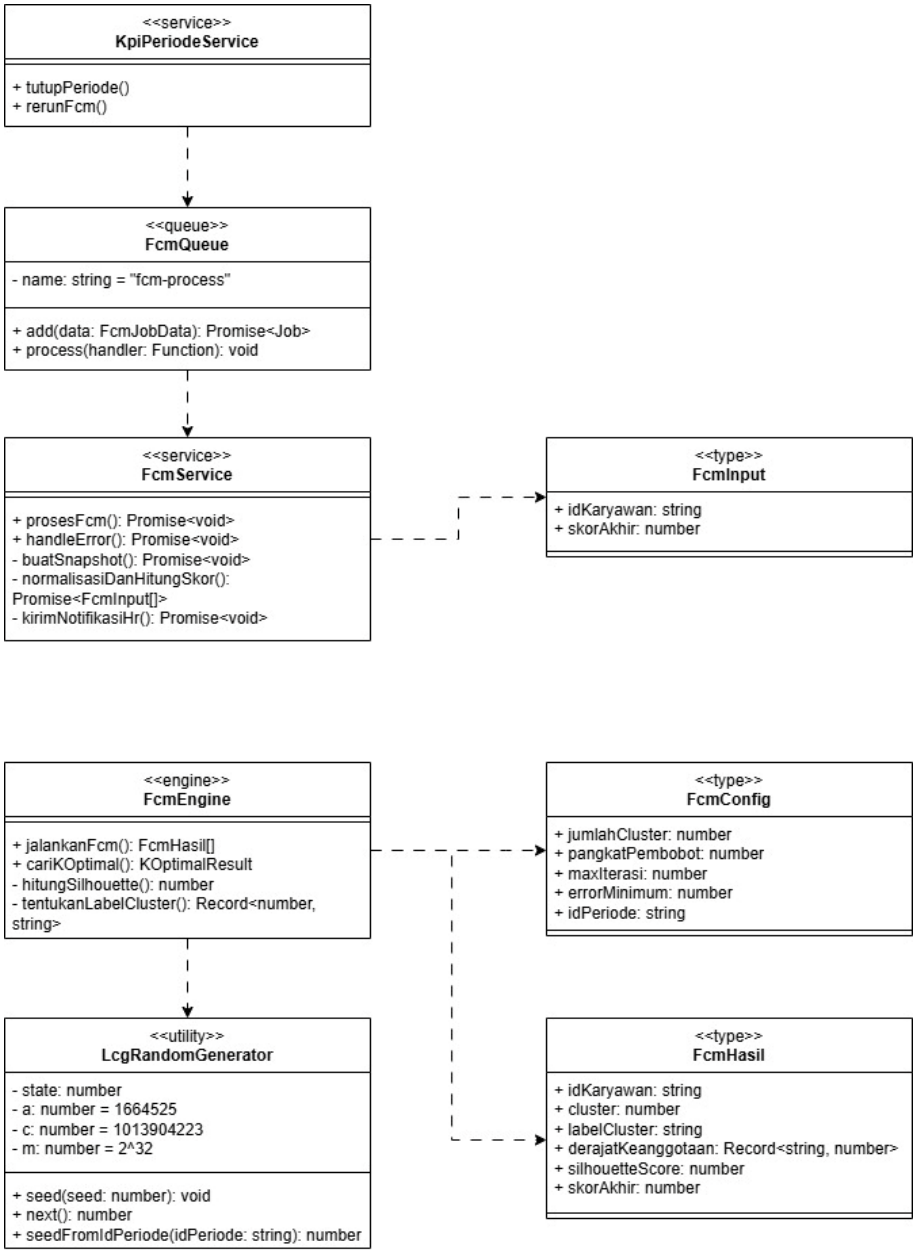
Gambar 55. *Class Diagram Decorator Pattern*

Class diagram kedua ini menggambarkan implementasi *Decorator Pattern* yang digunakan untuk melakukan resolusi variabel KPI secara berlapis. Pola ini diterapkan karena setiap karyawan dapat memiliki variabel KPI yang berasal dari tiga cakupan berbeda yaitu global, departemen, dan jabatan, di mana ketiga cakupan tersebut perlu diakses secara fleksibel dengan dukungan terhadap kemampuan *skip* per *layer* dan deduplikasi prioritas. Diagram ini terdiri dari enam *class*, di mana *KpiVariabelDecorator* adalah *class* utama yang berperan sebagai *decorator* dengan *method* utama *resolve()* yang menerima parameter *skipGlobal* dan *skipDepartemen* untuk mengontrol *layer* mana yang aktif, serta memiliki tiga *method* privat yaitu *layerGlobal()*, *layerDepartemen()*, dan *layerJabatan()*, dan *method mergeWithDeduplication()* yang menggabungkan hasil ketiga *layer* dengan prioritas jabatan terhadap departemen dan departemen terhadap global. Tiga *concrete strategy* yaitu *DefaultStrategy*, *JobSpecificStrategy*, dan *MitraStrategy* ditampilkan kembali sebagai pemanggil *decorator* dengan kombinasi parameter yang berbeda sesuai jenis karyawan, serta dua *entity* yang diakses oleh *decorator* yaitu *RefKpiVariabelAssignment* dan *KpiPenilaian*, dan *class CakupanAssignment* sebagai *enumeration* yang mendefinisikan tiga nilai cakupan yang valid.

Pada implementasi sistem, *KpiVariabelDecorator* direalisasikan dalam bentuk fungsi modular bernama *resolveVariabelDecorator* dan bukan dalam bentuk *class wrapper* sebagaimana pola *Decorator* klasik, untuk menyederhanakan komposisi dan menghindari *overhead* instansiasi karena fungsi ini bersifat *stateless*. Meskipun struktur implementasinya berbeda, peran dan tanggung jawab arsitektural tetap identik yaitu menambahkan kemampuan resolusi *multi-layer* yang dapat dikontrol secara dinamis. Relasi antar *class* pada diagram ini seluruhnya menggunakan notasi *dependency* yang menggambarkan bahwa setiap *concrete strategy* menggunakan *decorator*, *decorator* membaca data dari kedua *entity*, dan *entity RefKpiVariabelAssignment*

memiliki keterkaitan tipe dengan *enumeration CakupanAssignment* melalui atribut cakupan.

3. *Class Diagram* FCM dan LCG



Gambar 56. *Class Diagram* FCM dan LCG

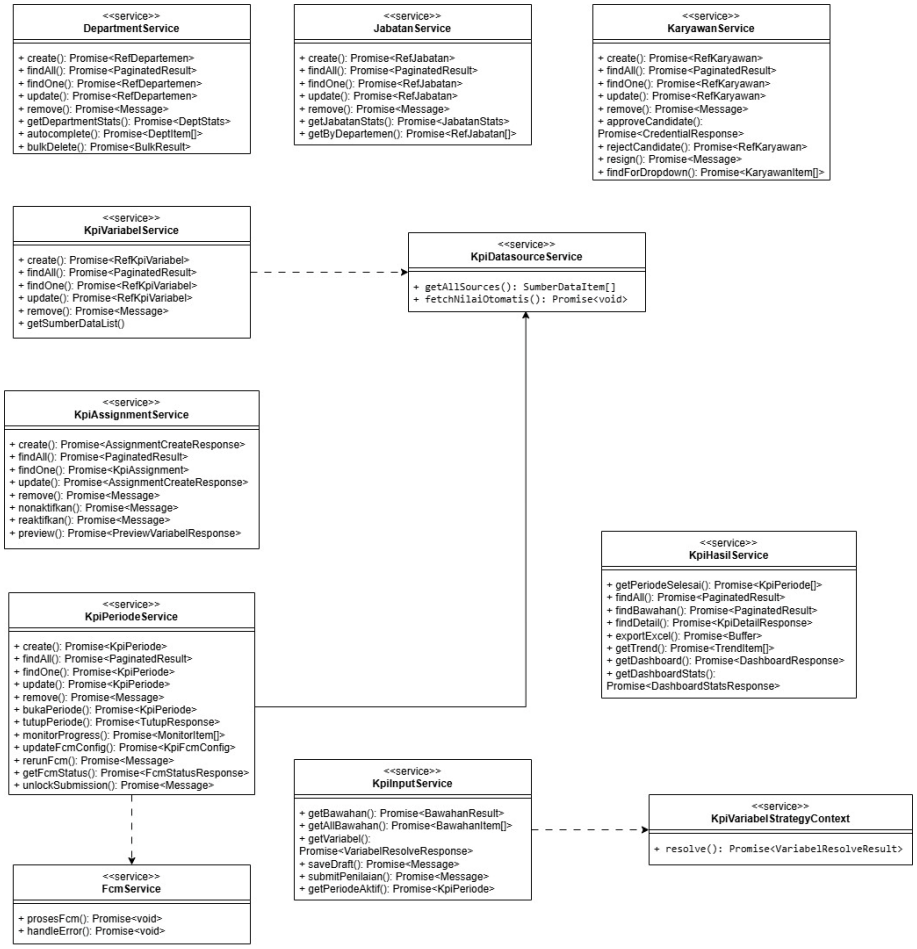
Class diagram ketiga ini menggambarkan arsitektur algoritma *Fuzzy C-Means* yang digunakan untuk klasterisasi kinerja karyawan, beserta komponen *Linear Congruential Generator* yang menjamin sifat deterministik hasil klasterisasi. Algoritma *Fuzzy C-Means* dipilih karena karakteristiknya yang memberikan nilai derajat keanggotaan

terhadap setiap *cluster* yang lebih sesuai dengan realita kinerja karyawan yang umumnya bersifat gradual, sementara integrasi *Linear Congruential Generator* dilakukan untuk mengatasi sifat acak dari inisialisasi matriks partisi awal sehingga hasil klasterisasi dapat direproduksi secara konsisten untuk keperluan audit dan validasi penelitian. Diagram ini terdiri dari delapan *class* yang dikelompokkan menjadi dua bagian, di mana bagian pertama adalah lima *class* utama yang menggambarkan alur eksekusi yaitu *KpiPeriodeService* yang bertindak sebagai *trigger* saat periode KPI ditutup, *FcmQueue* sebagai antrian pemrosesan *asynchronous* yang menerima *job* dari *service* dan mendistribusikannya ke *worker* agar proses klasterisasi tidak menghalangi alur HTTP utama, *FcmService* sebagai *orchestrator* yang melakukan persiapan data berupa *snapshot* konfigurasi, normalisasi nilai, dan penghitungan skor akhir, *FcmEngine* yang berisi logika murni algoritma dengan empat *method* inti yaitu *jalankanFcm()*, *cariKOptimal()*, *hitungSilhouette()*, dan *tentukanLabelCluster()*, serta *LcgRandomGenerator* sebagai *utility* yang menghasilkan bilangan acak deterministik dengan parameter klasik dari *Numerical Recipes*. Bagian kedua adalah tiga tipe data yang berfungsi sebagai kontrak antar komponen yaitu *FcmInput* yang dihasilkan oleh *service* dan dikonsumsi oleh *engine*, *FcmConfig* yang membawa parameter konfigurasi algoritma, serta *FcmHasil* yang merupakan keluaran akhir berisi *cluster*, label, derajat keanggotaan, dan *Silhouette Score* per karyawan.

Pada implementasi sistem, *FcmEngine* dan *LcgRandomGenerator* direalisasikan dalam bentuk fungsi modular pada *file* terpisah dan bukan sebagai *class instantiable* untuk menjaga sifat *stateless* dan menghindari *overhead* instansiasi, namun dimodelkan sebagai *class* konseptual pada *class diagram* untuk menampilkan tanggung jawab arsitektural dan komposisi antar komponen secara visual. Pemisahan tegas antara *FcmService* sebagai *orchestrator* yang berinteraksi dengan *database* dan *FcmEngine* sebagai algoritma murni tanpa *side effect* memberikan dua manfaat utama, yaitu kemudahan dalam pengujian

unit karena *engine* dapat dijalankan tanpa *dependency database* serta kejelasan tanggung jawab yang sesuai dengan prinsip *Separation of Concerns*. Sifat deterministik yang diberikan oleh *LcgRandomGenerator* dijamin melalui pembangkitan *seed* dari *idPeriode* menggunakan 8 karakter heksadesimal pertama, sehingga setiap periode menghasilkan urutan bilangan acak yang sama setiap kali algoritma dijalankan ulang.

4. *Class Diagram Service Architecture Overview*



Gambar 57. *Class Diagram Service Architecture Overview*

Class diagram keempat ini menggambarkan gambaran arsitektur *service layer* secara keseluruhan, di mana ketiga pola desain yang telah dijelaskan pada sub-bab sebelumnya berintegrasi menjadi satu sistem yang utuh. Diagram ini berfokus pada bagaimana setiap *service* berinteraksi melalui mekanisme *dependency injection* yang dikelola oleh *framework* NestJS, yang menjadi salah satu bukti penerapan

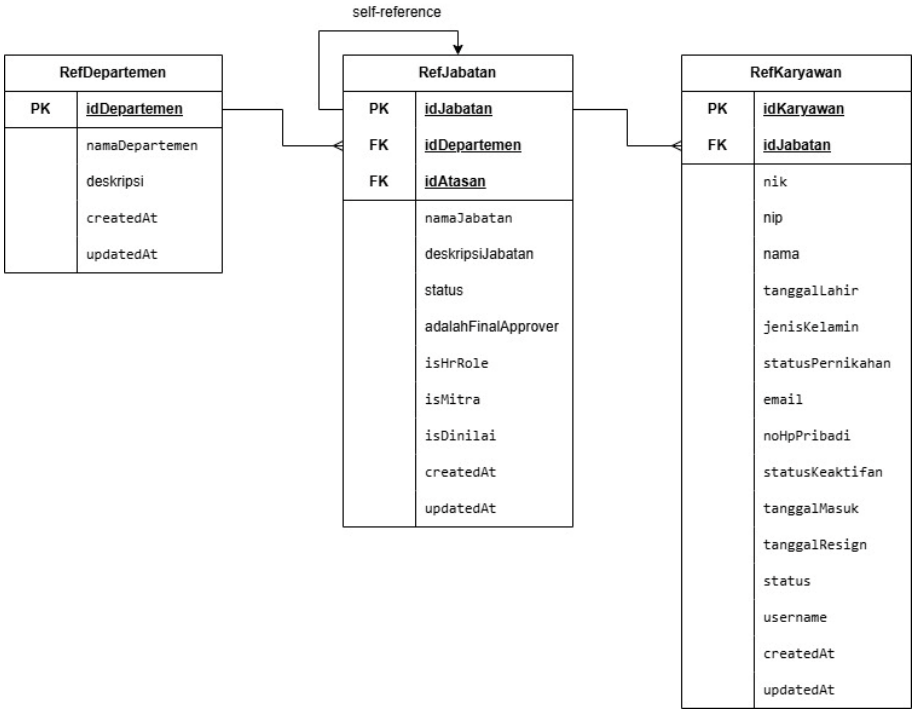
composable architecture. Diagram ini terdiri dari sebelas *class* dengan tanggung jawab spesifik, yaitu *DepartmentService*, *JabatanService*, dan *KaryawanService* untuk mengelola *master data* struktur organisasi, *KpiVariabelService* untuk pengelolaan *master* variabel KPI, *KpiAssignmentService* untuk *assignment* variabel ke *scope* tertentu, *KpiPeriodeService* untuk *lifecycle* periode KPI beserta konfigurasi FCM, *KpiInputService* untuk *input* penilaian, *FcmService* sebagai *orchestrator* proses klasterisasi *asynchronous*, dan *KpiHasilService* untuk akses hasil klasterisasi, serta dua *class* sebagai *dependency* yaitu *KpiVariabelStrategyContext* dan *KpiDatasourceService* yang menyediakan integrasi dengan sumber data otomatis.

Relasi antar *service* pada diagram ini ditampilkan berdasarkan *dependency injection* yang terjadi melalui *constructor* sesuai *pattern* NestJS, di mana terdapat empat relasi yaitu *KpiPeriodeService* menggunakan *FcmService* melalui *Bull Queue* untuk pemrosesan *asynchronous*, *KpiPeriodeService* menggunakan *KpiDatasourceService* untuk pengambilan nilai otomatis, *KpiInputService* menggunakan *KpiVariabelStrategyContext* untuk resolusi variabel per karyawan, serta *KpiVariabelService* menggunakan *KpiDatasourceService* untuk validasi sumber data. Beberapa keputusan desain perlu disampaikan, yaitu parameter *method* dengan tipe DTO tidak ditampilkan pada *signature* untuk menjaga keterbacaan, kompartemen atribut sengaja dikosongkan karena atribut implementasi hanya berupa *dependency injection* seperti *PrismaService* yang bukan *state* domain, dan *entity* serta struktur data dipisahkan ke sub-bab 4.2.9 dalam bentuk *Entity Relationship Diagram*.

4.2.9 Entity Relationship Diagram

Sub-bab ini menyajikan rancangan *Entity Relationship Diagram* (ERD) sistem yang dikembangkan, yang digunakan untuk menggambarkan struktur *entity* dalam basis data meliputi atribut yang dimiliki setiap *entity*, *primary key*, *foreign key*, beserta relasi antar *entity*. Mengingat sistem yang dibangun terdiri dari beberapa modul fungsional dengan karakteristik domain yang berbeda, ERD pada penelitian ini dipecah menjadi lima sub-ERD berdasarkan area tanggung jawab masing-masing modul, yaitu sub-ERD pertama menyajikan struktur *entity* untuk organisasi dan *user management* yang menjadi fondasi referensi bagi modul-modul lainnya, sub-ERD kedua menampilkan struktur *entity* untuk modul KPI yang merupakan inti dari penelitian ini, sub-ERD ketiga menggambarkan struktur *entity* untuk modul autentikasi dan *permission*, sub-ERD keempat menyajikan struktur *entity* untuk modul presensi dan jadwal kerja karyawan, serta sub-ERD kelima menampilkan struktur *entity* untuk modul *approval*, izin, dan notifikasi yang menangani alur pengajuan dan pemberitahuan dalam sistem.

1. ERD Organisasi & *User Management*



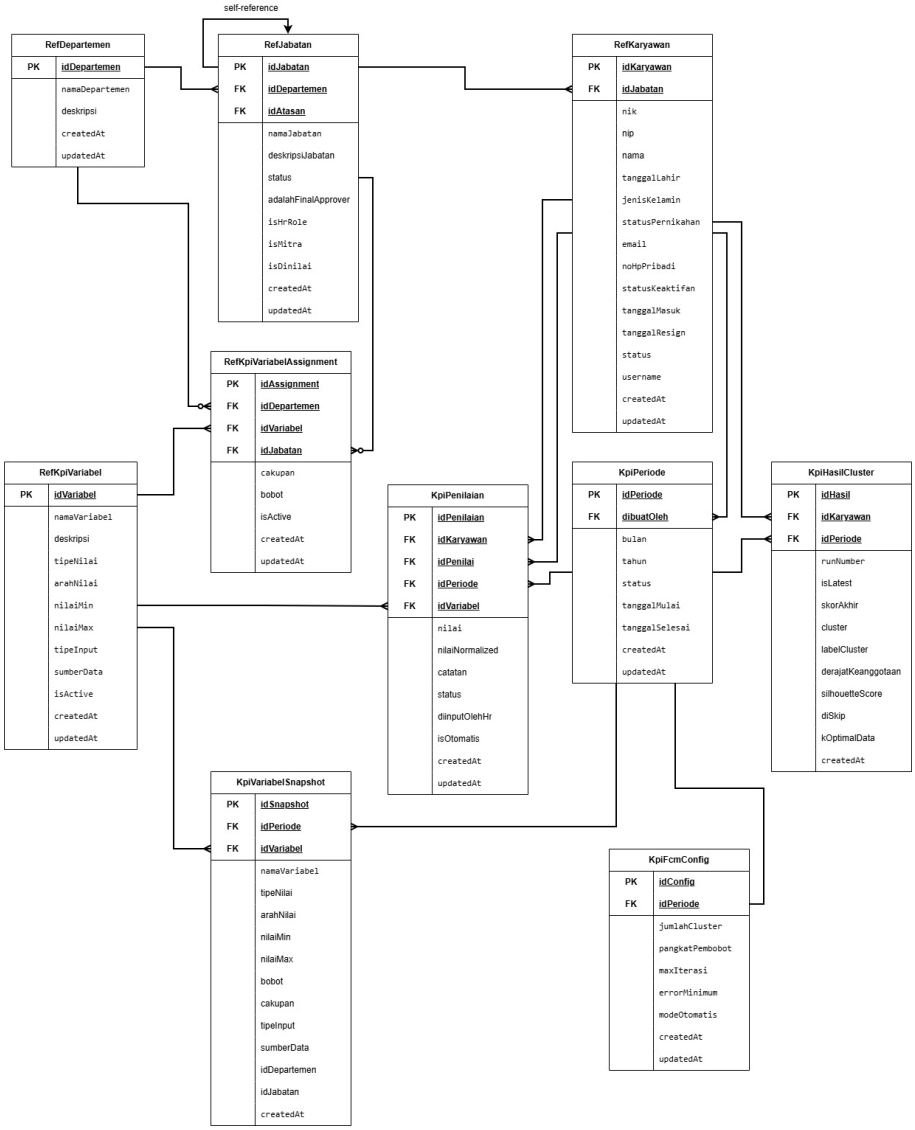
Gambar 58. ERD Organisasi & *User Management*

Sub-ERD pertama ini menggambarkan struktur *entity* yang menjadi fondasi referensi bagi seluruh modul sistem, yaitu data organisasi perusahaan dan data karyawan sebagai pengguna sistem. Sub-ERD ini disusun pertama karena ketiga *entity* di dalamnya menjadi acuan bagi hampir seluruh *entity* di sub-ERD berikutnya.

Sub-ERD ini terdiri dari tiga *entity* utama, yaitu *RefDepartemen* yang menyimpan data *master* departemen sebagai dasar pengelompokan jabatan dan *assignment* variabel KPI pada cakupan departemen, *RefJabatan* yang menyimpan data *master* jabatan dan merupakan *entity* paling krusial karena tidak hanya merepresentasikan posisi karyawan tetapi juga menjadi dasar pemilihan strategi resolusi variabel KPI melalui *flag isMitra* dan *isDinilai* serta menentukan peran HR melalui *flag isHrRole*, dan *RefKaryawan* yang menyimpan data *master* karyawan mencakup informasi identitas, jabatan yang diduduki, status keaktifan, serta atribut autentikasi untuk *login* ke sistem. Selain itu, *RefJabatan* juga memiliki relasi *self-reference* melalui atribut *idAtasan* untuk merepresentasikan hierarki atasan-bawahan dalam struktur organisasi.

Pola relasi pada sub-ERD ini mengikuti hierarki organisasi dari yang paling abstrak ke yang paling konkret, di mana satu departemen membawahi banyak jabatan, satu jabatan dapat diduduki oleh banyak karyawan, dan satu jabatan juga dapat membawahi banyak jabatan lain melalui relasi *self-reference* yang memungkinkan sistem membangun struktur organisasi berjenjang yang fleksibel tanpa batasan kedalaman hierarki.

2. ERD KPI



Gambar 59. ERD KPI

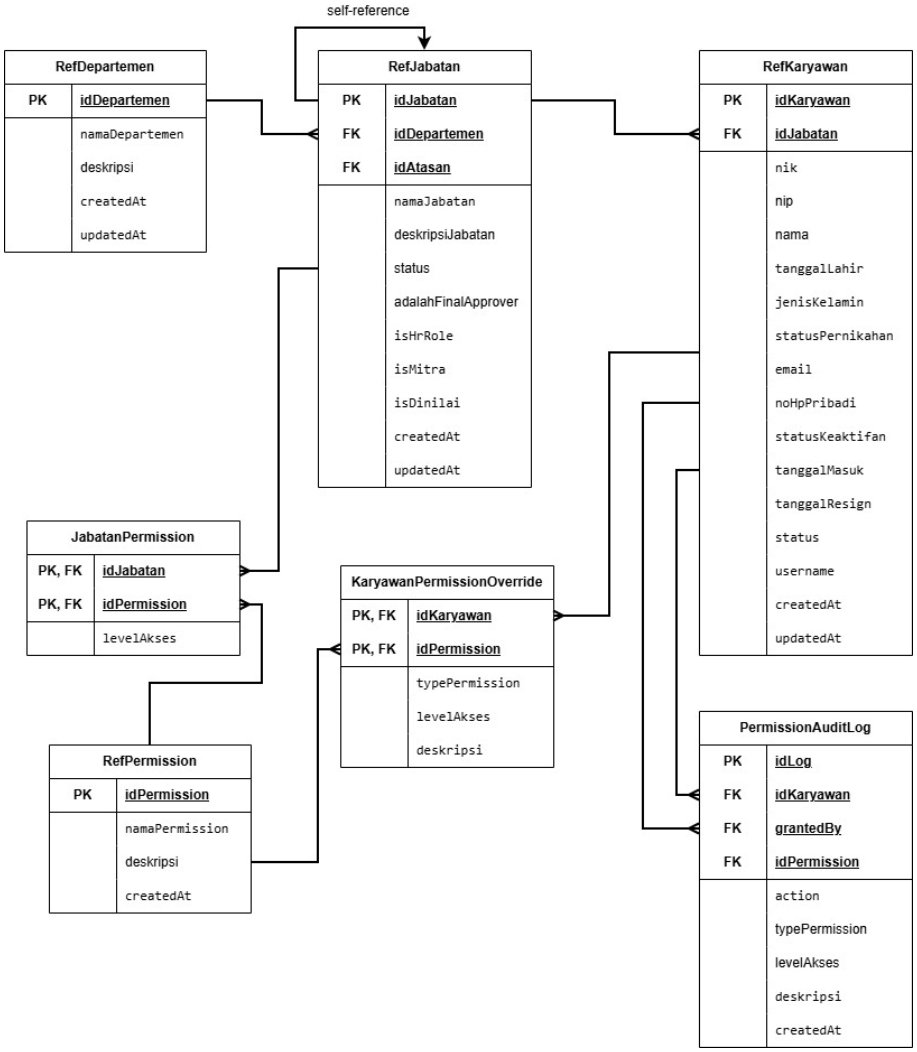
Sub-ERD kedua ini menggambarkan struktur *entity* untuk modul KPI yang merupakan inti dari penelitian skripsi ini, yang mencakup tujuh *entity* utama untuk mendukung seluruh alur penilaian KPI mulai dari definisi variabel, *assignment* variabel ke *scope* tertentu, periode penilaian, hingga hasil klasterisasi karyawan menggunakan algoritma *Fuzzy C-Means*. Pada bagian *master* variabel, *RefKpiVariabel* menyimpan *master* variabel KPI yang dapat ditambahkan dan dikonfigurasi secara dinamis oleh HR tanpa perlu mengubah kode program, di mana setiap variabel memiliki atribut tipe nilai, arah nilai, batas minimum dan maksimum, serta tipe *input* yang menentukan

apakah nilai diinput manual atau diambil secara otomatis, sedangkan *RefKpiVariabelAssignment* menjadi *entity* penghubung yang merepresentasikan implementasi tiga cakupan *assignment* (global, departemen, dan jabatan) melalui atribut cakupan beserta *idDepartemen* dan *idJabatan* yang bersifat opsional. Pada bagian periode dan konfigurasi, *KpiPeriode* menyimpan periode penilaian KPI beserta status *lifecycle* periode mulai dari *draft*, aktif, ditutup, *processing*, hingga selesai atau *error*, sedangkan *KpiFcmConfig* menyimpan parameter konfigurasi algoritma *Fuzzy C-Means* per periode yang mencakup jumlah *cluster*, pangkat pembobot, maksimum iterasi, *error* minimum, dan *flag* mode otomatis untuk pencarian *k* optimal. Pada bagian data *transactional*, *KpiPenilaian* mencatat hasil *input* penilaian per karyawan per variabel per periode beserta nilai aktual, nilai normalisasi, dan penilai yang melakukan *input*, *KpiVariabelSnapshot* menyimpan *snapshot* konfigurasi variabel pada saat periode ditutup sehingga hasil klasterisasi tetap konsisten meskipun konfigurasi *master* variabel diubah di kemudian hari (krusial untuk *reproducibility*), dan *KpiHasilCluster* menyimpan hasil klasterisasi karyawan per periode beserta *cluster*, label *cluster*, derajat keanggotaan *fuzzy*, dan nilai *Silhouette Coefficient*.

Pola relasi pada sub-ERD ini menunjukkan alur hidup data KPI yang mengalir dari konfigurasi menuju eksekusi, di mana *KpiPeriode* berperan sebagai *entity* pusat yang menjadi acuan bagi sebagian besar *entity transactional*, sehingga satu periode dapat memiliki banyak penilaian, banyak *snapshot* variabel, dan banyak hasil *cluster* sesuai jumlah karyawan yang dinilai dan jumlah *run* klasterisasi yang dilakukan. *Entity KpiPenilaian* memiliki dua relasi terpisah ke *RefKaryawan* yaitu sebagai karyawan yang dinilai dan sebagai karyawan yang menilai, di mana kedua peran tersebut dapat berbeda karena penilaian dilakukan oleh atasan atau oleh HR sebagai pengganti atasan jika diperlukan, sedangkan *KpiVariabelSnapshot* menyimpan atribut *idDepartemen* dan *idJabatan* sebagai data historis tanpa relasi

formal ke *entity* referensi agar *snapshot* tetap valid sebagai catatan historis meskipun data *master* organisasi mengalami perubahan di kemudian hari.

3. ERD *Auth* & Permission



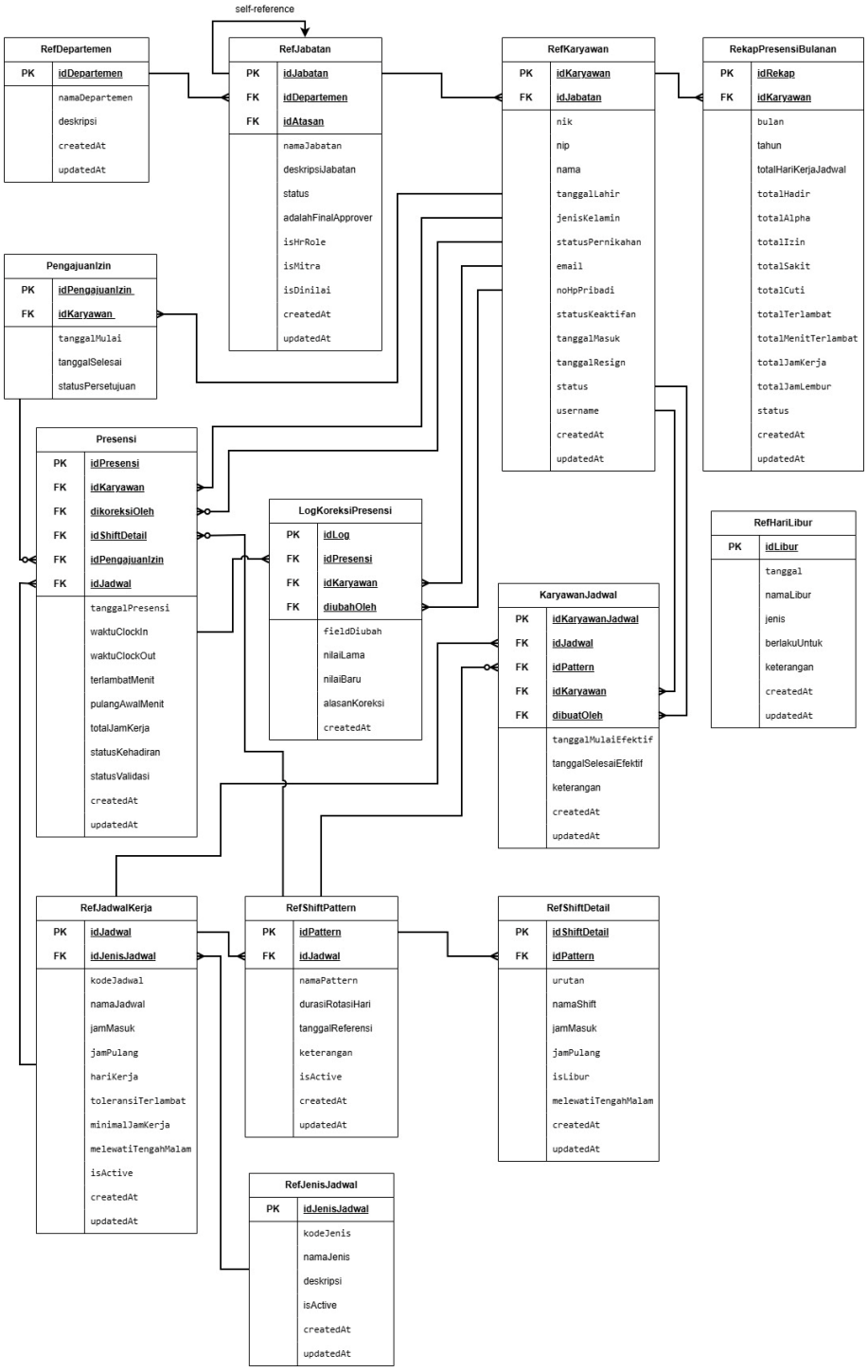
Gambar 60. ERD *Auth* & Permission

Sub-ERD ketiga ini menggambarkan struktur *entity* untuk sistem autentikasi dan otorisasi sistem, yang menerapkan pola *permission* berlapis untuk memungkinkan pengelolaan hak akses secara fleksibel baik melalui jabatan secara *default* maupun melalui *override* per individu karyawan jika diperlukan. *RefPermission* menyimpan *master permission* yang mendefinisikan jenis-jenis hak akses dalam sistem seperti hak untuk mengelola KPI, menginput penilaian, atau mengakses modul tertentu. *JabatanPermission* adalah *entity junction* yang menjadi

lapis pertama pengaturan *permission* yang mengaitkan setiap jabatan dengan *permission* yang dimilikinya beserta *level* akses berupa *bitmask* untuk operasi *read*, *write*, dan *delete*, sehingga semua karyawan yang menduduki jabatan tertentu akan secara otomatis mewarisi seluruh *permission* yang ditetapkan pada jabatan tersebut. *KaryawanPermissionOverride* adalah *entity* lapis kedua yang memungkinkan HR memberikan *permission* khusus per individu karyawan baik berupa penambahan *permission* yang tidak dimiliki oleh jabatannya maupun pencabutan *permission default* jabatan, di mana atribut *typePermission* membedakan kedua peran ini (*true* berarti *grant* dan *false* berarti *revoke*). *PermissionAuditLog* mencatat seluruh aktivitas perubahan *permission* untuk keperluan akuntabilitas dan *traceability* mencakup informasi tentang karyawan yang *permission*-nya diubah, jenis aksi yang dilakukan, *level* akses yang berubah, HR yang melakukan aksi, serta alasan perubahan.

Pola relasi pada sub-ERD ini mencerminkan arsitektur *permission* berlapis yang dianut sistem, di mana *RefPermission* berperan sebagai *master* yang menjadi acuan bagi tiga *entity transactional* yaitu *JabatanPermission*, *KaryawanPermissionOverride*, dan *PermissionAuditLog*. *Entity JabatanPermission* dan *KaryawanPermissionOverride* masing-masing menggunakan *composite primary key* sebagai mekanisme untuk menjamin tidak ada duplikasi *permission* per jabatan atau per karyawan, sedangkan *entity PermissionAuditLog* memiliki dua relasi terpisah ke *RefKaryawan* yaitu sebagai karyawan yang *permission*-nya berubah dan sebagai karyawan HR yang melakukan aksi perubahan, agar *audit log* dapat menjawab pertanyaan "siapa mengubah *permission* siapa" secara akurat.

4. ERD Presensi & Jadwal Kerja



Gambar 61. ERD Presensi & Jadwal Kerja

Sub-ERD keempat ini menggambarkan struktur *entity* untuk modul presensi dan jadwal kerja karyawan dan merupakan sub-ERD paling kompleks dalam sistem karena mencakup hierarki konfigurasi jadwal yang berjenjang, data presensi harian dengan banyak konteks, serta

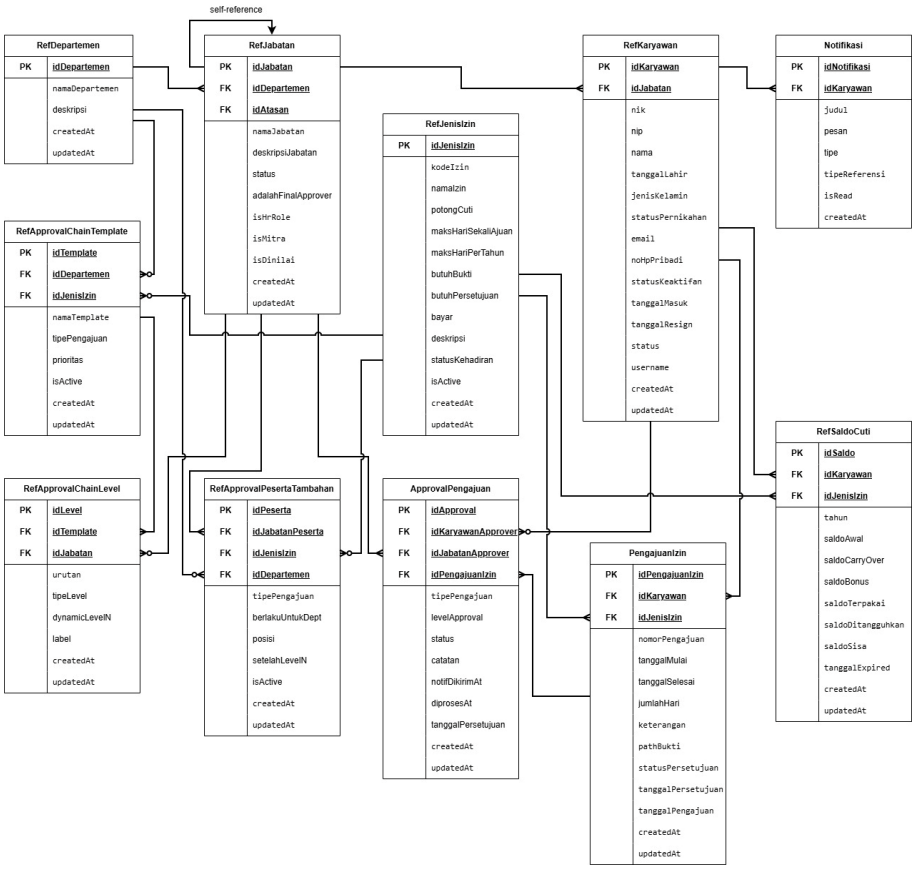
mekanisme koreksi dan rekapitulasi presensi bulanan. Pada bagian *master* jadwal, *RefJenisJadwal* menyimpan klasifikasi jenis jadwal yang didukung sistem yaitu reguler, *shift*, *flexible*, *project*, dan *hybrid*, *RefJadwalKerja* menyimpan detail konfigurasi jadwal seperti jam masuk, jam pulang, hari kerja, toleransi keterlambatan, dan minimal jam kerja, sedangkan *RefShiftPattern* dan *RefShiftDetail* membentuk hierarki tambahan untuk jadwal tipe *shift* di mana *pattern* mendefinisikan durasi rotasi *shift* dan *detail* mendefinisikan urutan *shift* dalam satu siklus rotasi. *RefHariLibur* adalah *entity standalone* yang menyimpan kalender hari libur tanpa relasi formal dengan *entity* lain karena dipakai oleh *business logic* saat menghitung presensi.

Pada bagian *assignment* dan presensi, *KaryawanJadwal* adalah *entity junction* yang mengaitkan karyawan dengan jadwal kerja yang berlaku untuknya beserta tanggal mulai dan tanggal selesai efektif, sedangkan *Presensi* adalah *entity* inti yang mencatat presensi harian karyawan beserta informasi waktu *clock in* dan *clock out*, total jam kerja, keterlambatan, status kehadiran, sumber presensi, lokasi GPS, foto, serta status validasi. Pada bagian audit dan rekapitulasi, *LogKoreksiPresensi* mencatat seluruh perubahan data presensi untuk keperluan jejak audit mencakup *field* yang diubah, nilai lama, nilai baru, alasan koreksi, dan HR yang melakukan perubahan, sedangkan *RekapPresensiBulanan* menyimpan ringkasan presensi per karyawan per bulan untuk keperluan pelaporan dan integrasi ke modul lain di masa mendatang.

Pola relasi pada sub-ERD ini menunjukkan dua aliran utama, yaitu aliran konfigurasi jadwal yang berjenjang dari *RefJenisJadwal* menuju *RefJadwalKerja*, kemudian menuju *RefShiftPattern* dan *RefShiftDetail* untuk jadwal tipe *shift*, dan akhirnya teraplikasikan ke karyawan melalui *KaryawanJadwal*, serta aliran pencatatan presensi yang berpusat pada *entity Presensi* dengan banyak relasi opsional ke jadwal yang berlaku, *shift detail* jika *applicable*, dan pengajuan izin jika presensi tersebut terkait dengan izin yang disetujui. Beberapa *entity*

dalam sub-ERD ini memiliki dua relasi terpisah ke *RefKaryawan* yang merepresentasikan dua peran berbeda, yaitu *KaryawanJadwal* memiliki relasi sebagai karyawan yang di-*assign* jadwal dan sebagai HR yang membuat penjadwalan, *Presensi* memiliki relasi sebagai karyawan pemilik presensi dan sebagai HR yang melakukan koreksi presensi, sedangkan *LogKoreksiPresensi* memiliki relasi sebagai karyawan yang presensi-nya dikoreksi dan sebagai HR yang melakukan koreksi.

5. ERD *Approval*, Izin & Notifikasi



Gambar 62. ERD *Approval*, Izin & Notifikasi

Sub-ERD kelima ini menggambarkan struktur *entity* untuk modul *approval*, izin, dan notifikasi sistem yang menerapkan pola *approval chain* yang konfiguratif, di mana HR dapat mengatur alur persetujuan secara dinamis sesuai dengan kebutuhan organisasi tanpa perlu mengubah kode program. Pada bagian *master* izin dan saldo, *RefJenisIzin* menyimpan klasifikasi jenis izin yang didukung sistem beserta aturan-aturannya seperti maksimum hari per ajuan, kewajiban

upload bukti, dan tipe pembayaran, sedangkan *RefSaldoCuti* menyimpan saldo cuti per karyawan per jenis izin per tahun mencakup saldo awal, *carry over* dari tahun sebelumnya, bonus, saldo terpakai, dan saldo sisa. Pada bagian pengajuan, *PengajuanIzin* menyimpan data pengajuan izin oleh karyawan beserta tanggal mulai, tanggal selesai, jumlah hari, keterangan, dan status persetujuan, di mana setiap pengajuan memiliki nomor pengajuan unik yang dapat dilacak melalui sistem.

Pada bagian konfigurasi *approval chain*, *RefApprovalChainTemplate* menyimpan *template chain approval* per tipe pengajuan yang dapat dispesifikasikan untuk jenis izin tertentu atau departemen tertentu, *RefApprovalChainLevel* menyimpan *level-level approval* dalam suatu *template* beserta tipe *level* yang menentukan apakah *approver* adalah jabatan spesifik, *role* HR, atau jabatan dinamis berdasarkan hierarki bawahan-atasan, sedangkan *RefApprovalPesertaTambahan* menyimpan peserta *approval* tambahan di luar *chain template default* yang dapat ditempatkan sebelum *chain*, sesudah *chain*, atau setelah *level* tertentu. Pada bagian eksekusi *approval*, *ApprovalPengajuan* mencatat status *approval* per *level* untuk setiap pengajuan mencakup *level approval*, jabatan *approver*, karyawan *approver* yang melakukan aksi, status, catatan, serta waktu pemrosesan, sehingga satu pengajuan akan menghasilkan beberapa *record approval* sesuai jumlah *level* dalam *chain* yang berlaku. *Notifikasi* menyimpan pemberitahuan yang ditujukan kepada karyawan mencakup judul, pesan, tipe notifikasi, serta status sudah dibaca atau belum, dan berperan sebagai penghubung antar modul karena notifikasi dapat dipicu oleh berbagai aktivitas sistem seperti pengajuan izin yang membutuhkan *approval*, hasil klasterisasi KPI yang sudah selesai diproses, atau perubahan *permission* yang melibatkan karyawan.

Pola relasi pada sub-ERD ini menunjukkan arsitektur *approval chain* yang fleksibel, di mana *RefApprovalChainTemplate* menjadi *entity* konfigurasi utama yang memiliki banyak *level* dan dapat

dilengkapi peserta tambahan. Saat pengajuan diajukan oleh karyawan, sistem akan membaca *template* yang sesuai dan menghasilkan beberapa *record ApprovalPengajuan* sesuai dengan *level* dalam *template*, di mana setiap *record approval* merujuk pada jabatan *approver* yang menentukan siapa yang berhak menyetujui pada *level* tersebut dan opsional menyimpan karyawan *approver* spesifik yang melakukan aksi persetujuan atau penolakan. Selain itu, *RefSaldoCuti* memiliki relasi ganda ke *RefKaryawan* dan *RefJenisIzin* dengan kombinasi *unique constraint* yang mencakup tahun, sehingga setiap karyawan hanya memiliki satu *record* saldo per jenis izin per tahun untuk memungkinkan sistem melacak saldo cuti secara akurat sekaligus memudahkan perhitungan saldo lintas tahun.

4.3 Implementasi

Sub-bab ini menjelaskan implementasi sistem yang telah dirancang pada sub-bab sebelumnya, mencakup penerapan pola desain *Strategy* dan *Decorator* yang menjadi inti arsitektur sistem, pra-pemrosesan data sebelum klasterisasi, implementasi algoritma *Fuzzy C-Means* beserta komponen pendukungnya, serta perhitungan *Silhouette Coefficient* sebagai metrik validasi kualitas klasterisasi. Implementasi sistem dibangun menggunakan *framework* NestJS pada sisi *backend* dengan bahasa pemrograman *TypeScript*, di mana arsitektur *dependency injection* yang dianut oleh *framework* digunakan secara konsisten untuk mengelola komposisi antar komponen, sedangkan basis data yang digunakan adalah PostgreSQL dengan ORM Prisma sebagai abstraksi akses data, dan pemrosesan klasterisasi yang berpotensi memakan waktu dijalankan secara *asynchronous* menggunakan *Bull Queue* agar tidak menghalangi alur permintaan utama dari pengguna.

4.3.1 Implementasi *Strategy Pattern*

Sub-bab ini menjelaskan implementasi *Strategy Pattern* pada sistem yang telah dirancang pada sub-bab 4.2.8. Pola ini diterapkan untuk memilih strategi resolusi variabel KPI per karyawan secara dinamis pada saat *runtime*, di mana implementasi dilakukan menggunakan *framework* NestJS dengan bahasa *TypeScript*.

1. *Interface IKpiVariabelStrategy*

Implementasi diawali dengan mendefinisikan *interface IKpiVariabelStrategy* yang menjadi kontrak bagi seluruh *concrete strategy*. *Interface* ini hanya berisi satu *method* utama, yaitu *resolve()*, yang menerima *idKaryawan* dan *idPeriode* sebagai parameter dan mengembalikan hasil resolusi variabel dalam tipe *VariabelResolveResult*.

```
// Filename: src/kpi-input/strategies/kpi-variabel-
strategy.interface.ts
export interface VariabelResolveItem {
  idAssignment: string;
  idVariabel: string;
  namaVariabel: string;
  tipeNilai: string;
  arahNilai: string;
  nilaiMin: number;
  nilaiMax: number;
  bobot: number;
  cakupan: string;
  tipeInput: string;
  sumberData: string | null;
  isOtomatis: boolean;
  nilaiExisting: number | null;
  catatanExisting: string | null;
}

export interface VariabelResolveResult {
  karyawan: {
    idKaryawan: string;
    nama: string;
    namaJabatan: string;
    namaDepartemen: string;
  };
  variabel: VariabelResolveItem[];
  totalBobot: number;
  statusPenilaian: string;
}

export interface IKpiVariabelStrategy {
  resolve(
    idKaryawan: string,
    idPeriode: string,
  ): Promise<VariabelResolveResult>;
}
```

Tipe *VariabelResolveResult* yang menjadi keluaran *method* berisi tiga komponen utama, yaitu data karyawan yang dinilai, daftar variabel KPI beserta atributnya, total bobot variabel, serta status penilaian. *Interface* ini memberikan dua manfaat utama. Pertama, kontrak yang seragam menjamin bahwa setiap *concrete strategy* dapat dipanggil dengan cara yang sama oleh *context*. Kedua, penambahan *strategy* baru di masa mendatang dapat dilakukan dengan mengimplementasikan *interface* ini tanpa perlu mengubah *class* yang sudah ada.

2. *Concrete Strategy DefaultStrategy*

DefaultStrategy digunakan untuk karyawan yang berasal dari jabatan tanpa *assignment* variabel pada cakupan jabatan. Strategi ini melakukan resolusi pada dua *layer*, yaitu *layer* global dan *layer* departemen.

```
// Filename: src/kpi-input/strategies/default.strategy.ts
@Injectable()
export class DefaultStrategy implements IKpiVariabelStrategy {
  constructor(private prisma: PrismaService) {}

  async resolve(
    idKaryawan: string,
    idPeriode: string,
  ): Promise<VariabelResolveResult> {
    // Ambil data karyawan
    const karyawan = await this.prisma.refKaryawan.findUnique({
      where: { idKaryawan },
      select: {
        idKaryawan: true,
        nama: true,
        jabatan: {
          select: {
            idJabatan: true,
            namaJabatan: true,
            idDepartemen: true,
            departemen: {
              select: { namaDepartemen: true },
            },
          },
        },
      },
    });

    if (!karyawan) {
```

```
      throw new NotFoundException('Karyawan tidak ditemukan');
    }

    // Decorator resolve dengan idJabatan = null
    // → Layer jabatan tidak diproses
    const variabel = await resolveVariabelDecorator(
      this.prisma,
      null,
      karyawan.jabatan.idDepartemen,
      idPeriode,
      idKaryawan,
    );

    // Hitung total bobot
    const totalBobot = variabel.reduce((sum, v) => sum + v.bobot, 0);

    // Tentukan status penilaian
    const idVariabelList = variabel.map((v) => v.idVariabel);

    const penilaians = await this.prisma.kpiPenilaian.findMany({
      where: {
        idPeriode,
        idKaryawan,
        idVariabel: { in: idVariabelList },
      },
      select: { status: true },
    });

    let statusPenilaian = 'belum_diisi';

    if (penilaians.length > 0) {
      const semuaSubmitted = penilaians.every((p) => p.status ===
'submitted');
      statusPenilaian = semuaSubmitted ? 'submitted' : 'draft';
    }

    return {
      karyawan: {
        idKaryawan: karyawan.idKaryawan,
        nama: karyawan.nama,
        namaJabatan: karyawan.jabatan.namaJabatan,
        namaDepartemen:
karyawan.jabatan.departemen.namaDepartemen,
      },
      variabel,
      totalBobot,
      statusPenilaian,
    };
```

```
  }
}
```

Strategi ini memanggil fungsi *resolveVariabelDecorator* dengan parameter *idJabatan* bernilai *null*, sehingga *layer* jabatan tidak diproses. *Layer* yang aktif adalah *layer* global yang berlaku untuk seluruh karyawan dan *layer* departemen yang spesifik per departemen karyawan. Setelah memperoleh daftar variabel, *strategy* menghitung total bobot sebagai indikator kesesuaian konfigurasi, kemudian menentukan status penilaian berdasarkan keberadaan dan status *record* pada tabel *KpiPenilaian*.

3. *Concrete Strategy JobSpecificStrategy*

JobSpecificStrategy digunakan untuk karyawan yang berasal dari jabatan dengan *assignment* variabel pada cakupan jabatan. Strategi ini melakukan resolusi pada tiga *layer* secara penuh, yaitu *layer* global, *layer* departemen, dan *layer* jabatan.

```
// Filename: src/kpi-input/strategies/job-specific.strategy.ts
@Injectable()
export class JobSpecificStrategy implements IKpiVariabelStrategy {
  constructor(private prisma: PrismaService) {}

  async resolve(
    idKaryawan: string,
    idPeriode: string,
  ): Promise<VariabelResolveResult> {
    // Ambil data karyawan
    const karyawan = await this.prisma.refKaryawan.findUnique({
      where: { idKaryawan },
      select: {
        idKaryawan: true,
        nama: true,
        jabatan: {
          select: {
            idJabatan: true,
            namaJabatan: true,
            idDepartemen: true,
            departemen: {
              select: { namaDepartemen: true },
            },
          },
        },
      },
```

```
    });

    if (!karyawan) {
      throw new NotFoundException('Karyawan tidak ditemukan');
    }

    // Decorator resolve 3 layer
    // idJabatan di-pass → Layer 3 jabatan aktif
    const variabel = await resolveVariabelDecorator(
      this.prisma,
      karyawan.jabatan.idJabatan,
      karyawan.jabatan.idDepartemen,
      idPeriode,
      idKaryawan,
    );

    // Hitung total bobot
    const totalBobot = variabel.reduce((sum, v) => sum + v.bobot, 0);

    // Tentukan status penilaian
    const idVariabelList = variabel.map((v) => v.idVariabel);

    const penilaians = await this.prisma.kpiPenilaian.findMany({
      where: {
        idPeriode,
        idKaryawan,
        idVariabel: { in: idVariabelList },
      },
      select: { status: true },
    });

    let statusPenilaian = 'belum_diisi';

    if (penilaians.length > 0) {
      const semuaSubmitted = penilaians.every((p) => p.status ===
'submitted');
      statusPenilaian = semuaSubmitted ? 'submitted' : 'draft';
    }

    return {
      karyawan: {
        idKaryawan: karyawan.idKaryawan,
        nama: karyawan.nama,
        namaJabatan: karyawan.jabatan.namaJabatan,
        namaDepartemen:
karyawan.jabatan.departemen.namaDepartemen,
      },
      variabel,
```

```
      totalBobot,
      statusPenilaian,
    };
  }
}
```

Strategi ini memanggil fungsi *resolveVariabelDecorator* dengan parameter *idJabatan* yang valid sehingga ketiga *layer* aktif. *Layer* global, *layer* departemen, dan *layer* jabatan akan diakumulasikan dan dideduplikasi berdasarkan prioritas, di mana variabel yang muncul pada *layer* jabatan akan menggantikan variabel dengan nama yang sama pada *layer* departemen, dan variabel pada *layer* departemen akan menggantikan variabel pada *layer* global. Mekanisme deduplikasi ini diterapkan oleh fungsi *decorator* yang akan dijelaskan pada sub-bab 4.3.2.

4. *Concrete Strategy MitraStrategy*

MitraStrategy digunakan secara khusus untuk karyawan dari jabatan dengan *flag isMitra* aktif, yang dalam konteks studi kasus penelitian ini adalah karyawan dengan jabatan *Driver* di PT Arete Logistik Indonesia. Strategi ini melakukan resolusi hanya pada *layer* jabatan, melewati *layer* global dan *layer* departemen, karena karyawan mitra dievaluasi dengan parameter operasional yang sepenuhnya berbeda dari karyawan tetap.

```
// Filename: src/kpi-input/strategies/mitra.strategy.ts
@Injectable()
export class MitraStrategy implements IKpiVariabelStrategy {
  constructor(private prisma: PrismaService) {}

  async resolve(
    idKaryawan: string,
    idPeriode: string,
  ): Promise<VariabelResolveResult> {
    // Ambil data karyawan
    const karyawan = await this.prisma.refKaryawan.findUnique({
      where: { idKaryawan },
      select: {
        idKaryawan: true,
        nama: true,
        jabatan: {
          select: {
```

```
          idJabatan: true,
          namaJabatan: true,
          idDepartemen: true,
          departemen: {
            select: { namaDepartemen: true },
          },
        },
      },
    },
  });

  if (!karyawan) {
    throw new NotFoundException('Karyawan tidak ditemukan');
  }

  // Decorator resolve 1 layer (jabatan saja)
  // skipGlobal = true → layer global dilewati
  // skipDepartemen = true → layer departemen dilewati
  const variabel = await resolveVariabelDecorator(
    this.prisma,
    karyawan.jabatan.idJabatan,
    karyawan.jabatan.idDepartemen,
    idPeriode,
    idKaryawan,
    true, // skipGlobal
    true, // skipDepartemen
  );

  // Hitung total bobot
  const totalBobot = variabel.reduce((sum, v) => sum + v.bobot, 0);

  // Tentukan status penilaian
  const idVariabelList = variabel.map((v) => v.idVariabel);

  const penilaians = await this.prisma.kpiPenilaian.findMany({
    where: {
      idPeriode,
      idKaryawan,
      idVariabel: { in: idVariabelList },
    },
    select: { status: true },
  });

  let statusPenilaian = 'belum_diisi';

  if (penilaians.length > 0) {
    const semuaSubmitted = penilaians.every((p) => p.status ===
'submitted');
```

```
      statusPenilaian = semuaSubmitted ? 'submitted' : 'draft';
    }

    return {
      karyawan: {
        idKaryawan: karyawan.idKaryawan,
        nama: karyawan.nama,
        namaJabatan: karyawan.jabatan.namaJabatan,
        namaDepartemen:
karyawan.jabatan.departemen.namaDepartemen,
      },
      variabel,
      totalBobot,
      statusPenilaian,
    };
  }
}
```

Inti dari *MitraStrategy* terletak pada pemanggilan fungsi *resolveVariabelDecorator* dengan kombinasi argumen *skipGlobal* dan *skipDepartemen* yang keduanya bernilai *true*. Kombinasi ini memastikan bahwa hanya variabel pada cakupan jabatan yang akan di-*resolve*, di mana hal ini sesuai dengan karakteristik kerja karyawan mitra yang tidak memiliki keterikatan dengan KPI global maupun KPI departemen.

5. Pola Bersama Ketiga *Concrete Strategy*

Ketiga *concrete strategy* mengikuti struktur implementasi yang serupa, yaitu mengambil data karyawan beserta jabatan dan departemennya, memanggil fungsi *resolveVariabelDecorator* dengan kombinasi parameter yang spesifik untuk masing-masing *strategy*, menghitung total bobot dari seluruh variabel yang teresolusi, kemudian menentukan status penilaian dengan tiga nilai yaitu *belum_diisi*, *draft*, dan *submitted* berdasarkan keberadaan dan status *record* pada tabel *KpiPenilaian*. Status *submitted* ditandai ketika seluruh variabel telah memiliki *record* dengan status *submitted*, status *draft* ditandai ketika sebagian variabel masih dalam status *draft*, dan status *belum_diisi* ditandai ketika belum ada *record* sama sekali untuk periode dan karyawan yang bersangkutan.

Tabel berikut menyajikan perbandingan kombinasi parameter yang diteruskan ke fungsi *decorator* oleh ketiga *strategy*.

Tabel 12. perbandingan kombinasi parameter

Strategy	idJabatan	*skipGlobal*	skipDepartemen	Layer Aktif
DefaultStrategy	*null*	*false*	*false*	*Global* + Departemen
JobSpecific Strategy	*valid*	*false*	*false*	*Global* + Departemen + Jabatan
MitraStrategy	*valid*	*true*	*true*	Jabatan saja

Perbedaan kombinasi parameter inilah yang membedakan perilaku masing-masing strategi dalam meresolusi variabel KPI per karyawan.

6. *Context KpiVariabelStrategyContext*

Pemilihan strategy yang tepat dilakukan oleh *KpiVariabelStrategyContext* yang menerapkan logika pemilihan dua tingkat berdasarkan karakteristik jabatan karyawan.

```
// Filename: src/kpi-input/strategies/kpi-variabel-strategy.context.ts
@Injectable()
export class KpiVariabelStrategyContext {
  constructor(
    private prisma: PrismaService,
    private defaultStrategy: DefaultStrategy,
    private jobSpecificStrategy: JobSpecificStrategy,
    private mitraStrategy: MitraStrategy,
  ) {}

  private async selectStrategy(
    idJabatan: string,
    isMitra: boolean,
  ): Promise<IKpiVariabelStrategy> {
    // Pemeriksaan 1: Flag mitra
    if (isMitra) {
      return this.mitraStrategy;
    }

    // Pemeriksaan 2: Keberadaan assignment cakupan jabatan
    const assignmentJabatan = await
this.prisma.refKpiVariabelAssignment.count({
      where: {
        isActive: true,
        cakupan: 'jabatan',
        idJabatan,
      },
```

```
    });

    if (assignmentJabatan > 0) {
      return this.jobSpecificStrategy;
    }

    // Default: 2 layer (global + departemen)
    return this.defaultStrategy;
  }

  async resolve(
    idKaryawan: string,
    idPeriode: string,
  ): Promise<VariabelResolveResult> {
    // Ambil idJabatan + isMitra karyawan untuk selector
    const karyawan = await this.prisma.refKaryawan.findUnique({
      where: { idKaryawan },
      select: {
        jabatan: {
          select: {
            idJabatan: true,
            isMitra: true,
          },
        },
      },
    });

    if (!karyawan) {
      return this.defaultStrategy.resolve(idKaryawan, idPeriode);
    }

    // Pilih strategy secara dinamis berdasarkan jabatan & flag mitra
    const strategy = await this.selectStrategy(
      karyawan.jabatan.idJabatan,
      karyawan.jabatan.isMitra,
    );

    return strategy.resolve(idKaryawan, idPeriode);
  }
}
```

Tingkat pertama pemeriksaan pada *method selectStrategy()* memeriksa apakah karyawan berasal dari jabatan dengan *flag isMitra* aktif, di mana karyawan mitra akan langsung diarahkan ke *MitraStrategy*, sedangkan tingkat kedua memeriksa keberadaan *assignment* dengan cakupan jabatan untuk jabatan karyawan tersebut di basis data, di mana keberadaan minimal satu *assignment* aktif akan

mengarahkan ke *JobSpecificStrategy* sedangkan ketiadaan *assignment* jabatan akan mengarahkan ke *DefaultStrategy*. Pendekatan ini memiliki keunggulan dalam hal adaptivitas karena tidak ada *hardcode* nama jabatan dalam logika pemilihan *strategy*, sehingga sistem secara otomatis menyesuaikan *strategy* yang dipakai berdasarkan konfigurasi data di basis data, di mana ketika HR menambahkan *assignment* cakupan jabatan untuk jabatan tertentu, sistem akan langsung beralih menggunakan *JobSpecificStrategy* tanpa perlu intervensi kode atau konfigurasi tambahan.

4.3.2 Implementasi *Decorator Pattern*

Sub-bab ini menjelaskan implementasi *Decorator Pattern* pada sistem yang telah dirancang pada sub-bab 4.2.8, di mana pola ini diterapkan untuk menggabungkan resolusi variabel KPI dari tiga *layer* cakupan yaitu global, departemen, dan jabatan, dengan setiap *layer* dapat ditambah atau dilewati secara fleksibel sesuai kebutuhan *strategy* yang memanggilnya. Berbeda dengan implementasi klasik *Decorator Pattern* yang menggunakan *class wrapper* berlapis, *decorator* pada sistem ini diimplementasikan sebagai sebuah fungsi modular bernama *resolveVariabelDecorator* yang menerima parameter untuk mengontrol *layer* mana yang aktif, di mana pendekatan fungsional ini dipilih karena *Decorator Pattern* pada konteks ini lebih cocok dijalankan sebagai *pipeline* pengayaan data dan lebih sesuai dengan paradigma fungsional yang umum digunakan pada ekosistem *TypeScript* dan NestJS.

1. Fungsi *resolveVariabelDecorator*

   Fungsi *resolveVariabelDecorator* menerima tujuh parameter yang menentukan perilaku resolusi.

```
// Filename: src/kpi-input/strategies/kpi-variabel-decorator.ts
export async function resolveVariabelDecorator(
  prisma: PrismaService,
  idJabatan: string | null,
  idDepartemen: string,
  idPeriode: string,
  idKaryawan: string,
  skipGlobal = false,
  skipDepartemen = false,
```

```
): Promise<VariabelResolveItem[]>
```

Parameter *idJabatan* bertipe *string* | *null*, di mana nilai *null* menandakan bahwa *layer* jabatan tidak diaktifkan. Parameter *skipGlobal* dan *skipDepartemen* adalah parameter *boolean* opsional dengan nilai *default false*, yang memungkinkan pemanggil untuk melewati *layer* global dan *layer* departemen sesuai kebutuhan. Kombinasi parameter inilah yang memungkinkan ketiga *concrete strategy* memanfaatkan satu fungsi *decorator* yang sama dengan perilaku yang berbeda-beda.

2. *Layer* 1: Cakupan *Global*

Layer pertama mengambil seluruh *assignment* variabel dengan cakupan *global* yang berlaku untuk seluruh karyawan tanpa memandang departemen dan jabatan.

```
const layerGlobal = skipGlobal
 ? []
 : await prisma.refKpiVariabelAssignment.findMany({
    where: { isActive: true, cakupan: 'global' },
    include: {
     variabel: {
      select: {
       idVariabel: true,
       namaVariabel: true,
       tipeNilai: true,
       arahNilai: true,
       nilaiMin: true,
       nilaiMax: true,
       tipeInput: true,
       sumberData: true,
      },
     },
    },
   });
```

Layer ini akan dilewati ketika parameter *skipGlobal* bernilai *true*, di mana pada kasus tersebut variabel *layerGlobal* akan diisi *array* kosong. Pelewatan *layer* ini dilakukan oleh *MitraStrategy* yang memerlukan resolusi pada *layer* jabatan saja.

3. *Layer* 2: Cakupan Departemen

Layer kedua mengambil *assignment* variabel dengan cakupan departemen yang spesifik untuk departemen karyawan yang dinilai.

```
const layerDepartemen = skipDepartemen
 ? []
 : await prisma.refKpiVariabelAssignment.findMany({
    where: { isActive: true, cakupan: 'departemen', idDepartemen },
    include: {
     variabel: {
      select: {
       idVariabel: true,
       namaVariabel: true,
       tipeNilai: true,
       arahNilai: true,
       nilaiMin: true,
       nilaiMax: true,
       tipeInput: true,
       sumberData: true,
      },
     },
    },
   });
```

Sama halnya dengan *layer* global, *layer* departemen akan dilewati ketika parameter *skipDepartemen* bernilai *true*. *Layer* ini hanya mengambil *assignment* dengan *idDepartemen* yang sesuai dengan departemen karyawan, sehingga setiap karyawan hanya menerima variabel KPI yang relevan dengan departemennya.

4. *Layer* 3: Cakupan Jabatan

Layer ketiga mengambil *assignment* variabel dengan cakupan jabatan yang spesifik untuk jabatan karyawan yang dinilai.

```
const layerJabatan = idJabatan
 ? await prisma.refKpiVariabelAssignment.findMany({
    where: { isActive: true, cakupan: 'jabatan', idJabatan },
    include: {
     variabel: {
      select: {
       idVariabel: true,
       namaVariabel: true,
       tipeNilai: true,
       arahNilai: true,
       nilaiMin: true,
       nilaiMax: true,
```

```
          tipeInput: true,
          sumberData: true,
        },
      },
    },
  })
: [];
```

Berbeda dengan *layer* global dan departemen yang dikontrol dengan parameter *skip*, *layer* jabatan dikontrol melalui parameter *idJabatan*. Ketika *idJabatan* bernilai *null*, *layer* ini tidak diproses. Mekanisme ini memungkinkan *DefaultStrategy* melewati *layer* jabatan dengan cara meneruskan nilai *null* pada parameter ini.

5. Mekanisme Penggabungan *Layer*

Setelah ketiga *layer* di-*resolve*, hasil dari masing-masing *layer* digabungkan dengan menerapkan prioritas jabatan lebih tinggi dari departemen, dan departemen lebih tinggi dari global. Penggabungan dilakukan menggunakan struktur data *Map* dengan *idVariabel* sebagai *key*.

```
const variabelMap = new Map<string, (typeof layerGlobal)[0]>();

// Layer 1: Global (prioritas terendah)
for (const a of layerGlobal) {
  variabelMap.set(a.idVariabel, a);
}

// Layer 2: Departemen (override global)
for (const a of layerDepartemen) {
  variabelMap.set(a.idVariabel, a);
}

// Layer 3: Jabatan (prioritas tertinggi, override semua)
for (const a of layerJabatan) {
  variabelMap.set(a.idVariabel, a);
}

const semuaAssignment = Array.from(variabelMap.values());
```

Penggunaan *Map* memanfaatkan sifat natural dari struktur data ini, yaitu setiap *key* yang di-*set* ulang akan menggantikan nilai sebelumnya, di mana dengan urutan iterasi dari *layer* prioritas terendah ke tertinggi, variabel yang muncul pada *layer* global akan otomatis tergantikan oleh

layer departemen jika nama variabelnya sama, dan *layer* departemen akan tergantikan oleh *layer* jabatan dengan mekanisme yang sama, sehingga menghasilkan daftar variabel unik dengan konfigurasi dari *layer* paling spesifik yang berlaku untuk karyawan tersebut. Implementasi fungsional ini menghasilkan satu fungsi yang dapat dipanggil oleh ketiga *concrete strategy* dengan kombinasi parameter yang berbeda, yaitu *DefaultStrategy* memanggil dengan *idJabatan* bernilai *null* tanpa parameter *skip*, *JobSpecificStrategy* memanggil dengan *idJabatan* valid tanpa parameter *skip*, dan *MitraStrategy* memanggil dengan *idJabatan* valid beserta *skipGlobal* dan *skipDepartemen* bernilai *true*, sehingga satu fungsi *decorator* berhasil menangani tiga kombinasi resolusi yang berbeda tanpa duplikasi logika sebagai inti manfaat dari pola *Decorator* yang diimplementasikan secara fungsional.

4.3.3 Pra-pemrosesan Data

Sub-bab ini menjelaskan implementasi pra-pemrosesan data sebelum data diteruskan ke algoritma *Fuzzy C-Means* untuk klasterisasi, yang diperlukan karena algoritma membutuhkan data dalam rentang yang seragam untuk mencegah variabel dengan skala besar mendominasi perhitungan, sekaligus mengakomodasi karakteristik variabel KPI yang memiliki arah nilai berbeda-beda di mana sebagian variabel lebih baik bernilai tinggi dan sebagian lainnya lebih baik bernilai rendah. Pra-pemrosesan pada sistem ini terdiri dari tiga langkah utama yang dijalankan secara berurutan, yaitu pembuatan *snapshot* konfigurasi variabel, normalisasi nilai dengan metode *min-max* yang memperhatikan arah nilai, dan perhitungan skor akhir berbobot per karyawan, di mana ketiga langkah ini diorkestrasi oleh *method prosesFcm()* pada *FcmService* yang dipanggil secara *asynchronous* oleh *Bull Queue Worker* sehingga proses tidak menghalangi respons sistem terhadap permintaan pengguna lain.

1. Pembuatan *Snapshot* Variabel

Langkah pertama pra-pemrosesan adalah membuat *snapshot* konfigurasi seluruh variabel KPI aktif yang akan digunakan untuk periode tersebut. *Snapshot* ini disimpan pada tabel *KpiVariabelSnapshot* dan menjadi sumber referensi konfigurasi yang konsisten sepanjang proses klasterisasi.

```
// Filename: src/kpi-fcm/fcm.service.ts
private async buatSnapshot(idPeriode: string): Promise<void> {
  const assignments = await
this.prisma.refKpiVariabelAssignment.findMany({
    where: { isActive: true },
    include: {
      variabel: {
        select: {
          namaVariabel: true,
          tipeNilai: true,
          arahNilai: true,
          nilaiMin: true,
          nilaiMax: true,
        },
      },
    },
  });

  if (assignments.length === 0) return;

  await this.prisma.kpiVariabelSnapshot.createMany({
    data: assignments.map((a) => ({
      idPeriode,
      idVariabel: a.idVariabel,
      namaVariabel: a.variabel.namaVariabel,
      tipeNilai: a.variabel.tipeNilai,
      arahNilai: a.variabel.arahNilai,
      nilaiMin: a.variabel.nilaiMin,
      nilaiMax: a.variabel.nilaiMax,
      bobot: a.bobot,
      cakupan: a.cakupan,
      idDepartemen: a.idDepartemen ?? null,
      idJabatan: a.idJabatan ?? null,
    })),
    skipDuplicates: true,
  });
}
```

Snapshot variabel dibuat dengan mengambil seluruh *RefKpiVariabelAssignment* aktif beserta data variabel dari tabel *RefKpiVariabel*, kemudian menyalinnya ke tabel *KpiVariabelSnapshot* dalam satu transaksi *batch insert*. Pembuatan *snapshot* menjamin *reproducibility* hasil klasterisasi karena konfigurasi yang dipakai selama proses tidak akan terpengaruh oleh perubahan *master* variabel yang mungkin dilakukan HR di kemudian hari. Pada *method prosesFcm()*, *snapshot* hanya dibuat ketika proses dijalankan pertama kali untuk periode tersebut, di mana pengecekan dilakukan melalui *flag isRerun* agar tidak terjadi duplikasi *snapshot* saat HR menjalankan ulang klasterisasi.

2. Pengambilan Data Penilaian dan Deduplikasi *Snapshot*

Langkah kedua dimulai dengan mengambil data penilaian yang berstatus *submitted* untuk setiap karyawan yang akan diproses, beserta *snapshot* variabel yang relevan dengan jabatan dan departemen karyawan tersebut.

```
// Filename: src/kpi-fcm/fcm.service.ts
const penilaians = await this.prisma.kpiPenilaian.findMany({
  where: {
    idPeriode,
    idKaryawan,
    status: 'submitted',
  },
  select: {
    idVariabel: true,
    nilai: true,
  },
});

if (penilaians.length === 0) continue;

// Ambil jabatan + departemen karyawan untuk deduplikasi
const karyawan = await this.prisma.refKaryawan.findUnique({
  where: { idKaryawan },
  select: {
    jabatan: {
      select: {
        idJabatan: true,
        idDepartemen: true,
      },
```

```
    },
  },
});

if (!karyawan) continue;

const idJabatanKaryawan = karyawan.jabatan.idJabatan;
const idDepartemenKaryawan = karyawan.jabatan.idDepartemen;

// Ambil snapshot variabel untuk karyawan ini
const idVariabelList = penilaians.map((p) => p.idVariabel);

const snapshots = await
this.prisma.kpiVariabelSnapshot.findMany({
  where: {
    idPeriode,
    idVariabel: { in: idVariabelList },
    OR: [
      { cakupan: 'global' },
      { cakupan: 'departemen', idDepartemen:
idDepartemenKaryawan },
      { cakupan: 'jabatan', idJabatan: idJabatanKaryawan },
    ],
  },
  select: {
    idVariabel: true,
    cakupan: true,
    idDepartemen: true,
    idJabatan: true,
    arahNilai: true,
    nilaiMin: true,
    nilaiMax: true,
    bobot: true,
  },
});
```

Pengambilan data penilaian dibatasi hanya pada *record* dengan status *submitted* untuk memastikan bahwa data yang masih dalam status *draft* tidak ikut dihitung. Pengambilan *snapshot* variabel menggunakan klausa *OR* untuk mengambil tiga jenis *snapshot* sekaligus, yaitu *snapshot* dengan cakupan global, cakupan departemen yang sesuai dengan karyawan, dan cakupan jabatan yang sesuai dengan karyawan. Setelah *snapshot* diperoleh, dilakukan deduplikasi dengan menerapkan prioritas yang sama dengan *decorator pattern*, yaitu jabatan lebih tinggi dari departemen, dan departemen lebih tinggi dari global.

```
const snapshotMap = new Map<string, (typeof snapshots)[0]>();

// Global dulu (prioritas terendah)
for (const s of snapshots.filter((s) => s.cakupan === 'global')) {
  snapshotMap.set(s.idVariabel, s);
}
// Departemen override global
for (const s of snapshots.filter((s) => s.cakupan === 'departemen'))
{
  snapshotMap.set(s.idVariabel, s);
}
// Jabatan override departemen dan global
for (const s of snapshots.filter((s) => s.cakupan === 'jabatan')) {
  snapshotMap.set(s.idVariabel, s);
}
```

Mekanisme deduplikasi ini mengikuti pola yang sama dengan *resolveVariabelDecorator* pada sub-bab 4.3.2, di mana *Map* digunakan untuk *override* otomatis ketika nilai dari *layer* lebih tinggi diiterasi setelah *layer* lebih rendah. Konsistensi mekanisme deduplikasi antara fase resolusi variabel saat *input* dan fase pra-pemrosesan saat klasterisasi menjamin bahwa variabel yang dipakai untuk menghitung skor akhir karyawan identik dengan variabel yang ditampilkan kepada penilai saat *input*.

3. Normalisasi *Min-Max*

Langkah ketiga adalah normalisasi nilai aktual ke dalam rentang nol sampai satu menggunakan metode *min-max* yang memperhatikan arah nilai variabel. Untuk variabel dengan arah nilai lebih tinggi lebih baik, formula yang digunakan adalah:

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Keterangan :

- x_{norm} = nilai variabel setelah dinormalisasi
- x = nilai aktual variabel karyawan
- x_{min} = nilai minimum variabel yang dikonfigurasi
- x_{max} = nilai maksimum variabel yang dikonfigurasi

Untuk variabel dengan arah nilai lebih rendah lebih baik, formula yang digunakan adalah inversinya:

$$x_{norm} = \frac{x_{max} - x}{x_{max} - x_{min}}$$

Keterangan :

- x_{norm} = nilai variabel setelah dinormalisasi
- x = nilai aktual variabel karyawan
- x_{min} = nilai minimum variabel yang dikonfigurasi
- x_{max} = nilai maksimum variabel yang dikonfigurasi

Dengan formula ini, nilai aktual yang baik akan selalu menghasilkan nilai normalisasi mendekati satu, dan nilai aktual yang buruk akan selalu menghasilkan nilai normalisasi mendekati nol, terlepas dari arah nilai variabelnya. Berikut implementasi kedua formula tersebut.

```
// Filename: src/kpi-fcm/fcm.service.ts (bagian normalisasi)
const nilai = Number(p.nilai);
const min = Number(snapshot.nilaiMin);
const max = Number(snapshot.nilaiMax);
const bobot = Number(snapshot.bobot);

let nilaiNormalized: number;

if (max === min) {
  // Edge case: max = min → normalized = 1.0
  nilaiNormalized = 1.0;
} else if (snapshot.arahNilai === 'lebih_tinggi_lebih_baik') {
  nilaiNormalized = (nilai - min) / (max - min);
} else {
  // lebih_rendah_lebih_baik
  nilaiNormalized = (max - nilai) / (max - min);
}

// Clamp antara 0 dan 1
nilaiNormalized = Math.max(0, Math.min(1, nilaiNormalized));
```

Implementasi normalisasi mencakup dua penanganan *edge case* yang penting. Pertama, ketika nilai maksimum dan minimum sama, normalisasi akan menghasilkan pembagian dengan nol yang tidak terdefinisi, sehingga sistem secara langsung mengembalikan nilai 1.0 sebagai *default*. Kedua, hasil normalisasi di-*clamp* ke dalam rentang nol sampai satu untuk menangani kemungkinan adanya nilai aktual di luar batas yang dideklarasikan pada variabel, misalnya ketika penilai memberikan nilai lebih tinggi dari nilai maksimum yang dikonfigurasi.

4. Perhitungan Skor Akhir Berbobot

Setelah seluruh nilai dinormalisasi, langkah selanjutnya adalah menghitung skor akhir berbobot per karyawan menggunakan formula akumulasi tertimbang:

$$skor_{akhir} = \sum_{i=1}^{n} \frac{x_{norm_i} \times bobot_i}{100}$$

Keterangan :

- $skor_{akhir}$ = skor akhir karyawan dalam rentang 0 hingga 1
- x_{norm_i} = nilai normalisasi variabel ke-i
- $bobot_i$ = bobot variabel ke-i dalam persentase
- n = jumlah variabel KPI yang berlaku untuk karyawan

di mana x_{norm_i} adalah nilai normalisasi variabel ke-i, $bobot_i$ adalah bobot variabel ke-i dalam satuan persen, dan n adalah jumlah variabel yang berlaku untuk karyawan tersebut. Pembagian dengan 100 dilakukan karena bobot pada sistem ini disimpan dalam satuan persen dengan rentang nol sampai seratus, sehingga perlu dikonversi ke satuan desimal sebelum dijumlahkan.

```
// Filename: src/kpi-fcm/fcm.service.ts (bagian akumulasi skor)
let skorAkhir = 0;

for (const p of penilaians) {
  const snapshot = snapshotMap.get(p.idVariabel);
  if (!snapshot) continue;

  // ... (normalisasi nilai)

  // Akumulasi skor akhir
  skorAkhir += (nilaiNormalized * bobot) / 100;
}

hasil.push({ idKaryawan, skorAkhir });
```

Skor akhir per karyawan akan berada dalam rentang nol sampai satu, dengan asumsi total bobot seluruh variabel yang berlaku untuk karyawan tersebut adalah seratus. Skor inilah yang akan menjadi *input*

bagi algoritma *Fuzzy C-Means* pada langkah berikutnya, di mana setiap karyawan diwakili oleh satu nilai skalar yang merepresentasikan kinerja keseluruhannya selama periode penilaian.

5. Persistensi Nilai Normalisasi

Selain dipakai untuk menghitung skor akhir, nilai normalisasi juga disimpan kembali ke tabel KpiPenilaian melalui kolom nilaiNormalized. Persistensi ini dilakukan agar nilai normalisasi dapat ditelusuri kembali tanpa perlu menghitung ulang.

```
// Filename: src/kpi-fcm/fcm.service.ts (bagian update)
const penilaianRecord = await this.prisma.kpiPenilaian.findFirst({
  where: { idPeriode, idKaryawan, idVariabel: p.idVariabel },
  select: { idPenilaian: true },
});

if (penilaianRecord) {
  nilaiNormalizedList.push({
    idPenilaian: penilaianRecord.idPenilaian,
    nilaiNormalized,
  });
}

// Update nilai_normalized di kpi_penilaian
for (const item of nilaiNormalizedList) {
  await this.prisma.kpiPenilaian.update({
    where: { idPenilaian: item.idPenilaian },
    data: { nilaiNormalized: item.nilaiNormalized },
  });
}
```

Persistensi nilai normalisasi mendukung reproducibility dan transparansi sistem, di mana setiap nilai normalisasi yang terlibat dalam perhitungan skor akhir dapat dilihat dan diverifikasi melalui antarmuka pengguna. Pendekatan ini juga memungkinkan audit perhitungan jika di kemudian hari terdapat kebutuhan untuk memeriksa konsistensi antara nilai aktual yang diinput penilai dengan skor akhir yang dihasilkan sistem.

4.3.4 Implementasi Algoritma FCM

Sub-bab ini menjelaskan implementasi algoritma *Fuzzy C-Means* pada sistem yang telah dirancang pada sub-bab 4.2.8. Algoritma *Fuzzy C-Means* dipilih karena kemampuannya menghasilkan klasterisasi dengan derajat keanggotaan, di mana setiap karyawan tidak hanya diklasifikasikan secara biner ke dalam satu *cluster* melainkan memiliki derajat keanggotaan pada seluruh *cluster* dengan total sebesar satu, sehingga sesuai dengan realitas kinerja karyawan yang seringkali berada pada zona transisi antar tingkat kinerja. Implementasi algoritma dipisahkan ke dalam sebuah modul khusus bernama *fcm.engine.ts* yang berisi fungsi-fungsi murni tanpa *dependency* eksternal untuk menjaga algoritma agar bersifat *stateless* dan dapat diuji secara independen, di mana algoritma bekerja pada ruang satu dimensi karena seluruh variabel KPI telah diagregasi menjadi satu skor akhir berbobot pada tahap pra-pemrosesan tanpa mengurangi validitas matematis algoritma. Penelitian ini menggunakan mode manual dengan jumlah *cluster* yang ditentukan oleh HR melalui konfigurasi periode KPI, dan pembahasan pada sub-bab ini mengikuti enam langkah algoritma yang telah diuraikan pada sub-bab 2.3.2 dengan tambahan tahap persiapan data *input* pada awal proses serta tahap pasca-iterasi untuk penentuan *cluster* final dan pelabelan otomatis.

1. Persiapan Data *Input* dan Tipe

   Sebelum membahas langkah-langkah algoritma, terlebih dahulu didefinisikan tipe data yang menjadi kontrak antara *FcmEngine* dan pemanggilnya. Tipe-tipe ini memastikan bahwa data masuk dan data keluar memiliki struktur yang konsisten dan dapat divalidasi pada saat kompilasi.

```
// Filename: src/kpi-fcm/fcm.engine.ts
export interface FcmInput {
  idKaryawan: string;
  skorAkhir: number;
}

export interface FcmHasil {
  idKaryawan: string;
  cluster: number;
```

```
  labelCluster: string;
  derajatKeanggotaan: Record<string, number>;
  silhouetteScore: number;
  skorAkhir: number;
}

export interface FcmConfig {
  jumlahCluster: number;
  pangkatPembobot: number;
  maxIterasi: number;
  errorMinimum: number;
  idPeriode: string;
}
```

Tipe *FcmInput* merepresentasikan satu karyawan yang akan diklasterkan, di mana setiap karyawan diwakili oleh sebuah skor akhir yang sudah dinormalisasi pada sub-bab 4.3.3, dengan pendekatan satu skor per karyawan dipilih karena seluruh variabel KPI telah diagregasi menjadi satu skor akhir berbobot pada tahap pra-pemrosesan sehingga algoritma *Fuzzy C-Means* dapat bekerja pada ruang satu dimensi yang lebih efisien. Tipe *FcmHasil* merepresentasikan hasil klasterisasi per karyawan mencakup nomor *cluster*, label *cluster* yang ditentukan secara otomatis, derajat keanggotaan untuk setiap *cluster* yang disimpan sebagai *map* dengan kunci label *cluster*, nilai *Silhouette Coefficient* global, dan skor akhir karyawan untuk referensi, sedangkan tipe *FcmConfig* menyimpan parameter algoritma yang dapat dikonfigurasi oleh HR mencakup jumlah *cluster*, pangkat pembobot fuzzifikasi, maksimum iterasi, batas *error* untuk kriteria konvergensi, dan identitas periode yang digunakan sebagai sumber *seed* deterministik.

Pada awal eksekusi fungsi jalankanFcm, data input diekstraksi menjadi sebuah vektor satu dimensi yang berisi skor akhir dari seluruh karyawan.

```
const n = data.length;
const x = data.map((d) => d.skorAkhir);
```

Variabel *n* menyimpan jumlah karyawan yang akan dikelompokkan, sedangkan vektor *x* menyimpan skor akhir seluruh karyawan dalam

urutan yang sama dengan *input*. Indeks vektor *x* menjadi referensi untuk seluruh perhitungan berikutnya, di mana posisi karyawan ke-*i* pada vektor *x* selalu konsisten dengan posisi karyawan tersebut pada matriks keanggotaan dan hasil akhir.

2. Inisialisasi Parameter

Langkah pertama algoritma adalah menentukan seluruh parameter awal yang akan digunakan selama proses iterasi. Parameter-parameter tersebut diterima dari *FcmConfig* yang dikirim oleh *FcmService*, mencakup jumlah *cluster*, pangkat pembobot, maksimum iterasi, dan batas *error* terkecil yang menjadi kriteria konvergensi.

```
// Filename: src/kpi-fcm/fcm.engine.ts (bagian awal jalankanFcm)
const {
  jumlahCluster: c,
  pangkatPembobot: w,
  maxIterasi,
  errorMinimum,
  idPeriode,
} = config;
```

Selain parameter dari *config*, fungsi objektif awal diinisialisasi dengan nilai nol, mengikuti konvensi $P_0 = 0$ pada sub-bab 2.3.2.

```
let prevObjektif = 0;
```

Variabel *prevObjektif* ini akan menyimpan nilai fungsi objektif dari iterasi sebelumnya untuk perhitungan selisih pada Langkah 6, di mana iterasi pertama akan membandingkan fungsi objektif yang dihitung pada Langkah 4 dengan nilai nol ini. Bagian penting dari inisialisasi parameter pada sistem adalah penyiapan generator bilangan acak deterministik yang dibutuhkan oleh Langkah 2, mengingat *reproducibility* hasil klasterisasi merupakan salah satu kontribusi penting pada sistem ini di mana eksekusi algoritma dengan data *input* yang sama harus menghasilkan *output* yang identik pada setiap kali dijalankan. Pencapaian *reproducibility* tersebut tidak dapat dilakukan dengan generator bilangan acak bawaan *JavaScript* yaitu *Math.random()* karena fungsi tersebut tidak menyediakan mekanisme penyimpanan *state* dan tidak dapat di-*seed* secara eksplisit, sehingga

sistem mengimplementasikan generator bilangan acak deterministik menggunakan algoritma *Linear Congruential Generator* (LCG) yang bekerja berdasarkan rumus rekursif:

$$S_{n+1} = (a \,.\, S_n + c_{LCG}) mod\ m$$

Di mana adalah *state* generator pada iterasi ke-*n*, adalah konstanta pengali, adalah konstanta penambah (dibedakan dari jumlah *cluster*), dan adalah modulus yang membatasi rentang nilai. Setiap nilai yang dihasilkan kemudian dinormalisasi ke dalam rentang nol sampai satu dengan membaginya terhadap *m*, sehingga keluaran LCG dapat dipakai sebagai pengganti *Math.random()*.

```
function buatLcg(seed: number): () => number {
  const a = 1664525;
  const c = 1013904223;
  const m = Math.pow(2, 32);
  let state = seed >>> 0; // unsigned 32-bit

  return () => {
    state = (a * state + c) >>> 0;
    return state / m; // return [0, 1)
  };
}
```

Implementasi *buatLcg* menggunakan parameter klasik yang dipopulerkan oleh buku *Numerical Recipes* yaitu $a = 1664525$, $c_{LCG} = 1013904223$, dan $m = 2^{32}$,di mana kombinasi parameter ini menghasilkan periode penuh sepanjang 2^{32} angka sebelum nilai mulai berulang sehingga cukup panjang untuk seluruh kebutuhan klasterisasi pada skala jumlah karyawan yang realistis. Operator *>>> 0* digunakan untuk memaksa hasil perhitungan tetap berada dalam rentang *unsigned 32-bit* karena *JavaScript* secara *default* menggunakan tipe *number 64-bit* yang dapat menyebabkan presisi tidak konsisten ketika nilai *overflow*, sedangkan fungsi *buatLcg* mengembalikan sebuah fungsi tertutup (*closure*) yang menyimpan *state* internalnya sehingga setiap pemanggilan akan menghasilkan nilai acak deterministik yang berbeda dan memungkinkan setiap pemanggilan *jalankanFcm* memiliki

instance LCG yang independen tanpa saling mengganggu *state* generator pada eksekusi paralel.

Untuk menjamin *reproducibility*, nilai *seed* awal diturunkan dari identitas periode menggunakan *helper* seedDariIdPeriode.

```
function seedDariIdPeriode(idPeriode: string): number {
  return parseInt(idPeriode.replace(/-/g, '').substring(0, 8), 16);
}
```

Helper ini mengambil delapan karakter pertama dari identitas periode setelah karakter *dash* dihilangkan, kemudian mengonversinya menjadi bilangan bulat dengan basis heksadesimal. Pendekatan ini menjamin bahwa periode dengan identitas yang sama akan selalu menghasilkan *seed* yang sama, sehingga eksekusi algoritma untuk periode tersebut akan selalu menghasilkan hasil klasterisasi yang identik.

```
const seed = seedDariIdPeriode(idPeriode);
const random = buatLcg(seed);
```

Variabel *random* yang dihasilkan adalah fungsi generator yang akan dipanggil berulang kali pada Langkah 2 untuk membangkitkan nilai-nilai acak yang diperlukan oleh proses inisialisasi matriks keanggotaan.

3. Pembangkitan Partisi Awal (μ_{ik})

Langkah kedua adalah membangkitkan matriks partisi awal μ yang berukuran $n \times c$, di mana n adalah jumlah karyawan dan c adalah jumlah *cluster*. Matriks ini menyimpan derajat keanggotaan setiap karyawan terhadap setiap *cluster*, dengan batasan bahwa total derajat keanggotaan setiap karyawan harus sama dengan satu. Batasan tersebut dapat dirumuskan sebagai:

$$\sum_{k=1}^{c} \mu_{ik} = 1, \quad \forall i \in \{1,2,\dots,n\}$$

di mana adalah derajat keanggotaan karyawan ke-i pada *cluster* ke-k. Dengan batasan ini, derajat keanggotaan menjadi terkalibrasi sebagai probabilitas relatif yang menunjukkan seberapa besar karyawan tergolong ke setiap *cluster*, di mana jumlah totalnya selalu menjamin

bahwa karyawan tersebut sepenuhnya terklasifikasi pada gabungan seluruh *cluster*.

```
// Filename: src/kpi-fcm/fcm.engine.ts (bagian pembangkitan partisi awal)
const mu: number[][] = [];

for (let i = 0; i < n; i++) {
  const row: number[] = [];
  let total = 0;

  for (let k = 0; k < c; k++) {
    const val = random();
    row.push(val);
    total += val;
  }

  // Normalisasi: total per karyawan = 1
  mu.push(row.map((v) => v / total));
}
```

Pembangkitan partisi awal dilakukan dalam dua tahap untuk setiap karyawan, yaitu tahap pertama membangkitkan *c* nilai acak menggunakan fungsi *random* yang sudah di-*seed* dengan LCG pada Langkah 1 lalu menyimpannya sementara dalam *array row* sekaligus menghitung jumlah totalnya, sedangkan tahap kedua membagi setiap nilai pada *row* dengan total tersebut sehingga seluruh elemen pada *row* dijamin berjumlah satu setelah normalisasi. Pendekatan normalisasi proporsional ini dipilih karena lebih sederhana dan lebih cepat serta secara matematis tetap memenuhi batasan total satu yang dipersyaratkan oleh algoritma *Fuzzy C-Means*, dan karena nilai acak dibangkitkan dengan LCG yang deterministik, matriks partisi awal akan selalu identik untuk periode yang sama yang menjadi fondasi dari sifat *reproducible* algoritma.

4. Perhitungan Pusat Klaster (V_k)

   Langkah ketiga adalah menghitung pusat *cluster* untuk setiap *cluster* ke-*k*. Pusat *cluster* dihitung sebagai rata-rata tertimbang dari seluruh data karyawan, di mana bobot setiap karyawan ditentukan oleh derajat keanggotaannya terhadap *cluster* tersebut, dipangkatkan dengan pangkat pembobot *w*.

Mengacu pada formula umum perhitungan pusat *cluster* pada sub-bab 2.3.2, dengan penyesuaian bahwa sistem bekerja pada ruang satu dimensi sehingga *subscript j* untuk atribut tidak diperlukan, formula menjadi:

$$V_k = \frac{\sum_{i=1}^{n} (\mu_{ik})^w x_i}{\sum_{i=1}^{n} (\mu_{ik})^w}$$

di mana V_k adalah pusat cluster ke-k, X_i adalah skor akhir karyawan ke-i, μ_{ik} adalah derajat keanggotaan karyawan ke-i pada cluster ke-k, dan w adalah pangkat pembobot.

```
// Filename: src/kpi-fcm/fcm.engine.ts (bagian perhitungan
centroid)
const centroid: number[] = [];

for (let k = 0; k < c; k++) {
  let pembilang = 0;
  let penyebut = 0;

  for (let i = 0; i < n; i++) {
    const muPangkat = Math.pow(mu[i][k], w);
    pembilang += muPangkat * x[i];
    penyebut += muPangkat;
  }

  centroid.push(penyebut === 0 ? 0 : pembilang / penyebut);
}
```

Implementasi perhitungan *centroid* dilakukan dengan *loop* ganda. *Loop* luar mengiterasi seluruh *cluster*, sedangkan *loop* dalam mengiterasi seluruh karyawan untuk menghitung pembilang dan penyebut dari formula. Variabel muPangkat menyimpan hasil dari $(\mu_{ik})^w$ yang dipakai bersama untuk pembilang dan penyebut, sehingga perhitungan pangkat tidak perlu diulang.

Penanganan kasus penyebut bernilai nol dilakukan dengan mengembalikan nol sebagai *centroid*. Kasus ini terjadi ketika seluruh derajat keanggotaan untuk *cluster* ke-k bernilai nol, yang secara teoretis tidak akan terjadi karena matriks partisi telah dinormalisasi pada pembangkitan partisi awal. Namun, penanganan ini tetap

diimplementasikan sebagai pengaman terhadap kemungkinan pembagian dengan nol akibat pembulatan *floating point* pada iterasi-iterasi lanjut.

5. Perhitungan Fungsi Objektif (P_t)

Langkah selanjutnya adalah menghitung fungsi objektif pada iterasi ke-t yang mengukur total varian dalam cluster. Nilai fungsi objektif ini akan dipakai sebagai indikator konvergensi, di mana algoritma dianggap konvergen ketika selisih nilai fungsi objektif antara dua iterasi berurutan sudah cukup kecil.

Mengacu pada formula umum perhitungan fungsi objektif pada sub-bab 2.3.2, dengan penyesuaian bahwa jarak antara data dan pusat cluster pada ruang satu dimensi dihitung sebagai nilai absolut selisih keduanya, formula menjadi:

$$P_t = \sum_{i=1}^{n} \sum_{k=1}^{c} (\mu_{ik})^w \, . \, d(X_i, V_k)^2$$

di mana P_t adalah fungsi objektif pada iterasi ke-t, $d(X_i, V_k)$ adalah jarak Euclidean antara skor akhir karyawan ke-i dengan pusat *cluster* ke-k, μ_{ik} adalah derajat keanggotaan karyawan ke-i pada *cluster* ke-k, dan w adalah pangkat pembobot. Karena sistem bekerja pada ruang satu dimensi, jarak Euclidean disederhanakan menjadi nilai absolut selisih, yaitu $d(X_i, V_k) = | X_i - V_k |$.

```
// Filename: src/kpi-fcm/fcm.engine.ts (helper jarak Euclidean)
function jarakEuclidean(a: number, b: number): number {
  return Math.abs(a - b);
}
```

Helper jarak Euclidean mengimplementasikan jarak antara dua titik pada ruang satu dimensi sebagai nilai absolut selisih, yang secara matematis ekuivalen dengan jarak *Euclidean* pada ruang berdimensi lebih tinggi ketika hanya satu dimensi yang dipertimbangkan. Pemisahan *helper* ini juga membuat algoritma lebih mudah diperluas ke ruang *multi-dimensi* di masa mendatang dengan mengubah implementasi *helper* tanpa menyentuh kode utama.

Perhitungan fungsi objektif diimplementasikan dengan *loop* ganda yang mengakumulasikan kontribusi setiap pasangan karyawan dan *cluster*.

```
// Filename: src/kpi-fcm/fcm.engine.ts (bagian perhitungan fungsi objektif)
let objektif = 0;

for (let i = 0; i < n; i++) {
  for (let k = 0; k < c; k++) {
    const jarak = jarakEuclidean(x[i], centroid[k]);
    objektif += Math.pow(mu[i][k], w) * Math.pow(jarak, 2);
  }
}
```

Variabel *objektif* menampung akumulasi kontribusi seluruh pasangan karyawan dan *cluster*. Pada setiap iterasi *loop*, jarak antara karyawan ke-*i* dan *centroid* ke-*k* dihitung, kemudian dikuadratkan dan dikalikan dengan derajat keanggotaan yang sudah dipangkatkan dengan *w*. Nilai *objektif* yang dihasilkan akan dibandingkan dengan *prevObjektif* pada Langkah 6 untuk menentukan apakah algoritma sudah konvergen.

6. Pembaruan Matriks Partisi (μ_{ik})

Langkah selanjutnya adalah memperbarui matriks partisi μ untuk iterasi berikutnya berdasarkan pusat *cluster* yang baru dihitung pada perhitungan pusat *cluster*. Pembaruan ini menjamin bahwa derajat keanggotaan setiap karyawan akan bergerak ke arah *cluster* yang lebih sesuai dengan posisinya pada ruang data.

Mengacu pada formula umum pembaruan matriks partisi pada sub-bab 2.3.2, dengan penyesuaian bahwa sistem bekerja pada ruang satu dimensi sehingga subscript j untuk atribut tidak diperlukan, formula menjadi:

$$\mu_{ik} = \frac{\left(\sum_{j=1}^{m}(x_{ij} - v_{kj})^2\right)^{\frac{-1}{w-1}}}{\sum_{k=1}^{c}\left(\sum_{j=1}^{m}(x_{ij} - v_{kj})^2\right)^{\frac{-1}{w-1}}}$$

di mana μ_{ik} adalah derajat keanggotaan baru karyawan ke-i pada *cluster* ke-k, X_i adalah skor akhir karyawan ke-i, V_k adalah pusat *cluster*

ke-k, dan iterator l berjalan dari 1 sampai c untuk menjumlahkan kontribusi seluruh *cluster*.

Pada implementasi kode, formula tersebut dimanipulasi secara aljabar menjadi bentuk ekuivalen yang lebih efisien secara komputasi. Pembilang dan penyebut dibagi dengan pembilang itu sendiri, sehingga rumus menjadi:

$$\mu_{ik} = \frac{1}{\sum_{l=1}^{c} \left(\frac{d(X_i, V_k)}{d(X_i, V_l)} \right)^{\frac{2}{w-1}}}$$

Bentuk ini matematis identik dengan formula sebelumnya, tetapi lebih efisien karena pemangkatan dilakukan satu kali setelah rasio dibentuk, bukan dua kali secara terpisah pada pembilang dan penyebut. di mana μ_{ik} adalah derajat keanggotaan baru karyawan ke-i pada *cluster* ke-k, $d(X_i, V_k)$ adalah jarak antara karyawan ke-i dengan pusat *cluster* ke-k, dan iterator l berjalan dari 1 sampai c untuk menjumlahkan kontribusi seluruh *cluster*.

Sebelum membahas implementasi formula utama, perlu diperhatikan adanya kasus khusus ketika jarak antara karyawan dan salah satu *centroid* bernilai nol. Pada kasus tersebut, formula utama akan menghasilkan pembagian dengan nol, sehingga sistem memerlukan penanganan eksplisit untuk mempertahankan stabilitas algoritma.

```
// Filename: src/kpi-fcm/fcm.engine.ts
for (let i = 0; i < n; i++) {
  let tepat = -1;

  for (let k = 0; k < c; k++) {
    if (jarakEuclidean(x[i], centroid[k]) === 0) {
      tepat = k;
      break;
    }
  }

  if (tepat >= 0) {
    // Edge case: xi = Vk → μik = 1, cluster lain = 0
    for (let k = 0; k < c; k++) {
```

```
      mu[i][k] = k === tepat ? 1.0 : 0.0;
    }
  } else {
    // Update normal
    for (let k = 0; k < c; k++) {
      let total = 0;

      for (let l = 0; l < c; l++) {
        const ratio =
          jarakEuclidean(x[i], centroid[k]) /
          jarakEuclidean(x[i], centroid[l]);
        total += Math.pow(ratio, 2 / (w - 1));
      }

      mu[i][k] = 1 / total;
    }
  }
}
```

Implementasi pembaruan dilakukan dalam dua jalur logika, yaitu jalur pertama menangani kasus khusus ketika skor karyawan persis sama dengan salah satu *centroid* di mana sistem akan langsung menetapkan derajat keanggotaan 1.0 untuk *cluster* tersebut dan 0.0 untuk *cluster* lainnya karena secara matematis jika karyawan tepat berada di pusat *cluster* maka karyawan tersebut sepenuhnya menjadi anggota *cluster* tersebut, sedangkan jalur kedua adalah pembaruan normal menggunakan formula utama di mana untuk setiap kombinasi karyawan ke-*i* dan *cluster* ke-*k* sistem menghitung total dari rasio jarak yang dipangkatkan dengan $\frac{2}{w-1}$ kemudian mengambil kebalikan dari total tersebut sebagai derajat keanggotaan baru, sehingga penggunaan pangkat $\frac{2}{w-1}$ menghasilkan derajat keanggotaan yang lebih halus seiring kenaikan nilai pangkat pembobot *w* (lebih kabur ketika *w* lebih besar dan lebih tegas ketika *w* mendekati satu). Setelah seluruh elemen matriks μ diperbarui, matriks ini siap dipakai sebagai *input* untuk iterasi berikutnya pada perhitungan pusat *cluster*, di mana *centroid* baru akan dihitung berdasarkan derajat keanggotaan yang sudah diperbarui ini.

7. Pengecekan Konvergensi

Langkah keenam adalah memeriksa apakah algoritma sudah konvergen, yaitu kondisi di mana perubahan fungsi objektif antara dua iterasi berurutan sudah cukup kecil sehingga iterasi dapat dihentikan. Kriteria konvergensi dirumuskan sebagai:

$$|p_t - p_{t-1}| < \xi$$

Di mana P_t adalah fungsi objektif pada iterasi saat ini, P_{t-1} adalah fungsi objektif pada iterasi sebelumnya, dan ξ adalah batas error terkecil yang ditentukan oleh HR melalui parameter *errorMinimum*. Jika kondisi tersebut terpenuhi, iterasi dihentikan dan algoritma dianggap konvergen. Jika tidak, nilai P_t disimpan sebagai P_{t-1} untuk iterasi berikutnya, kemudian algoritma kembali ke Langkah 3 untuk menghitung *centroid* baru.

```
// Filename: src/kpi-fcm/fcm.engine.ts (bagian pengecekan
konvergensi)
if (Math.abs(objektif - prevObjektif) < errorMinimum) {
  onProgress?.(90, `FCM konvergen pada iterasi ke-${t + 1}`);
  break;
}

prevObjektif = objektif;
```

Implementasi pengecekan konvergensi menggunakan fungsi *Math.abs* untuk menghitung nilai absolut selisih antara fungsi objektif iterasi saat ini dengan iterasi sebelumnya, di mana apabila selisih tersebut lebih kecil dari *errorMinimum* perulangan iterasi dihentikan dengan pernyataan *break*, sedangkan apabila kondisi konvergensi belum terpenuhi nilai objektif disimpan ke variabel *prevObjektif* untuk dibandingkan pada iterasi berikutnya dan *loop* akan berlanjut ke perhitungan pusat *cluster* baru. Selain kriteria konvergensi melalui ξ, algoritma juga dibatasi oleh parameter *maxIterasi* yang berfungsi sebagai pengaman untuk mencegah perulangan tak terbatas pada kasus data yang tidak dapat mencapai konvergensi, dengan nilai *default* ditetapkan sebesar 100 iterasi. Pada implementasi kode, pengecekan konvergensi diletakkan setelah perhitungan fungsi objektif dan sebelum

pembaruan matriks partisi, di mana urutan ini berbeda dengan pemaparan pada sub-bab 2.3.2 dan dimaksudkan untuk efisiensi komputasi karena apabila konvergensi sudah tercapai pembaruan matriks partisi tidak perlu dilakukan lagi, namun secara matematis kedua urutan menghasilkan hasil yang identik karena iterasi terakhir yang konvergen tidak akan mengubah matriks partisi secara signifikan.

8. Perhitungan *Centroid Final*

Setelah perulangan utama selesai, *centroid final* dihitung sekali lagi berdasarkan matriks partisi terakhir. Perhitungan ulang ini dilakukan karena pada implementasi ini pembaruan matriks partisi pada pembaruan matriks partisi mengubah nilai μ tanpa langsung memperbarui centroid, sehingga centroid yang dipakai untuk pelabelan harus dihitung kembali dari matriks μ yang sudah konvergen.

```
// Filename: src/kpi-fcm/fcm.engine.ts (bagian centroid final)
const centroidFinal: number[] = [];

for (let k = 0; k < c; k++) {
  let pembilang = 0;
  let penyebut = 0;

  for (let i = 0; i < n; i++) {
    const muPangkat = Math.pow(mu[i][k], w);
    pembilang += muPangkat * x[i];
    penyebut += muPangkat;
  }

  centroidFinal.push(penyebut === 0 ? 0 : pembilang / penyebut);
}
```

Formula yang digunakan untuk menghitung *centroid final* identik dengan formula pada perhitungan pusat klaster, di mana perbedaannya hanya pada matriks partisi yang dipakai, yaitu matriks μ akhir setelah konvergensi. Hasil *centroid final* disimpan dalam *array centroidFinal* yang akan menjadi input untuk penentuan label *cluster* otomatis.

9. Pelabelan *Cluster* Otomatis

Pelabelan *cluster* dilakukan secara otomatis berdasarkan urutan nilai *centroid*, di mana *cluster* dengan *centroid* tertinggi diberi label "*High Performer*", *cluster* dengan *centroid* terendah diberi label "*Low*

Performer", dan *cluster* di antaranya diberi label sesuai dengan urutan dari nilai tertinggi ke terendah.

```
// Filename: src/kpi-fcm/fcm.engine.ts (helper pelabelan)
function tentukanLabelCluster(centroid: number[]):
Record<number, string> {
  const indexed = centroid.map((nilai, idx) => ({ idx, nilai }));
  const sorted = [...indexed].sort((a, b) => b.nilai - a.nilai);

  const namaLabelPerJumlah: Record<number, string[]> = {
    2: ['High Performer', 'Low Performer'],
    3: ['High Performer', 'Standard', 'Low Performer'],
    4: ['High Performer', 'Above Average', 'Below Average', 'Low
Performer'],
    5: [
      'High Performer',
      'Above Average',
      'Standard',
      'Below Average',
      'Low Performer',
    ],
  };

  const namaLabel =
    namaLabelPerJumlah[centroid.length] ??
    Array.from({ length: centroid.length }, (_, i) =>
      i === 0
        ? 'High Performer'
        : i === centroid.length - 1
          ? 'Low Performer'
          : `Level ${centroid.length - i}`,
    );

  const labels: Record<number, string> = {};
  sorted.forEach((item, rank) => {
    labels[item.idx] = namaLabel[rank] ?? `Cluster ${item.idx + 1}`;
  });

  return labels;
}
```

Implementasi *tentukanLabelCluster* memetakan setiap indeks *cluster* ke label kategorisnya berdasarkan *ranking* nilai *centroid*, di mana tahap pertama menyimpan pasangan indeks dan nilai *centroid* pada *array indexed* kemudian menghasilkan *array sorted* yang berisi pasangan tersebut dalam urutan menurun berdasarkan nilai *centroid*,

tahap kedua mendefinisikan daftar nama label berdasarkan jumlah *cluster* dengan sistem menyediakan label *predefined* untuk jumlah *cluster* dua sampai lima dan secara otomatis menghasilkan label generik dengan format "*Level X*" untuk *cluster* di tengah apabila jumlah *cluster* di luar rentang tersebut, sedangkan tahap ketiga menggabungkan hasil *sorting* dengan daftar nama label di mana label pada peringkat ke-*rank* diberikan kepada *cluster* dengan indeks yang sesuai, sehingga pendekatan ini menjamin bahwa label *cluster* selalu konsisten dengan urutan nilai *centroid* terlepas dari urutan numerik indeks *cluster* yang dihasilkan oleh inisialisasi acak.

10. Penentuan *Cluster* Dominan per Karyawan

Setelah label *cluster* ditentukan, langkah selanjutnya adalah menentukan *cluster* dominan untuk setiap karyawan berdasarkan derajat keanggotaan tertinggi.

```
// Filename: src/kpi-fcm/fcm.engine.ts (bagian penentuan cluster
dominan)
const clusterKaryawan: number[] = mu.map((row) => {
  let maxVal = -1;
  let maxIdx = 0;

  row.forEach((val, idx) => {
    if (val > maxVal) {
      maxVal = val;
      maxIdx = idx;
    }
  });

  return maxIdx;
});
```

Cluster dominan untuk karyawan ke-i adalah *cluster* ke-k yang memiliki μ_{ik} tertinggi pada baris ke-i dari matriks partisi. Implementasi menggunakan *loop* pencarian nilai maksimum sederhana yang mengembalikan *indeks* dengan derajat keanggotaan terbesar. *Indeks cluster* dominan ini kemudian digabungkan dengan label hasil pelabelan otomatis untuk menghasilkan hasil akhir yang dapat ditampilkan kepada pengguna.

11. Penyusunan Hasil Akhir

Hasil akhir disusun sebagai *array* FcmHasil yang berisi informasi per karyawan beserta *cluster*, label, dan derajat keanggotaan.

```
// Filename: src/kpi-fcm/fcm.engine.ts (bagian susun hasil akhir)
const hasil: FcmHasil[] = data.map((d, i) => {
  const cluster = clusterKaryawan[i];
  const derajat: Record<string, number> = {};

  for (let k = 0; k < c; k++) {
    derajat[labelMap[k] ?? `Cluster ${k + 1}`] =
      Math.round(mu[i][k] * 10000) / 10000;
  }

  return {
    idKaryawan: d.idKaryawan,
    cluster: cluster + 1,
    labelCluster: labelMap[cluster] ?? `Cluster ${cluster + 1}`,
    derajatKeanggotaan: derajat,
    silhouetteScore: Math.round(scGlobal * 10000) / 10000,
    skorAkhir: d.skorAkhir,
  };
});
```

Pada penyusunan hasil, derajat keanggotaan setiap karyawan terhadap seluruh *cluster* disimpan sebagai *map* dengan kunci berupa label *cluster*, sehingga penerima hasil dapat mengakses derajat keanggotaan secara natural tanpa perlu mengetahui indeks numerik *cluster*. Pembulatan ke empat desimal dilakukan untuk menyederhanakan tampilan tanpa kehilangan presisi yang signifikan. *Field cluster* menggunakan penomoran berbasis satu yang lebih natural untuk pengguna akhir, sedangkan *field labelCluster* mengisi label kategoris yang sudah ditentukan secara otomatis pada tahap pelabelan. Nilai *silhouetteScore* yang disertakan pada setiap hasil merupakan nilai global yang dihitung secara terpisah dan akan dibahas pada sub-bab 4.3.5.

4.3.5 Implementasi *Silhouette Coefficient*

Sub-bab ini menjelaskan implementasi *Silhouette Coefficient* pada sistem yang digunakan sebagai metrik validasi kualitas klasterisasi hasil algoritma *Fuzzy C-Means* pada sub-bab 4.3.4. *Silhouette Coefficient* dipilih

karena merupakan metode evaluasi internal yang tidak memerlukan informasi label *ground truth* sehingga sesuai dengan konteks klasterisasi kinerja karyawan yang merupakan permasalahan *unsupervised learning*, sekaligus menyediakan interpretasi yang intuitif berupa tingkat kekompakan internal *cluster* dan tingkat keterpisahan antar *cluster* sehingga kualitas hasil klasterisasi dapat dievaluasi secara kuantitatif. Implementasi *Silhouette Coefficient* pada sistem ini berada pada modul *fcm.engine.ts* melalui fungsi *hitungSilhouette* yang dijalankan setelah tahap penentuan *cluster* dominan selesai, di mana hasil perhitungan berupa nilai *Silhouette Coefficient* global yang merepresentasikan kualitas klasterisasi keseluruhan akan disertakan pada setiap hasil klasterisasi per karyawan untuk dilaporkan kepada HR sebagai indikator validitas hasil, dan pembahasan pada sub-bab ini mengikuti empat tahap perhitungan yang telah diuraikan pada sub-bab 2.3.3 dengan penyesuaian bahwa sistem bekerja pada ruang satu dimensi sehingga jarak *Euclidean* disederhanakan menjadi nilai absolut selisih antara dua skor akhir karyawan.

1. Perhitungan Rata-rata Jarak *Intra-Cluster* a(i)

   Tahap pertama adalah menghitung rata-rata jarak antara karyawan ke-i dengan seluruh karyawan lain yang berada dalam cluster yang sama dengan karyawan ke-i. Nilai ini, dinotasikan sebagai $a(i)$, merepresentasikan tingkat kekompakan karyawan terhadap anggota *cluster* sendiri, di mana nilai $a(i)$ yang kecil menunjukkan bahwa karyawan berada dekat dengan anggota *cluster* lainnya.

   Mengacu pada formula umum perhitungan rata-rata jarak *intra-cluster* pada sub-bab 2.3.3, dengan penyesuaian bahwa sistem bekerja pada ruang satu dimensi, formula menjadi:

$$a(i) = \frac{1}{|A| - 1} \sum_{j \in A, j \neq i} d(X_i, X_j)$$

   Di mana A adalah *cluster* yang berisi karyawan ke-i, $| A |$ adalah jumlah anggota *cluster* A, X_i adalah skor akhir karyawan ke-i, X_j adalah skor akhir karyawan ke-j, dan $d(X_i, X_j) = | X_i - X_j |$ adalah jarak antara

karyawan ke-i dan karyawan ke-j yang dihitung sebagai nilai absolut selisih dua skor pada ruang satu dimensi. Pembagi $|A| - 1$ digunakan karena karyawan ke-i sendiri dikeluarkan dari perhitungan, sehingga hanya anggota cluster A selain karyawan ke-i yang dirata-rata.

```
// Filename: src/kpi-fcm/fcm.engine.ts
const clusterI = clusterKaryawan[i];

const anggotaClusterI = Array.from({ length: n }, (_, idx) =>
idx).filter(
  (idx) => clusterKaryawan[idx] === clusterI && idx !== i,
);

let ai = 0;
if (anggotaClusterI.length > 0) {
  ai =
    anggotaClusterI.reduce(
      (sum, j) => sum + jarakEuclidean(x[i], x[j]),
      0,
    ) / anggotaClusterI.length;
}
```

Implementasi perhitungan rata-rata jarak intra-*cluster a(i)* dilakukan dalam tiga langkah, yaitu langkah pertama mengidentifikasi *cluster* yang berisi karyawan ke-*i* melalui pembacaan *array clusterKaryawan* yang menyimpan indeks *cluster* setiap karyawan, langkah kedua menyusun daftar *anggotaClusterI* yang berisi indeks seluruh karyawan yang berada pada *cluster* yang sama dengan karyawan ke-*i* dengan pengecualian indeks karyawan ke-*i* sendiri agar tidak ikut dihitung, dan langkah ketiga menghitung rata-rata jarak *Euclidean* antara karyawan ke-*i* dengan setiap anggota *cluster* yang sudah disusun pada *anggotaClusterI* di mana akumulasi jarak dilakukan menggunakan operasi *reduce* kemudian dibagi dengan jumlah anggota *cluster*. Penanganan kasus *cluster* dengan hanya satu anggota dilakukan dengan memberikan nilai awal nol pada variabel *ai* kemudian pembagian dengan jumlah anggota hanya dilakukan jika *anggotaClusterI.length* lebih besar dari nol, karena kasus ini terjadi ketika karyawan ke-*i* adalah satu-satunya anggota *cluster* sehingga tidak ada anggota lain untuk dibandingkan, di mana pemberian nilai nol pada *a(i)* untuk kasus ini

secara matematis benar dan secara semantik mencerminkan bahwa karyawan tunggal pada *cluster* tidak memiliki tingkat kekompakan yang relevan untuk diukur.

2. Perhitungan Rata-rata Jarak ke *Cluster* Lain d(i, C)

   Tahap kedua adalah menghitung rata-rata jarak antara karyawan ke-*i* dengan seluruh karyawan yang berada pada *cluster* lain selain *cluster* asalnya. Nilai ini, dinotasikan sebagai *d(i,C)*, dihitung untuk setiap *cluster C* yang bukan *cluster* asal karyawan ke-*i*, sehingga menghasilkan beberapa nilai rata-rata jarak yang akan digunakan pada penentuan tetangga terdekat untuk menentukan tetangga terdekat.

   Mengacu pada formula umum perhitungan rata-rata jarak ke *cluster* lain pada sub-bab 2.3.3, dengan penyesuaian untuk ruang satu dimensi, formula menjadi:

$$d(i, C) = \frac{1}{|C|} \sum_{j \in C} d(X_i, X_j)$$

   Di mana *C* adalah *cluster* lain yang bukan *cluster* asal karyawan ke-*i*, adalah jumlah anggota *cluster C*, dan adalah jarak antara karyawan ke-*i* dan karyawan ke-*j* pada ruang satu dimensi. Berbeda dengan tahap perhitungan rata-rata jarak intra-*cluster*, pembagi pada formula ini adalah tanpa pengurangan, karena karyawan ke-*i* tidak berada pada *cluster C* sehingga seluruh anggota *cluster C* ikut diperhitungkan tanpa pengecualian.

3. Penentuan Tetangga Terdekat b(i)

   Tahap ketiga adalah memilih nilai rata-rata jarak terkecil dari seluruh nilai $d(i, C)$ yang dihitung pada tahap perhitungan rata-rata jarak ke *cluster* lain. Nilai minimum ini, dinotasikan sebagai $b(i)$, merepresentasikan jarak pemisahan terdekat antara karyawan ke-i dengan *cluster* lain selain *cluster* asalnya, di mana nilai $b(i)$ yang besar menunjukkan bahwa karyawan benar-benar terpisah dari *cluster* lain.

Mengacu pada formula penentuan tetangga terdekat pada sub-bab 2.3.3, formula menjadi:

$$b(i) = \min_{C \neq A} d(i, C)$$

Di mana A adalah *cluster* asal karyawan ke-i, dan iterasi minimum dilakukan terhadap seluruh *cluster* C selain *cluster* A. Hasil $b(i)$ ini akan menjadi salah satu komponen utama dalam perhitungan *Silhouette Coefficient* pada perhitungan *silhouette coefficient*.

Secara konseptual tahap perhitungan rata-rata jarak ke *cluster* lain dan tahap penentuan tetangga terdekat adalah dua langkah perhitungan yang berbeda, di mana tahap perhitungan rata-rata jarak ke *cluster* lain menghasilkan beberapa nilai rata-rata jarak ke setiap *cluster* lain dan tahap penentuan tetangga terdekat memilih nilai minimum dari nilai-nilai tersebut. Namun pada implementasi sistem, kedua tahap digabung dalam satu perulangan untuk efisiensi memori, di mana nilai minimum diperbarui secara bertahap selama perulangan tanpa menyimpan seluruh nilai rata-rata terlebih dahulu.

```
// Filename: src/kpi-fcm/fcm.engine.ts
let bi = Infinity;

for (let k = 0; k < c; k++) {
  if (k === clusterI) continue;

  const anggotaClusterK = Array.from({ length: n }, (_, idx) =>
idx).filter(
    (idx) => clusterKaryawan[idx] === k,
  );

  if (anggotaClusterK.length === 0) continue;

  const rataJarak =
    anggotaClusterK.reduce(
      (sum, j) => sum + jarakEuclidean(x[i], x[j]),
      0,
    ) / anggotaClusterK.length;

  if (rataJarak < bi) bi = rataJarak;
}
```

Variabel *bi* diinisialisasi dengan nilai *Infinity* agar setiap nilai rata-rata jarak yang dihitung pada iterasi pertama otomatis menjadi nilai minimum sementara, kemudian perulangan dilakukan untuk seluruh *cluster* ke-*k* di mana *cluster* yang sama dengan *cluster* asal karyawan ke-*i* dilewati melalui pernyataan *continue* agar tidak ikut diperhitungkan. Untuk setiap *cluster* lain, sistem menyusun daftar *anggotaClusterK* yang berisi indeks seluruh karyawan pada *cluster* tersebut, dan apabila *cluster* ditemukan kosong perulangan langsung melanjutkan ke *cluster* berikutnya sebagai pengaman terhadap kemungkinan *cluster* kosong. Setelah daftar anggota *cluster* diperoleh, sistem menghitung rata-rata jarak *rataJarak* antara karyawan ke-*i* dengan seluruh anggota *cluster* tersebut menggunakan operasi *reduce* tanpa pengecualian karyawan tertentu karena karyawan ke-*i* tidak berada pada *cluster* ini, kemudian sistem membandingkan nilai *rataJarak* dengan *bi* saat ini di mana apabila *rataJarak* lebih kecil maka nilai *bi* diperbarui menjadi *rataJarak*, sehingga mekanisme ini secara efektif mengimplementasikan operasi minimum pada formula tahap penentuan tetangga terdekat tanpa memerlukan *array* tambahan untuk menyimpan seluruh nilai rata-rata jarak ke setiap *cluster*.

4. Perhitungan *Silhouette Coefficient* s(i)

   Tahap keempat adalah menggabungkan nilai $a(i)$ dari tahap perhitungan rata-rata jarak *intra-cluster* dan $b(i)$ dari tahap penentuan tetangga terdekat menjadi nilai *Silhouette Coefficient* akhir untuk karyawan ke-i. Nilai ini, dinotasikan sebagai $s(i)$, merepresentasikan tingkat keseuaian penempatan karyawan ke-i pada *cluster* asalnya dengan mempertimbangkan tingkat kekompakan *internal cluster* sekaligus tingkat keterpisahan dengan *cluster* lain.

   Mengacu pada formula perhitungan *Silhouette Coefficient* pada sub-bab 2.3.3, formula menjadi:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

Di mana pembilang merupakan selisih antara nilai separasi *b(i)* dengan nilai kohesi *a(i)*, dan penyebut merupakan nilai maksimum di antara keduanya yang berfungsi sebagai faktor normalisasi. Nilai *s(i)* yang dihasilkan akan berada dalam rentang antara minus satu sampai satu, di mana nilai mendekati satu menunjukkan bahwa karyawan ditempatkan pada *cluster* yang sangat sesuai, nilai mendekati nol menunjukkan bahwa karyawan berada pada zona transisi antar *cluster*, dan nilai mendekati minus satu menunjukkan bahwa karyawan kemungkinan ditempatkan pada *cluster* yang kurang sesuai.

```
// Filename: src/kpi-fcm/fcm.engine.ts
const si =
  bi === Infinity ? 0 : (bi - ai) / Math.max(ai, bi === Infinity ? 0 :
bi);

silhouettes.push(isNaN(si) ? 0 : si);
```

Implementasi tahap perhitungan *silhouette coefficient* mencakup dua penanganan *edge case* yang penting, yaitu penanganan pertama adalah pengecekan apakah *bi* masih bernilai *Infinity* yang merupakan nilai inisialisasi pada tahap penentuan tetangga terdekat, di mana kondisi ini terjadi ketika tidak ada *cluster* lain selain *cluster* asal karyawan ke-*i* (misalnya pada kasus seluruh karyawan terkonsentrasi pada satu *cluster* yang sama) sehingga nilai *s(i)* ditetapkan menjadi nol karena *Silhouette Coefficient* tidak dapat dihitung tanpa adanya *cluster* pembanding, sedangkan penanganan kedua adalah pengecekan terhadap hasil perhitungan yang mungkin menghasilkan nilai *NaN* akibat pembagian nol dengan nol ketika *a(i)* dan *b(i)* sama-sama bernilai nol, di mana penanganan dengan operator *isNaN* mengembalikan nilai nol pada kasus ini sebagai pengaman agar tidak ada nilai *NaN* yang masuk ke perhitungan rata-rata global. Hasil perhitungan *s(i)* yang valid kemudian disimpan ke *array silhouettes* melalui operasi *push*, di mana *array* ini akan menjadi sumber data untuk perhitungan *Silhouette Coefficient* global pada tahap selanjutnya.

5. Perhitungan *Silhouette Coefficient Global*

Setelah *Silhouette Coefficient s(i)* dihitung untuk seluruh karyawan, sistem menghitung nilai *Silhouette Coefficient* global yang merepresentasikan kualitas klasterisasi secara keseluruhan. Nilai global ini dihitung sebagai rata-rata aritmatika sederhana dari seluruh nilai *s(i)*.

$$SC = \frac{1}{n}\sum_{i=1}^{n} s(i)$$

Di mana *SC* adalah nilai *Silhouette Coefficient* global, *n* adalah jumlah karyawan, dan *s(i)* adalah nilai *Silhouette Coefficient* karyawan ke-*i*. Nilai *SC* global ini menjadi indikator tunggal untuk mengevaluasi apakah klasterisasi yang dihasilkan memiliki kualitas yang baik atau memerlukan penyesuaian parameter.

```
// Filename: src/kpi-fcm/fcm.engine.ts (bagian perhitungan SC
global)
return silhouettes.length > 0
  ? silhouettes.reduce((sum, s) => sum + s, 0) / silhouettes.length
  : 0;
```

Implementasi perhitungan *SC* global dilakukan dengan operasi *reduce* untuk menjumlahkan seluruh nilai pada *array silhouettes* kemudian dibagi dengan panjang *array* tersebut, di mana penanganan kasus *array* kosong dilakukan dengan mengembalikan nilai nol apabila tidak ada karyawan yang diproses sebagai pengaman terhadap pemanggilan fungsi dengan *input* kosong meskipun secara teoretis tidak akan terjadi pada eksekusi normal. Nilai *SC* global yang dihasilkan kemudian disimpan pada *field silhouetteScore* di setiap *record FcmHasil* sehingga seluruh karyawan dalam satu periode klasterisasi memiliki nilai *SC* global yang sama, di mana penempatan nilai global pada setiap *record* bertujuan memudahkan akses oleh antarmuka pengguna tanpa perlu *query* terpisah, sehingga HR dapat langsung melihat nilai *SC* global pada halaman hasil klasterisasi tanpa perlu navigasi ke halaman lain.

6. Interpretasi Nilai *Silhouette*

Setelah nilai *Silhouette Coefficient global* dihitung, langkah berikutnya adalah menginterpretasikan nilai tersebut untuk menentukan kualitas struktur klasterisasi yang dihasilkan. Interpretasi dilakukan dengan membandingkan nilai SC *global* terhadap rentang kategori yang telah ditetapkan, sehingga hasil klasterisasi dapat dikategorikan menjadi salah satu dari empat tingkat kualitas struktur.

Tabel berikut menyajikan kategorisasi kualitas struktur klasterisasi berdasarkan nilai *Silhouette Coefficient*, sebagaimana telah dijelaskan pada sub-bab 2.3.3.

Tabel 13. Tabel Nilai *Silhouette*

Nilai *Silhouette* (SC)	Kriteria Struktur (*Structure Quality*)
$0{,}7 < SC \leq 1$	*Close Structure* (Struktur Kuat)
$0{,}5 < SC \leq 0{,}7$	*Medium Structure* (Struktur Sedang)
$0{,}25 < SC \leq 0{,}5$	*Tensile Structure* (Struktur Lemah)
$SC \leq 0{,}25$	*Unstructured* (Tidak Terstruktur)

Kategori *Close Structure* dengan nilai *SC* di atas 0,7 menunjukkan bahwa *cluster* yang terbentuk memiliki tingkat kekompakan internal yang tinggi sekaligus tingkat keterpisahan antar *cluster* yang jelas sehingga hasil klasterisasi dapat langsung dipakai sebagai dasar pengambilan keputusan SDM oleh HR tanpa perlu penyesuaian parameter algoritma, kategori *Medium Structure* dengan nilai *SC* antara 0,5 sampai 0,7 menunjukkan bahwa *cluster* yang terbentuk masih cukup baik tetapi terdapat sebagian karyawan yang berada pada zona transisi antar *cluster* sehingga HR perlu memberikan perhatian khusus pada karyawan dengan nilai *s(i)* rendah, kategori *Tensile Structure* dengan nilai *SC* antara 0,25 sampai 0,5 menunjukkan bahwa *cluster* yang terbentuk memiliki tingkat keterpisahan yang lemah sehingga HR perlu mempertimbangkan untuk menyesuaikan parameter algoritma seperti jumlah *cluster* atau pangkat pembobot, sedangkan kategori *Unstructured* dengan nilai *SC* di bawah 0,25 menunjukkan bahwa *cluster* yang terbentuk tidak memiliki struktur yang jelas sehingga HR

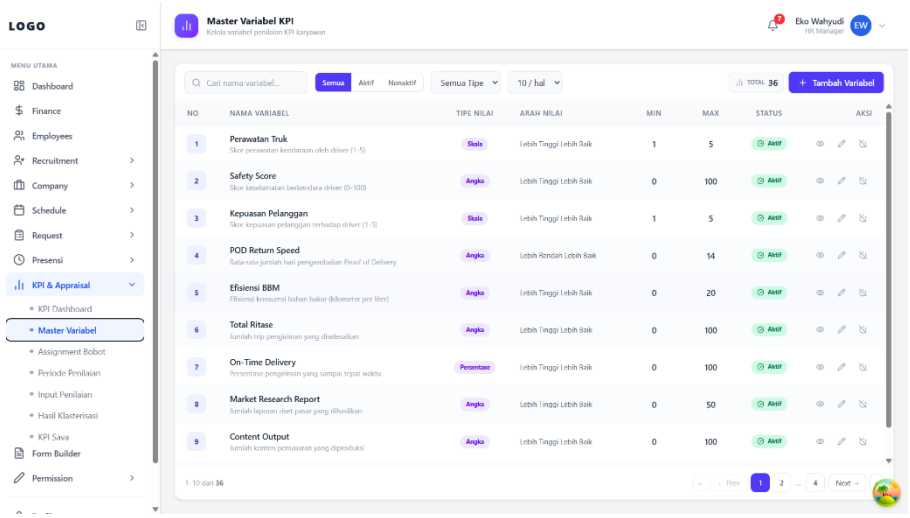
sebaiknya tidak menggunakan hasil klasterisasi sebagai dasar pengambilan keputusan dan perlu mengevaluasi ulang kualitas data masukan, konfigurasi variabel KPI, atau parameter algoritma. Pada implementasi sistem, kategorisasi ini ditampilkan secara visual pada antarmuka HR melalui label berwarna yang sesuai dengan kategori hasil untuk memudahkan HR dalam menginterpretasikan hasil klasterisasi secara cepat tanpa perlu mengingat rentang nilai *SC* untuk setiap kategori, sekaligus menjadi panduan apakah hasil klasterisasi memerlukan tindak lanjut berupa penyesuaian parameter.

4.3.6 Implementasi Antarmuka Pengguna

Sub-bab ini menjelaskan implementasi antarmuka pengguna sistem yang menghubungkan pengguna dengan logika *backend* yang telah dijelaskan pada sub-bab sebelumnya. Pembahasan difokuskan pada tujuh halaman utama yang berhubungan dengan kontribusi penelitian, yaitu Master Variabel, *Assignment* Bobot, Periode Penilaian, *Input* Penilaian, *KPI Dashboard*, Hasil Klasterisasi, dan KPI Saya. Setiap halaman disajikan dalam dua bagian, yaitu tampilan antarmuka beserta fitur utama dan implementasi kode *backend* yang menjalankan operasi pada halaman tersebut. Urutan pembahasan mengikuti alur penggunaan sistem secara kronologis, mulai dari konfigurasi variabel di awal sampai konsumsi hasil klasterisasi oleh karyawan di akhir.

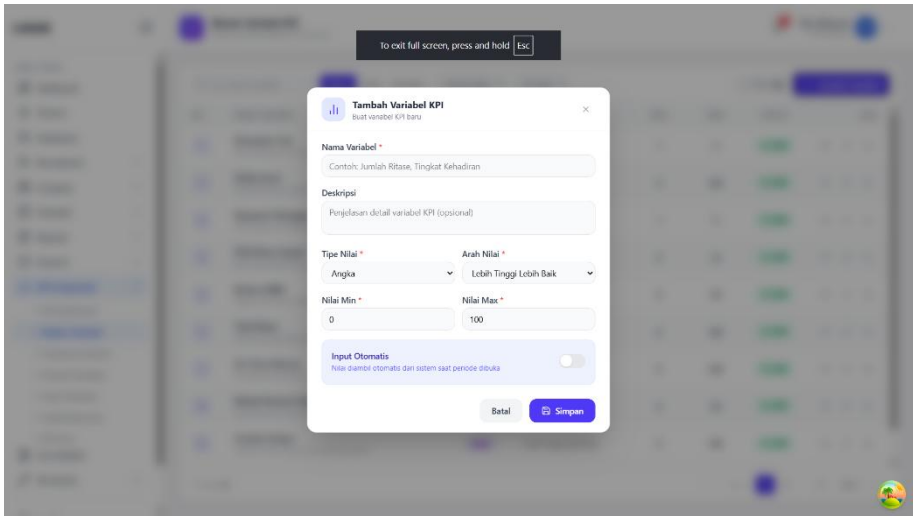
1. Master Variabel

   Halaman Master Variabel digunakan oleh HR untuk mengelola variabel KPI yang akan dipakai dalam penilaian kinerja karyawan. Halaman ini menjadi titik awal alur konfigurasi sistem, di mana HR perlu mendefinisikan variabel KPI terlebih dahulu sebelum dapat melanjutkan ke tahap *assignment* dan periode penilaian.



Gambar 63. Halaman Daftar Master Variabel KPI

Tampilan utama menampilkan daftar variabel KPI dalam bentuk tabel dengan kolom nama variabel, tipe nilai, arah nilai, nilai minimum, nilai maksimum, dan status, yang dilengkapi dengan fitur pencarian, *filter* status aktif dan nonaktif, *filter* tipe nilai, paginasi, serta tombol aksi pada setiap baris untuk melihat detail, mengubah, dan menonaktifkan variabel. Formulir tambah variabel disajikan dalam bentuk *modal dialog* yang memuat *field* nama, deskripsi, tipe nilai dengan pilihan Angka, Skala, dan Persentase, arah nilai dengan pilihan Lebih Tinggi Lebih Baik dan Lebih Rendah Lebih Baik, serta nilai minimum dan maksimum, di mana atribut arah nilai inilah yang dipakai oleh fungsi normalisasi pada sub-bab 4.3.3 untuk menentukan formula *min-max* yang sesuai.



Gambar 64. *Modal* Tambah Variabel KPI

Implementasi *backend* untuk pengelolaan *master* variabel terdapat pada *KpiVariabelService* yang berisi enam *method*, yaitu *create*, *findAll*, *findOne*, *update*, *remove*, dan *getSumberDataList*. *Method create* bertanggung jawab atas penambahan variabel baru beserta validasi.

```
// Filename: src/kpi-variabel/kpi-variabel.service.ts
async create(dto: CreateKpiVariabelDto) {
  // Validasi 1: Rentang nilai harus valid
  if (dto.nilaiMax <= dto.nilaiMin) {
    throw new BadRequestException(
      'nilaiMax harus lebih besar dari nilaiMin'
    );
  }

  // Validasi 2: Sumber data wajib untuk tipe input otomatis
  if (dto.tipeInput === 'otomatis') {
    if (!dto.sumberData) {
      throw new BadRequestException(
        'sumberData wajib diisi jika tipeInput otomatis'
      );
    }
    const isRegistered = this.datasourceService
      .getAllSources()
      .find((s) => s.key === dto.sumberData);
    if (!isRegistered) {
      throw new BadRequestException(
        `sumberData '${dto.sumberData}' tidak tersedia di registry`
      );
    }
  }
```

```
  // Validasi 3: Cek duplikasi nama variabel
  const existing = await this.prisma.refKpiVariabel.findFirst({
    where: { namaVariabel: dto.namaVariabel },
  });
  if (existing) {
    throw new ConflictException(
      `Variabel KPI dengan nama "${dto.namaVariabel}" sudah ada`
    );
  }

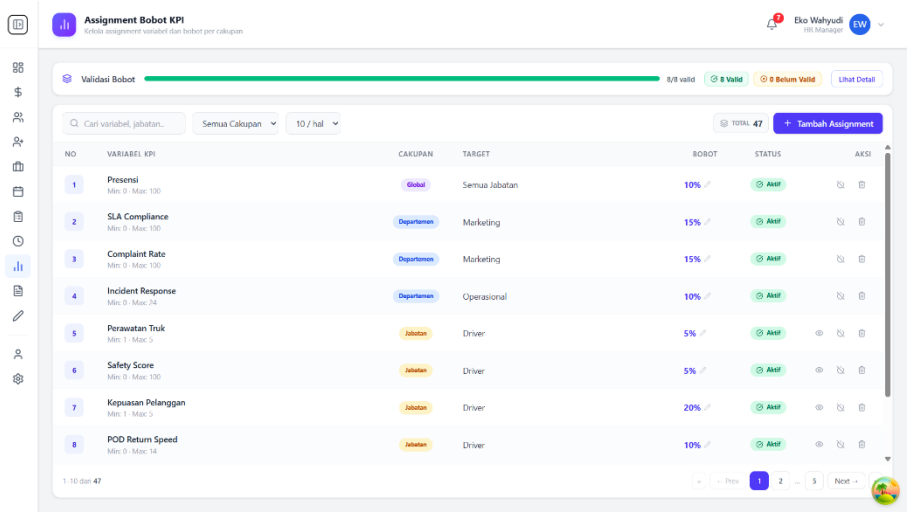
  // Simpan variabel KPI ke database
  return this.prisma.refKpiVariabel.create({ data: { ...dto } });
}
```

Method create menjalankan tiga validasi utama sebelum menyimpan data, yaitu pemeriksaan nilai maksimum harus lebih besar dari nilai minimum agar rentang nilai variabel selalu valid, validasi *field sumberData* yang wajib diisi dan terdaftar pada *registry datasource* apabila tipe *input* adalah otomatis, serta pemeriksaan *unique* pada nama variabel untuk mencegah duplikasi. *Field tipeInput default* bernilai manual apabila tidak dispesifikasikan, dan *field sumberData* diisi *null* apabila tipe *input* bukan otomatis. Fitur *input* otomatis disediakan untuk mendukung integrasi variabel KPI dengan modul lain dalam sistem, meskipun detail fitur ini berada di luar lingkup utama penelitian.

Method findAll, *findOne*, *update*, *remove*, dan *getSumberDataList* melengkapi operasi pengelolaan variabel KPI sesuai dengan pola CRUD standar. *Method remove* secara khusus menerapkan mekanisme *soft delete* dengan mengubah status *isActive* menjadi *false* setelah memeriksa bahwa variabel tidak sedang digunakan pada *assignment* aktif, karena variabel KPI memiliki keterkaitan historis dengan data penilaian dan *snapshot* variabel periode sebelumnya yang akan kehilangan referensi jika dihapus permanen.

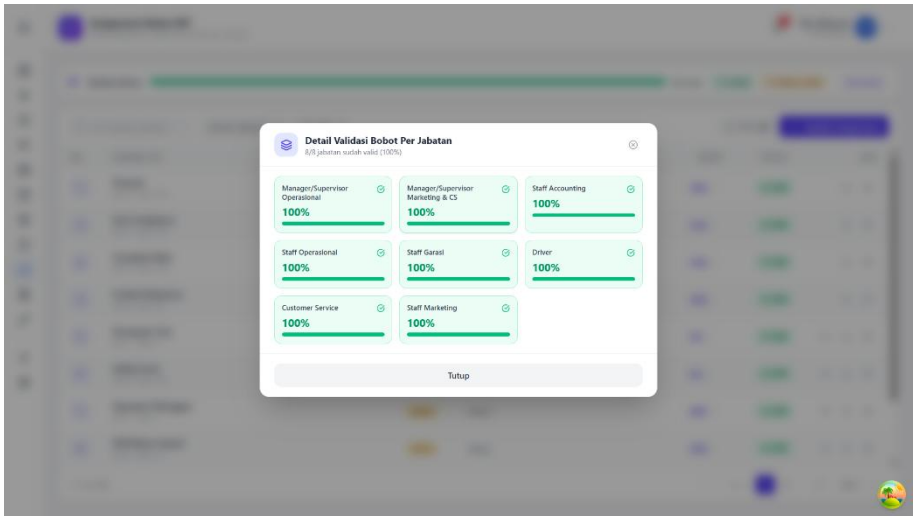
2. *Assignment* Bobot

Halaman *Assignment* Bobot digunakan oleh HR untuk mengelola penempatan variabel KPI ke dalam tiga cakupan, yaitu global, departemen, dan jabatan, beserta bobot masing-masing variabel. Halaman ini merupakan implementasi konkret dari *Decorator Pattern* yang telah dijelaskan pada sub-bab 4.3.2.



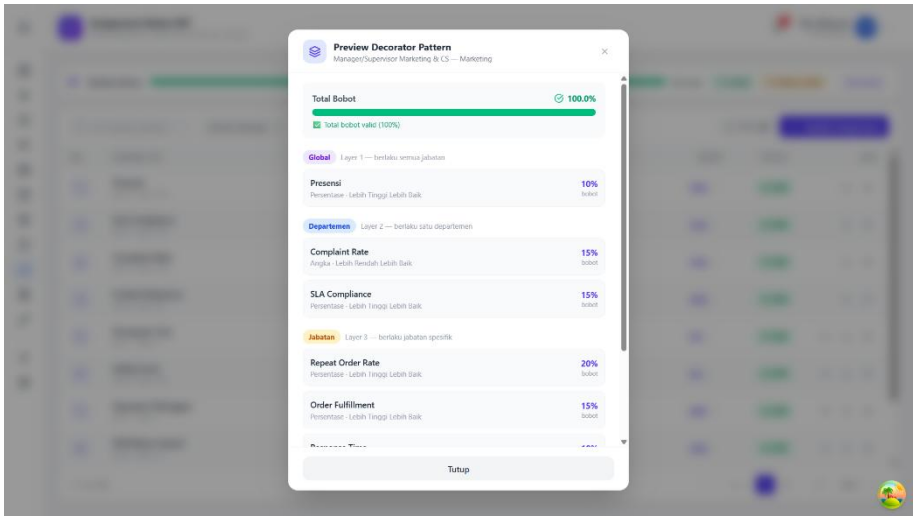
Gambar 65. Halaman Daftar *Assignment* Bobot

Tampilan utama menampilkan *banner* validasi bobot pada bagian atas yang memantau total bobot per jabatan harus sama dengan seratus persen, sesuai dengan persyaratan formula skor akhir pada sub-bab 4.3.3. Tabel di bawah *banner* menampilkan daftar *assignment* dengan *badge* berwarna berbeda untuk setiap cakupan, yaitu ungu untuk global, biru untuk departemen, dan kuning untuk jabatan. Nilai bobot pada setiap baris dapat diubah secara *inline* melalui ikon *edit* di samping nilai. Apabila HR menekan tombol Lihat Detail pada *banner*, sistem menampilkan *modal* validasi per jabatan untuk pemeriksaan menyeluruh.



Gambar 66. *Modal* Detail Validasi Bobot Per Jabatan

Fitur paling menarik pada halaman ini adalah *modal Preview Decorator Pattern* yang dapat diakses melalui tombol mata pada baris *assignment* dengan cakupan jabatan. *Modal* ini menampilkan visualisasi langsung dari fungsi *resolveVariabelDecorator* pada sub-bab 4.3.2, di mana variabel dikelompokkan berdasarkan *layer* asalnya pada cakupan global, departemen, dan jabatan, beserta total bobot hasil deduplikasi.



Gambar 67. *Modal* Preview Decorator Pattern

Implementasi *backend* untuk pengelolaan *assignment* bobot terdapat pada *KpiAssignmentService* yang berisi delapan *method*, yaitu *create*, *findAll*, *findOne*, *update*, *remove*, *nonaktifkan*, *reaktifkan*, dan *preview*.

Method create bertanggung jawab atas penambahan *assignment* baru beserta validasi berlapis.

```
// Filename: src/kpi-assignment/kpi-assignment.service.ts
async create(dto: CreateKpiAssignmentDto) {
  // Validasi 1: Periode aktif (integrity guard)
  await this.cekPeriodeAktif();

  // Validasi 2: Kesesuaian cakupan dengan referensi
  this.validasiCakupan(dto.cakupan, dto.idDepartemen,
dto.idJabatan);

  // Validasi 3: Variabel KPI ada dan aktif
  const variabel = await this.prisma.refKpiVariabel.findUnique({
    where: { idVariabel: dto.idVariabel },
    select: { idVariabel: true, isActive: true, namaVariabel: true },
  });
  if (!variabel) {
    throw new NotFoundException(
      `Variabel KPI dengan ID ${dto.idVariabel} tidak ditemukan`
    );
  }
  if (!variabel.isActive) {
    throw new BadRequestException(
      `Variabel KPI "${variabel.namaVariabel}" tidak aktif`
    );
  }

  // Validasi 4: Referensi departemen/jabatan (detail lihat lampiran)
  // ... validasi keberadaan departemen atau jabatan sesuai cakupan

  // Validasi 5: Cek duplikasi berdasarkan kombinasi unique
  const duplikat = await
this.prisma.refKpiVariabelAssignment.findFirst({
    where: {
      idVariabel: dto.idVariabel,
      cakupan: dto.cakupan,
      idDepartemen: dto.idDepartemen ?? null,
      idJabatan: dto.idJabatan ?? null,
    },
  });
  if (duplikat) {
    throw new ConflictException('Variabel sudah di-assign ke
cakupan ini');
  }

  // Simpan assignment ke database
```

```
  const assignment = await
this.prisma.refKpiVariabelAssignment.create({
    data: { ...dto },
    include: {
      variabel: { select: { namaVariabel: true } },
      departemen: { select: { namaDepartemen: true } },
      jabatan: { select: { namaJabatan: true } },
    },
  });

  // Hitung total bobot real-time untuk validasi 100%
  let totalBobot = 0;
  if (dto.cakupan === CakupanAssignment.jabatan &&
dto.idJabatan) {
    totalBobot = await hitungTotalBobot(this.prisma, dto.idJabatan);
  }

  return {
    assignmentBaru: assignment,
    totalBobotSaatIni: totalBobot,
    isValid: Math.abs(totalBobot - 100) < 0.01,
  };
}
```

Method create menjalankan rangkaian validasi sebelum menyimpan *assignment* baru, yaitu pemeriksaan periode aktif melalui *cekPeriodeAktif*, validasi kesesuaian *field* cakupan dengan identitas departemen atau jabatan, validasi keberadaan variabel KPI dan statusnya, validasi referensi departemen atau jabatan, dan pemeriksaan duplikasi berdasarkan kombinasi *unique*. Pemeriksaan periode aktif berfungsi sebagai *integrity guard* yang mencegah perubahan konfigurasi di tengah proses penilaian sehingga *reproducibility* hasil klasterisasi tetap terjaga, kemudian setelah validasi terpenuhi sistem menyimpan *assignment* dan mengembalikan *response* yang menyertakan *flag isValid* berdasarkan total bobot *real-time* sehingga antarmuka pengguna dapat langsung menampilkan status validitas.

Method *preview* merupakan *method* paling representatif terhadap kontribusi penelitian karena mengimplementasikan visualisasi *Decorator Pattern* secara langsung.

```
// Filename: src/kpi-assignment/kpi-assignment.service.ts
async preview(idJabatan: string) {
```

```
  const jabatan = await this.prisma.refJabatan.findUnique({
    where: { idJabatan },
    select: {
      idJabatan: true,
      namaJabatan: true,
      idDepartemen: true,
      isMitra: true,
      isDinilai: true,
      departemen: { select: { namaDepartemen: true } },
    },
  });
  if (!jabatan) {
    throw new NotFoundException(
      `Jabatan dengan ID ${idJabatan} tidak ditemukan`
    );
  }

  // Early return untuk jabatan penilai (tidak dinilai)
  if (!jabatan.isDinilai) {
    return {
      jabatan: { /* data jabatan */ },
      isDinilai: false,
      isMitra: jabatan.isMitra,
      variabelList: [],
      totalBobot: 0,
      isValid: true,
    };
  }

  // Layer 1 — Global (dilewati jika Mitra)
  const layerGlobal = jabatan.isMitra
    ? []
    : await this.prisma.refKpiVariabelAssignment.findMany({
        where: { isActive: true, cakupan: CakupanAssignment.global
},
        include: { variabel: { select: { /* atribut variabel */ } } },
      });

  // Layer 2 — Departemen (dilewati jika Mitra)
  const layerDepartemen = jabatan.isMitra
    ? []
    : await this.prisma.refKpiVariabelAssignment.findMany({
        where: {
          isActive: true,
          cakupan: CakupanAssignment.departemen,
          idDepartemen: jabatan.idDepartemen,
        },
        include: { variabel: { select: { /* atribut variabel */ } } },
```

```
    });

  // Layer 3 — Jabatan
  const layerJabatan = await
this.prisma.refKpiVariabelAssignment.findMany({
    where: {
      isActive: true,
      cakupan: CakupanAssignment.jabatan,
      idJabatan,
    },
    include: { variabel: { select: { /* atribut variabel */ } } },
  });

  // Deduplikasi dengan Map — prioritas jabatan > departemen >
global
  const variabelMap = new Map();
  for (const a of layerGlobal) {
    variabelMap.set(a.idVariabel, { ...a, layer: 'global' });
  }
  for (const a of layerDepartemen) {
    variabelMap.set(a.idVariabel, { ...a, layer: 'departemen' });
  }
  for (const a of layerJabatan) {
    variabelMap.set(a.idVariabel, { ...a, layer: 'jabatan' });
  }

  const variabelList = Array.from(variabelMap.values());
  const totalBobot = variabelList.reduce(
    (sum, a) => sum + Number(a.bobot),
    0
  );

  return {
    jabatan: { /* data jabatan */ },
    isDinilai: true,
    isMitra: jabatan.isMitra,
    variabelList,
    totalBobot,
    isValid: Math.abs(totalBobot - 100) < 0.01,
  };
}
```

Method preview mengambil informasi jabatan termasuk *flag isMitra* dan *isDinilai*, kemudian melakukan *early return* untuk jabatan tidak dinilai seperti Komisaris atau Direktur. Untuk jabatan yang dinilai, sistem mengambil tiga *layer assignment* dengan *handling* khusus untuk jabatan mitra yang melewati *layer* global dan departemen sesuai logika

MitraStrategy pada sub-bab 4.3.1, sedangkan deduplikasi dilakukan menggunakan *Map* dengan urutan iterasi global, departemen, dan jabatan secara berurutan sehingga variabel pada *layer* dengan prioritas tertinggi akan menggantikan variabel dengan nama yang sama pada *layer* lebih rendah. *Field layer* yang disertakan pada setiap variabel memungkinkan antarmuka pengguna mengelompokkan variabel berdasarkan *layer* asalnya pada *modal Preview*.

Method preview didukung oleh *helper function* hitungTotalBobot yang digunakan oleh *method create* dan *update* untuk menghitung total bobot *real-time*.

```
// Filename: src/kpi-assignment/kpi-assignment.service.ts
async function hitungTotalBobot(
  prisma: PrismaService,
  idJabatan: string,
): Promise<number> {
  const jabatan = await prisma.refJabatan.findUnique({
    where: { idJabatan },
    select: { idDepartemen: true, isMitra: true },
  });
  if (!jabatan) return 0;

  // Jabatan Mitra: hanya layer jabatan, skip global & departemen
  // Jabatan Non-Mitra: tiga layer (global, departemen, jabatan)
  const whereCondition = jabatan.isMitra
    ? { cakupan: CakupanAssignment.jabatan, idJabatan }
    : {
        OR: [
          { cakupan: CakupanAssignment.global },
          {
            cakupan: CakupanAssignment.departemen,
            idDepartemen: jabatan.idDepartemen,
          },
          { cakupan: CakupanAssignment.jabatan, idJabatan },
        ],
      };

  const assignments = await
prisma.refKpiVariabelAssignment.findMany({
    where: {
      isActive: true,
      ...whereCondition,
    },
    select: { bobot: true },
```

```
  });

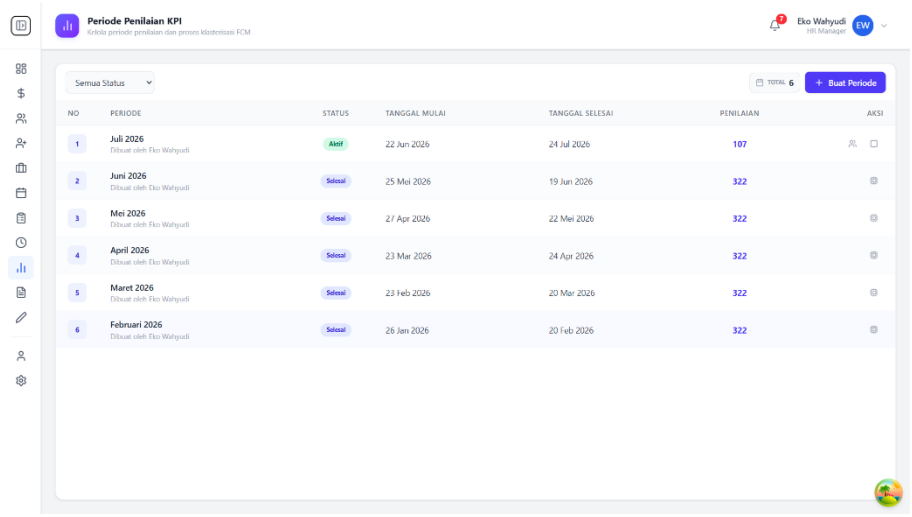
  return assignments.reduce((sum, a) => sum + Number(a.bobot),
0);
}
```

Helper hitungTotalBobot mengimplementasikan logika yang konsisten dengan *strategy* pada sub-bab 4.3.1, di mana jabatan mitra hanya memperhitungkan bobot dari *layer* jabatan, sedangkan jabatan non-mitra memperhitungkan bobot dari tiga *layer* sekaligus. Konsistensi logika antara *helper* ini, *method preview*, dan implementasi *strategy* menjamin total bobot yang dihitung saat konfigurasi sesuai dengan variabel yang muncul saat penilaian.

Method findAll dan *findOne* melengkapi operasi pembacaan dengan dukungan *filter*, paginasi, dan *include* relasi. *Method update* hanya memperbolehkan perubahan *field* bobot dengan pemeriksaan periode aktif terlebih dahulu untuk mencegah inkonsistensi data, sedangkan *method remove*, *nonaktifkan*, dan *reaktifkan* melengkapi operasi penghapusan dan reaktivasi dengan ketiganya menjalankan *cekPeriodeAktif* sebelum eksekusi.

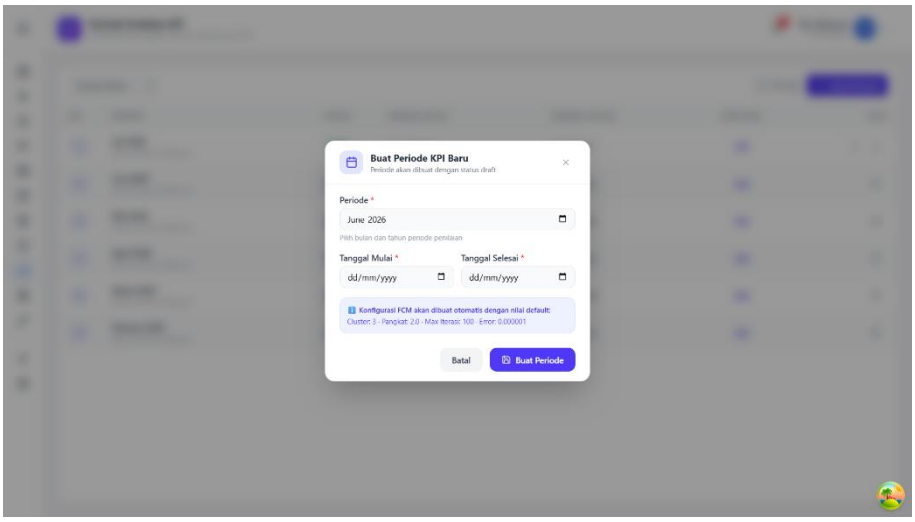
3. Periode Penilaian

Halaman Periode Penilaian digunakan oleh HR untuk mengelola periode penilaian KPI beserta konfigurasi parameter *Fuzzy C-Means* yang akan dipakai saat klasterisasi. Halaman ini menjadi pusat orkestrasi seluruh proses penilaian, mulai dari membuka periode untuk dimulainya *input* nilai, memantau *progress* pengisian, sampai memicu eksekusi algoritma klasterisasi setelah periode ditutup.



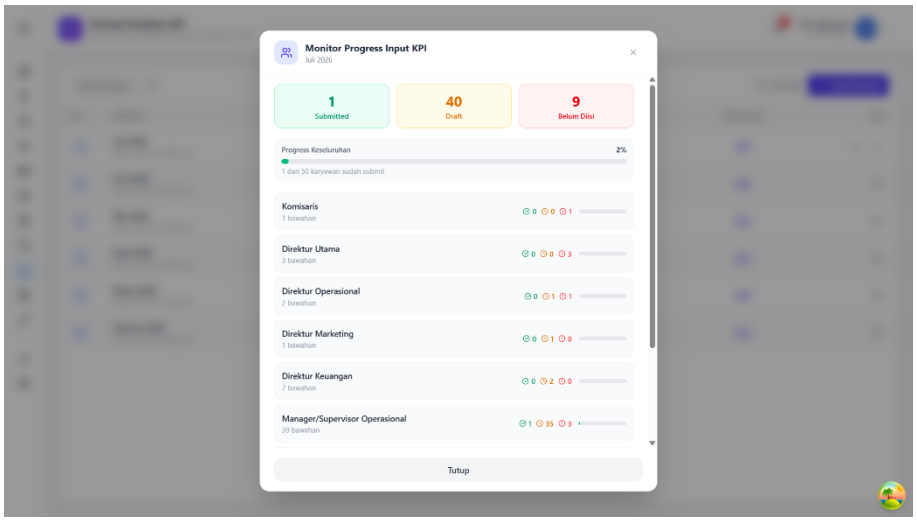
Gambar 68. Halaman Daftar Periode Penilaian KPI

Tampilan utama menampilkan daftar periode dengan kolom periode, status, tanggal mulai, tanggal selesai, jumlah penilaian, dan aksi. Setiap periode memiliki lima status yang merepresentasikan *lifecycle*, yaitu *draft* saat periode baru dibuat, aktif saat periode dibuka untuk *input* nilai, ditutup saat periode ditutup dan menunggu eksekusi FCM, *processing* saat algoritma FCM sedang dijalankan oleh *Bull Queue*, dan selesai saat klasterisasi berhasil. Status *error* dapat muncul apabila eksekusi FCM gagal. Aksi yang tersedia pada setiap baris berbeda sesuai status periode, di mana periode aktif memiliki aksi *Monitor Progress* untuk memantau pengisian nilai, sedangkan periode selesai memiliki aksi *FCM Status* untuk melihat hasil dan melakukan *rerun*.



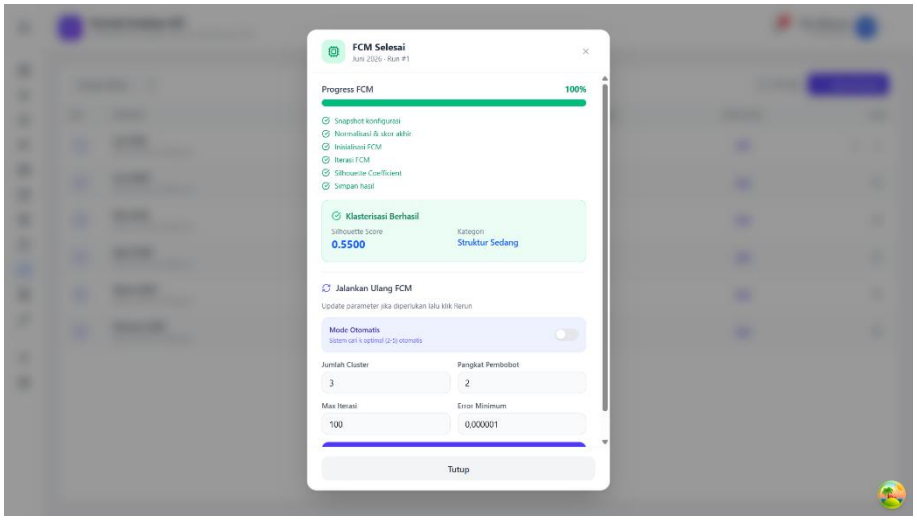
Gambar 69. *Modal* Buat Periode KPI Baru

Saat HR membuat periode baru, sistem menampilkan *modal* dengan *field* bulan, tahun, tanggal mulai, dan tanggal selesai. Pada bagian bawah *modal* terdapat informasi bahwa konfigurasi FCM akan dibuat otomatis dengan nilai *default* jumlah *cluster* sebanyak tiga, pangkat pembobot dua, maksimum iterasi seratus, dan *error* minimum nol koma nol nol nol nol nol satu. Nilai *default* ini sesuai dengan konfigurasi yang ditetapkan sebagai standar implementasi sistem pada sub-bab 4.3.4, di mana jumlah *cluster* sebanyak tiga dipilih untuk menghasilkan kategorisasi *High Performer*, *Standard*, dan *Low Performer*.



Gambar 70. *Modal* Monitor Progress Input KPI

Saat periode dalam status aktif, HR dapat memantau *progress* pengisian nilai melalui *modal Monitor Progress* yang menampilkan statistik agregat dan rincian per jabatan atasan. Statistik agregat mencakup jumlah karyawan dengan status *submitted*, *draft*, dan belum diisi, dilengkapi dengan *progress bar* keseluruhan. Rincian per jabatan menampilkan setiap jabatan atasan beserta jumlah bawahan yang dinilai dan status pengisian masing-masing, sehingga HR dapat segera mengidentifikasi atasan yang masih perlu menyelesaikan penilaian.



Gambar 71. *Modal* FCM

Setelah periode ditutup dan algoritma FCM selesai dieksekusi, HR dapat membuka *modal FCM Status* untuk melihat hasil dan opsi *rerun*. *Modal* menampilkan *progress* eksekusi yang sudah selesai pada enam langkah utama, yaitu *snapshot* konfigurasi, normalisasi dan skor akhir, inisialisasi FCM, iterasi FCM, perhitungan *Silhouette Coefficient*, dan penyimpanan hasil. Hasil klasterisasi ditampilkan beserta nilai *Silhouette Score* dan kategori struktur sesuai interpretasi pada sub-bab 4.3.5, sedangkan bagian Jalankan Ulang FCM memungkinkan HR melakukan eksekusi ulang dengan opsi mengubah konfigurasi atau mempertahankannya, di mana apabila *rerun* dilakukan dengan konfigurasi yang identik hasil yang dihasilkan akan identik dengan eksekusi sebelumnya karena *Linear Congruential Generator* pada sub-bab 4.3.4 menggunakan *seed* deterministik dari identitas periode. *Modal* ini juga menyediakan *toggle* Mode Otomatis untuk pencarian *k* optimal secara otomatis, meskipun pembahasan detail fitur ini berada di luar lingkup utama penelitian.

Implementasi *backend* untuk pengelolaan periode penilaian terdapat pada *KpiPeriodeService* yang berisi dua belas *method* untuk menangani *lifecycle* periode dan eksekusi FCM. *Method create* bertanggung jawab atas pembuatan periode baru beserta konfigurasi FCM *default*.

```
// Filename: src/kpi-periode/kpi-periode.service.ts
async create(dto: CreateKpiPeriodeDto, idKaryawan: string) {
```

```
  // Validasi 1: Rentang tanggal periode
  const tanggalMulai = new Date(dto.tanggalMulai);
  const tanggalSelesai = new Date(dto.tanggalSelesai);
  if (tanggalSelesai <= tanggalMulai) {
    throw new BadRequestException(
      'tanggalSelesai harus lebih besar dari tanggalMulai'
    );
  }

  // Validasi 2: Cek duplikasi periode bulan-tahun
  const existing = await this.prisma.kpiPeriode.findUnique({
    where: { bulan_tahun: { bulan: dto.bulan, tahun: dto.tahun } },
  });
  if (existing) {
    throw new ConflictException(
      `Periode KPI untuk bulan ${dto.bulan}/${dto.tahun} sudah ada`
    );
  }

  // Transaksi atomik: buat periode + konfigurasi FCM default
  const periode = await this.prisma.$transaction(async (tx) => {
    const p = await tx.kpiPeriode.create({
      data: {
        bulan: dto.bulan,
        tahun: dto.tahun,
        tanggalMulai,
        tanggalSelesai,
        dibuatOleh: idKaryawan,
        status: StatusPeriodeKpi.draft,
      },
    });

    // Konfigurasi FCM default sesuai sub-bab 4.3.4
    await tx.kpiFcmConfig.create({
      data: {
        idPeriode: p.idPeriode,
        jumlahCluster: 3,
        pangkatPembobot: 2.0,
        maxIterasi: 100,
        errorMinimum: 0.000001,
      },
    });

    return p;
  });

  return this.findOne(periode.idPeriode);
}
```

Method create melakukan validasi tanggal selesai harus lebih besar dari tanggal mulai dan pemeriksaan *unique constraint* pada kombinasi bulan dan tahun untuk mencegah duplikasi periode. Setelah validasi terpenuhi, periode beserta konfigurasi FCM *default* dibuat dalam satu transaksi *database*, sehingga apabila salah satu operasi gagal, keduanya akan di-*rollback* secara otomatis. Periode dibuat dengan status awal *draft* yang masih memungkinkan perubahan konfigurasi sebelum periode dibuka.

Method bukaPeriode mengubah status periode dari *draft* menjadi aktif dan mengirim notifikasi kepada seluruh atasan untuk memulai pengisian nilai.

```
// Filename: src/kpi-periode/kpi-periode.service.ts
async bukaPeriode(idPeriode: string) {
  const periode = await this.findOne(idPeriode);

  // Validasi 1: Status harus draft (state machine)
  if (periode.status !== StatusPeriodeKpi.draft) {
    throw new BadRequestException(
      'Hanya periode berstatus draft yang bisa dibuka'
    );
  }

  // Validasi 2: Tidak ada periode lain yang aktif (mutual exclusion)
  const periodeAktif = await this.prisma.kpiPeriode.findFirst({
    where: { status: StatusPeriodeKpi.aktif },
    select: { idPeriode: true, bulan: true, tahun: true },
  });
  if (periodeAktif) {
    throw new ConflictException(
      `Periode ${periodeAktif.bulan}/${periodeAktif.tahun} sedang
aktif`
    );
  }

  // Validasi 3: Total bobot semua jabatan harus 100%
  const jabatanInvalid = await this.validasiBobotSemuaJabatan();
  if (jabatanInvalid.length > 0) {
    throw new BadRequestException({
      message: 'Validasi bobot gagal — beberapa jabatan memiliki
total bobot ≠ 100%',
      jabatanInvalid,
    });
  }
```

```
  // Update status periode menjadi aktif
  await this.prisma.kpiPeriode.update({
    where: { idPeriode },
    data: { status: StatusPeriodeKpi.aktif },
  });

  // Fire-and-forget: fetch nilai otomatis untuk variabel
tipeInput='otomatis'
  void
this.datasourceService.fetchNilaiOtomatis(idPeriode).catch((err) =>
{
    console.error(`Gagal fetch nilai otomatis: ${err.message}`);
  });

  // Kirim notifikasi batch ke semua atasan aktif
  const atasanAktif = await this.prisma.refKaryawan.findMany({
    where: {
      statusKeaktifan: true,
      status: 'aktif' as const,
      jabatan: { idAtasan: { not: null } },
    },
    select: { idKaryawan: true },
  });

  if (atasanAktif.length > 0) {
    await this.prisma.notifikasi.createMany({
      data: atasanAktif.map((k) => ({
        idKaryawan: k.idKaryawan,
        judul: 'Periode KPI Dibuka',
        pesan: `Periode penilaian KPI bulan
${periode.bulan}/${periode.tahun} telah dibuka`,
        tipe: 'kpi_periode_dibuka',
        tipeReferensi: 'kpi_periode',
        idReferensi: idPeriode,
      })),
    });
  }

  return this.findOne(idPeriode);
}
```

Method bukaPeriode melakukan validasi berlapis sebelum mengubah status periode. Pemeriksaan pertama memastikan periode masih berstatus *draft*, pemeriksaan kedua memastikan tidak ada periode lain yang sedang aktif karena sistem hanya mengizinkan satu periode aktif pada satu waktu, dan pemeriksaan ketiga memvalidasi total bobot

semua jabatan yang dinilai harus sama dengan seratus persen melalui *helper validasiBobotSemuaJabatan*. Validasi bobot ini menggunakan logika resolusi tiga *layer* yang konsisten dengan *Decorator Pattern* pada sub-bab 4.3.2 dan *helper* hitungTotalBobot pada sub-bagian *Assignment* Bobot. Setelah seluruh validasi terpenuhi, sistem mengubah status periode menjadi aktif, melakukan *fetch* nilai otomatis untuk variabel dengan tipe *input* otomatis, dan mengirim notifikasi kepada seluruh atasan aktif.

Method tutupPeriode mengubah status periode dari aktif menjadi ditutup, kemudian mengirim job ke *Bull Queue* untuk eksekusi algoritma FCM secara asinkron.

```
// Filename: src/kpi-periode/kpi-periode.service.ts
async tutupPeriode(idPeriode: string, dto: TutupKpiPeriodeDto) {
  const periode = await this.findOne(idPeriode);

  // Validasi: hanya periode aktif yang bisa ditutup
  if (periode.status !== StatusPeriodeKpi.aktif) {
    throw new BadRequestException(
      'Hanya periode berstatus aktif yang bisa ditutup'
    );
  }

  // Identifikasi karyawan yang belum dinilai
  const karyawanAktif = await this.prisma.refKaryawan.findMany({
    where: { statusKeaktifan: true, status: 'aktif' as const },
    select: { idKaryawan: true, nama: true },
  });

  const sudahSubmit = await this.prisma.kpiPenilaian.findMany({
    where: {
      idPeriode,
      status: 'submitted',
      idKaryawan: { in: karyawanAktif.map((k) => k.idKaryawan) },
    },
    select: { idKaryawan: true },
    distinct: ['idKaryawan'],
  });

  const idSudahSubmit = new Set(sudahSubmit.map((p) =>
p.idKaryawan));
  const karyawanBelumDinilai = karyawanAktif.filter(
    (k) => !idSudahSubmit.has(k.idKaryawan)
  );
```

```
  // Mekanisme force close: warning jika ada yang belum dinilai
  if (karyawanBelumDinilai.length > 0 && !dto.forceClose) {
    return {
      warning: true,
      karyawanBelumDinilai,
      totalBelum: karyawanBelumDinilai.length,
      totalKaryawan: karyawanAktif.length,
    };
  }

  // Karyawan yang di-skip dari klasterisasi FCM
  const idKaryawanDiskip = karyawanBelumDinilai.map((k) =>
k.idKaryawan);

  // Ubah status menjadi ditutup
  await this.prisma.kpiPeriode.update({
    where: { idPeriode },
    data: { status: StatusPeriodeKpi.ditutup },
  });

  // Kirim job ke Bull Queue dengan retry mechanism
  await this.fcmQueue.add(
    'fcm-process',
    { idPeriode, idKaryawanDiskip },
    {
      attempts: 3,
      backoff: { type: 'exponential', delay: 5000 },
      removeOnComplete: false,
      removeOnFail: false,
    },
  );

  // Ubah status menjadi processing sebagai indikasi FCM berjalan
  await this.prisma.kpiPeriode.update({
    where: { idPeriode },
    data: { status: StatusPeriodeKpi.processing },
  });

  return {
    warning: false,
    status: StatusPeriodeKpi.processing,
    karyawanDiskip: idKaryawanDiskip.length,
  };
}
```

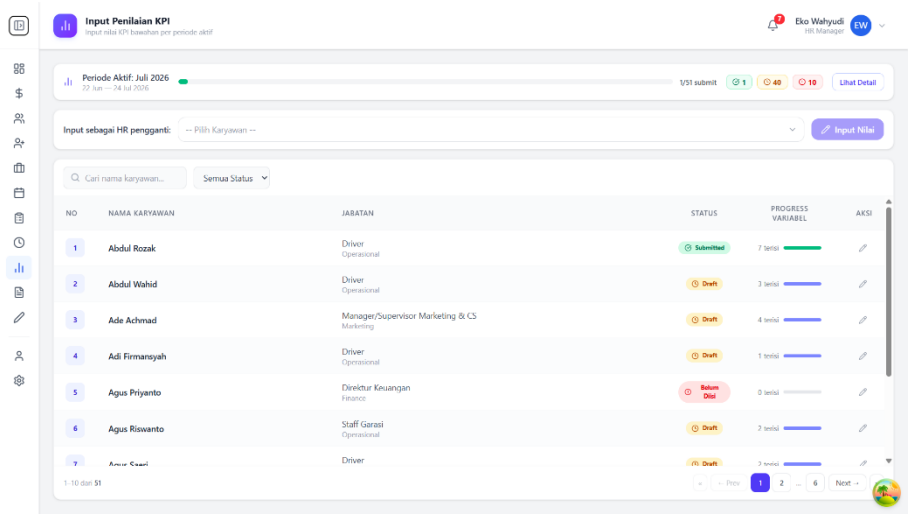
Method tutupPeriode memulai dengan pemeriksaan status periode harus aktif, kemudian mengidentifikasi karyawan yang belum dinilai

dengan membandingkan daftar karyawan aktif dengan daftar karyawan berstatus *submitted*. Apabila terdapat karyawan yang belum dinilai dan parameter *forceClose* bernilai *false*, sistem mengembalikan *warning* untuk konfirmasi HR. Apabila HR menyetujui penutupan dengan *force close* atau seluruh karyawan sudah dinilai, sistem mengubah status menjadi ditutup, mengirim *job* ke *Bull Queue* dengan *retry policy* tiga percobaan dan *exponential backoff*, kemudian mengubah status menjadi *processing*. *Job* yang dikirim ke *queue* inilah yang memicu eksekusi algoritma FCM dan perhitungan *Silhouette Coefficient* sesuai sub-bab 4.3.4 dan 4.3.5.

Sembilan *method* lainnya melengkapi operasi pengelolaan periode. *Method findAll* dan *findOne* mengambil data periode beserta konfigurasi FCM, *method update*, *remove*, dan *updateFcmConfig* hanya memperbolehkan perubahan pada periode berstatus *draft*, sedangkan *method monitorProgress* mengambil statistik pengisian nilai per jabatan atasan untuk ditampilkan pada *modal Monitor Progress*. *Method getFcmStatus* melakukan *polling* terhadap *Bull Queue* untuk memantau *progress* eksekusi FCM, *method rerunFcm* memungkinkan eksekusi ulang dengan opsi mengubah konfigurasi atau mempertahankannya, dan *method unlockSubmission* memungkinkan HR membuka kembali penilaian *submitted* untuk koreksi. *Helper* internal *resolveVariabelPerJabatan* dan *validasiBobotSemuaJabatan* mendukung validasi bobot saat pembukaan periode dengan logika yang konsisten dengan pengelolaan *assignment*.

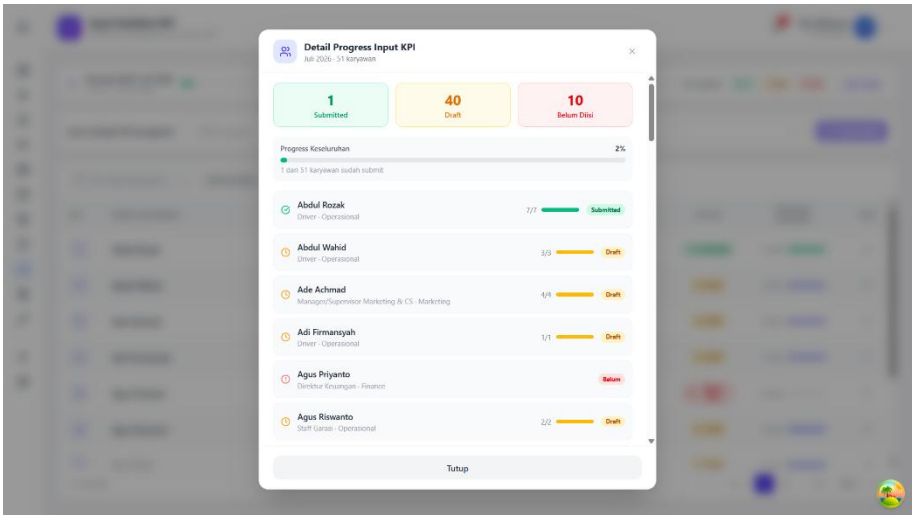
4. *Input* Penilaian

   Halaman *Input* Penilaian digunakan oleh Atasan untuk mengisi nilai KPI bawahannya pada periode penilaian yang sedang aktif. Halaman ini merupakan titik konvergensi seluruh kontribusi penelitian karena pemanggilan *Strategy Pattern* pada sub-bab 4.3.1 yang secara dinamis memanggil *Decorator Pattern* pada sub-bab 4.3.2 terjadi pada halaman ini.



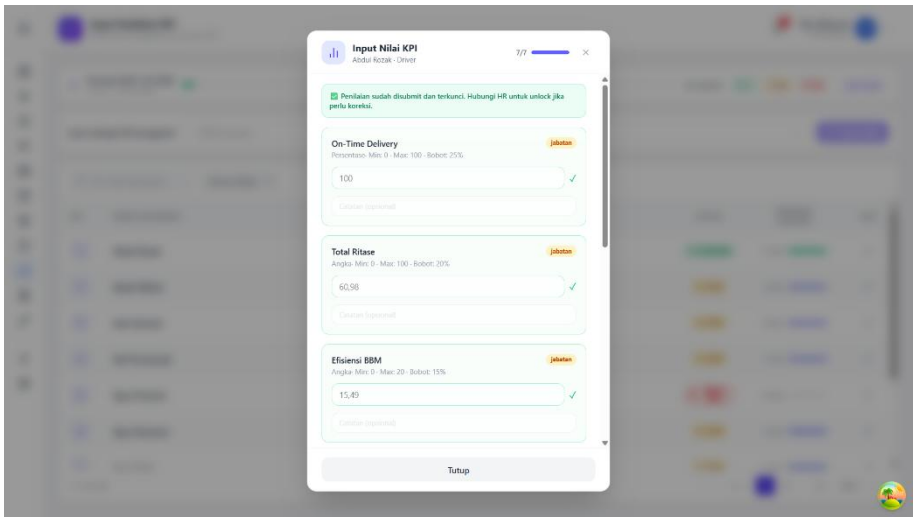
Gambar 72. Halaman *Input* Penilaian KPI

Tampilan utama menampilkan *banner* periode aktif beserta statistik *progress submission* dan *badge* ringkasan status *submitted*, *draft*, dan belum diisi. Di bawah *banner* terdapat *section* khusus untuk HR pengganti yang memungkinkan HR melakukan *input* nilai sebagai pengganti atasan, sesuai dengan implementasi URS-HR-15. Tabel utama menampilkan daftar bawahan dengan kolom nama karyawan, jabatan, status penilaian, dan *progress* variabel, dilengkapi dengan *filter* pencarian dan *filter* status. *Badge* status pada setiap baris memberikan indikator visual berupa hijau untuk *submitted*, kuning untuk *draft*, dan merah untuk belum diisi.



Gambar 73. *Modal* Detail *Progress* Input KPI

Atasan dapat membuka *modal* Detail *Progress* untuk melihat status pengisian seluruh bawahannya secara menyeluruh, dengan rincian status *submitted*, *draft*, atau belum diisi per karyawan beserta *progress bar*.



Gambar 74. *Modal Input* Nilai KPI untuk Karyawan dengan Jabatan *Driver*

Ketika Atasan menekan tombol aksi pada baris bawahan, sistem menampilkan *modal Input* Nilai yang berisi daftar variabel KPI yang berlaku untuk karyawan tersebut. Pada contoh tampilan, karyawan Abdul Rozak dengan jabatan *Driver* menampilkan variabel *On-Time Delivery*, Total Ritase, Efisiensi BBM, dan variabel lainnya yang seluruhnya memiliki *badge layer* asal jabatan. Tampilan ini merupakan bukti visual bahwa *MitraStrategy* pada sub-bab 4.3.1 bekerja sesuai rancangan, di mana karyawan dengan jabatan mitra hanya menggunakan variabel dari *layer* jabatan tanpa mengikutsertakan *layer* global dan departemen. Untuk karyawan dengan jabatan reguler yang memiliki *assignment* cakupan jabatan, *modal* akan menampilkan variabel dari tiga *layer* sekaligus sesuai resolusi *JobSpecificStrategy*, sedangkan untuk karyawan dengan jabatan reguler tanpa *assignment* cakupan jabatan, *modal* akan menampilkan variabel dari *layer* global dan departemen sesuai resolusi *DefaultStrategy*. Setiap variabel dilengkapi dengan informasi tipe nilai, rentang nilai minimum dan maksimum, bobot, serta *field input* nilai dan catatan opsional.

Implementasi *backend* untuk pengelolaan *input* penilaian terdapat pada *KpiInputService* yang berisi enam *method* utama dan beberapa *helper* validasi. *Method getVariabel* merupakan titik integrasi paling representatif terhadap kontribusi penelitian karena memanggil *Strategy Context* untuk resolusi variabel secara dinamis.

```
// Filename: src/kpi-input/kpi-input.service.ts
async getVariabel(
  query: QueryVariabelDto,
  idJabatanJwt: string,
  permissions: Record<string, number>,
) {
  // Validasi periode aktif
  await this.cekPeriodeAktif(query.idPeriode);

  // Validasi akses berdasarkan relasi atasan-bawahan atau HR
override
  await this.validasiAkses(query.idKaryawan, idJabatanJwt,
permissions);

  // Strategy Context memilih strategy yang tepat secara dinamis
  return this.strategyContext.resolve(query.idKaryawan,
query.idPeriode);
}
```

Method getVariabel hanya melakukan dua pemeriksaan, yaitu validasi status periode harus aktif dan validasi akses pengguna terhadap karyawan target, kemudian mendelegasikan resolusi variabel ke *strategyContext.resolve*. Pemanggilan terhadap *context* inilah yang menjadi jembatan antara *endpoint* dengan *Strategy Pattern*, di mana *context* akan secara dinamis memilih *strategy* yang sesuai berdasarkan karakteristik jabatan karyawan. *Helper validasiAkses* memeriksa *permission manage_kpi* pengguna, di mana pengguna dengan *permission* tersebut diizinkan mengakses karyawan manapun sebagai implementasi URS-HR-15 yaitu HR sebagai pengganti atasan, sedangkan pengguna lain hanya diizinkan mengakses karyawan yang berada di bawah jabatannya secara langsung melalui relasi *idAtasan*.

Method saveDraft bertanggung jawab atas penyimpanan nilai KPI dengan status *draft* beserta seluruh validasi yang diperlukan.

```
// Filename: src/kpi-input/kpi-input.service.ts
```

```
async saveDraft(
  dto: SaveDraftKpiDto,
  idJabatanJwt: string,
  idKaryawanJwt: string,
  permissions: Record<string, number>,
) {
  // Validasi prasyarat: periode aktif dan akses pengguna
  await this.cekPeriodeAktif(dto.idPeriode);
  await this.validasiAkses(dto.idKaryawan, idJabatanJwt,
permissions);

  // Cek penilaian belum disubmit (tidak bisa diedit jika sudah
submit)
  const sudahSubmit = await this.prisma.kpiPenilaian.findFirst({
    where: {
      idPeriode: dto.idPeriode,
      idKaryawan: dto.idKaryawan,
      status: 'submitted',
    },
    select: { idPenilaian: true },
  });
  if (sudahSubmit) {
    throw new BadRequestException(
      'Penilaian sudah disubmit dan tidak bisa diedit'
    );
  }

  // Resolve variabel berlaku via Strategy Pattern untuk validasi
  const resolved = await this.strategyContext.resolve(
    dto.idKaryawan,
    dto.idPeriode,
  );
  const idVariabelValid = new Set(resolved.variabel.map((v) =>
v.idVariabel));
  const hasManageKpi =
    (permissions['manage_kpi'] ?? 0) &
PERMISSION_LEVEL.UPDATE;

  // Validasi setiap nilai: variabel valid, dalam rentang, bukan
otomatis
  for (const item of dto.nilaiList) {
    if (!idVariabelValid.has(item.idVariabel)) {
      throw new BadRequestException(
        `Variabel ${item.idVariabel} tidak berlaku untuk karyawan ini`
      );
    }
    // ... validasi rentang min-max dan pengecekan variabel otomatis
  }
```

```
  // UPSERT per variabel untuk mendukung penyimpanan bertahap
  let jumlahTersimpan = 0;
  for (const item of dto.nilaiList) {
    await this.prisma.kpiPenilaian.upsert({
      where: {
        idPeriode_idKaryawan_idVariabel: {
          idPeriode: dto.idPeriode,
          idKaryawan: dto.idKaryawan,
          idVariabel: item.idVariabel,
        },
      },
      update: {
        nilai: item.nilai,
        catatan: item.catatan ?? null,
        status: 'draft',
      },
      create: {
        idPeriode: dto.idPeriode,
        idKaryawan: dto.idKaryawan,
        idPenilai: idKaryawanJwt,
        idVariabel: item.idVariabel,
        nilai: item.nilai,
        catatan: item.catatan ?? null,
        status: 'draft',
        diinputOlehHr: !!hasManageKpi,
      },
    });
    jumlahTersimpan++;
  }

  return { message: 'Draft berhasil disimpan', jumlahTersimpan };
}
```

Method saveDraft menjalankan rangkaian validasi sebelum menyimpan nilai, yaitu pemeriksaan status periode dan validasi akses pengguna, pemeriksaan bahwa penilaian belum dalam status *submitted* untuk mencegah *edit* setelah dikunci, validasi bahwa setiap variabel yang dikirim berada dalam daftar variabel yang berlaku menurut hasil resolusi *strategy*, validasi bahwa setiap nilai berada dalam rentang minimum dan maksimum yang ditetapkan, dan validasi bahwa variabel dengan tipe *input* otomatis tidak boleh diedit manual. Penyimpanan dilakukan dengan operasi *upsert* per variabel, sehingga apabila nilai untuk kombinasi periode, karyawan, dan variabel sudah ada, sistem

akan memperbarui nilai yang ada, sedangkan apabila belum ada, sistem akan membuat *record* baru dengan status *draft*.

Method submitPenilaian mengubah status seluruh *draft* penilaian karyawan menjadi *submitted* untuk mengunci nilai dari perubahan lebih lanjut.

```
// Filename: src/kpi-input/kpi-input.service.ts
async submitPenilaian(
  dto: SubmitKpiDto,
  idJabatanJwt: string,
  permissions: Record<string, number>,
) {
  // Validasi prasyarat: periode aktif dan akses pengguna
  await this.cekPeriodeAktif(dto.idPeriode);
  await this.validasiAkses(dto.idKaryawan, idJabatanJwt,
permissions);

  // Resolve variabel wajib via Strategy Pattern
  const resolved = await this.strategyContext.resolve(
    dto.idKaryawan,
    dto.idPeriode,
  );
  const idVariabelWajib = resolved.variabel.map((v) =>
v.idVariabel);

  // Validasi 1: Semua variabel wajib sudah terisi di draft
  const draftAda = await this.prisma.kpiPenilaian.findMany({
    where: {
      idPeriode: dto.idPeriode,
      idKaryawan: dto.idKaryawan,
      status: 'draft',
      idVariabel: { in: idVariabelWajib },
    },
    select: { idVariabel: true },
  });
  const idVariabelTerisi = new Set(draftAda.map((d) =>
d.idVariabel));
  const variabelKosong = idVariabelWajib.filter(
    (id) => !idVariabelTerisi.has(id),
  );
  if (variabelKosong.length > 0) {
    throw new BadRequestException({
      message: 'Semua variabel wajib diisi sebelum submit',
      // ... daftar nama variabel kosong untuk feedback ke user
    });
  }
```

```
  // Validasi 2: Variabel otomatis sudah diisi oleh sistem
  const variabelOtomatis = resolved.variabel.filter(
    (v) => v.tipeInput === 'otomatis',
  );
  for (const v of variabelOtomatis) {
    const nilaiOtomatis = await this.prisma.kpiPenilaian.findFirst({
      where: {
        idPeriode: dto.idPeriode,
        idKaryawan: dto.idKaryawan,
        idVariabel: v.idVariabel,
        isOtomatis: true,
      },
      select: { idPenilaian: true },
    });
    if (!nilaiOtomatis) {
      throw new BadRequestException(
        `Nilai otomatis untuk variabel "${v.namaVariabel}" belum
tersedia`
      );
    }
  }

  // Transisi atomik status draft → submitted
  await this.prisma.kpiPenilaian.updateMany({
    where: {
      idPeriode: dto.idPeriode,
      idKaryawan: dto.idKaryawan,
      status: 'draft',
    },
    data: { status: 'submitted' },
  });

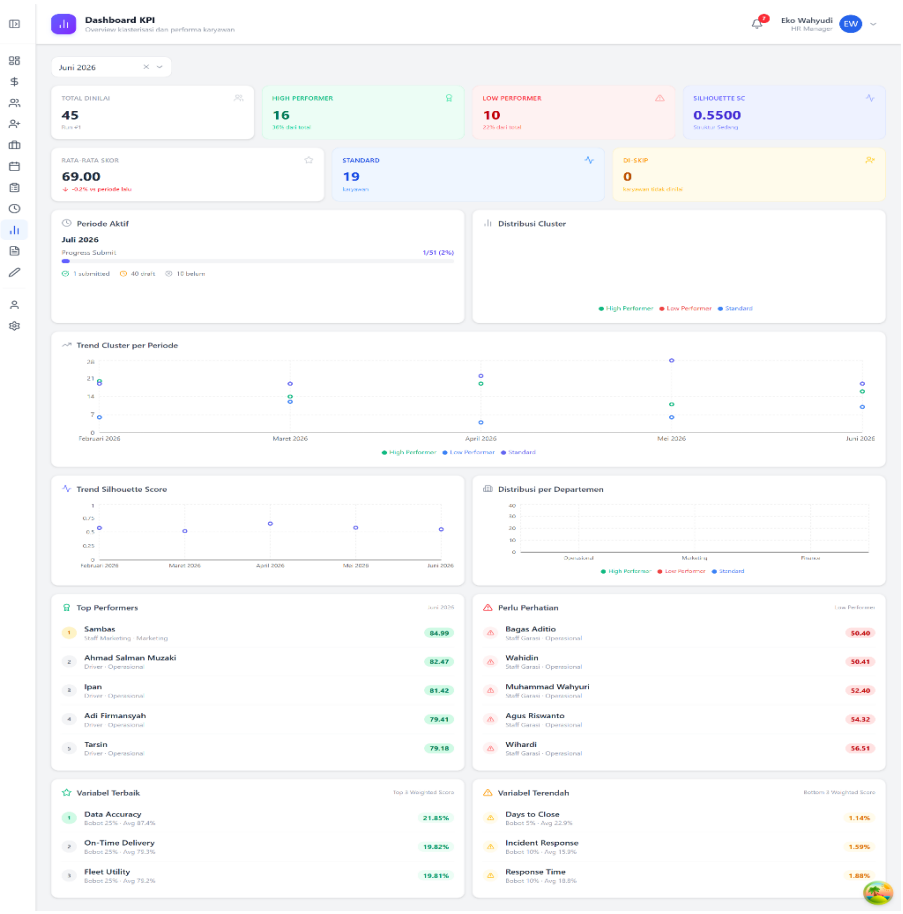
  return { message: 'Penilaian berhasil dikunci' };
}
```

Method submitPenilaian melakukan pemeriksaan kelengkapan dengan membandingkan daftar variabel yang berlaku menurut hasil resolusi *strategy* dengan daftar variabel yang sudah memiliki nilai *draft*, dan apabila terdapat variabel yang belum diisi sistem mengembalikan pesan *error* beserta daftar nama variabel yang kosong. Sistem juga memeriksa bahwa variabel dengan tipe *input* otomatis sudah memiliki nilai yang di-*fetch* oleh sistem, kemudian setelah seluruh pemeriksaan terpenuhi sistem mengubah status seluruh *draft* penilaian menjadi *submitted* dengan operasi *updateMany*, sehingga nilai tidak dapat

diubah lagi tanpa proses *unlock* oleh HR. Tiga *method* lainnya melengkapi operasi pembacaan dan informasi periode, yaitu *method getBawahan* yang mengambil daftar bawahan dengan paginasi, *filter* status, *summary* jumlah karyawan per status, dan dukungan pencarian berdasarkan nama, *method getAllBawahan* yang mengambil seluruh bawahan tanpa paginasi, serta *method getPeriodeAktif* yang mengambil periode dalam rentang tanggal aktif dengan memperhatikan zona waktu Asia/Jakarta.

5. KPI *Dashboard*

Halaman KPI *Dashboard* digunakan oleh HR untuk memantau gambaran keseluruhan klasterisasi kinerja karyawan beserta tren lintas periode. Halaman ini berfungsi sebagai pusat informasi yang merangkum hasil klasterisasi dalam berbagai bentuk visualisasi, mulai dari statistik agregat, distribusi *cluster*, tren historis, sampai analisis variabel paling dan kurang berkontribusi.



Gambar 75. Halaman KPI Dashboard

Tampilan utama dimulai dengan *dropdown* pemilih periode yang hanya menampilkan periode berstatus selesai, kemudian di bawahnya terdapat sembilan kartu statistik yang menampilkan total karyawan dinilai, jumlah karyawan per kategori *cluster* (*High Performer*, *Standard*, *Low Performer*), nilai *Silhouette Coefficient* beserta kategori struktur sesuai sub-bab 4.3.5, rata-rata skor akhir beserta perbandingan periode sebelumnya, dan jumlah karyawan yang di-*skip*. Bagian visualisasi tren menampilkan tiga panel grafik yaitu *Trend Cluster* per Periode, *Trend Silhouette Score*, dan Distribusi per Departemen sebagai alat *monitoring* lintas waktu, sedangkan bagian bawah menampilkan empat panel *action items* yaitu *Top Performers*, Perlu Perhatian, Variabel Terbaik, dan Variabel Terendah, di mana dua panel terakhir merupakan visualisasi langsung dari formula skor akhir tertimbang pada sub-bab 4.3.3.

Implementasi *backend* untuk *dashboard* terdapat pada *KpiHasilService* yang berisi empat *method*, yaitu *getPeriodeSelesai* untuk *dropdown selector*, *getTrend* untuk visualisasi tren, *getDashboard* untuk *top performers* dan perlu perhatian, serta *getDashboardStats* sebagai *method* paling komprehensif yang menggabungkan rata-rata skor, perbandingan periode, distribusi departemen, dan analisis variabel *weighted score* dalam satu pemanggilan.

```
// Filename: src/kpi-hasil/kpi-hasil.service.ts
async getDashboardStats(idPeriode: string) {
  await this.cekPeriodeSelesai(idPeriode);

  // ... ambil rata-rata skor periode dan perbandingan periode
sebelumnya
  // ... agregasi distribusi cluster per departemen

  // Analisis variabel — weighted score (sesuai sub-bab 4.3.3)
  const penilaianList = await this.prisma.kpiPenilaian.findMany({
    where: { idPeriode, status: 'submitted' },
    select: { idVariabel: true, nilaiNormalized: true },
  });
```

```
  const snapshotList = await
this.prisma.kpiVariabelSnapshot.findMany({
    where: { idPeriode },
    select: { idVariabel: true, namaVariabel: true, bobot: true },
  });

  // Deduplikasi snapshot per variabel
  const snapshotMap = new Map();
  for (const s of snapshotList) {
    if (!snapshotMap.has(s.idVariabel)) {
      snapshotMap.set(s.idVariabel, {
        namaVariabel: s.namaVariabel,
        bobot: Number(s.bobot),
      });
    }
  }

  // Agregasi rata-rata nilai ternormalisasi per variabel
  const variabelAgg = new Map();
  for (const p of penilaianList) {
    if (!p.nilaiNormalized) continue;
    if (!variabelAgg.has(p.idVariabel)) {
      variabelAgg.set(p.idVariabel, { total: 0, count: 0 });
    }
    const agg = variabelAgg.get(p.idVariabel);
    agg.total += Number(p.nilaiNormalized);
    agg.count += 1;
  }

  // Hitung weighted score = (nilai ternormalisasi rata-rata × bobot) /
100
  const variabelList = Array.from(variabelAgg.entries())
    .map(([idVariabel, agg]) => {
      const snapshot = snapshotMap.get(idVariabel);
      const nilaiNormalized = agg.total / agg.count;
      const bobot = snapshot?.bobot ?? 0;
      return {
        namaVariabel: snapshot?.namaVariabel ?? '-',
        nilaiNormalized,
        bobot,
        weightedScore: (nilaiNormalized * bobot) / 100,
      };
    })
    .sort((a, b) => b.weightedScore - a.weightedScore);

  // Top 3 variabel terbaik dan bottom 3 variabel terendah
  const analisisVariabel = {
    top3: variabelList.slice(0, 3),
```

```
    bottom3: variabelList.slice(-3).reverse(),
  };

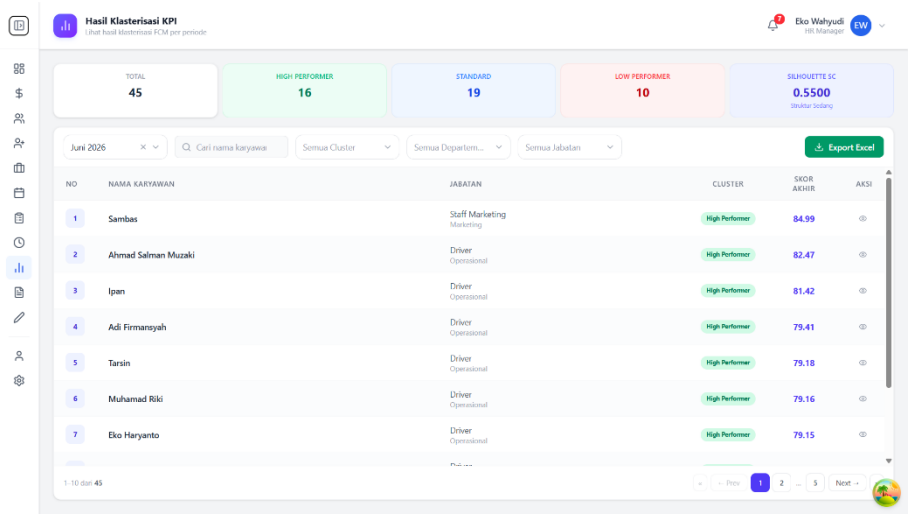
  return {
    // ... rataRataSkor, perbandinganPeriode, distribusiDepartemen
    analisisVariabel,
  };
}
```

Method getDashboardStats memulai dengan validasi periode harus berstatus selesai dan mengambil seluruh hasil klasterisasi dengan *filter isLatest* untuk memastikan data merupakan *run* terbaru. Rata-rata skor dihitung dari karyawan yang tidak di-*skip*, perbandingan periode sebelumnya menggunakan selisih dengan toleransi 0.001, distribusi per departemen dihitung dengan mengelompokkan hasil berdasarkan nama departemen, dan analisis variabel *weighted score* dihitung dengan mengalikan rata-rata nilai *normalized* per variabel dengan bobot kemudian membagi seratus sebagai visualisasi langsung dari formula skor akhir tertimbang pada sub-bab 4.3.3.

Tiga *method* lainnya melengkapi pengelolaan *dashboard*, yaitu *getPeriodeSelesai* yang mengambil daftar periode selesai untuk *dropdown selector*, *getTrend* yang mengambil distribusi *cluster* dan nilai *Silhouette Coefficient* untuk seluruh periode selesai sebagai data *line chart*, dan *getDashboard* yang mengambil lima karyawan dengan skor tertinggi sebagai *Top Performers* dan lima karyawan dengan label *Low Performer* sebagai panel Perlu Perhatian. *Helper kategoriSc* yang digunakan oleh *method-method* ini melakukan konversi nilai *Silhouette Coefficient* ke kategori struktur yang konsisten dengan sub-bab 4.3.5.

6. Hasil Klasterisasi

Halaman Hasil Klasterisasi menampilkan *output* akhir proses *Fuzzy C-Means* yang telah dijalankan pada periode yang telah ditutup. Halaman ini menjadi muara visual dari seluruh sub-bab implementasi sebelumnya, di mana hasil normalisasi pada sub-bab 4.3.3, klasterisasi pada sub-bab 4.3.4, dan validasi pada sub-bab 4.3.5 disajikan dalam satu antarmuka yang dapat diakses oleh HR maupun atasan.



Gambar 76. Halaman Hasil Klasterisasi KPI

Bagian atas halaman menampilkan lima kartu statistik ringkas hasil klasterisasi yang meliputi total karyawan dinilai, jumlah *High Performer*, *Standard*, *Low Performer*, dan nilai *Silhouette Coefficient* global beserta kategori strukturnya, dengan susunan yang konsisten dengan KPI *Dashboard* dan label *cluster* yang diperoleh dari *output method tentukanLabelCluster* pada sub-bab 4.3.4. Bagian *filter* menyediakan kombinasi *dropdown* periode, pencarian nama, serta *filter* berdasarkan *cluster*, departemen, dan jabatan, dilengkapi tombol *Export Excel*, sedangkan tabel di bawahnya menampilkan daftar karyawan diurutkan berdasarkan skor akhir tertinggi dengan kolom nama, jabatan, *badge cluster* berwarna, skor akhir, dan ikon aksi untuk membuka *modal* detail berisi rincian nilai dan kontribusi setiap variabel.

Modal detail per karyawan dihasilkan oleh *method findDetail*, yang menampilkan rincian nilai aktual, nilai normalisasi, bobot, serta kontribusi setiap variabel terhadap skor akhir.

```
// Filename: src/kpi-hasil/kpi-hasil.service.ts
async findDetail(idPeriode: string, idKaryawan: string, user:
AuthUser) {
  await this.cekPeriodeSelesai(idPeriode);

  // Validasi akses: Atasan hanya bisa lihat detail bawahan langsung
  if (user.role === 'atasan') {
    const isBawahan = await this.prisma.refKaryawan.findFirst({
```

```
      where: { idKaryawan, idAtasan: user.idKaryawan },
    });
    if (!isBawahan) {
      throw new ForbiddenException(
        'Anda tidak memiliki akses ke detail karyawan ini'
      );
    }
  }

  // Ambil hasil klasterisasi karyawan
  const hasil = await this.prisma.kpiHasilCluster.findFirst({
    where: { idPeriode, idKaryawan },
    include: {
      karyawan: {
        select: {
          namaLengkap: true,
          jabatan: { include: { departemen: true } },
        },
      },
    },
  });
  if (!hasil) throw new NotFoundException('Hasil tidak ditemukan');

  // Ambil penilaian dan snapshot untuk perhitungan kontribusi
  const penilaians = await this.prisma.kpiPenilaian.findMany({
    where: { idPeriode, idKaryawan, status: 'submitted' },
    select: { idVariabel: true, nilai: true, nilaiNormalized: true },
  });
  const snapshots = await
this.prisma.kpiVariabelSnapshot.findMany({
    where: {
      idPeriode,
      idVariabel: { in: penilaians.map((p) => p.idVariabel) },
    },
    select: {
      idVariabel: true,
      namaVariabel: true,
      tipeNilai: true,
      arahNilai: true,
      bobot: true,
    },
  });

  // Hitung kontribusi skor per variabel sesuai formula sub-bab 4.3.3
  const detailVariabel = penilaians.map((p) => {
    const snapshot = snapshots.find((s) => s.idVariabel ===
p.idVariabel);
    const nilaiNorm = Number(p.nilaiNormalized ?? 0);
```

```
    const bobot = Number(snapshot?.bobot ?? 0);
    const kontribusiSkor = (nilaiNorm * bobot) / 100;
    return {
      namaVariabel: snapshot?.namaVariabel,
      tipeNilai: snapshot?.tipeNilai,
      arahNilai: snapshot?.arahNilai,
      nilaiAktual: Number(p.nilai),
      nilaiNormalized: nilaiNorm,
      bobot,
      kontribusiSkor,
    };
  });

  return {
    karyawan: hasil.karyawan,
    skorAkhir: hasil.skorAkhir,
    labelCluster: hasil.labelCluster,
    derajatKeanggotaan: hasil.derajatKeanggotaan,
    detailVariabel,
  };
}
```

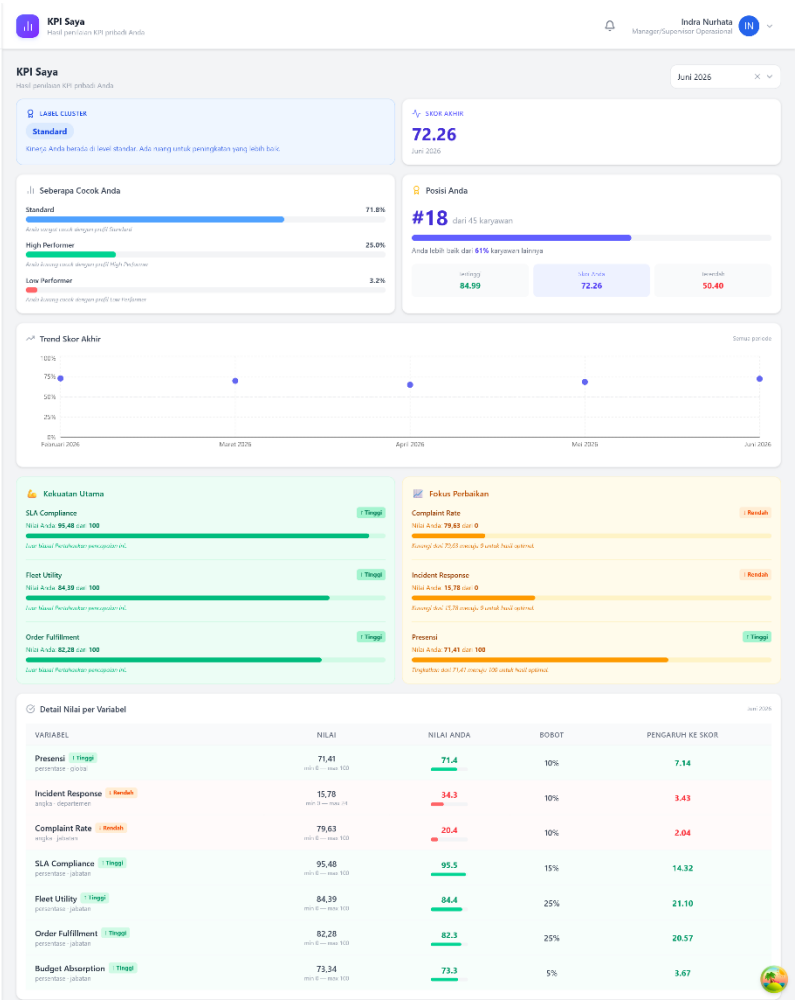
Method findDetail menampilkan dua aspek penting sistem, yaitu validasi akses dan perhitungan kontribusi skor. Validasi pada awal *method* memastikan kontrol akses berbasis peran, di mana atasan hanya dapat melihat bawahan langsungnya melalui pengecekan *idAtasan*, sedangkan HR memiliki akses penuh. Perhitungan *kontribusiSkor* memvisualisasikan formula skor akhir pada sub-bab 4.3.3 melalui perkalian nilai normalisasi dengan bobot dibagi seratus, dan penggunaan *KpiVariabelSnapshot* menjamin konsistensi historis sehingga nama, arah nilai, dan bobot yang ditampilkan merupakan konfigurasi saat periode diproses, bukan data *master* terkini.

Method lainnya pada *KpiHasilService* melengkapi operasi pengelolaan hasil klasterisasi. *Method findBawahan* mengambil daftar hasil klasterisasi yang difilter berdasarkan *idAtasan* pengguna yang *login* untuk membatasi pengawasan sesuai URS-A-08, sedangkan *method exportExcel* menghasilkan *file Excel* berisi tiga *sheet* terpisah, yaitu Ringkasan yang menampilkan daftar karyawan dengan skor akhir, label *cluster*, dan derajat keanggotaan, Detail Nilai yang memuat nilai aktual, nilai normalisasi, bobot, serta kontribusi skor per variabel per

karyawan, dan Statistik FCM yang menyajikan nilai *Silhouette Coefficient*, kategori struktur, dan distribusi jumlah karyawan per *cluster*.

7. KPI Saya

Halaman KPI Saya merupakan antarmuka yang diakses oleh karyawan untuk melihat hasil penilaian KPI dirinya sendiri pada setiap periode yang telah selesai diproses. Berbeda dengan halaman Hasil Klasterisasi yang menampilkan seluruh karyawan dari perspektif HR atau bawahan dari perspektif atasan, halaman ini hanya menampilkan data karyawan yang sedang *login* beserta posisinya relatif terhadap rekan kerja, sebagai implementasi URS-K-06 yang memungkinkan karyawan melihat hasil klasterisasi kinerja dirinya sendiri beserta derajat keanggotaan terhadap masing-masing *cluster*.



Gambar 77. Halaman KPI Saya

Bagian atas halaman menampilkan dua kartu utama sebagai ringkasan langsung bagi karyawan, yaitu kartu pertama yang menyajikan *badge* warna label *cluster* (*High*, *Standard*, atau *Low Performer*) beserta interpretasi tekstual yang ramah, dan kartu kedua yang menampilkan skor akhir dan periode penilaian, dengan penentuan label dan interpretasi merujuk pada *method tentukanLabelCluster* sub-bab 4.3.4 untuk menjaga konsistensi data antara halaman HR, atasan, dan karyawan. Di bawahnya terdapat kontribusi unik halaman ini berupa panel "Seberapa Cocok Anda" yang menyajikan tiga *progress bar* derajat keanggotaan karyawan pada setiap *cluster* beserta persentasenya, sebagai visualisasi konkret konsep *fuzzy* pada *Fuzzy C-Means* agar aspek matematis derajat keterikatan antar-*cluster* dapat dipahami oleh pengguna tanpa latar belakang statistik.

Sementara itu, panel "Posisi Anda" di sisi kanan menyajikan peringkat karyawan di antara seluruh rekan kerja pada periode tersebut menggunakan format *ranking* dan persentil, dilengkapi tiga *mini card* yang membandingkan skor karyawan terhadap skor tertinggi dan terendah pada periode terpilih. Data posisi ini didukung oleh *line chart Tren Skor Akhir* di bagian tengah halaman yang memvisualisasikan lima periode terakhir sebagai indikator perkembangan personal, serta dua panel interpretasi otomatis yaitu "Kekuatan Utama" yang menampilkan tiga variabel berkontribusi tertinggi dan "Fokus Perbaikan" yang menampilkan tiga variabel area perbaikan beserta pesan saran berdasarkan arah nilai variabel.

Di bagian bawah halaman terdapat tabel Detail Nilai per Variabel yang menyajikan seluruh variabel KPI yang dinilai pada periode terpilih beserta arah nilai, cakupan, rentang minimum-maksimum, nilai aktual yang divisualisasikan menggunakan *progress bar* berwarna sesuai performa, bobot variabel, dan kolom Pengaruh ke Skor. Kolom Pengaruh ke Skor secara transparan memvisualisasikan formula skor akhir tertimbang pada sub-bab 4.3.3 dari perspektif karyawan, sehingga karyawan dapat melihat dengan jelas bagaimana setiap variabel

berkontribusi terhadap skor akhir yang diperoleh. Implementasi tabel ini ditangani oleh *method findDetail* pada *KpiSayaService*.

```
// Filename: src/kpi-saya/kpi-saya.service.ts
async findDetail(idKaryawan: string, idPeriode: string) {
  // Validasi periode harus berstatus selesai
  const periode = await this.prisma.kpiPeriode.findUnique({
    where: { idPeriode },
    select: { status: true, bulan: true, tahun: true },
  });
  if (!periode || periode.status !== 'selesai') {
    throw new NotFoundException(
      'Detail nilai hanya tersedia setelah periode selesai'
    );
  }

  // Ambil hasil klasterisasi karyawan
  const hasil = await this.prisma.kpiHasilCluster.findFirst({
    where: { idPeriode, idKaryawan, isLatest: true, diSkip: false },
  });
  if (!hasil) {
    throw new NotFoundException('Hasil KPI tidak ditemukan');
  }

  // Ambil penilaian, data karyawan, dan snapshot variabel
  const penilaians = await this.prisma.kpiPenilaian.findMany({
    where: { idPeriode, idKaryawan, status: 'submitted' },
    select: { idVariabel: true, nilai: true, nilaiNormalized: true,
catatan: true },
  });

  const karyawan = await this.prisma.refKaryawan.findUnique({
    where: { idKaryawan },
    select: {
      idJabatan: true,
      jabatan: { select: { idDepartemen: true } },
    },
  });

  const snapshots = await
this.prisma.kpiVariabelSnapshot.findMany({
    where: { idPeriode, idVariabel: { in: penilaians.map((p) =>
p.idVariabel) } },
    select: { /* atribut variabel beserta cakupan, idJabatan,
idDepartemen */ },
  });
```

```
  // Deduplikasi snapshot dengan prioritas: jabatan > departemen >
global
  const snapshotMap = new Map();
  for (const s of snapshots) {
    const existing = snapshotMap.get(s.idVariabel);
    if (!existing) {
      snapshotMap.set(s.idVariabel, s);
      continue;
    }
    if (s.cakupan === 'jabatan' && s.idJabatan ===
karyawan?.idJabatan) {
      snapshotMap.set(s.idVariabel, s);
    } else if (
      s.cakupan === 'departemen' &&
      s.idDepartemen === karyawan?.jabatan?.idDepartemen &&
      existing.cakupan === 'global'
    ) {
      snapshotMap.set(s.idVariabel, s);
    }
  }

  // Hitung kontribusi skor per variabel sesuai formula sub-bab 4.3.3
  const nilaiDetail = penilaians.map((p) => {
    const snapshot = snapshotMap.get(p.idVariabel);
    const bobot = snapshot ? Number(snapshot.bobot) : 0;
    const nilaiNorm = p.nilaiNormalized ?
Number(p.nilaiNormalized) : 0;
    return {
      namaVariabel: snapshot?.namaVariabel ?? '-',
      cakupan: snapshot?.cakupan ?? '-',
      nilaiAktual: Number(p.nilai),
      nilaiNormalized: nilaiNorm,
      bobot,
      kontribusiSkor: (nilaiNorm * bobot) / 100,
      catatan: p.catatan ?? null,
    };
  });

  return {
    periode: { bulan: periode.bulan, tahun: periode.tahun },
    hasil: {
      labelCluster: hasil.labelCluster,
      skorAkhir: Number(hasil.skorAkhir),
      derajatKeanggotaan: hasil.derajatKeanggotaan,
      silhouetteScore: Number(hasil.silhouetteScore),
      kategoriSc: this.kategoriSc(Number(hasil.silhouetteScore)),
    },
    nilaiDetail,
```

```
  };
}
```

Method findDetail menerapkan validasi berlapis (status periode selesai, ketersediaan hasil klasterisasi, serta *filter isLatest* dan *diSkip*) untuk memastikan hanya data hasil klasterisasi terbaru yang ditampilkan jika terjadi proses ulang. Deduplikasi *snapshot* mengadopsi pola prioritas *Decorator Pattern* pada sub-bab 4.3.2, di mana cakupan jabatan menggantikan departemen dan global, dan departemen menggantikan global. Penggunaan *KpiVariabelSnapshot* menjamin konsistensi data yang diproses pada periode tersebut, bukan konfigurasi *master* terkini HR, sedangkan perhitungan *kontribusiSkor* pada setiap entri langsung ditampilkan pada kolom Pengaruh ke Skor di tabel agar karyawan mengetahui kontribusi tiap variabel terhadap skor akhirnya.

Method lainnya pada *KpiSayaService* melengkapi operasi pembacaan data KPI karyawan. *Method findRiwayat* mengambil tren skor akhir karyawan antar periode sebagai sumber data *line chart Tren Skor Akhir*, *method getRanking* menghitung posisi karyawan beserta persentil untuk panel "Posisi Anda", dan *helper kategoriSc* digunakan untuk mengonversi nilai *Silhouette Coefficient* global menjadi kategori struktur klasterisasi yang konsisten dengan sub-bab 4.3.5.

4.4 Pengujian

4.4.1 Pengujian Fungsional

Pengujian *Black Box* dilakukan untuk memvalidasi bahwa setiap fungsionalitas sistem berjalan sesuai dengan kebutuhan pengguna yang telah didefinisikan pada *User Requirement Specification* (URS) di sub-bab sebelumnya. Pengujian dilakukan dengan pendekatan skenario uji yang dipetakan secara langsung ke setiap kode URS, sehingga *traceability* antara kebutuhan dan hasil pengujian dapat dipertanggungjawabkan. Setiap URS diuji dengan dua sampai tiga skenario yang mencakup alur normal (*positive case*) dan validasi penanganan kesalahan (*negative case*). Hasil pengujian disajikan dalam tiga tabel terpisah berdasarkan aktor pengguna, yaitu HR, Atasan, dan Karyawan.

Tabel 14. *Black Box Testing* — Aktor HR

No	Kode URS	Deskripsi URS	Skenario Pengujian	Hasil yang Diharapkan	Hasil
1	URS-HR-01	HR dapat mengakses sistem dengan melakukan *login* menggunakan kredensial yang valid	HR memasukkan *username* dan *password* yang valid pada halaman *login*	Sistem mengarahkan HR ke halaman *dashboard* dan menyimpan sesi *login*	Valid
2	URS-HR-01	HR dapat mengakses sistem dengan melakukan *login* menggunakan kredensial yang valid	HR memasukkan *username* yang valid tetapi *password* salah	Sistem menampilkan pesan kesalahan dan tetap berada di halaman *login*	Valid
3	URS-HR-02	HR dapat keluar dari sistem dengan melakukan *logout* untuk mengakhiri sesi	HR menekan tombol *logout* pada navigasi sistem	Sistem mengakhiri sesi dan mengarahkan HR ke halaman *login*	Valid
4	URS-HR-03	HR dapat mengelola data pengguna sistem meliputi penambahan, pengubahan, dan penonaktifan akun pengguna	HR menambahkan akun pengguna baru dengan data yang lengkap dan valid	Sistem menyimpan data akun pengguna baru dan menampilkannya di daftar akun	Valid

5	URS-HR-03	HR dapat mengelola data pengguna sistem meliputi penambahan, pengubahan, dan penonaktifan akun pengguna	HR mengubah status keaktifan akun pengguna menjadi nonaktif	Sistem memperbarui status keaktifan akun dan akun tidak dapat melakukan *login*	Valid
6	URS-HR-03	HR dapat mengelola data pengguna sistem meliputi penambahan, pengubahan, dan penonaktifan akun pengguna	HR menambahkan akun pengguna dengan *username* yang sudah terdaftar	Sistem menampilkan pesan kesalahan duplikasi *username* dan tidak menyimpan data	Valid
7	URS-HR-04	HR dapat mengelola data departemen yang menjadi dasar *assignment* variabel KPI pada cakupan departemen dalam sistem	HR menambahkan departemen baru dengan nama yang valid	Sistem menyimpan data departemen baru dan menampilkannya di daftar departemen	Valid
8	URS-HR-04	HR dapat mengelola data departemen yang menjadi dasar *assignment* variabel KPI pada cakupan departemen dalam sistem	HR mengubah nama departemen yang sudah ada	Sistem memperbarui nama departemen dan menyimpannya	Valid
9	URS-HR-05	HR dapat mengelola data jabatan yang menjadi dasar pemilihan logika penilaian secara dinamis dalam sistem	HR menambahkan jabatan baru dengan menentukan departemen induk dan *flag* isMitra	Sistem menyimpan data jabatan baru beserta atribut yang dipilih	Valid
10	URS-HR-05	HR dapat mengelola data jabatan yang menjadi dasar pemilihan logika penilaian secara	HR mengubah *flag* isMitra pada jabatan yang sudah ada	Sistem memperbarui *flag* isMitra dan sistem secara otomatis akan memilih	Valid

		dinamis dalam sistem		strategy yang sesuai pada proses penilaian berikutnya	
11	URS-HR-06	HR dapat mengelola jadwal kerja karyawan sebagai dasar perhitungan presensi	HR menambahkan jadwal kerja baru dengan tipe regular dan rentang jam yang valid	Sistem menyimpan data jadwal kerja baru	Valid
12	URS-HR-06	HR dapat mengelola jadwal kerja karyawan sebagai dasar perhitungan presensi	HR menetapkan jadwal kerja kepada karyawan tertentu dengan periode efektif yang valid	Sistem menyimpan penetapan jadwal kerja dan menampilkannya pada profil karyawan	Valid
13	URS-HR-07	HR dapat mengelola data presensi karyawan	HR melihat daftar presensi karyawan berdasarkan *filter* tanggal dan karyawan	Sistem menampilkan data presensi yang sesuai dengan *filter* yang diberikan	Valid
14	URS-HR-07	HR dapat mengelola data presensi karyawan	HR melakukan koreksi data presensi karyawan dengan menyertakan alasan koreksi	Sistem menyimpan data presensi yang dikoreksi beserta *log* riwayat koreksi	Valid
15	URS-HR-08	HR dapat mengkonfigurasi *chain approval* untuk pengajuan izin karyawan	HR menambahkan *template chain approval* dengan beberapa *level approver*	Sistem menyimpan *template chain approval* beserta level-levelnya	Valid
16	URS-HR-08	HR dapat mengkonfigurasi *chain approval* untuk pengajuan izin karyawan	HR menambahkan peserta tambahan pada *chain approval* di posisi setelah level tertentu	Sistem menyimpan peserta tambahan dengan posisi yang ditentukan	Valid

17	URS-HR-09	HR dapat mengelola variabel KPI meliputi penambahan, pengubahan, dan penonaktifan variabel secara dinamis tanpa mengubah kode program	HR menambahkan variabel KPI baru dengan nama, arah nilai, batas minimum, dan batas maksimum	Sistem menyimpan variabel KPI baru dan menampilkannya di daftar variabel	Valid
18	URS-HR-09	HR dapat mengelola variabel KPI meliputi penambahan, pengubahan, dan penonaktifan variabel secara dinamis tanpa mengubah kode program	HR mengubah arah nilai variabel dari lebih_tinggi_lebih_baik menjadi lebih_rendah_lebih_baik	Sistem memperbarui arah nilai variabel dan perubahan akan berlaku pada perhitungan klasterisasi berikutnya	Valid
19	URS-HR-09	HR dapat mengelola variabel KPI meliputi penambahan, pengubahan, dan penonaktifan variabel secara dinamis tanpa mengubah kode program	HR menonaktifkan variabel KPI yang tidak relevan	Sistem mengubah status variabel menjadi nonaktif dan variabel tidak muncul di pemilihan *assignment*	Valid
20	URS-HR-10	HR dapat mengelola *assignment* variabel KPI ke tiga cakupan yaitu *global*, departemen, dan jabatan	HR menambahkan *assignment* variabel ke cakupan *global* dengan bobot tertentu	Sistem menyimpan *assignment* variabel pada cakupan *global* beserta bobot yang ditentukan	Valid
21	URS-HR-10	HR dapat mengelola *assignment* variabel KPI ke tiga cakupan yaitu *global*, departemen, dan jabatan	HR menambahkan *assignment* variabel yang sama ke cakupan departemen dengan bobot berbeda	Sistem menyimpan *assignment* pada cakupan departemen dan sistem akan menggunakan bobot departemen untuk karyawan	Valid

				di departemen tersebut sesuai mekanisme *decorator*	
22	URS-HR-10	HR dapat mengelola *assignment* variabel KPI ke tiga cakupan yaitu *global*, departemen, dan jabatan	HR menambahkan *assignment* variabel ke cakupan jabatan untuk jabatan tertentu	Sistem menyimpan *assignment* dan sistem akan menggunakan *strategy JobSpecificStrategy* untuk jabatan tersebut	Valid
23	URS-HR-11	HR dapat membuka dan menutup periode penilaian secara terkontrol	HR membuat periode penilaian baru dengan tanggal mulai dan tanggal selesai yang valid	Sistem menyimpan periode penilaian dengan status terbuka	Valid
24	URS-HR-11	HR dapat membuka dan menutup periode penilaian secara terkontrol	HR menutup periode penilaian yang sedang aktif	Sistem mengubah status periode menjadi tertutup dan input nilai KPI tidak dapat dilakukan lagi	Valid
25	URS-HR-12	HR dapat mengkonfigurasi parameter FCM meliputi jumlah *cluster*, pangkat pembobot, error minimum, dan maksimum iterasi	HR mengkonfigurasi parameter FCM dengan jumlah *cluster* 3, pangkat pembobot 2, errorMinimum 0,001, dan maksimum iterasi 100	Sistem menyimpan konfigurasi FCM pada periode yang dipilih	Valid
26	URS-HR-12	HR dapat mengkonfigurasi parameter FCM meliputi jumlah *cluster*, pangkat pembobot, error minimum, dan maksimum iterasi	HR mencoba mengkonfigurasi pangkat pembobot dengan nilai kurang dari atau sama dengan 1	Sistem menampilkan pesan kesalahan validasi karena pangkat pembobot harus lebih besar dari 1	Valid

27	URS-HR-13	HR dapat menjalankan proses klasterisasi FCM setelah periode ditutup	HR menjalankan proses klasterisasi FCM pada periode yang sudah ditutup	Sistem memproses klasterisasi secara asinkron via *Bull Queue* dan menampilkan progress pemrosesan	Valid
28	URS-HR-13	HR dapat menjalankan proses klasterisasi FCM setelah periode ditutup	HR mencoba menjalankan klasterisasi pada periode yang masih terbuka	Sistem menampilkan pesan kesalahan dan tidak menjalankan proses klasterisasi	Valid
29	URS-HR-14	HR dapat melihat hasil klasterisasi kinerja seluruh karyawan beserta nilai *Silhouette Coefficient*	HR membuka halaman hasil klasterisasi setelah proses FCM selesai	Sistem menampilkan daftar karyawan beserta *cluster*, label *cluster*, derajat keanggotaan, dan nilai *Silhouette Coefficient global*	Valid
30	URS-HR-14	HR dapat melihat hasil klasterisasi kinerja seluruh karyawan beserta nilai *Silhouette Coefficient*	HR memfilter hasil klasterisasi berdasarkan *cluster* tertentu	Sistem menampilkan hanya karyawan yang termasuk pada *cluster* yang dipilih	Valid
31	URS-HR-15	HR dapat mengisi nilai KPI karyawan sebagai pengganti atasan jika diperlukan	HR mengisi nilai KPI karyawan tertentu melalui halaman input penilaian	Sistem menyimpan nilai KPI yang diinput oleh HR dengan keterangan input dilakukan oleh HR	Valid
32	URS-HR-15	HR dapat mengisi nilai KPI karyawan sebagai pengganti atasan jika diperlukan	HR *mensubmit* penilaian yang telah diisi sebagai pengganti atasan	Sistem mengubah status penilaian menjadi *submitted* dan data nilai tidak dapat diubah	Valid

				lagi tanpa proses koreksi	

Tabel 15. *Black Box Testing* — Aktor Atasan

No	Kode URS	Deskripsi URS	Skenario Pengujian	Hasil yang Diharapkan	Hasil
1	URS-A-01	Atasan dapat mengakses sistem dengan melakukan *login* menggunakan kredensial yang valid	Atasan memasukkan *username* dan *password* yang valid pada halaman *login*	Sistem mengarahkan Atasan ke halaman *dashboard* dan menyimpan sesi *login*	Valid
2	URS-A-01	Atasan dapat mengakses sistem dengan melakukan *login* menggunakan kredensial yang valid	Atasan memasukkan kredensial yang tidak valid	Sistem menampilkan pesan kesalahan dan tetap berada di halaman *login*	Valid
3	URS-A-02	Atasan dapat keluar dari sistem dengan melakukan *logout* untuk mengakhiri sesi	Atasan menekan tombol *logout* pada navigasi sistem	Sistem mengakhiri sesi dan mengarahkan Atasan ke halaman *login*	Valid
4	URS-A-03	Atasan dapat memproses pengajuan izin bawahan berdasarkan *chain approval* yang telah dikonfigurasi	Atasan menyetujui pengajuan izin bawahan pada *level approval* yang sesuai	Sistem mencatat persetujuan dan meneruskan pengajuan ke *level approver* berikutnya jika ada	Valid
5	URS-A-03	Atasan dapat memproses pengajuan izin bawahan berdasarkan *chain approval* yang telah dikonfigurasi	Atasan menolak pengajuan izin bawahan dengan menyertakan catatan penolakan	Sistem mencatat penolakan beserta catatan dan mengubah status pengajuan menjadi ditolak	Valid
6	URS-A-04	Atasan dapat melihat daftar bawahan beserta status penilaian pada periode yang sedang aktif	Atasan membuka halaman daftar bawahan pada periode yang sedang aktif	Sistem menampilkan daftar bawahan beserta status penilaian masing-masing yaitu	Valid

				belum_diisi, *draft*, atau *submitted*	
7	URS-A-04	Atasan dapat melihat daftar bawahan beserta status penilaian pada periode yang sedang aktif	Atasan mengubah periode aktif yang dilihat melalui filter periode	Sistem menampilkan daftar bawahan beserta status penilaian sesuai dengan periode yang dipilih	Valid
8	URS-A-05	Atasan dapat melihat variabel KPI yang berlaku untuk setiap bawahan secara otomatis berdasarkan jabatan dan departemennya	Atasan membuka halaman input penilaian untuk bawahan dengan jabatan reguler	Sistem menampilkan variabel KPI dari *layer global*, departemen, dan jabatan sesuai dengan resolusi *JobSpecificStrategy*	Valid
9	URS-A-05	Atasan dapat melihat variabel KPI yang berlaku untuk setiap bawahan secara otomatis berdasarkan jabatan dan departemennya	Atasan membuka halaman input penilaian untuk bawahan dengan jabatan mitra	Sistem menampilkan variabel KPI hanya dari layer jabatan sesuai dengan resolusi *MitraStrategy*	Valid
10	URS-A-05	Atasan dapat melihat variabel KPI yang berlaku untuk setiap bawahan secara otomatis berdasarkan jabatan dan departemennya	Atasan membuka halaman *input* penilaian untuk bawahan dengan jabatan tanpa *assignment* cakupan jabatan	Sistem menampilkan variabel KPI dari *layer global* dan departemen sesuai dengan resolusi *DefaultStrategy*	Valid
11	URS-A-06	Atasan dapat mengisi dan menyimpan *draft* nilai KPI per variabel untuk setiap bawahannya	Atasan mengisi nilai KPI per variabel dan menyimpannya sebagai *draft*	Sistem menyimpan nilai KPI dengan status draft dan dapat diubah kembali	Valid
12	URS-A-06	Atasan dapat mengisi dan menyimpan *draft* nilai KPI per	Atasan mengisi nilai KPI dengan nilai di luar rentang	Sistem menampilkan pesan kesalahan validasi rentang nilai	Valid

		variabel untuk setiap bawahannya	minimum dan maksimum variabel		
13	URS-A-07	Atasan dapat *mengsubmit* penilaian untuk mengunci data nilai KPI bawahan	Atasan *mensubmit* penilaian yang sudah lengkap untuk seluruh variabel	Sistem mengubah status penilaian menjadi *submitted* dan nilai tidak dapat diubah lagi	Valid
14	URS-A-07	Atasan dapat *mengsubmit* penilaian untuk mengunci data nilai KPI bawahan	Atasan mencoba *mensubmit* penilaian yang masih ada variabel kosong	Sistem menampilkan pesan kesalahan validasi kelengkapan dan tidak mengubah status penilaian	Valid
15	URS-A-08	Atasan dapat melihat hasil klasterisasi kinerja bawahannya berdasarkan periode yang dipilih	Atasan membuka halaman hasil klasterisasi pada periode yang sudah selesai diproses	Sistem menampilkan hasil klasterisasi hanya untuk bawahan dari Atasan yang *login*	Valid
16	URS-A-08	Atasan dapat melihat hasil klasterisasi kinerja bawahannya berdasarkan periode yang dipilih	Atasan memilih periode lain melalui *filter* periode	Sistem menampilkan hasil klasterisasi bawahan sesuai dengan periode yang dipilih	Valid

Tabel 16. *Black Box Testing* — Aktor Karyawan

No	Kode URS	Deskripsi URS	Skenario Pengujian	Hasil yang Diharapkan	Hasil
1	URS-K-01	Karyawan dapat mengakses sistem dengan melakukan *login* menggunakan kredensial yang valid	Karyawan memasukkan *username* dan *password* yang valid pada halaman *login*	Sistem mengarahkan Karyawan ke halaman *dashboard* dan menyimpan sesi *login*	Valid
2	URS-K-01	Karyawan dapat mengakses sistem dengan melakukan *login*	Karyawan memasukkan kredensial yang tidak valid	Sistem menampilkan pesan kesalahan	Valid

		menggunakan kredensial yang valid		dan tetap berada di halaman *login*	
3	URS-K-02	Karyawan dapat keluar dari sistem dengan melakukan *logout* untuk mengakhiri sesi	Karyawan menekan tombol *logout* pada navigasi sistem	Sistem mengakhiri sesi dan mengarahkan Karyawan ke halaman *login*	Valid
4	URS-K-03	Karyawan dapat mengajukan izin melalui sistem	Karyawan mengajukan izin dengan jenis izin, tanggal, dan keterangan yang valid	Sistem menyimpan pengajuan izin dengan status menunggu *approval* dan memulai alur *chain approval*	Valid
5	URS-K-03	Karyawan dapat mengajukan izin melalui sistem	Karyawan mengajukan izin tanpa mengisi salah satu *field* wajib	Sistem menampilkan pesan kesalahan validasi *field* wajib	Valid
6	URS-K-03	Karyawan dapat mengajukan izin melalui sistem	Karyawan mengajukan izin dengan jumlah hari melebihi saldo cuti yang tersedia	Sistem menampilkan pesan kesalahan saldo cuti tidak mencukupi	Valid
7	URS-K-04	Karyawan dapat melihat status pengajuan izin dan data presensi dirinya sendiri	Karyawan membuka halaman riwayat pengajuan izin	Sistem menampilkan daftar pengajuan izin Karyawan beserta status *approval* terkini	Valid
8	URS-K-04	Karyawan dapat melihat status pengajuan izin dan data presensi dirinya sendiri	Karyawan membuka halaman data presensi pada rentang tanggal tertentu	Sistem menampilkan data presensi Karyawan sesuai rentang tanggal yang dipilih	Valid
9	URS-K-05	Karyawan dapat melihat variabel KPI yang berlaku untuk jabatannya pada periode yang sedang aktif	Karyawan membuka halaman KPI saya pada periode yang sedang aktif	Sistem menampilkan daftar variabel KPI yang berlaku untuk jabatan Karyawan beserta bobot masing-masing	Valid
10	URS-K-05	Karyawan dapat melihat variabel	Karyawan dengan jabatan	Sistem menampilkan	Valid

		KPI yang berlaku untuk jabatannya pada periode yang sedang aktif	mitra membuka halaman KPI saya	variabel KPI hanya dari *layer* jabatan sesuai resolusi *MitraStrategy*	
11	URS-K-06	Karyawan dapat melihat hasil klasterisasi kinerja dirinya sendiri beserta derajat keanggotaan terhadap masing-masing *cluster*	Karyawan membuka halaman hasil klasterisasi pada periode yang sudah selesai diproses	Sistem menampilkan *cluster* dominan, *label cluster*, dan derajat keanggotaan Karyawan terhadap seluruh *cluster*	Valid
12	URS-K-06	Karyawan dapat melihat hasil klasterisasi kinerja dirinya sendiri beserta derajat keanggotaan terhadap masing-masing *cluster*	Karyawan mencoba mengakses hasil klasterisasi karyawan lain	Sistem menolak akses dan hanya menampilkan hasil klasterisasi Karyawan yang sedang *login*	Valid

4.4.2 Pengujian Integrasi Modul

Pengujian integrasi modul dilakukan untuk memvalidasi bahwa interaksi antar modul dalam sistem berjalan sesuai rancangan, di mana *output* suatu modul menjadi *input* modul berikutnya dengan benar dan konsisten, dengan menguji alur kerja *end-to-end* yang melibatkan beberapa modul sekaligus dari modul awal hingga akhir. Pengujian pada sistem ini mengidentifikasi empat alur kerja utama yang merepresentasikan *use case* inti penelitian, yaitu konfigurasi sistem oleh HR sampai terbentuknya hasil klasterisasi (melibatkan modul *master* variabel, *assignment*, periode penilaian, *input* KPI, FCM, dan hasil klasterisasi), autentikasi pengguna sampai penampilan variabel KPI di antarmuka *input* penilaian (melibatkan modul autentikasi, *Strategy Pattern*, *Decorator Pattern*, dan antarmuka pengguna), pengajuan izin oleh karyawan sampai notifikasi hasil *approval* (melibatkan modul izin, *chain approval*, dan notifikasi), serta *snapshot* konfigurasi periode sampai eksekusi *FCM Engine* (melibatkan modul *snapshot*, pra-pemrosesan, dan *FCM Engine*).

Tabel 17. Pengujian Integrasi Modul

No	Nama Alur Integrasi	Modul yang Terlibat	Skenario Pengujian	Hasil yang Diharapkan	Hasil
1	Konfigurasi sistem sampai hasil klasterisasi	Master Variabel, Assignment, Periode, Input KPI, FCM, Hasil Klasterisasi	HR menambah variabel KPI baru, menetapkan assignment ke 3 cakupan dengan bobot total 100%, membuat periode dengan konfigurasi FCM, atasan input nilai KPI seluruh bawahan, HR menutup periode dan menjalankan klasterisasi	Sistem menghasilkan hasil klasterisasi dengan label High Performer, Standard, dan Low Performer beserta derajat keanggotaan dan nilai Silhouette Coefficient	Valid
2	Autentikasi sampai tampilan variabel KPI	Autentikasi, Strategy Pattern, Decorator Pattern, UI Input Penilaian	Atasan login menggunakan kredensial valid, membuka halaman input penilaian untuk bawahan dengan jabatan Driver	Sistem mengarahkan ke dashboard, memilih MitraStrategy berdasarkan flag isMitra, decorator mengembalikan variabel hanya dari layer jabatan, UI menampilkan variabel yang sesuai untuk Driver	Valid
3	Autentikasi sampai tampilan variabel KPI	Autentikasi, Strategy Pattern, Decorator Pattern, UI Input Penilaian	Atasan login dan membuka halaman input penilaian untuk bawahan dengan jabatan reguler yang memiliki assignment cakupan jabatan	Sistem memilih JobSpecificStrategy, decorator menggabungkan variabel dari 3 layer dengan deduplikasi sesuai prioritas, UI menampilkan	Valid

				variabel hasil resolusi	
4	Pengajuan izin sampai notifikasi approval	Izin, Chain Approval, Notifikasi	Karyawan mengajukan izin dengan jenis izin yang memiliki konfigurasi chain approval, sistem memproses pengajuan melalui chain approval, atasan menyetujui pengajuan pada setiap level	Sistem membuat record approval per level sesuai template, status pengajuan berubah sesuai progress approval, karyawan menerima notifikasi setiap perubahan status	Valid
5	Snapshot konfigurasi sampai eksekusi FCM Engine	Snapshot Periode, Pra-pemrosesan, FCM Engine	HR menjalankan klasterisasi pada periode yang sudah ditutup, sistem membuat snapshot konfigurasi variabel, melakukan normalisasi nilai dan perhitungan skor akhir, kemudian menjalankan algoritma FCM melalui Bull Queue	Sistem membuat record KpiVariabelSnapshot, menghitung nilai normalisasi dan menyimpannya ke kpi_penilaian, FCM Engine menjalankan 6 langkah algoritma sesuai konfigurasi periode, hasil klasterisasi tersimpan beserta Silhouette Coefficient	Valid
6	Reproducibility eksekusi FCM	LCG, FCM Engine, Snapshot Periode	HR menjalankan klasterisasi pada periode yang sama sebanyak beberapa kali secara berturut-turut tanpa mengubah konfigurasi atau data input	Sistem menghasilkan hasil klasterisasi yang identik pada setiap eksekusi, termasuk cluster setiap karyawan, derajat keanggotaan, dan nilai	Valid

				Silhouette Coefficient	

4.4.3 Pengujian *Validitas Cluster*

Pengujian validitas *cluster* dilakukan untuk memvalidasi dua aspek penting dari implementasi algoritma *Fuzzy C-Means* pada sistem, yaitu sensitivitas hasil klasterisasi terhadap parameter konfigurasi yang memvalidasi pemilihan jumlah *cluster* (c) dan pangkat pembobot (w) sebagai parameter *default* sistem, serta *reproducibility* hasil pada eksekusi berulang dengan konfigurasi yang sama yang memvalidasi efektivitas *Linear Congruential Generator* (LCG) pada sub-bab 4.3.4 dalam menjamin konsistensi hasil. Pengujian dilakukan menggunakan data simulasi yang dihasilkan dari mekanisme *seeders* sistem dan dirancang untuk mengilustrasikan karakteristik distribusi kinerja yang realistis pada konteks operasional logistik, dilaksanakan pada periode April 2026 dengan 45 karyawan yang tersebar di seluruh departemen dan jabatan yang ada di sistem.

1. Skenario Pengujian Sensitivitas Parameter

   Pengujian sensitivitas parameter dilakukan dengan menjalankan algoritma *Fuzzy C-Means* menggunakan dua belas kombinasi konfigurasi parameter yang berbeda, mencakup variasi jumlah *cluster* (c) pada nilai 2, 3, 4, dan 5 serta pangkat pembobot (w) pada nilai 1.5, 2.0, dan 2.5, di mana c=5 sengaja ditambahkan di luar batasan awal sistem yang menerapkan c=3 sebagai *default* untuk memberikan pembanding terhadap pilihan jumlah *cluster* yang lebih besar. Setiap konfigurasi dievaluasi dengan menghitung nilai *Silhouette Coefficient* beserta kategori strukturnya berdasarkan ambang batas pada sub-bab 4.3.5, di mana konfigurasi *default* sistem (c=3, w=2.0) dievaluasi bersamaan dengan konfigurasi alternatif untuk memvalidasi apakah pilihan *default* tersebut menghasilkan kualitas separasi yang memadai.

Tabel 18. Hasil Pengujian Sensitivitas Parameter

No	Konfigurasi	Jumlah *Cluster* (c)	Pangkat Pembobot (w)	*Silhouette Coefficient*	Kategori Struktur
1	*Config A*	2	1,5	0.5758	Struktur Sedang
2	*Config B*	2	2,0	0.5737	Struktur Sedang
3	*Config C*	2	2,5	0.5737	Struktur Sedang
4	*Config D*	3	1,5	0.6522	Struktur Sedang
5	*Config E*	3	2,0	0.6522	Struktur Sedang
6	*Config F*	3	2,5	0.6522	Struktur Sedang
7	*Config G*	4	1,5	0.6044	Struktur Sedang
8	*Config H*	4	2,0	0.6044	Struktur Sedang
9	*Config I*	4	2,5	0.6014	Struktur Sedang
10	*Config J*	5	1,5	0.5820	Struktur Sedang
11	*Config K*	5	2,0	0.6650	Struktur Sedang
12	*Config L*	5	2,5	0.5675	Struktur Sedang

2. Analisis dan Justifikasi Pemilihan Jumlah *Cluster*

Hasil pengujian sensitivitas parameter menunjukkan bahwa seluruh konfigurasi menghasilkan *Silhouette Coefficient* (SC) pada kategori Struktur Sedang, sehingga semua konfigurasi memiliki separasi *cluster* yang valid dan dapat diinterpretasikan. Meskipun demikian, terdapat tiga pertimbangan utama yang menjadi dasar pemilihan $c = 3$ sebagai parameter *default* sistem.

Dari sisi matematis, konfigurasi $c = 3$ menghasilkan SC sebesar 0.6522 yang stabil pada seluruh variasi pangkat pembobot, hanya berselisih 0.0128 atau 1.96 persen dari SC tertinggi yang dicapai $c = 5$ dengan $w = 2.0$ (0.6650), sehingga perbedaannya tidak signifikan secara praktis. Konfigurasi $c = 3$ juga secara konsisten mengungguli $c = 2$ yang berkisar pada 0.5737–0.5758 dengan selisih 12.0 persen,

serta $c = 4$ yang berkisar pada 0.6014–0.6044 dengan selisih 7.9 persen.

Dari sisi *robustness*, konfigurasi $c = 5$ menunjukkan sensitivitas tinggi terhadap pangkat pembobot dengan rentang SC 0.5675–0.6650 atau variasi sebesar 0.0975, sehingga pemilihan w yang kurang tepat pada $c = 5$ dapat menghasilkan kualitas *cluster* yang lebih buruk dibandingkan $c = 3$ dengan kombinasi w manapun. Sebaliknya, $c = 3$ menunjukkan stabilitas sempurna dengan SC identik pada seluruh variasi w yang diuji.

Dari sisi kebutuhan bisnis, $c = 3$ selaras dengan kebutuhan PT Arete Logistik Indonesia yang mengkategorikan kinerja karyawan ke dalam tiga tingkat, yaitu *High Performer*, *Standard*, dan *Low Performer* yang telah diterapkan oleh HR dalam pengambilan keputusan terkait penghargaan, pengembangan, dan tindak lanjut perbaikan. Konfigurasi $c = 2$ dinilai terlalu biner sehingga tidak memberikan granularitas yang cukup untuk membedakan karyawan dengan kinerja menonjol dari kinerja rata-rata, sementara $c = 4$ atau $c = 5$ berpotensi terlalu granular sehingga menyulitkan interpretasi manajemen. Berdasarkan ketiga aspek tersebut, pemilihan $c = 3$ dengan $w = 2.0$ sebagai parameter *default* sistem terbukti memiliki dasar yang kuat baik dari sisi matematis, *robustness* terhadap variasi parameter, maupun kesesuaian dengan kebutuhan bisnis.

3. Skenario Pengujian *Reproducibility*

Pengujian dilakukan dengan menjalankan algoritma *Fuzzy C-Means* sebanyak lima kali pada konfigurasi default sistem (c=3, w=2.0) untuk periode April 2026, dengan mencatat nilai *Silhouette Coefficient* dan label cluster pada setiap *run* untuk dua karyawan sampel representatif, yaitu Ali Usman (Driver) yang penilaian KPI-nya menggunakan *MitraStrategy* dan Vina Safanah (Staff Accounting) yang menggunakan *JobSpecificStrategy* sesuai implementasi pada sub-bab 4.3.1. Pemilihan kedua sampel ini didasarkan pada dua pertimbangan, yaitu keduanya mewakili dua *strategy* berbeda dari *Strategy Pattern* yang

diimplementasikan pada sistem sehingga pengujian *reproducibility* sekaligus memvalidasi konsistensi hasil lintas mekanisme resolusi variabel KPI, serta keduanya mewakili dua label cluster yang berbeda yaitu *High Performer* dan *Standard* sehingga pengujian memvalidasi konsistensi pelabelan cluster pada tingkat kinerja yang berbeda.

Tabel 19. Hasil Pengujian *Reproducibility*

No	***Run* ke-**	***Silhouette Coefficient***	***Cluster* Sampel Ali Usman**	***Cluster* Sampel Vina Safanah**	**Status**
1	1	0.6522	*High Performer*	*Standard*	*Baseline*
2	2	0.6522	*High Performer*	*Standard*	Identik
3	3	0.6522	*High Performer*	*Standard*	Identik
4	4	0.6522	*High Performer*	*Standard*	Identik
5	5	0.6522	*High Performer*	*Standard*	Identik

4. Analisis *Reproducibility*

Hasil pengujian *reproducibility* menunjukkan bahwa kelima eksekusi algoritma menghasilkan *output* yang identik secara presisi pada seluruh dimensi yang diukur, yaitu nilai *Silhouette Coefficient* sebesar 0.6522 tanpa variasi desimal, label *cluster* Ali Usman secara konsisten sebagai *High Performer*, dan label *cluster* Vina Safanah secara konsisten sebagai *Standard*. Konsistensi ini membuktikan bahwa implementasi *Linear Congruential Generator* pada sub-bab 4.3.4 berhasil menjamin sifat deterministik algoritma *Fuzzy C-Means*, di mana *seed* yang diturunkan dari *idPeriode* menghasilkan deret bilangan acak yang sama pada setiap eksekusi.

Konsistensi *reproducibility* ini memiliki tiga implikasi praktis bagi sistem, yaitu kemampuan melakukan *rerun* klasterisasi tanpa hasil yang berbeda, hasil yang dapat ditelusuri kembali (*auditable*), dan perlindungan dari risiko inkonsistensi pada implementasi FCM dengan inisialisasi acak murni. Konsistensi hasil lintas dua *strategy* yang berbeda pada karyawan sampel juga memvalidasi bahwa integrasi antara *Strategy Pattern* dan algoritma klasterisasi berjalan secara konsisten dan dapat direproduksi.

BAB V PENUTUP

5.1 Kesimpulan

Berdasarkan hasil penelitian dan pembahasan yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Sistem klasterisasi kinerja karyawan berbasis algoritma *Fuzzy C-Means* berhasil dikembangkan menggunakan *Composable Architecture* yang mengintegrasikan *Strategy Pattern* dan *Decorator Pattern*. *Strategy Pattern* melalui tiga variasi strategi (*DefaultStrategy*, *JobSpecificStrategy*, dan *MitraStrategy*) terbukti efektif mengatasi perbedaan karakteristik resolusi variabel KPI antar jabatan, sedangkan *Decorator Pattern* dalam bentuk fungsional memberikan komposabilitas pada resolusi variabel sehingga setiap strategi dapat memilih kombinasi *layer* yang sesuai tanpa duplikasi kode, dengan mekanisme deduplikasi berdasarkan prioritas jabatan, departemen, dan *global*.
2. Integrasi *Linear Congruential Generator* (LCG) dengan *seed* deterministik dari identitas periode penilaian berhasil menjamin sifat *reproducibility* algoritma *Fuzzy C-Means*. Pengujian dengan lima eksekusi berulang pada konfigurasi *default* menghasilkan nilai *Silhouette Coefficient* dan label *cluster* yang identik, sehingga sistem tidak rentan terhadap variabilitas hasil yang umum terjadi pada implementasi *Fuzzy C-Means* dengan inisialisasi acak murni.
3. Pengujian validitas *cluster* dengan dua belas konfigurasi parameter menghasilkan nilai *Silhouette Coefficient* sebesar 0,6522 untuk konfigurasi tiga *cluster* yang stabil pada seluruh variasi pangkat pembobot, berada pada kategori Struktur Sedang, mengungguli konfigurasi dua *cluster* sebesar 12,0 persen dan empat *cluster* sebesar 7,9 persen, sekaligus menunjukkan stabilitas yang lebih baik dibandingkan konfigurasi lima *cluster* yang sensitif terhadap variasi parameter.
4. Pengujian *Black Box* terhadap 29 kebutuhan pengguna dengan 60 skenario uji yang dipetakan langsung ke kode URS menghasilkan status valid pada seluruh skenario, sehingga memvalidasi bahwa sistem telah memenuhi seluruh kebutuhan fungsional yang ditetapkan untuk ketiga aktor pengguna

(HR, atasan, dan karyawan) sesuai spesifikasi yang disepakati dengan PT Arete Logistik Indonesia.

5.2 Saran

Berdasarkan hasil penelitian dan pembahasan yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Disarankan untuk mengembangkan integrasi sistem dengan modul presensi dan modul izin yang sudah disediakan struktur datanya pada sistem, sehingga variabel KPI yang berkaitan dengan kehadiran dan kedisiplinan karyawan dapat dihitung secara otomatis tanpa intervensi manual dari atasan. Integrasi ini akan meningkatkan objektivitas penilaian dan mengurangi beban administratif pada saat periode penilaian berlangsung.
2. Untuk meningkatkan akurasi penilaian dan mengakomodasi konteks kerja yang dinamis, direkomendasikan untuk menambahkan mekanisme penilaian *360-degree feedback* yang memungkinkan karyawan dinilai tidak hanya oleh atasan langsung, tetapi juga oleh rekan kerja, bawahan, dan diri sendiri. Implementasi mekanisme ini perlu mempertimbangkan pembobotan yang berbeda untuk setiap sumber penilaian agar tetap selaras dengan struktur hierarki organisasi.
3. Untuk meningkatkan interpretabilitas hasil klasterisasi bagi manajemen, direkomendasikan untuk menambahkan fitur *explainable clustering* yang menjelaskan secara naratif mengapa seorang karyawan berada pada cluster tertentu beserta variabel KPI yang paling memengaruhi penempatan tersebut. Fitur ini akan membantu HR dan atasan dalam memberikan umpan balik yang konkret kepada karyawan terkait area kinerja yang perlu ditingkatkan.
4. Disarankan untuk mengembangkan sistem rekomendasi pengembangan karyawan berdasarkan hasil klasterisasi, yang secara otomatis menghubungkan label cluster dan area perbaikan karyawan dengan program pelatihan, *mentoring*, atau pengembangan karier yang sesuai. Sistem rekomendasi ini akan menjadikan hasil klasterisasi tidak hanya sebagai laporan kinerja, tetapi juga sebagai *trigger* untuk tindak lanjut pengembangan yang konkret.

DAFTAR PUSTAKA

Ali Quaosar, G. M. A., & Rahman, Md. S. (2021). Human Resource Information Systems (HRIS) of Developing Countries in 21st Century: Review and Prospects. *Journal of Human Resource and Sustainability Studies*, *09*(03), 470–483. https://doi.org/10.4236/jhrss.2021.93030

Anindita, A., & Rahayu, W. I. (2021). SISTEM PENDUKUNG KEPUTUSAN PENILAIAN KINERJA KARYAWAN PADA KANDATEL BONE MENGGUNAKAN METODE SAW. *Antivirus : Jurnal Ilmiah Teknik Informatika*, *15*(1), 44–61. https://doi.org/10.35457/antivirus.v15i1.1358

Ardiansyah, Risnita, & Jailani, M. S. (2023). *Teknik Pengumpulan Data Dan Instrumen Penelitian Ilmiah Pendidikan Pada Pendekatan Kualitatif dan Kuantitatif*. http://ejournal.yayasanpendidikandzurriyatulquran.id/index.php/ihsan

Fariz, M., Lazuardy, S., & Anggraini, D. (2022). Modern Front End Web Architectures with React.Js and Next.Js. *International Research Journal of Advanced Engineering and Science*, *7*(1), 132–141.

Fatinah, H. N. (2022). HUMAN RESOURCE INFORMATION SYSTEM : Fungsi, Peran, Manfaat, dan Penerapan Pada Wordpress. Dalam *Applied Business and Administration Journal* (Vol. 1).

Fitriana, H. (2022). *Pengaruh Penilaian Kinerja Terhadap Produktivitas Kerja Karyawan*. https://jurnal.smartindo.org/index.php/ekobil

Hapsari, R. K., Indriyani, T., & Sulaksono, D. H. (2025). *BUKU AJAR DATA MINING CV. LUMINARY PRESS INDONESIA*. www.luminarypress.id

Hossain, Q., Ikbal, Md. Z., & Rahman, Md. M. (2025). A META DATA-DRIVEN DECISION SUPPORT IN HUMAN CAPITAL MANAGEMENT: REVIEWING HRIS AND PREDICTIVE ANALYTICS INTEGRATION. *ASRC Procedia: Global Perspectives in Science and Scholarship*, *01*(01), 215–246. https://doi.org/10.63125/xgew7q22

Kevin, A., & Kurniawan, H. C. (2023). Analisis Pengaruh Design Pattern Terhadap Pemeliharaan Perangkat Lunak Learning Management System. *Jurnal Telematika*, *18*(1).

Kumar, S., & Padur, R. (2025). The Future of Enterprise ERP Modernization with AI: From Monolithic Systems to Generative, Composable, and Autonomous Platforms Journal of Artificial Intelligence, Machine Learning and Data Science. *J Artif Intell Mach Learn & Data Sci 2025*, *3*(1), 2958–2961. https://doi.org/10.51219/JAIMLD/shravan

Kurmangali, A., Rana, M. E., & Rahman, W. N. W. A. (2022). *Impact of Abstract Factory and Decorator Design Patterns on Software Maintainability: Empirical Evaluation using CK Metrics*. IEEE.

Makris, A., Tserpes, K., Spiliopoulos, G., Zissis, D., & Anagnostopoulos, D. (2021). MongoDB Vs PostgreSQL: A comparative study on performance aspects. *GeoInformatica*, *25*(2), 243–268. https://doi.org/10.1007/s10707-020-00407-w

Mohlala, T. T. N., Mehlwana, L. L. M., Nekhavhambe, U. P., Thango, B., & Matshaka, L. (2024). *Strategic Innovation in HRIS and AI for Enhancing Workforce Productivity in SMEs: A Systematic Review*. https://doi.org/10.20944/preprints202409.1996.v1

Muhamad Saepuloh, A., & Seliwati. (2022). PERANCANGAN SISTEM INFORMASI MANAJEMEN PROYEK DENGAN MENGGUNAKAN SOFTWARE NEST.JS BERBASIS WEB DI. *Jurnal Informatika dan Komputer*. http://journal.piksi.ac.id/index.php/INFOKOM

Nasrullah, A. H., Fajar, A. M., Taufiq, M. A., Rahmat, N., & Adiba, F. (2024). Evaluation Of Fuzzy C-Means Method For District Clustering. *Media of Computer Science*, *1*(2), 117–128. https://doi.org/10.69616/mcs.v1i2.203

Nugraha, A., Nurdiawan, O., & Dwilestari, G. (2022). PENERAPAN DATA MINING METODE K-MEANS CLUSTERING UNTUK ANALISA PENJUALAN PADA TOKO YANA SPORT. Dalam *Jurnal Mahasiswa Teknik Informatika)* (Vol. 6, Nomor 2).

Nurwahidah, A., Sawal, A., Afifudin, M. T., & Sari, H. (2021). PERANCANGAN KEY PERFORMANCE INDICATOR (KPI) SEBAGAI DASAR PENGUKURAN KINERJA KARYAWAN DI GUDANG SPAREPART PADA PT XYZ. *ARIKA*, *15*(2).

Ramdany, S. (2024). Penerapan UML Class Diagram dalam Perancangan Sistem Informasi Perpustakaan Berbasis Web. *Journal of Industrial and Engineering System*, *5*(1), 30–41. https://doi.org/10.31599/2e9afp31

Rizal, A., Novitasari, D. C. R., & Hafiyusholeh, Moh. (2022). Pengelompokan Karyawan Berdasarkan Kesalehan Menggunakan Perbandingan Fuzzy C-Means, K-Means, dan Probabilistic Distance Clustering. *Jurnal Fourier*, *11*(2), 69–77. https://doi.org/10.14421/fourier.2022.112.69-77

Rochman, E. M. S., Miswanto, & Suprajitno, H. (2022). COMPARISON OF CLUSTERING IN TUBERCULOSIS USING FUZZY C-MEANS AND K-MEANS METHODS. *Communications in Mathematical Biology and Neuroscience*, *2022*. https://doi.org/10.28919/cmbn/7335

Ruan, Y., & Joe-Wong, C. (2022). *FedSoft: Soft Clustered Federated Learning with Proximal Local Updating*. www.aaai.org

Rustiawan, I., Purwati, S., Kraugusteeliana, K., & Ady Bakri, A. (2023). *Teknik Penilaian Kinerja Karyawan Menggunakan Metode Behaviour Anchor Rating Scale dalam Pemeringkatan Karyawan Terbaik* (Vol. 2, Nomor 3). https://ejournal.sidyanusa.org/index.php/jkdn

Rusum, G. P., & Anasuri, S. (2023). Composable Enterprise Architecture: A New Paradigm for Modular Software Design. *International Journal of Emerging Research in Engineering and Technology*, *4*. https://doi.org/10.63282/3050-922x.ijeret-v4i1p111

Su, R., & Li, X. (2024). Modular Monolith: Is This the Trend in Software Architecture? *Proceedings - 2024 IEEE/ACM International Workshop New Trends in Software Architecture, SATrends 2024*, 10–13. https://doi.org/10.1145/3643657.3643911

Suhada, M. D., & Ali, H. (t.t.). *Komponen yang mempengaruhi Human Resource Information System : Database, Software dan Regulasi*. https://doi.org/10.38035/jgit.v1i3

Syukron, H., Fauzi Fayyad, M., Junita Fauzan, F., Ikhsani, Y., & Rizkya Gurning, U. (2022). *Perbandingan K-Means K-Medoids dan Fuzzy C-Means untuk Pengelompokan Data Pelanggan dengan Model LRFM*. *2*, 76–83.

Teknologi, J. (2021). *Rancang Bangun Sistem Human Resource Management pada PT.Batang Hari Barisan dengan Berbasis WEB*. *Diterbitkan*(2), 31–43. https://doi.org/10.37034/komtekinfo.v8i3.213

Togatorop, P. R., Simanjuntak, R. P., Manurung, S. B., & Silalahi, M. C. (2021). PEMBANGKIT ENTITY RELATIONSHIP DIAGRAM DARI SPESIFIKASI KEBUTUHAN MENGGUNAKAN NATURAL LANGUAGE PROCESSING UNTUK BAHASA INDONESIA. *Jurnal Komputer dan Informatika*, *9*(2), 196–206. https://doi.org/10.35508/jicon.v9i2.5051

Trias Hanggara, B., Hasan Nasrullah, M., & Pramono, D. (2022). *J-INTECH (Journal of Information and Technology) Analisis Perbandingan Performa Framework NestJS dan Lumen Pada Studi Kasus Aplikasi Berbasis REST API*. https://creativecommons.org/licenses/by-sa/4.0/

Wu, W. T., Li, Y. J., Feng, A. Z., Li, L., Huang, T., Xu, A. D., & Lyu, J. (2021). Data mining in clinical big data: the frequently used databases, steps, and methodological models. Dalam *Military Medical Research* (Vol. 8, Nomor 1). BioMed Central Ltd. https://doi.org/10.1186/s40779-021-00338-z

Wulandari, L., & Yogantara, B. O. (2022). Algorithm Analysis of K-Means and Fuzzy C-Means for Clustering Countries Based on Economy and Health. *Faktor Exacta*, *15*(2). https://doi.org/10.30998/faktorexacta.v15i2.12106

Xiong, W., Legrand, E., Åberg, O., & Lagerström, R. (2022). Cyber security threat modeling based on the MITRE Enterprise ATT&CK Matrix. *Software and*

Systems Modeling, *21*(1), 157–177. https://doi.org/10.1007/s10270-021-00898-7

Zahro, I. H., Rosyidah, U. A., & Handayani, L. (2024). Implementasi Algoritma Fuzzy C-Means untuk Pengelompokkan Provinsi di Indonesia Berdasarkan Kualitas Perguruan Tinggi. *BIOS : Jurnal Teknologi Informasi dan Rekayasa Komputer*, *5*(1), 80–86. https://doi.org/10.37148/bios.v5i1.102